

Este texto foi construído para suprir, em parte, a falta de acesso gratuito a livro em português. São as minhas notas de aula da disciplina Linguagens Formais e Computabilidade. O objetivo é que auxilie você a acompanhar as aulas e permita uma consulta rápida, sem ter de retornar aos vídeos. Devido à velocidade da escrita, para tentar acompanhar o conteúdo das aulas, provavelmente terá erros.

Todo comentário é bem vindo!

Jânio Canuto

Motivação

Esta disciplina trata fundamentalmente do estudo dos modelos teóricos e das limitações da “computação”. É uma área que tem suas origens na década de 30 (perceba que os computadores eletrônicos nem existiam ainda), quando matemáticos estavam tentando entender o significado preciso de uma “computação”. A questão essencial era se todo problema matemático podia ser resolvido de forma sistemática, ou seja, se há um “algoritmo” para resolver qualquer problema. Essa área pode ser dividida em três subáreas: Teoria dos Autômatos, Teoria da Computabilidade e Teoria da Complexidade.

Perceba que no parágrafo anterior eu usei os termos computação e algoritmo entre aspas. Isso porque apesar de termos noções intuitivas sobre o que são, os matemáticos precisavam de definições precisas desses termos para poder fornecer provas para as questões propostas. Intuitivamente, um algoritmo é uma sequência finita de operações, cada uma das quais executada em tempo finito, que resolve um dado problema. A execução do algoritmo é a computação.

Dessa descrição surge um problema, quem vai executar essas tarefas? Quais as capacidades das entidades que computam? Esse é o assunto tratado na Teoria dos Autômatos, os modelos teóricos de computação. Tendo um modelo adequado surge o segundo problema, qualquer problema pode ser resolvido por estas máquinas? Esse é o assunto central da Teoria da Computabilidade, classificar os problemas entre solucionáveis e não solucionáveis. Por fim, a Teoria da Complexidade faz uma classificação dos problemas solucionáveis em termos da “dificuldade” de sua computação, em termos da quantidade de operações ou memória necessárias para a sua execução (esse último assunto não faz parte da ementa desta disciplina, mas veremos se tivermos tempo).

Outra forma de entender essa disciplina é que o nosso objeto de estudo são coisas chamadas “Linguagens Formais”, como diz o título. Mas, para estudar esse objeto, precisaremos de duas ferramentas auxiliares que são as máquinas (autômatos), que são reconhecedores de linguagens, e gramáticas, que são geradores de linguagens. Além disso vamos tentar associar essas linguagens ao problema da Computabilidade.

O objetivo fundamental da disciplina é apresentar a vocês os conceitos fundamentais da teoria da computação. Ao fazer isso, objetivos específicos devem ser alcançados, que são: melhorar a habilidade de tratar conceitos formais abstratos, apresentar as diferentes classes de linguagens, preparar para o posterior estudo de Compiladores e Processamento de Linguagem.

Aproveitem o curso e atenção, se ligue aí que é hora da revisão! (by Telecurso 2000)

Revisão

Antes de iniciar os assuntos da disciplina propriamente dita, vamos para uma rápida revisão de alguns conceitos que serão fundamentais ao longo do curso.

Sequências (Tuplas)

Uma sequência é uma coleção de objetos com ordem definida e que permite a repetição de elementos. Uma sequência de k elementos é chamada de k -upla. Neste texto utilizaremos parênteses para a listagem dos elementos de uma sequência.

Curiosidade, o termo tupla vem da sequência de nomes: dupla, tripla, quádrupla, quártupla, sêxtupla, sétupla, óctupla, ...

Exemplo 1

- a) $()$ é uma 0-upla (ou tupla vazia ou sequência vazia)
 - b) (a) é uma 1-upla (ou monupla)
 - c) $(+, -)$ é uma 2-upla (ou dupla ou par ordenado)
 - d) $(0, 1, 2)$ é uma 3-upla (ou tripla ou trio ordenado).
 - e) $(\varsigma, \Delta, \text{率}, \text{妙})$ é uma 4-upla (ou quádrupla ou tétrade).
 - f) $(0, 2, 1)$ é uma 3-upla diferente daquela em (d), pois a ordem importa.
 - g) $(0, 1, 1, 2)$ é uma 4-upla diferente daquelas em (d) pois a repetição importa.
-

Subsequências

Dizemos que b é uma subsequência de a se b pode ser construído através da remoção de zero ou mais elementos de a .

Exemplo

- a) $(0, 1, 2)$ é uma subsequência de $(0, 1, 2)$.
 - b) $(1, 2, 3)$ é uma subsequência de $(0, 1, 2, 3, 4, 5)$.
 - c) $(0, 2, 5)$ é uma subsequência de $(0, 1, 2, 3, 4, 5)$.
-

Conjuntos

Um conjunto é uma coleção de objetos sem ordem definida e que não permite a repetição de objetos.

Pertinência

Os objetos que fazem parte de um conjunto são chamados de elementos ou membros do conjunto. Para indicar a relação de pertinência entre um elemento x e um conjunto A escrevemos:

- $x \in A$, lê-se “ x pertence a A ”; ou
- $x \notin A$, lê-se “ x não pertence a A ”.

Praticamente qualquer objeto pode ser elemento de um conjunto, com duas exceções:

- $A \notin A$ (um conjunto não pode ser elemento de si mesmo); e
- dados dois conjuntos A e B , se $A \in B$ então $B \notin A$.

Cardinalidade

A cardinalidade de um conjunto A , denotada por $|A|$, é a quantidade de elementos que fazem parte deste conjunto.

Igualdade

Dois conjuntos A e B são iguais se possuem os mesmos elementos.

Descrevendo Conjuntos

Conjuntos podem ser descritos por enumeração, textualmente ou por funções de predicado. Neste texto usaremos chaves para a listagem dos objetos de um conjuntos e letras maiúsculas para nomeá-los.

Na enumeração cada um dos objetos que estão no conjunto é escrito entre chaves ou implicitamente através do uso de elipses para indicar a repetição de um padrão.

Na descrição textual dizemos as propriedades que um objeto deve ter para fazer parte de um conjunto.

A definição através de funções de predicado é similar à textual, mas é mais precisa. Uma função de predicado é uma função que tem como resultado duas possibilidades: verdadeiro ou falso. De forma similar à descrição textual, a função de predicado descreve propriedades dos objetos. Os objetos para os quais a função é verdadeira fazem parte do conjunto os demais não fazem parte.

Exemplo 2

- $A = \{a, b, d, e\}$ é um conjunto de cardinalidade 4 descrito por enumeração.
 - $B = \{2, 4, 6, \dots, 20\}$ é um conjunto de cardinalidade 10 descrito por enumeração. As elipses são usadas para indicar a repetição de um padrão (a enumeração completa seria $B = \{2, 4, 6, 8, 10, 12, 14, 16, 18, 20\}$). Muito cuidado ao usar elipses para não gerar descrições ambíguas.
 - $C = \{1, 2, 3, \dots\}$ é um conjunto de cardinalidade ∞ descrito por enumeração. As elipses ao fim indicam a continuação do padrão *ad infinitum*.
 - $D = \{\text{Bares de Aracaju}\}$ é um conjunto definido textualmente.
 - $E = \{\text{Números pares não nulos menores que vinte}\}$ é uma descrição textual que gera um conjunto idêntico a B .
 - $F = \{x \in \mathbb{N} : x > 0, x \leq 20, x = 0 \pmod{2}\}$ é uma descrição por funções de predicado de um conjunto idêntico a B e E . Os dois pontos são lidos como “tal que” e as vírgulas como “e”. Portanto a frase pode ser lida como: “ F é formado por números naturais tais que são maiores que zero e menores ou iguais a 20 e o resto da divisão inteira por 2 é zero”.
-

Conjuntos Importantes

Alguns conjuntos são recorrentes na matemática e por isso têm nomes fixos. Eis alguns deles:

- $\emptyset = \{ \}$, conjunto vazio;
- $\mathbb{N} = \{1, 2, 3, \dots\}$, conjunto dos números naturais;
- $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$, conjunto dos números inteiros;
- $\mathbb{Q} = \{x/y: x \in \mathbb{Z}, y \in \mathbb{Z}, y \neq 0\}$, conjunto dos números racionais;
- $\mathbb{R} = \{\text{quantidades representadas por uma expansão decimal infinita}\}$, conjunto dos números reais.

Subconjuntos

Se A e B são conjuntos, então dizemos que A é subconjunto de B , denotado por $A \subseteq B$, se todos os elementos de A também são elementos de B . Todo conjunto é subconjunto de si mesmo e o conjunto vazio é subconjunto de todos os conjuntos, então, para um conjunto A qualquer, temos que $A \subseteq A$ e $\emptyset \subseteq A$.

Se A é subconjunto de B e $A \neq B$, dizemos que A é um subconjunto próprio de B , denotado por $A \subset B$.

Perceba que esta relação pode ser usada para definir a igualdade entre conjuntos: se $A \subseteq B$ e $B \subseteq A$ então $A = B$.

Conjunto Potência

O conjunto potência (ou conjunto de partes) de um conjunto A , denotado por $\mathcal{P}(A)$ é o conjunto de todos os subconjuntos de A . Ou seja, $\mathcal{P}(A) = \{X: X \subseteq A\}$. Por consequência da definição anterior, temos que $\emptyset \in \mathcal{P}(A)$ e $A \in \mathcal{P}(A)$.

Operações

Sejam A e B dois conjuntos, define-se então as seguintes operações:

- União: $A \cup B = \{x: x \in A \text{ ou } x \in B\}$;
- Interseção: $A \cap B = \{x: x \in A, x \in B\}$;
 - Dois conjuntos são ditos disjuntos se $A \cap B = \emptyset$
- Diferença: $A \setminus B = \{x: x \in A, x \notin B\}$;
- Produto Cartesiano: $A \times B = \{(x, y): x \in A, y \in B\}$
- Complemento: $\neg A = \{x: x \notin A\}$

Partições

Uma partição de um conjunto A é a divisão deste conjunto em subconjuntos

$A_1, A_2, \dots, A_k$ tais que $\bigcup_{i=1}^k A_i = A$ e $A_i \cap A_j = \emptyset, \forall i \neq j$.

Funções

Uma função é uma relação entre dois conjuntos que associa a cada elemento do primeiro exatamente um elemento do segundo. A função tem uma assinatura e uma definição (descrição).

Assinatura

A assinatura descreve quais os conjuntos que estão sendo relacionados, por exemplo, $f: X \rightarrow Y$ indica que a função f atribui a cada elemento do conjunto X (chamado de domínio da função) um elemento do conjunto Y (chamado de contradomínio da função).

Definição

A definição da função diz de que forma a associação entre o domínio e o contradomínio é realizada. O subconjunto de Y que de fato é usado é chamado de imagem da função.

A definição de uma função pode ser feita por enumeração (se seu domínio é finito), textualmente, por fórmula ou recursão.

Aridade

A quantidade de argumentos de uma função é chamada de aridade da função. Uma função com 2 argumentos é 2-ária (ou binária) e com k argumentos é k -ária. Outra forma de se definir isso é que o domínio de uma função k -ária é o produto cartesiano de k conjuntos.

O termo “função binária” às vezes também é usado para designar funções cujo contradomínio tem apenas dois elementos, como as funções Booleanas.

Tipos

Uma função $f: X \rightarrow Y$ é injetora se dois elementos distintos de X , x e x' , são mapeados em dois elementos distintos de Y , ou seja $f(x) \neq f(x')$.

Se para todos elemento y do contra-domínio existe um elemento x do domínio tal que $f(x) = y$, ou seja, se o contradomínio é igual à imagem, dizemos que a função é sobrejetora. Se a função é tanto sobrejetora quando injetora dizemos que ela é bijetora.

Uma função binária $f: X \times X \rightarrow \{\text{verdadeiro}, \text{falso}\}$ é dita de equivalência se é reflexiva, simétrica e transitiva. Isto é, dados três elementos a , b e c pertencentes a X temos que $f(a, a) = \text{verdadeiro}$, $f(a, b) = f(b, a)$ e se $f(a, b) = \text{verdadeiro}$ e $f(b, c) = \text{verdadeiro}$ então $f(a, c) = \text{verdadeiro}$.

Operações Booleanas

A operações booleanas são funções binárias sobre o conjunto $B = \{0, 1\}$, representando os valores lógicos falso e verdadeiro, respectivamente. As operações Booleanas básicas incluem a negação ($\neg$), a conjunção ($\wedge$), a disjunção ($\vee$), a disjunção exclusiva ($\oplus$), a equivalência ($\leftrightarrow$) e a implicação ($\rightarrow$). Estas funções estão definidas na tabela abaixo:

$\neg$	$\wedge$	$\vee$	$\oplus$	$\leftrightarrow$	$\rightarrow$
$\neg 0 = 1$	$0 \wedge 0 = 0$	$0 \vee 0 = 0$	$0 \oplus 0 = 0$	$0 \leftrightarrow 0 = 1$	$0 \rightarrow 0 = 1$
$\neg 1 = 0$	$0 \wedge 1 = 0$	$0 \vee 1 = 1$	$0 \oplus 1 = 1$	$0 \leftrightarrow 1 = 0$	$0 \rightarrow 1 = 1$
	$1 \wedge 0 = 0$	$1 \vee 0 = 1$	$1 \oplus 0 = 1$	$1 \leftrightarrow 0 = 0$	$1 \rightarrow 0 = 0$
	$1 \wedge 1 = 1$	$1 \vee 1 = 1$	$1 \oplus 1 = 0$	$1 \leftrightarrow 1 = 1$	$1 \rightarrow 1 = 1$

Provas

Uma prova, na matemática, é uma argumentação válida que determina que uma certa afirmação é verdade. A argumentação é considerada válida se ela não tem erros e cobre todas as possibilidades de aplicação da afirmação. A pergunta principal é “como fazer uma prova?”. Esta pergunta é o mesmo que perguntar “como resolver um problema?”, que claramente não tem resposta definitiva (a vida seria muito chata). A melhor maneira de resolver problemas (e encontrar provas) é o exercício desta tarefa em vários problemas. O matemático George Pólya tem um livro interessante sobre a resolução de problemas (How to Solve It, 1945). Aqui vão algumas dicas:

- Entenda completamente a afirmação a ser provada, sem isso não será possível fazer mais nada;
- Às vezes uma afirmação pode ser dividida em duas ou mais afirmações. Bi-implicações (“se e somente se”) podem ser divididas em duas implicações. Igualdade de conjuntos pode ser separada em duas provas de subconjuntos.
- Tente alguns casos simples;
- Escreva a prova que você (acha) que tem. Às vezes o raciocínio parece perfeito e só descobrimos as falhas quando colocamos no papel;
- Seja paciente e se expresse da forma mais clara e precisa possível.

Apesar de não existir método mágico para a prova de teoremas, conhecer algumas técnicas de prova também pode ajudar. Nas próximas seções vamos examinar algumas delas (atenção, alguns textos classificam as provas de forma diferente).

Prova Direta

Como o nome diz, basta abordar o teorema diretamente.

Exemplo

Se n é inteiro positivo ímpar, n^2 também é.

Prova

Podemos escrever $n = 2k + 1$. Daí temos $n^2 = (2k + 1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$. Como $2(2k^2 + 2k)$ é par e um número par somado a um número ímpar é ímpar, concluimos que n^2 é ímpar.

Prova por Contradição

Consiste em fazer uma hipótese contrária àquilo que se quer provar e chegar a uma impossibilidade (contradição). Daí conclui-se que a hipótese é falsa e o que você queria provar é verdadeiro.

Exemplo

$\sqrt{2}$ é irracional.

Prova

Suponha que $\sqrt{2}$ é racional. Então existem números inteiros a e b tais que $a/b = \sqrt{2}$. a e b são ambos positivos ou ambos negativos. Podemos assumir que são ambos maiores ou iguais a 1, sem perda de generalidade. Além disso, podemos assumir que o maior divisor comum entre eles é 1, se não for o caso os divisores comuns podem ser anulados na divisão. Tomando o quadrado dos dois lados, temos que $a^2 = 2b^2$, o que implica que a^2 é par e, consequentemente, a é par. Então podemos escrever a como $2k$ e daí temos $4k^2 = 2b^2$ que simplifica para $2k^2 = b^2$, que implica em b^2 ser par e consequentemente b ser par também. Então provamos que a e b devem ser par, mas sabemos que eles não são ambos pares porque não têm divisor comum. Logo, temos uma contradição e concluímos que a hipótese inicial, de que $\sqrt{2}$ é racional, é falsa e portanto o teorema é válido.

Prova Construtiva

Em provas de existência, consiste em apresentar um exemplo concreto ou um método para a criação de um objeto com a propriedade desejada.

Exemplo

Existem números irracionais x e y tais que x^y é racional.

Prova

Sejam $x = \sqrt{2}$ e $y = \log_2 k^2$, onde k é um número ímpar maior que 1. Temos então que $x^y = (\sqrt{2})^{\log_2 k^2} = (\sqrt{2})^{2\log_2 k} = 2^{\log_2 k} = k$, que é um número racional. Para completar a prova precisamos mostrar que $\log_2 k^2$ é irracional.

Suponha que $\log_2 k^2$, com k ímpar, é racional, então existem inteiros a e b tais que $a/b = \log_2 k^2$. Isso significa que $2^{a/b} = k^2$ e daí, temos que $(2^{a/b})^b = (k^2)^b$, que simplifica para $2^a = k^{2b}$. Mas 2^a é par e k^{2b} é ímpar, pois k é ímpar. Daí concluímos que é impossível que existam tais a e b e $\log_2 k^2$, com k ímpar, é irracional.

Note que nesse exemplo usamos uma prova por contradição dentro da prova construtiva, não é raro a mistura de técnicas de prova.

Prova Não-Construtiva

Também utilizada em provas de existência, e às vezes chamada de prova indireta de existência. Consiste em provar a existência de um objeto sem nunca construí-lo de fato.

Exemplo

Existem números irracionais x e y tais que x^y é racional.

Prova

Se $(\sqrt{2})^{\sqrt{2}}$ é racional, usamos $x = y = \sqrt{2}$ e a afirmação é válida neste caso.

Se $(\sqrt{2})^{\sqrt{2}}$ é irracional, usamos $x = (\sqrt{2})^{\sqrt{2}}$ e $y = \sqrt{2}$ e $x^y = \left((\sqrt{2})^{\sqrt{2}}\right)^{\sqrt{2}} = (\sqrt{2})^2 = 2$ e

a afirmação também é válida neste caso.

Como todos os casos foram testados, concluímos que o teorema é válido. Note que o teorema foi provado sem dar nenhum caso concreto de existência.

Princípio da Casa de Pombos

Este é um princípio simples com consequências surpreendentes. Se $n + 1$ ou mais objetos são colocados em n caixas, então pelo menos uma caixa tem dois ou mais objetos. Ou seja, se A e B são conjuntos com $|A| > |B|$, não existe função injetora de A em B .

Exemplo

Seja n um inteiro positivo. Toda sequência de $n^2 + 1$ números reais distintos tem uma subsequência de comprimento $n + 1$ que é crescente ou decrescente.

Prova

Seja $(a_1, a_2, \dots, a_{n^2+1})$ uma sequência arbitrária de $n^2 + 1$ números reais distintos. Seja C_i o comprimento da maior subsequência crescente que inicia em a_i e D_i a maior subsequência decrescente que inicia em a_i . Podemos então reescrever o teorema como: existe índice i tal que $C_i \geq n + 1$ ou $D_i \geq n + 1$. Por contradição, vamos assumir que $C_i \leq n$ e $D_i \leq n$ para todo i . Então existem n^2 possíveis pares (C_i, D_i) e temos $n^2 + 1$ elementos na sequência. Isso quer dizer que existem pelo menos dois índices s e t para os quais $(C_s, D_s) = (C_t, D_t)$. Vamos assumir que $s < t$. Como são distintos, ou $a_s < a_t$ ou $a_s > a_t$.

No primeiro caso, como $C_s = C_t$, poderíamos ter uma subsequência crescente de comprimento $C_t + 1$ começando em a_s . Tomando a_s seguido da subsequência de comprimento C_t que inicia em a_t . Então $C_s \neq C_t$, o que é uma contradição.

No segundo caso, como $D_s = D_t$, poderíamos ter uma subsequência decrescente de comprimento $D_t + 1$ começando em a_s . Tomando a_s seguido da subsequência de comprimento D_t que inicia em a_t . Então $D_s \neq D_t$, o que é uma contradição.

Prova por Indução

Essa é uma técnica importante e poderosa. Seja $P(n)$ uma proposição que depende de um número inteiro positivo n . Assuma que queremos provar que $P(n)$ é verdade para todo inteiro positivo. A prova por indução consiste em provar que $P(1)$ é válida (base de indução) e depois assumir que $P(k)$ é válido e mostrar que $P(k + 1)$ também é. (passo de indução).

Na verdade, é possível usar outra base e outro passo, mas a prova seria válida para outro conjunto, por exemplo se a base é a e o passo é $k + 3$ a prova vale para $n = a + 3m$, onde m é um natural. Se a base é 1 e o passo é $2k$, a prova é válida para todas as potências de 2 de n .

Exemplo

Para todo n inteiro positivo, $\sum_{i=1}^n i = \frac{n(n+1)}{2}$

Prova

Base: Se $n = 1$, os dois lados da igualdade são iguais a 1, e a proposição é válida.

Passo: Temos que provar que vale para $k + 1$, ou seja $\sum_{i=1}^{k+1} i = \frac{(k+1)(k+2)}{2}$.

$$\sum_{i=1}^{k+1} i = \sum_{i=1}^k i + (k + 1) = \frac{k(k+1)}{2} + (k + 1) = \frac{(k+1)(k+2)}{2}.$$

Prova por Enumeração

Esse tipo de prova às vezes é esquecido mas é igualmente válido. Ele é equivalente ao método de força bruta para resolução de problemas e pode ser usado sempre que o domínio for finito. Como dito antes, esse é um método válido mas quase sempre é ineficiente.

Exemplo

A fórmula lógica $G = (A \leftrightarrow B) \leftrightarrow ((A \rightarrow B) \wedge (B \rightarrow A))$ é uma tautologia.

Prova

A tabela verdade é a enumeração de todas as possibilidades sobre o domínio.

A	B	$C = A \rightarrow B$	$D = B \rightarrow A$	$E = C \wedge D$	$F = A \leftrightarrow B$	$G = E \leftrightarrow F$
-----	-----	-----------------------	-----------------------	------------------	---------------------------	---------------------------

0	0	1	1	1	1	1
0	1	1	0	0	0	1
1	0	0	1	0	0	1
1	1	1	1	1	1	1

Linguagens Formais

Vamos ver finalmente o que são as tais linguagens formais que vamos estudar. No entanto, antes vamos precisar ver alguns conceitos preliminares para poder defini-las.

Alfabeto

Um alfabeto é um conjunto finito de elementos, chamados símbolos. Neste texto os alfabetos serão representados por letras gregas maiúsculas (quase sempre Σ ou Γ).

Exemplo

- a) $\Sigma_1 = \{0, 1\}$
 - b) $\Sigma_2 = \{a, b, c, d\}$
 - c) $\Gamma_1 = \{if, while, for\}$
 - d) $\Gamma_2 = \{\leftarrow, +, 10, 01\}$
-

Note que um símbolo não precisa ser necessariamente um único caractere ou dígito.

Cadeia (*String*)

Dado um alfabeto Σ , uma cadeia (ou *string*, ou palavra) sobre Σ é uma sequência $s_1 s_2 \dots s_n$, onde $s_i \in \Sigma$ para $1 \leq i \leq n$. Neste texto usaremos letras gregas minúsculas para nomear as cadeias.

Exemplo

- a) $\alpha = 00101$ é uma cadeia sobre $\Sigma = \{0, 1\}$
 - b) $\beta = abracadabra$ é uma cadeia sobre $\Gamma = \{a, b, c, d, \dots, z\}$
 - c) $\gamma = 1 + 1 = 3$ é uma cadeia sobre $\Omega = \{0, 1, 2, 3, 4, 5, +, =\}$
 - o Note que o segundo símbolo “=” é parte da cadeia e não uma igualdade.
 - d) $\delta = 00101$ é uma cadeia sobre $\Pi = \{00, 10, 1\}$
-

Comprimento

O comprimento de uma cadeia ω , $|\omega|$, é o número de símbolos que a compõem.

Exemplo

- a) $\alpha = 00101$ é uma cadeia sobre $\Sigma = \{0, 1\}$; $|\alpha| = 5$
- b) $\beta = abracadabra$ é uma cadeia sobre $\Gamma = \{a, b, c, d, \dots, z\}$; $|\beta| = 11$
- c) $\gamma = 1 + 1 = 3$ é uma cadeia sobre $\Omega = \{0, 1, 2, 3, 4, 5, +, =\}$; $|\gamma| = 5$.

d) $\delta = 00101$ é uma cadeia sobre $\Pi = \{00, 10, 1\}$; $|\delta| = 3$

Concatenação

A concatenação da cadeia $\alpha = a_1 a_2 \cdots a_n$ com a cadeia $\beta = b_1 b_2 \cdots b_m$, denotada por $\alpha\beta$, é a cadeia $a_1 a_2 \cdots a_n b_1 b_2 \cdots b_m$.

Exemplo

- a) Se $\alpha = \text{casa}$ e $\beta = \text{mentos}$, $\alpha\beta = \text{casamentos}$;
 - b) Se $\alpha = 01$ e $\beta = 10$, $\alpha\beta = 0110$.
-

Cadeia Vazia

A cadeia vazia, denotada por ε , é a cadeia de comprimento zero. Note que, dado uma cadeia ω qualquer, $\varepsilon\omega = \omega\varepsilon = \omega$.

Cadeia Reversa

Dado uma cadeia $\alpha: a_1 a_2 \cdots a_n$, a cadeia reversa, denotada por $\alpha^{\mathcal{R}}$ ou α^{-1} , é a cadeia $a_n a_{n-1} \cdots a_2 a_1$. Se $\alpha = \alpha^{\mathcal{R}}$ dizemos que α é um palíndromo.

Exemplo

- a) Se $\alpha = \text{arara}$, $\alpha^{\mathcal{R}} = \text{arara}$. α é um palíndromo;
 - b) Se $\alpha = 12345$, $\alpha^{\mathcal{R}} = 54321$.
-

Subcadeia

β é uma subcadeia de ω se existem cadeias α e γ tais que $\omega = \alpha\beta\gamma$. Ou seja, uma subcadeia de ω é uma sequência formada por zero ou mais elementos consecutivos de ω . Note que ε é subcadeia de qualquer cadeia. E toda cadeia é subcadeia de si mesmo.

Exemplo

- a) Se $\alpha = \text{aracaju}$, $\alpha_1 = \text{caju}$ e $\alpha_2 = \text{aca}$ são subcadeias de α .
-

Potências de Cadeias

Se ω é uma cadeia, a k -ésima potência de ω é a concatenação de ω k vezes. Note que $\omega^0 = \varepsilon$ e $\omega^1 = \omega$. No caso da cadeia ter um único elemento, $\omega = x$, podemos escrever x^k por brevidade.

Exemplo

- a) Se $\alpha = 01$, $\alpha^3 = 010101$;
 - b) Se $\alpha = k$, $\alpha^{10} = kkkkkkkkkk$.
 - c) $10^2 1^3 01^2 = 100111011$
-

Potências de Alfabetos

Σ^k é o conjunto de todas as cadeias de comprimento k sobre o alfabeto Σ .

Exemplo

Seja $\Sigma = \{0, 1\}$

- a) $\Sigma^0 = \{\epsilon\}$;
 - b) $\Sigma^1 = \{0, 1\}$;
 - c) $\Sigma^2 = \{00, 01, 10, 11\}$;
 - d) $\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$.
-

Fecho de Kleene

Dado um alfabeto Σ , o fecho de Kleene de Σ , denotado por Σ^* , é o conjunto de todas as cadeias de todos os comprimentos sobre o alfabeto Σ . Ou seja, é a concatenação de 0 ou mais elementos do alfabeto. Resumindo, $\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i$.

Também é comum a definição do fecho positivo de Σ , denotado por Σ^+ , como sendo a concatenação de um ou mais elementos de Σ . Resumindo, $\Sigma^+ = \bigcup_{i=1}^{\infty} \Sigma^i$ ou simplesmente $\Sigma^+ = \Sigma^* \setminus \{\epsilon\}$.

Exemplo

Se $\Sigma = \{0, 1\}$, Σ^* é o conjunto de todos os números binários mais a cadeia vazia.

Linguagens

Uma linguagem L sobre um alfabeto Σ é um subconjunto de Σ^* . Ou seja, $L \subseteq \Sigma^*$.

Exemplo

- a) $L_1 = \{0, 1, 10\}$
 - b) $L_2 = \{x \in \{0, 1\}^* : x \text{ tem o mesmo número de 0s e 1s}\}$
 - c) $L_3 = \{a^n b^n c^n \in \{a, b, c\}^* : n \geq 0\}$
 - d) $L_4 = \{\epsilon\}$
 - e) $L_5 = \{\}$
 - f) $L_6 = \{x : x \text{ é um programa sintaticamente correto em Haskell}\}$
-

Linguagens e Problemas

Como dito no início do texto, alguns matemáticos estavam tentando descobrir se todos os problemas matemáticos têm solução algorítmica ou não. Mas, pela definição, não é tão óbvio qual a relação das linguagens com problemas matemáticos.

Os problemas vêm em vários tipos (futuramente falaremos mais sobre os tipos de problemas), dentre estes tipos, os Problemas de Decisão são aqueles expressos de tal forma que resposta é binária: “Sim” ou “Não”.

Considere o problema de determinar se um número natural é primo ou não. Um algoritmo para resolver este problema recebe como entrada um número natural e devolve uma resposta binária: “É primo” ou “Não é primo”. Considere a linguagem formada por todos os números primos $P = \{2, 3, 5, 7, 11, 13, 17, 19, \dots\}$. Decidir se um número é primo é o mesmo que decidir se ele pertence ou não a P .

Todo problema de decisão pode ser representado por uma linguagem que representam suas instâncias cuja resposta é “Sim”. Em outras palavras, as linguagens formalizam os problemas.

De um ponto de vista formal, a computação de um problema de decisão consiste em decidir se uma cadeia pertence ou não à linguagem que formaliza o problema.

Perceba que até agora só falamos de problemas de decisão, toda formalização deste ponto em diante tratará deste tipo de problema apenas. Ao fim da disciplina ficará claro porque não vamos nos preocupar com os outros tipos agora.

Classificação das Linguagens

As linguagens formais podem ser divididas em classes dependendo do seu poder de representação. A classificação mais utilizada foi desenvolvida em 1956 pelo linguista Noam Chomsky (*Three Models for the Description of Language*) e é chamada de hierarquia de Chomsky.

Como foi dito na motivação, nosso estudo das linguagens formais passará por duas construções auxiliares que são as máquinas e as gramáticas. A hierarquia de Chomsky classifica as linguagens formais de acordo com o tipo de gramática que gera aquela linguagem ou de máquina que reconhece aquela linguagem.

No momento, vou apenas adiantar que existem as Linguagens Regulares, as Linguagens Livres de Contexto, as Linguagens Recursivas (essa não está na hierarquia original) e as Linguagens Recursivamente Enumeráveis. Além disso essas classes são tais que Regulares $\subseteq$ Livres de Contexto $\subseteq$ Recursivas $\subseteq$ Recursivamente Enumeráveis.

As definições das classes de linguagens serão dadas em cada um dos capítulos seguintes em ordem crescente de tamanho.

Linguagens Regulares

Primeiramente, vamos estudar a classe das linguagens conhecidas como Regulares. Essa é a menor das classes e começaremos pelas máquinas que são capazes de “processar” tais linguagens.

Autômatos Finitos Determinísticos (DFA)

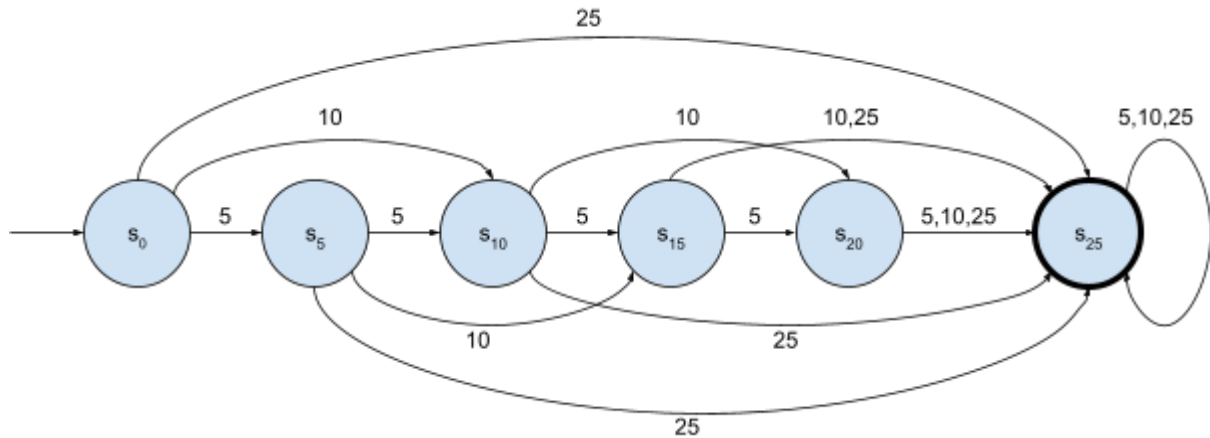
Provavelmente a maioria das pessoas tem uma ideia intuitiva do que é um autômato finito determinístico (DFA - *Deterministic Finite Automata*) mesmo sem nunca ter visto (ou mesmo entender) o significado dessa nomenclatura. Se você já estudou circuitos digitais provavelmente já viu um DFA.

Vamos começar com um problema para dar uma intuição do que seria esse tipo de máquina. Considere que queremos criar uma máquina que dá um pirulito se o cliente inserir 25 centavos. Por simplicidade, assumimos que a máquina não dá troco e foi construída de tal forma que só aceita moedas de 5, 10 e 25 centavos. Também para simplificar o exemplo, apenas a parte da máquina que decide dar o pirulito será modelada, a volta ao estado inicial (começar a contar de novo) será ignorada. Ou seja, precisamos contar as moedas e decidir se o cliente inseriu 25 centavos ou mais.



Para contar quanto dinheiro foi inserido, precisamos nos lembrar do saldo atual. Como a menor moeda é de 5 centavos e as outras são múltiplos de 5, o contador de saldo tem 6 valores possíveis: 0 centavos (antes do cliente iniciar), 5 centavos, 10 centavos, 15 centavos, 20 centavos, e 25 centavos. Perceba que não precisamos contar além de 25 centavos devido às simplificações sugeridas.

Conforme o cliente vai inserindo moedas o contador vai sendo atualizado, ou seja, cada interação do cliente com a máquina provoca uma alteração na memória. Vamos desenhar um diagrama representando a interação do usuário com a máquina. Os círculos representam a memória, o valor é o saldo atual. As setas indicam como a memória é alterada a cada interação, os valores sobre as setas indicam a moeda inserida, se há múltiplos valores quer dizer que qualquer um daqueles foi inserido.



Como o saldo 0 acontece antes do cliente iniciar a interação, inserimos uma seta para indicar por onde começa o processamento. Além disso, como o objetivo é decidir se pelo menos 25 centavos foram depositados, demos uma aparência diferente ao último círculo. Perceba que a máquina só precisa saber em qual círculo ela está agora, não precisa saber o valor numérico do saldo. Então, a quantidade de memória necessária para distinguir entre 6 possíveis casos é de apenas $\lceil \log_2 6 \rceil = 3$ bits.

Suponha que o cliente insere a seguinte sequência de moedas (5,10,5), a máquina parte do primeiro círculo, vai para o segundo, depois para o quarto e finalmente para o quinto; e o pirulito não é liberado. Se a sequência de moedas fosse (10,5,10), a sequência de círculos visitados seria 1, 3, 4, 6 e o pirulito seria fornecido.

Essa máquina pode ser vista como uma função que recebe uma sequência de elementos do conjunto $\{5, 10, 25\}$ como entrada e retorna “Sim” ou “Não”, $f; \{5, 10, 25\}^* \rightarrow \{\text{"Sim"}, \text{"Não"}\}$. Mesmo sem saber precisamente o que é, construímos um autômato finito determinístico.

Definição Formal

Para termos precisão em exatamente o que é ou não é um DFA, precisamos de uma definição formal. É possível ter mais de um estado final? É possível ter mais de um estado inicial? É preciso definir transições para todos os símbolos do alfabeto? Precisa ter um estado final? A transição precisa levar para um único estado? Todas essas questões desaparecem depois da definição formal.

Definição: Autômato Finito Determinístico (DFA)

Um autômato finito determinístico (DFA) é uma 5-upla $M = (Q, \Sigma, \delta, q_0, F)$, onde:

- Q é um conjunto finito, cujos elementos são chamados de *estados*;
- Σ é um conjunto finito, chamado *alfabeto*;
- $\delta: Q \times \Sigma \rightarrow Q$ é uma função, chamada de *função de transição*;
- $q_0 \in Q$, é chamado de *estado inicial*;
- $F \subseteq Q$, são os *estados de aceitação* (ou *estados finais*).

A função de transição é como se fosse o “programa” do autômato, ela diz o que o autômato faz em um passo. Sejam r um estado do autômato e a um símbolo do alfabeto. Se o DFA está no estado r e lê o símbolo a na entrada, ele transita do estado r para o estado $\delta(r, a)$.

Bem, agora vamos alterar um pouco o vocabulário utilizado no exemplo anterior. Projetamos uma máquina que tem 6 estados possíveis, o estado rotulado por “0” é o estado inicial da máquina. O usuário entra com uma cadeia de valores sobre do alfabeto $\{5, 10, 25\}$. Cada elemento dessa cadeia de entrada faz com que a máquina sofra uma transição de estado. O estado s_{25} é especial porque se a cadeia de entrada o leva até esse estado a máquina “Libera” caso contrário “Não Libera”.

A máquina de pirulito é, portanto, um DFA. Temos $Q = \{s_0, s_5, s_{10}, s_{15}, s_{20}, s_{25}\}$, $\Sigma = \{5, 10, 25\}$, $q_0 = s_0$, $F = \{s_{25}\}$ e δ é definido por enumeração na seguinte tabela:

δ	s_0	s_5	s_{10}	s_{15}	s_{20}	s_{25}
5	s_5	s_{10}	s_{15}	s_{20}	s_{25}	s_{25}
10	s_{10}	s_{15}	s_{20}	s_{25}	s_{25}	s_{25}
25	s_{25}	s_{25}	s_{25}	s_{25}	s_{25}	s_{25}

As linhas representam os símbolos de entrada e as colunas os estados o valor na segunda linha (rotulada como 10) e terceira coluna (rotulada como s_{10}) é s_{20} , isso deve ser lido como $\delta(s_{10}, 10) = s_{20}$. Ou, textualmente, se o estado atual é s_{10} e a entrada lida é o símbolo 10, o próximo estado é s_{20} .

Para representar DFA através de diagramas de estados, como no exemplo anterior, as seguintes convenções serão utilizadas: Os estados são representados por círculos. O estado inicial tem um seta apontando para ele que não vem de nenhum estado. Os estados de aceitação serão representados por círculos diferentes (aqui com a borda mais grossa, mas geralmente com borda dupla nos livros de teoria da computação). A função de transição é representada por setas que partem do estado atual e chegam no estado de destino, as setas são rotuladas pelo símbolo de entrada que provoca a transição.

Por conveniência, se dois símbolos distintos provocam a mesma transição, escreveremos os símbolos sobre a transição separados por vírgulas. Vou também usar a seguinte convenção para evitar o excesso de setas em exemplos maiores, se nenhuma seta sai do estado r com rótulo $a \in \Sigma$, então $\delta(r, a) = q_x$. Esse estado q_x é um estado que não será desenhado e do qual não é possível sair, ou seja, $\delta(q_x, s) = q_x, \forall s \in \Sigma$, além disso $q_x \notin F$ (essa é uma convenção minha, pode não ocorrer em outros textos).

Vamos agora definir como se dá a “computação” do autômato, ou seja, como é o seu funcionamento para uma dada entrada.

Definição: Computação em um DFA

Seja $M = (Q, \Sigma, \delta, q_0, F)$ um DFA e seja $\omega = w_1 w_2 \cdots w_n$ uma cadeia sobre Σ . Defina a sequência $s_0 s_1 \cdots s_n$ de estados da seguinte forma:

- $s_0 = q_0$;
- $s_i = \delta(s_{i-1}, w_i)$, para $i = 1, 2, \dots, n$.

Daí define-se:

- Se $s_n \in F$, diz-se que M *aceita* ω ;
- Se $s_n \notin F$, diz-se que M *rejeita* ω .

Note que ω pode ser a cadeia vazia, ε , e nesse caso $n = 0$ e a sequência de estados tem apenas um elemento, $s_0 = q_0$. A cadeia vazia é aceita por M se, e somente se, o estado inicial também é de aceitação, $q_0 \in F$. Perceba também que a entrada é lida apenas uma vez, da esquerda para a direita, não há a possibilidade de voltar ao início e ler novamente a entrada.

Essa definição diz que, para uma dada entrada, o DFA tem duas saídas possíveis, chamadas de *aceita* e *rejeita*. A partir daí vamos definir a linguagem do autômato.

Definição: Linguagem Reconhecida por um DFA

Seja $M = (Q, \Sigma, \delta, q_0, F)$ um DFA. A *linguagem reconhecida* por M , denotada por $\mathcal{L}(M)$, é o conjunto de todas as cadeias sobre Σ^* aceitas por M : $\mathcal{L}(M) = \{x \in \Sigma^* : M \text{ aceita } x\}$.

A linguagem reconhecida por M só é vazia, $\mathcal{L}(M) = \{\}$, se $F = \{\}$ ou se os estados de aceitação são inalcançáveis, isto é, nenhuma cadeia $\omega \in \Sigma^*$ leva o autômato de q_0 a um estado $x \in F$. Finalmente, as linguagens regulares podem ser definidas.

Definição: Linguagem Regular

Uma linguagem A é dita *regular* se existe um DFA M tal que $\mathcal{L}(M) = A$.

Por fim, apresento uma versão alternativa da computação em um DFA.

Definição: Computação em um DFA (versão recursiva)

Seja $M = (Q, \Sigma, \delta, q_0, F)$ um DFA e seja $\omega = w_1 w_2 \cdots w_n$ uma cadeia sobre Σ . Defina a função $\bar{\delta}: Q \times \Sigma^* \rightarrow Q$ como

- $\bar{\delta}(r, w) = r$, se $w = \varepsilon$

- $\bar{\delta}(r, w) = \delta(\bar{\delta}(r, \gamma), a)$, se $w = \gamma a$, onde γ é uma cadeia sobre Σ e $a \in \Sigma$.
 Se $\bar{\delta}(q_0, \omega) \in F$, dizemos que M aceita ω .
 Se $\bar{\delta}(q_0, \omega) \notin F$, dizemos que M rejeita ω .

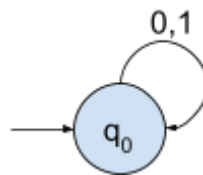
A função $\bar{\delta}(r, w)$ é o estado que M alcança quando está no estado r e lê a cadeia w da esquerda para a direita. Essa função é chamada de função de transição estendida.

Exemplos de DFA

Tudo que precisa ser definido acerca dos DFA já foi definido. Para assimilar melhor o seu funcionamento vamos examinar alguns exemplos. O projeto de autômatos finitos que reconhecem uma dada linguagem é tão difícil quanto resolver qualquer problema, envolve criatividade, esforço e um pouco de sorte. Vou tentar descrever o raciocínio em cada projeto mas lembre que o melhor é que vocês tentem por conta própria antes de ver as soluções. Todas as linguagens nos exemplos que seguem são sobre o alfabeto $\Sigma = \{0, 1\}$.

Exemplo 1: $L_1 = \{\}$

Para reconhecer a linguagem vazia, ou seja, não aceitar nada, basta que o autômato não tenha estados finais. Ele precisa ter pelo menos um estado que é o inicial, e nunca sai dele.

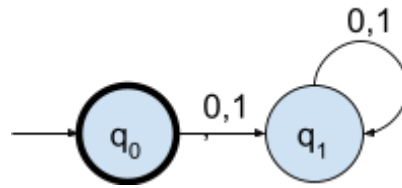


A descrição formal desse autômato é $D = (\{q_0\}, \Sigma, \delta, q_0, \emptyset)$. Com δ definido por:

δ	q_0
0	q_0
1	q_0

Exemplo 2: $L_2 = \{\varepsilon\}$

Para reconhecer a cadeia vazia o estado inicial deve ser um estado de aceitação. Para não aceitar nada além de ε é necessário sair desse estado assim que alguma entrada chegar (o estado com a borda mais grossa é o estado de aceitação, não vou mais usar a borda dupla porque é mais rápido desenhar assim).

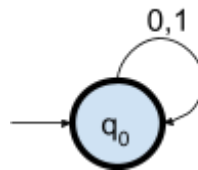


A descrição desse autômato é $D = (\{q_0, q_1\}, \Sigma, \delta, q_0, \{q_0\})$. Com δ definido por:

δ	q_0	q_1
0	q_1	q_1
1	q_1	q_1

Exemplo 3: $L_3 = \Sigma^*$

No exemplo anterior, se o autômato nunca sair do estado de aceitação qualquer coisa será aceita.

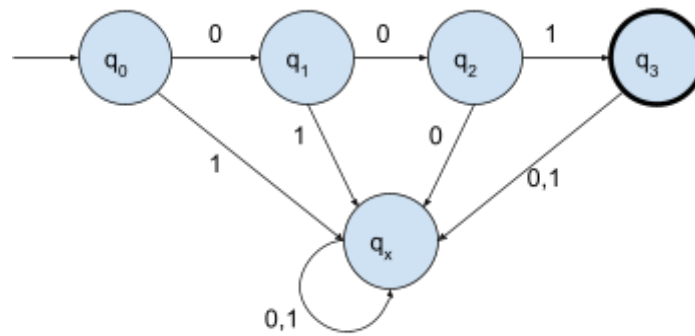


A descrição desse autômato é $D = (\{q_0\}, \Sigma, \delta, q_0, \{q_0\})$. Com δ definido por:

δ	q_0
0	q_0
1	q_0

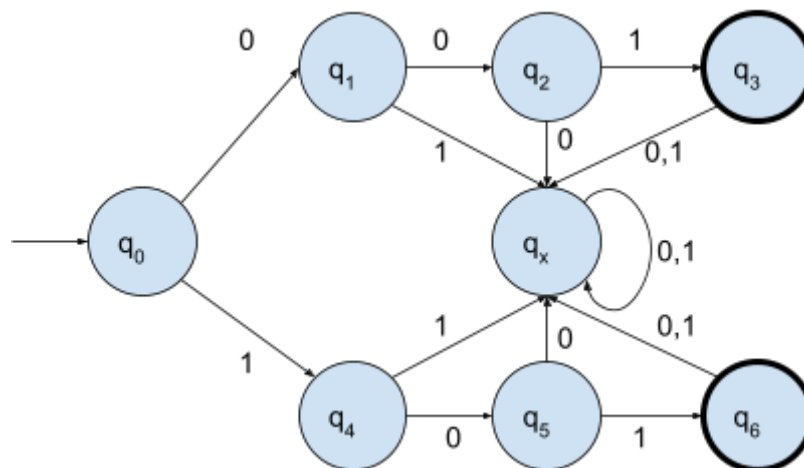
Exemplo 4: $L_4 = \{001, 101\}$

Para aceitar uma cadeia específica, ω , o autômato deve partir do estado inicial e transitar $|\omega|$ vezes a cada vez que acerta um símbolo da cadeia desejada. Se qualquer erro ocorrer o autômato deve rejeitar a entrada, portanto teremos um estado que serve para rejeitar sempre que um erro ocorrer. Lembre que queremos aceitar uma cadeia exata, então depois de chegar no estado de aceitação qualquer entrada também é considerada um erro. Vamos começar projetando um autômato que reconhece 001.

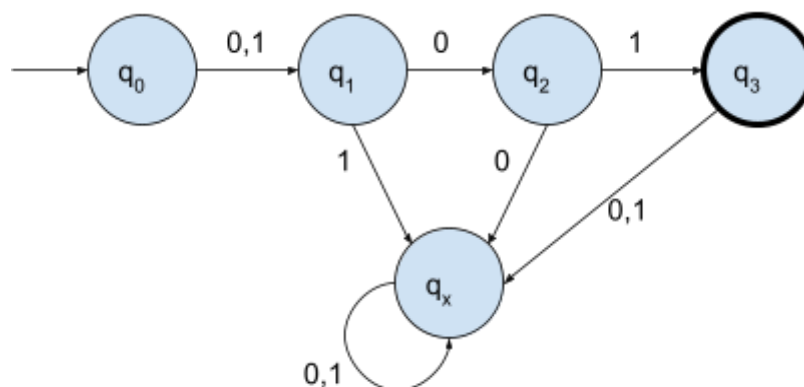


Perceba que a única cadeia que chega e fica em q_3 é 001. O estado q_x é um estado do qual é impossível sair e que não é de aceitação, alguns livros chamam esse tipo de estado de armadilha (*trap state*).

No entanto, ainda não alcançamos nosso objetivo. Vamos agora acrescentar um novo ramo neste autômato para aceitar a cadeia 101.



Essa é uma solução correta, mas muito mais complicada que o necessário. Note que as duas cadeias são idênticas a menos do primeiro símbolo. A solução abaixo é igualmente válida e muito mais simples.



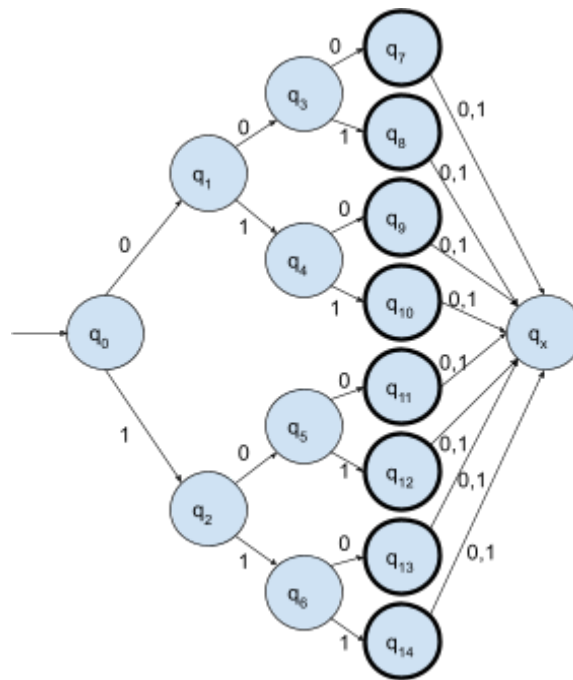
A descrição desse autômato é $D = (\{q_0, q_1, q_2, q_3, q_x\}, \Sigma, \delta, q_0, \{q_3\})$. Com δ definido por:

δ	q_0	q_1	q_2	q_3	q_x
0	q_1	q_2	q_x	q_x	q_x
1	q_1	q_x	q_3	q_x	q_x

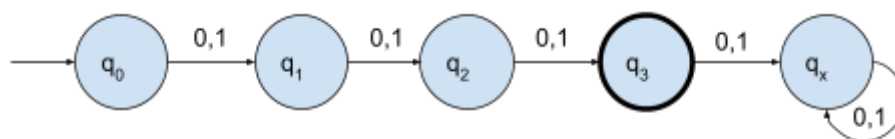
Toda linguagem finita pode ser projetada simplesmente enumerando cada uma das cadeias desejadas. Esse fato é similar à afirmação que fiz na revisão de que todo problema com domínio finito pode ser resolvido por força bruta ou que toda prova sobre um domínio finito pode ser realizada por enumeração.

Exemplo 5: $L_5 = \{x \in \Sigma^* : |x| \text{ é múltiplo de } 3\}$

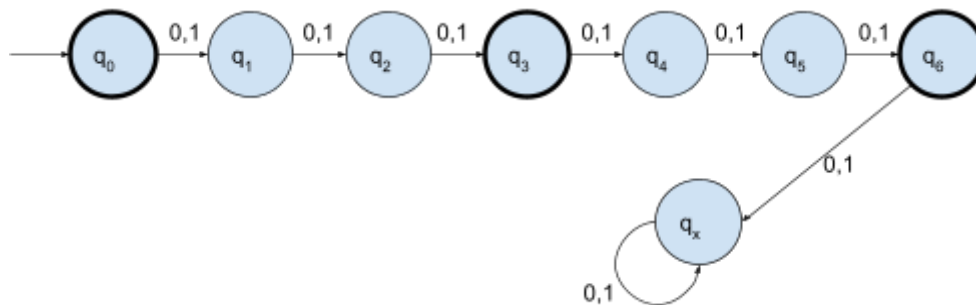
Vamos começar projetando um autômato que aceita as cadeias que têm comprimento exatamente igual a 3. Vou fazer de duas formas apenas para ilustrar como fazer uma enumeração.



Perceba que cada um dos 8 caminhos de q_0 a algum estado de aceitação representa uma cadeia aceita pelo autômato. Agora vamos resumir essa enumeração porque não nos importa quais os símbolos da cadeia de entrada, só interessa que o número de transições até um estado de aceitação é 3.

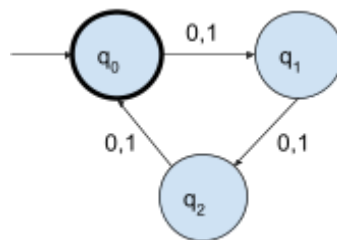


Atualmente temos um autômato que aceita qualquer coisa com comprimento 3 e rejeita todo o resto. Podemos tentar incluir os múltiplos de 3 acrescentando estados. Vamos aproveitar e incluir o estado inicial como estado de aceitação, já que 0 é múltiplo de 3.



Agora o autômato aceita cadeias de comprimento 0, 3 ou 6, mas essa idéia não vai dar certo. Queremos projetar um autômato *finito* e existem infinitos múltiplos de 3. Podemos aproveitar os estados que já temos.

Perceba que $n \equiv n + 3k \pmod{3}$, com $k \in \mathbb{N}$, ou seja, $1 \equiv 4 \equiv 7 \equiv 10 \equiv \dots \pmod{3}$, $2 \equiv 5 \equiv 8 \equiv 11 \equiv \dots \pmod{3}$ e $0 \equiv 3 \equiv 6 \equiv 9 \equiv \dots \pmod{3}$. Em outras palavras, se olharmos para o resto da divisão do comprimento da cadeia por 3, há apenas 3 valores possíveis, ou seja, precisamos de apenas 3 estados. Podemos finalmente determinar o autômato que reconhece a linguagem desejada. Neste caso o índice do estado representa o resto da divisão do comprimento da cadeia de entrada por 3.

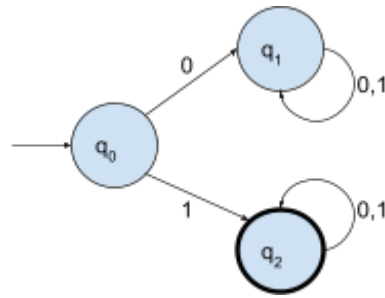


A descrição desse autômato é $D = (\{q_0, q_1, q_2\}, \Sigma, \delta, q_0, \{q_0\})$. Com δ definido por:

δ	q_0	q_1	q_2
0	q_1	q_2	q_0
1	q_1	q_2	q_0

Exemplo 6: $L_6 = \{x \in \Sigma^* : x \text{ começa com } 1\}$

Esse exemplo é bem direto, a partir do estado inicial a entrada 1 leve para o estado de aceitação e daí não saia mais. Se o primeiro símbolo for 0, vamos para um estado que não é de aceitação e daí não saímos.

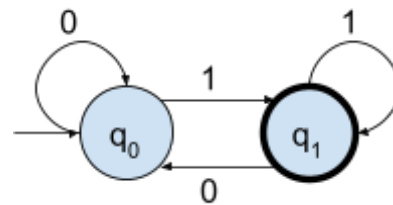


A descrição desse autômato é $D = (\{q_0, q_1, q_2\}, \Sigma, \delta, q_0, \{q_2\})$. Com δ definido por:

δ	q_0	q_1	q_2
0	q_1	q_1	q_2
1	q_2	q_1	q_2

Exemplo 7: $L_7 = \{x \in \Sigma^* : x \text{ termina com } 1\}$

Esse caso é um pouco mais interessante porque não sabemos de antemão qual o comprimento da cadeia que será entrada. Podemos simplesmente supor que o símbolo atual é o último da cadeia. Assim, sempre que o símbolo lido for 1 devemos ir para um estado de aceitação sempre que o símbolo lido for 0 devemos ir para um estado de rejeição. Portanto, bastam dois estados num autômato que reconhece essa linguagem “o último símbolo lido foi 1” e “o último símbolo lido não foi 1”.

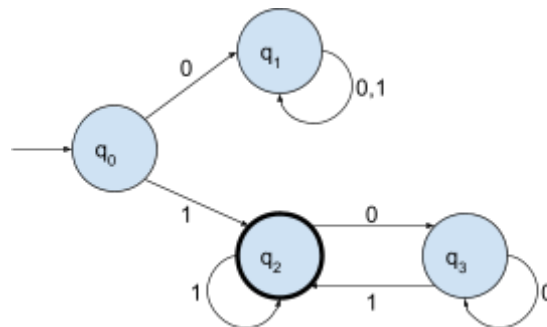


A descrição desse autômato é $D = (\{q_0, q_1\}, \Sigma, \delta, q_0, \{q_1\})$. Com δ definido por:

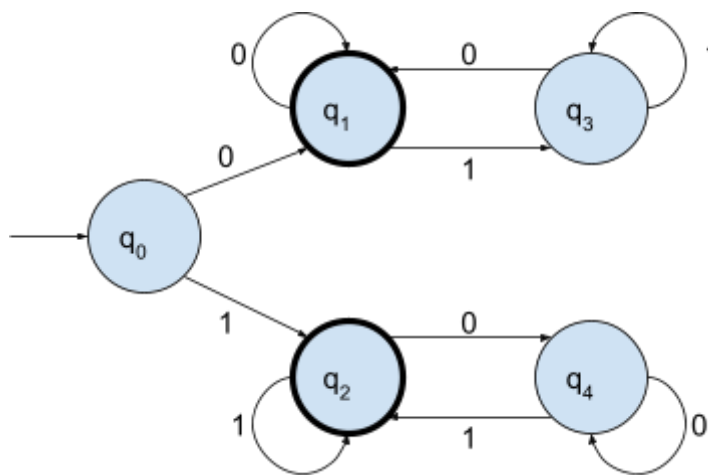
δ	q_0	q_1
0	q_0	q_0
1	q_1	q_1

Exemplo 8: $L_8 = \{x \in \Sigma^* : x \text{ começa e termina com o mesmo símbolo}\}$

Esse autômato é basicamente uma combinação dos dois autômatos vistos anteriormente. Para ilustrar isso, vamos primeiro fazer um autômato que aceita cadeias que começam e terminam com 1.



Neste caso q_2 é o estado de aceitação dos autômatos nos exemplos 6 e 7. Para reconhecer a linguagem desejada basta aplicar um raciocínio análogo no ramo superior.



A descrição desse autômato é $D = (\{q_0, q_1, q_2, q_3, q_4\}, \Sigma, \delta, q_0, \{q_1, q_2\})$. Com δ definido por:

δ	q_0	q_1	q_2	q_3	q_4
0	q_1	q_1	q_4	q_1	q_4
1	q_2	q_3	q_2	q_3	q_2

Exemplo

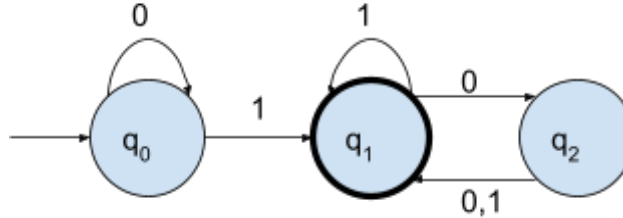
9:

$L_9 = \{x \in \Sigma^* : x \text{ contém pelo menos um 1 e termina com uma quantidade par de 0}\}$

Para saber se um autômato tem pelo menos um 1, basta sair do estado inicial quando o símbolo de entrada for 1 e permanecer nele enquanto for 0. Logo após a leitura do primeiro 1 o autômato está um novo estado que indica que há pelo menos um 1 e

nenhum 0 depois dele. Considerando que zero é par, esse deve ser um estado de aceitação. Qualquer ocorrência do símbolo 1 deve levar de volta a esse estado.

A outra possibilidade é que tem uma sequência de 0 depois do último 1, para isso vamos utilizar aritmética modular, como no exemplo 5. A cada ocorrência de dois 0 consecutivos voltamos a esse estado de aceitação.

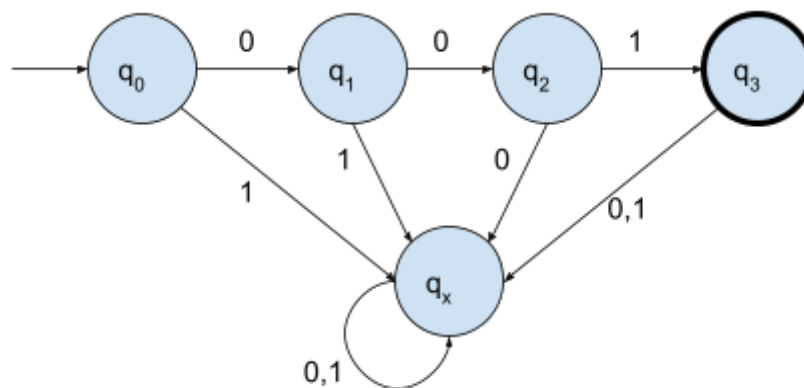


A descrição desse autômato é $D = (\{q_0, q_1, q_2\}, \Sigma, \delta, q_0, \{q_1\})$. Com δ definido por:

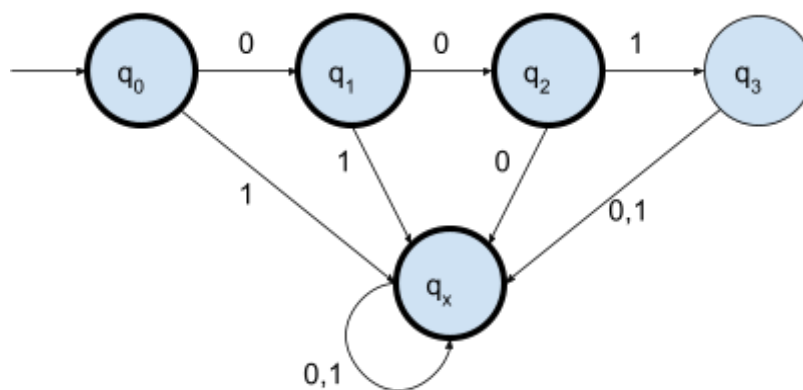
δ	q_0	q_1	q_2
0	q_0	q_2	q_1
1	q_1	q_1	q_1

Exemplo 10: $L_{10} = \{x \in \Sigma^* : x \neq 001\}$

No início do Exemplo 4 criamos um autômato que aceita apenas 001.



O nosso objetivo é projetar um autômato que aceita qualquer coisa exceto 001, ou seja, a linguagem desejada é o complemento da linguagem do autômato que temos no momento. Podemos obter facilmente a linguagem desejada invertendo os estados de aceitação e rejeição. O novo autômato rejeita apenas 001.



A descrição desse autômato é $D = (\{q_0, q_1, q_2, q_3, q_x\}, \Sigma, \delta, q_0, \{q_0, q_1, q_2, q_x\})$. Com δ definido por:

δ	q_0	q_1	q_2	q_3	q_x
0	q_1	q_2	q_x	q_x	q_x
1	q_x	q_x	q_3	q_x	q_x

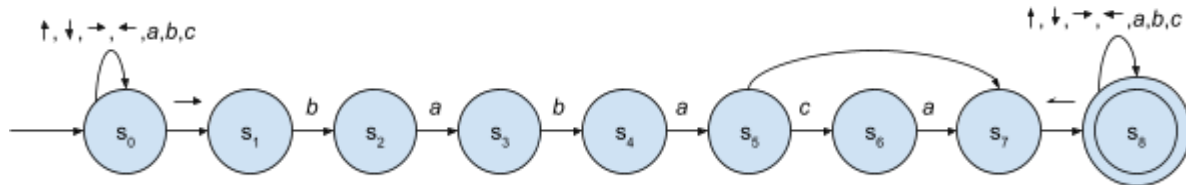
Autômatos Finitos Não-Determinísticos (NFA)

Os autômatos vistos anteriormente são chamados de determinísticos porque de um dado estado com uma dada entrada ocorre a transição para um estado definido unicamente pela função de transição. Vamos tentar dar superpoderes para nossas máquinas quebrando o determinismo e criando os Autômatos Finitos Não-Determinísticos (NFA - *Nondeterministic Finite Automata*).

Vamos iniciar novamente com um problema muito sério que precisa ser modelado. Um programador com tempo livre na empresa de jogos CapComp, criadora de clássicos como Script Fighter e Presidente Ivo, decidiu implementar um segredo nos jogos da empresa, se o usuário entrasse com uma certa combinação de botões alguma vantagem seria dada ao jogador (mais vidas, personagens secretos, fases secretas, mini-jogos).

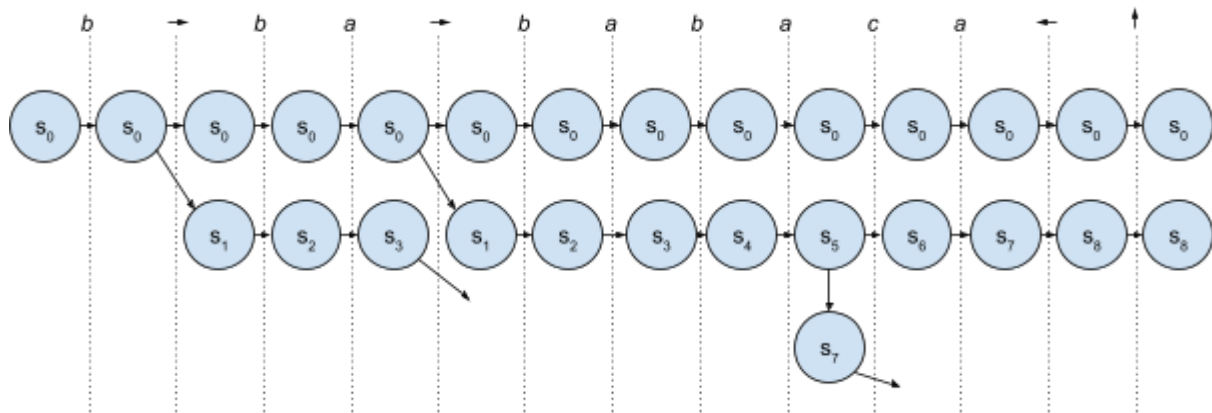


Os controladores do console de jogos têm os seguintes botões: $\uparrow, \downarrow, \leftarrow, \rightarrow, a, b, c$ e o programador escolheu dois códigos secretos: $\rightarrow baba \leftarrow$ e $\rightarrow babaca \leftarrow$. Então o que precisa ser feito é monitorar continuamente a entrada do usuário e, se um dos códigos ocorrer em algum momento, a recompensa é dada. Note que um autômato que aceita apenas os códigos secretos não é suficiente, o usuário pode entrar com qualquer coisa antes ou depois do código. É necessário um autômato que aceita qualquer cadeia que tenha o código como subcadeia. Minha sugestão de modelo segue abaixo.



A primeira vista, parece um DFA, mas tem duas grandes diferenças. Perceba a transição do estado inicial quando a entrada é $\rightarrow$ (as setas sobre as transições são do alfabeto, não confunda). Note que o autômato permanece no estado inicial mas, ao mesmo tempo, ele transita para o estado s_1 e, a partir desse ponto, está em dois estados simultaneamente. Além disso, o que acontece quando estamos em s_1 e chega uma entrada diferente de b ? O que é aquela transição sem rótulo de s_5 para s_7 ?

Vou dar agora uma descrição do funcionamento dessa máquina hipotética. Vamos assumir que a entrada do usuário foi $b \rightarrow ba \rightarrow babaca \leftarrow \uparrow$. Para facilitar o entendimento, vou apresentar um diagrama primeiro e explicá-lo depois.



Cada região, separadas pelas barras verticais, são os estados nos quais o autômato está após a leitura da entrada que rotula a barra. O autômato começa em s_0 e após ler b , continua em s_0 . Depois, ao ler $\rightarrow$, o autômato passa para dois estados simultâneos s_0 e s_1 . Como todo símbolo de entrada leva de s_0 de volta para s_0 , nunca sairemos desse estado. Após a leitura de $\rightarrow$ o jogador entrou com ba levando o estado s_1 até o estado s_3 , nesse ponto ele acertou 3 símbolos do código. Então o jogador pressiona $\rightarrow$ novamente duas coisas acontecem, o ramo que termina em s_3 *morre* (vai para lugar nenhum) e o ramo que estava em s_0 se bifurca para s_0 e s_1 , como tinha ocorrido da primeira vez que $\rightarrow$ foi lido.

Depois disso o jogador digita *baba* e chega ao estado s_5 que bifurca imediatamente, sem leitura da entrada, para o estado s_7 . Essa transição sem rótulo é uma transição vazia ou transição ε (na maioria dos livros vai estar rotulado com ε , mas a ferramenta de desenho do Google não permite inserir fórmulas ou caracteres especiais na figura então resolvi deixar sem nada). Neste ponto o jogador insere c , de modo que o ramo em s_7 morre e s_5 vai para s_6 , e finaliza o código inserindo $a \leftarrow$ e alcançando o estado s_8 . A partir desse ponto qualquer que seja a entrada não vai mais sair de s_8 .

Em resumo, o estado s_0 verifica a entrada do usuário continuamente e sempre que vê o símbolo $\rightarrow$ dá início ao processo de verificação do código, que inicia em s_1 . Se o jogador errar o código, o ramo de verificação morre e só s_0 fica ativo. Se a máquina chega até s_5 , ou seja, se o jogador realizou a sequência $\rightarrow baba$, há uma nova bifurcação porque há dois códigos possíveis. Se o próximo símbolo for $\leftarrow$ o estado final é alcançado, se for c o ramo alternativo morre e continua o ramo principal, indo para s_6 , se for qualquer outra coisa os dois ramos morrem e apenas s_0 fica ativo. Depois disso, se o jogador pressiona $a \leftarrow$ ele alcança o estado final.

Se qualquer um dos ramos termina em s_8 o jogador recebe a recompensa. Espero que você esteja convencido que essa máquina, de acordo com as regras de funcionamento descritas, dá a recompensa ao jogador sempre que um dos códigos secretos é subcadeia da sequência de entradas.

Definição Formal

Como fizemos para o DFA, vamos dar uma definição precisa do que é um NFA e como ele funciona. Nas definições que seguem $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$.

Definição: Autômato Finito Não-Determinístico (NFA)

Um autômato finito não-determinístico (NFA) é uma 5-upla $N = (Q, \Sigma, \delta, q_0, F)$, onde:

- Q é um conjunto finito, cujos elementos são chamados de *estados*;
- Σ é um conjunto finito, chamado *alfabeto*;
- $\delta: Q \times \Sigma_\varepsilon \rightarrow \mathcal{P}(Q)$ é uma função, chamada de *função de transição*;
- $q_0 \in Q$, é chamado de *estado inicial*;
- $F \subseteq Q$, são os *estados de aceitação* (ou *estados finais*).

Perceba que a única diferença na definição do DFA e do NFA é a sua função de transição. Essa nova definição permite que a entrada seja ε , dando origem às transições vazias, que ocorrem sem leitura da entrada. Ela também permite que a saída seja qualquer subconjunto de Q , então podemos ir para qualquer quantidade de estados, inclusive nenhum.

Como dito anteriormente, vou deixar as transições vazias sem rótulo nos diagramas de estado, mas na maioria dos livros você as verá rotuladas por ε .

O exemplo anterior, do jogo, é de fato um NFA e tem $Q = \{s_0, s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8\}$, $q_0 = s_0$, $\Sigma = \{a, b, c, \uparrow, \downarrow, \leftarrow, \rightarrow\}$, $F = \{s_8\}$ e δ é definido por enumeração na seguinte tabela:

δ	s_0	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8
a	$\{s_0\}$	$\emptyset$	$\{s_3\}$	$\emptyset$	$\{s_5\}$	$\emptyset$	$\{s_7\}$	$\emptyset$	$\{s_8\}$
b	$\{s_0\}$	$\{s_2\}$	$\emptyset$	$\{s_4\}$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\{s_8\}$
c	$\{s_0\}$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\{s_6\}$	$\emptyset$	$\emptyset$	$\{s_8\}$
$\uparrow$	$\{s_0\}$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\{s_8\}$
$\downarrow$	$\{s_0\}$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\{s_8\}$
$\leftarrow$	$\{s_0\}$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\{s_8\}$	$\{s_8\}$
$\rightarrow$	$\{s_0, s_1\}$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\{s_8\}$
ε	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\{s_7\}$	$\emptyset$	$\emptyset$	$\emptyset$

Como a função de transição foi modificada, precisamos atualizar a definição de computação para autômatos não-determinísticos.

Definição: Computação em um NFA

Seja $N = (Q, \Sigma, \delta, q_0, F)$ um NFA e seja $\omega = w_1 w_2 \cdots w_n$ onde $w_i \in \Sigma_\varepsilon$. Diz-se que N aceita ω se *existe* uma sequência $s_0 s_1 \cdots s_n$ de estados tal que:

- $s_0 = q_0$;
- $s_{i+1} \in \delta(s_i, w_{i+1})$, para $i = 0, 1, \dots, n - 1$.
- $s_n \in F$

Caso contrário, diz-se que N rejeita ω .

A linguagem do autômato continua sendo definida da mesma forma.

Definição: Linguagem Reconhecida por um NFA

Seja $N = (Q, \Sigma, \delta, q_0, F)$ um NFA. A *linguagem reconhecida* por N , denotada por $\mathcal{L}(N)$, é o conjunto de todas as cadeias sobre Σ^* aceitas por N : $\mathcal{L}(N) = \{x \in \Sigma^* : N \text{ aceita } x\}$.

Apresento agora uma versão alternativa da computação, da forma que executamos o procedimento de computação no exemplo anterior. Basicamente, o processo que se repete a cada leitura de um símbolo da entrada é aplicar a função de transição a cada um dos estados atuais, obtendo um novo conjunto de estado e, em seguida, efetuar todas as transições vazias possíveis partindo de cada um dos estados do novo conjunto.

O fecho vazio (ou fecho ε) de um estado r do NFA, $C_\varepsilon(r)$, é o conjunto de todos os estados de N que podem ser alcançados a partir de r através de zero ou mais transições vazias. Para um estado conjunto de estados, R , definimos $C_\varepsilon(R) = \bigcup_{r \in R} C_\varepsilon(r)$, ou seja, o fecho do conjunto é a união dos fechos de cada um dos seus elementos.

Definição: Computação em um NFA (alternativa)

Seja $N = (Q, \Sigma, \delta, q_0, F)$ um NFA e seja $\omega = w_1 w_2 \dots w_n$ uma cadeia sobre Σ . Defina uma sequência de conjuntos de estados $R_0 R_1 \dots R_n$ da seguinte forma:

- $R_0 = C_\varepsilon(q_0)$;
- $R_i = \bigcup_{r \in R_{i-1}} C_\varepsilon(\delta(r, w_i))$, para $i = 1, 2, \dots, n$.

Daí define-se:

- Se $R_n \cap F \neq \emptyset$, diz-se que N aceita ω ;
- Se $R_n \cap F = \emptyset$, diz-se que N rejeita ω .

Assim como fizemos para os DFA, segue uma versão recursiva da computação em um NFA utilizando uma função de transição estendida.

Definição: Computação em um NFA (versão recursiva)

Seja $N = (Q, \Sigma, \delta, q_0, F)$ um NFA e seja $\omega = w_1 w_2 \dots w_n$ uma cadeia sobre Σ . Defina a função $\bar{\delta}: Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$ como

- $\bar{\delta}(r, w) = C_\varepsilon(r)$, se $w = \varepsilon$
- $\bar{\delta}(r, w) = C_\varepsilon(\delta(\bar{\delta}(r, \gamma), a))$, se $w = \gamma a$, onde γ é uma cadeia sobre Σ e $a \in \Sigma$.

Se $\bar{\delta}(q_0, \omega) \cap F \neq \emptyset$, dizemos que M aceita ω .

Se $\bar{\delta}(q_0, \omega) \cap F = \emptyset$, dizemos que M rejeita ω .

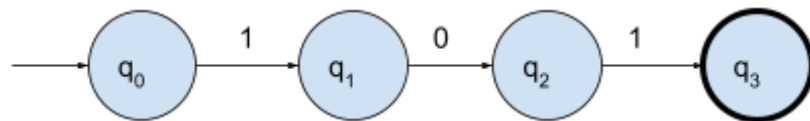
Exemplos de NFA

Como no caso dos DFA, vamos ver alguns exemplos de NFA para assimilar melhor seu funcionamento. Em muitas situações projetar um NFA é mais fácil que projetar um DFA por causa da liberdade extra que é dada pelo não-determinismo, no entanto, pelo menos

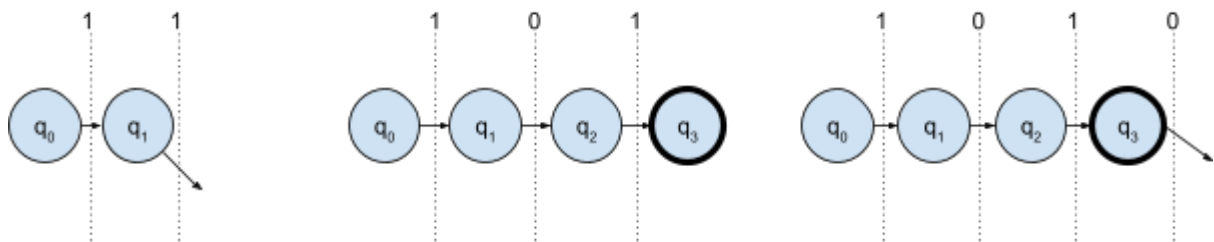
motivo, pode ser mais difícil de analisá-lo, ou seja, dado o autômato definir a linguagem que ele reconhece. Todas as linguagens nos exemplos que seguem são sobre o alfabeto $\Sigma = \{0, 1\}$.

Exemplo 1: $L_1 = \{101\}$

Para definir um NFA que aceita uma cadeia específica, basta explicitar essa cadeia através das transições. Não é necessário criar um estado de rejeição porque é possível simplesmente não especificar uma transição, fazendo o ramo de computação morrer, ou seja ir para o subconjunto vazio de estados.



Vamos analisar passo a passo o processamento das cadeias 11, 101 e 1010 por este autômato.



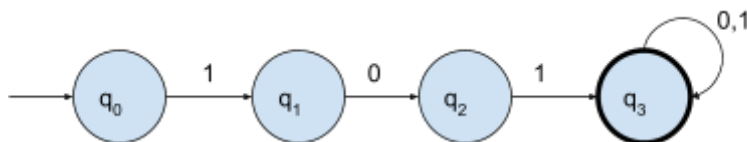
Somente a entrada 101 termina em um estado de aceitação.

A descrição desse autômato é $D = (\{q_0, q_1, q_2, q_3\}, \Sigma, \delta, q_0, \{q_3\})$. Com δ definido por:

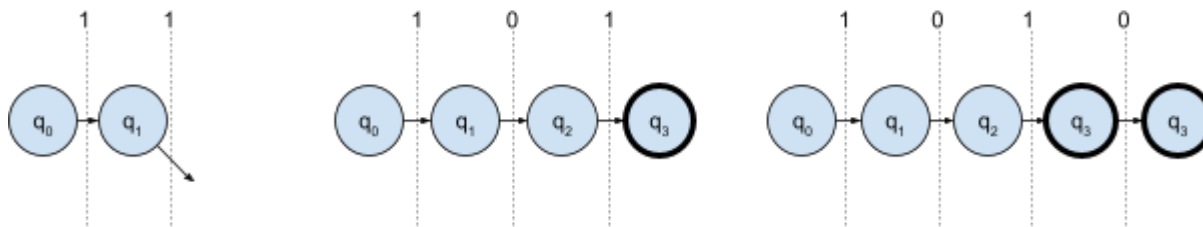
δ	q_0	q_1	q_2	q_3
0	$\emptyset$	$\{q_2\}$	$\emptyset$	$\emptyset$
1	$\{q_1\}$	$\emptyset$	$\{q_3\}$	$\emptyset$
ε	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Exemplo 2: $L_2 = \{x \in \Sigma^* : x \text{ começa com } 101\}$

Fizemos algo similar nos exemplos do DFA. Mais uma vez o projeto fica mais simples pois não é necessário indicar todas as transições. Basta focar no que é desejado, 101 seguido de qualquer coisa.



Vamos analisar passo a passo o processamento das cadeias 11, 101 e 1010 por este autômato.



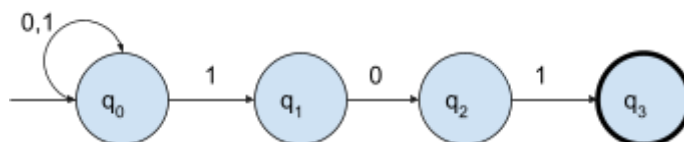
As entrada 101e 1010 são aceitas. De fato, após atingir o estado de aceitação qualquer coisa será aceita pois não é possível sair dele.

A descrição desse autômato é $D = (\{q_0, q_1, q_2, q_3\}, \Sigma, \delta, q_0, \{q_3\})$. Com δ definido por:

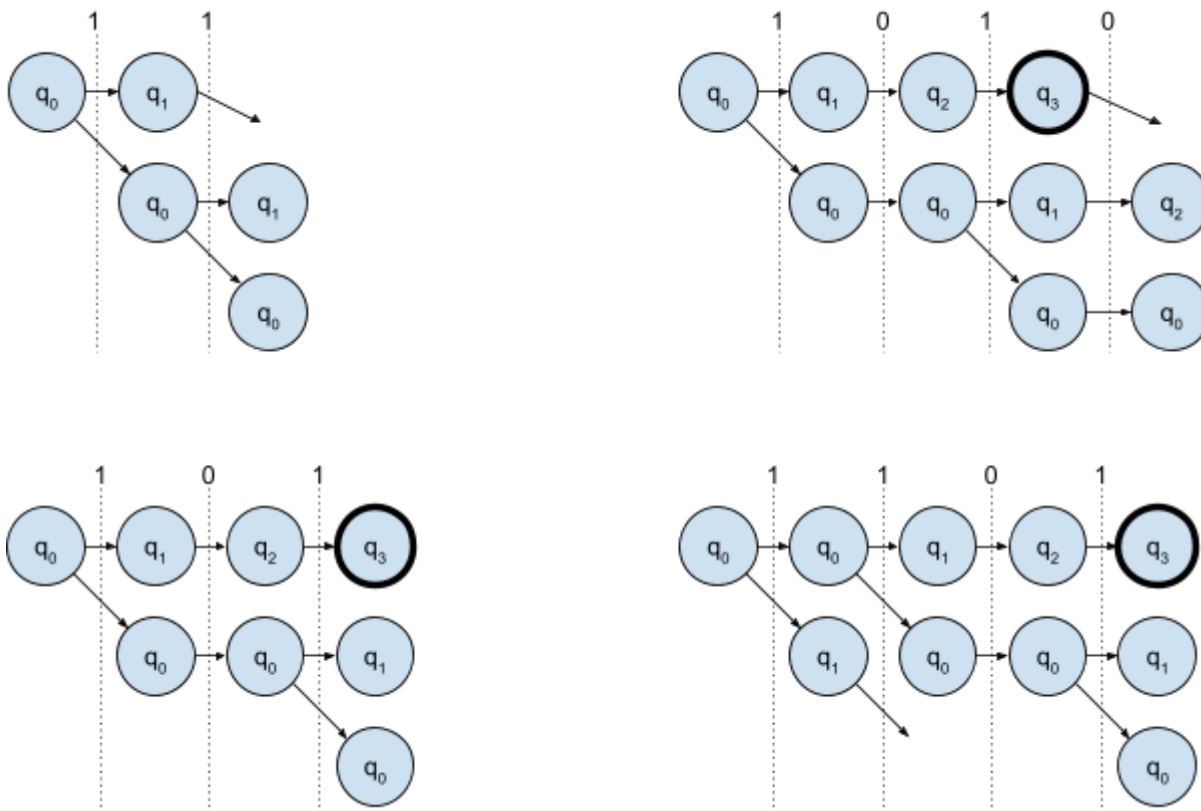
δ	q_0	q_1	q_2	q_3
0	$\emptyset$	$\{q_2\}$	$\emptyset$	$\{q_3\}$
1	$\{q_1\}$	$\emptyset$	$\{q_3\}$	$\{q_3\}$
ε	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Exemplo 3: $L_3 = \{x \in \Sigma^* : x \text{ termina com } 101\}$

Num raciocínio análogo ao do exemplo anterior, basta explicitar as transições desejadas qualquer coisa seguida de 101. Perceba que o estado inicial está sempre ativo pois nunca sabemos se os três símbolos seguintes são os três últimos. Também fizemos algo similar com o DFA.



Vamos analisar passo a passo o processamento das cadeias 11, 101, 1010 e 1101 por este autômato.



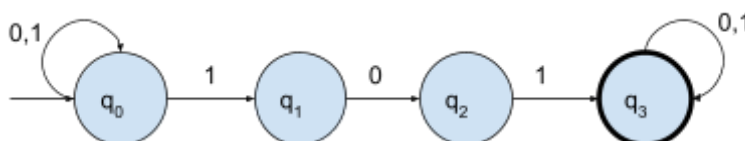
Apenas as entrada 101e 1101 são aceitas.

A descrição desse autômato é $D = (\{q_0, q_1, q_2, q_3\}, \Sigma, \delta, q_0, \{q_3\})$. Com δ definido por:

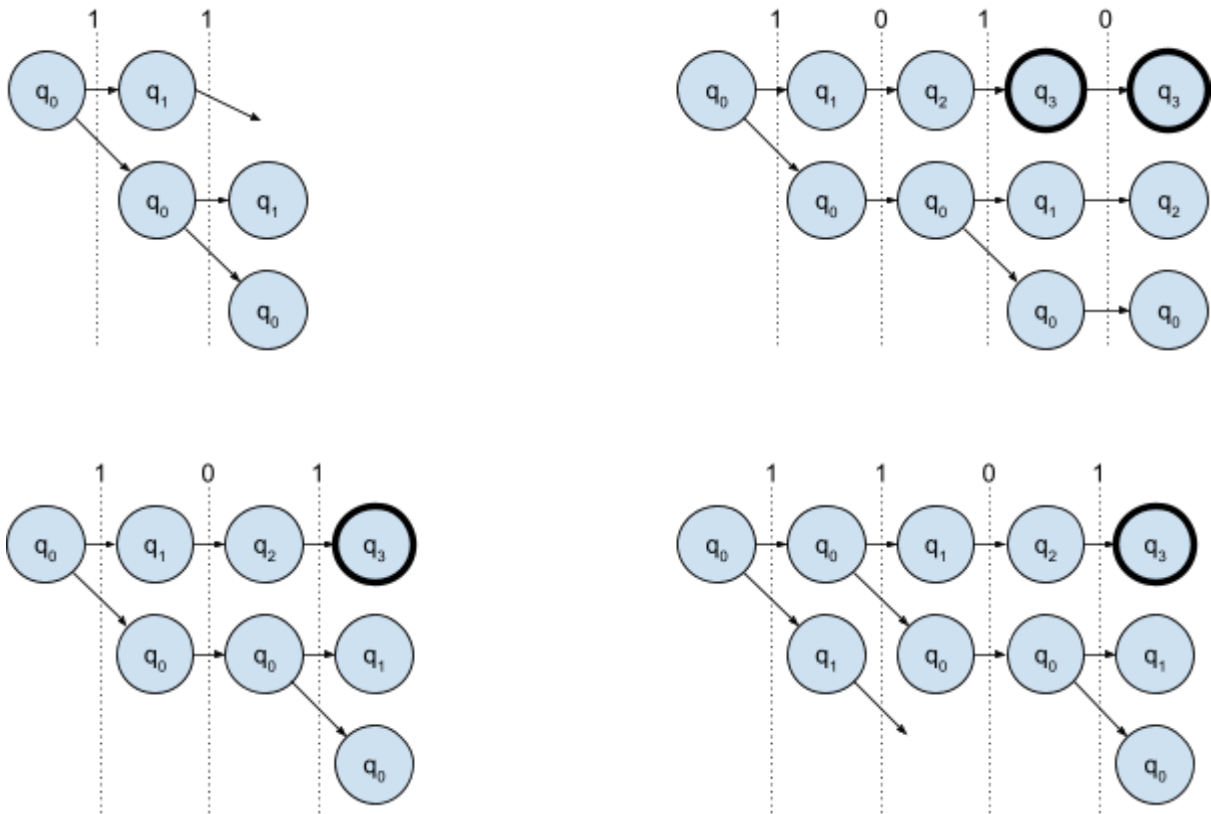
δ	q_0	q_1	q_2	q_3
0	$\{q_0\}$	$\{q_2\}$	$\emptyset$	$\emptyset$
1	$\{q_0, q_1\}$	$\emptyset$	$\{q_3\}$	$\emptyset$
ε	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Exemplo 4: $L_4 = \{x \in \Sigma^* : x \text{ contém } 101\}$

Uma mistura do raciocínio dos dois exemplos anteriores, queremos que haja 101 e pode ocorrer qualquer coisa antes ou depois.



Vamos analisar passo a passo o processamento das cadeias 11, 101, 1010 e 1101 por este autômato.



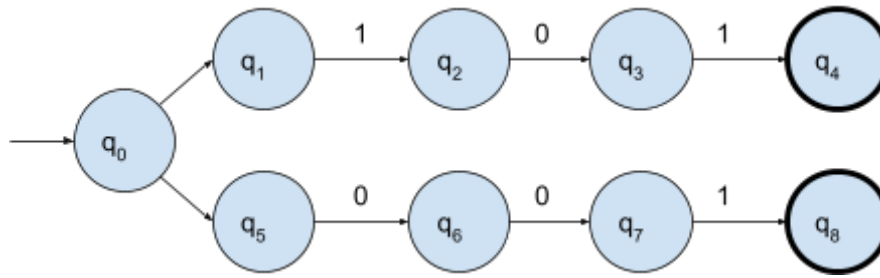
As entradas 101, 1010 e 1101 são aceitas. Ao contrário do caso anterior a ocorrência do 0 após o 101 não retira o autômato do estado de aceitação.

A descrição desse autômato é $D = (\{q_0, q_1, q_2, q_3\}, \Sigma, \delta, q_0, \{q_3\})$. Com δ definido por:

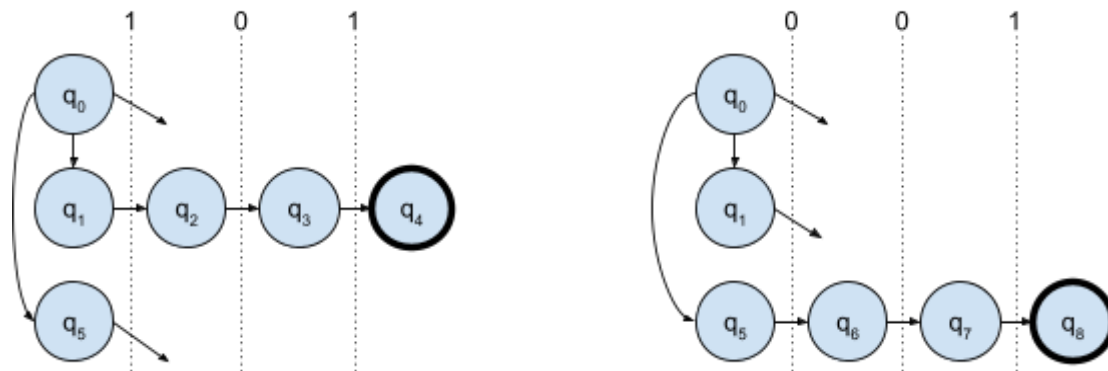
δ	q_0	q_1	q_2	q_3
0	$\{q_0\}$	$\{q_2\}$	$\emptyset$	$\{q_3\}$
1	$\{q_0, q_1\}$	$\emptyset$	$\{q_3\}$	$\{q_3\}$
ε	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Exemplo 5: $L_5 = \{101, 001\}$

Este exemplo ilustra uma facilidade dos NFA que é a possibilidade de escolha (alternativa). Já sabemos como projetar um NFA para 101 ou para 001, este último não foi feito, mas sabemos como fazer. Já fizemos um DFA para essa linguagem também, que funcionaria aqui. Mas para ilustrar o poder do NFA, vamos usar as transições vazias para simular múltiplos estados iniciais, cada um desses estados iniciais é o início de um autômato (um para 001 e outro para 101). Se qualquer um dos dois terminar no estado de aceitação o autômato que projetamos também vai aceitar, pela definição de computação de um NFA.



Vamos analisar passo a passo o processamento das cadeias 101 e 001 por este autômato.



O estado q_0 tem transições vazias para q_1 e q_5 , de modo que estes três estados são o ponto de partida. Com a leitura de qualquer símbolo q_0 desaparece. A depender de qual o primeiro símbolo lido o ramo superior ou inferior do autômato é seguido.

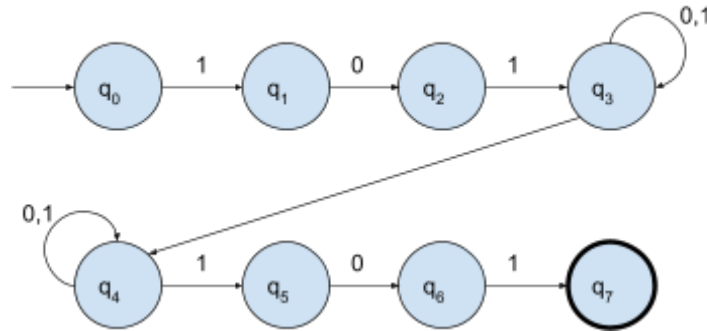
A descrição desse autômato é $D = (\{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8\}, \Sigma, \delta, q_0, \{q_4, q_8\})$.

Com δ definido por:

δ	q_0	q_1	q_2	q_3	q_4	q_5	q_6	q_7	q_8
0	$\emptyset$	$\emptyset$	$\{q_3\}$	$\emptyset$	$\emptyset$	$\{q_6\}$	$\{q_7\}$	$\emptyset$	$\emptyset$
1	$\emptyset$	$\{q_2\}$	$\emptyset$	$\{q_4\}$	$\emptyset$	$\emptyset$	$\emptyset$	$\{q_8\}$	$\emptyset$
ε	$\{q_1, q_5\}$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Exemplo 6: $L_6 = \{x \in \Sigma^* : x \text{ começa e termina com subcadeias } 101\}$

Mais uma vez, vamos usar os poderes do NFA para combinar dois autômatos o que inicia e o que termina com 101. A idéia básica é que é possível dividir a cadeia em algum ponto em duas subcadeias de modo que um autômato aceita a primeira cadeia e o outro autômato aceita a segunda cadeia. Basta conectar o estado final do primeiro autômato, que agora deixará de ser de aceitação, ao estado inicial do segundo. Desse modo o autômato só aceita se as duas condições forem satisfeitas em sequência.

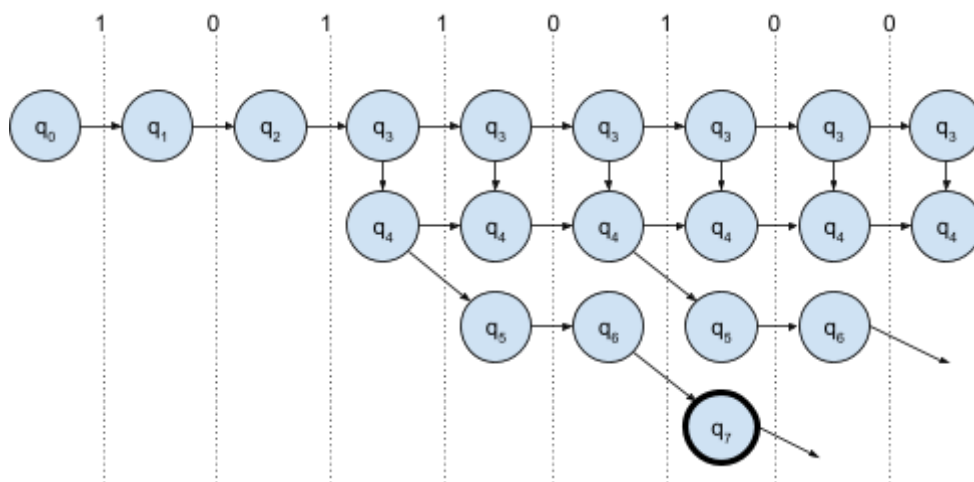


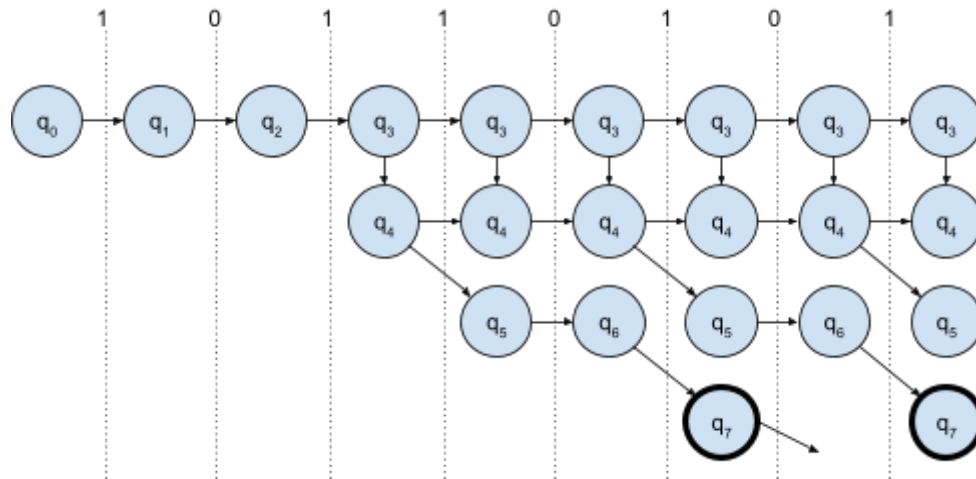
Poderia ser feito um autômato mais simples, com fusão dos estados q_3 e q_4 em um estado apenas, mas o objetivo é mostrar a facilidade de fazer a verificação sequencial por dois autômatos.

A descrição desse autômato é $D = (\{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7\}, \Sigma, \delta, q_0, \{q_7\})$. Com δ definido por:

δ	q_0	q_1	q_2	q_3	q_4	q_5	q_6	q_7
0	$\emptyset$	$\{q_2\}$	$\emptyset$	$\{q_3\}$	$\{q_4\}$	$\{q_6\}$	$\emptyset$	$\emptyset$
1	$\{q_1\}$	$\emptyset$	$\{q_3\}$	$\{q_3\}$	$\{q_4, q_5\}$	$\emptyset$	$\{q_7\}$	$\emptyset$
ε	$\emptyset$	$\emptyset$	$\emptyset$	$\{q_4\}$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

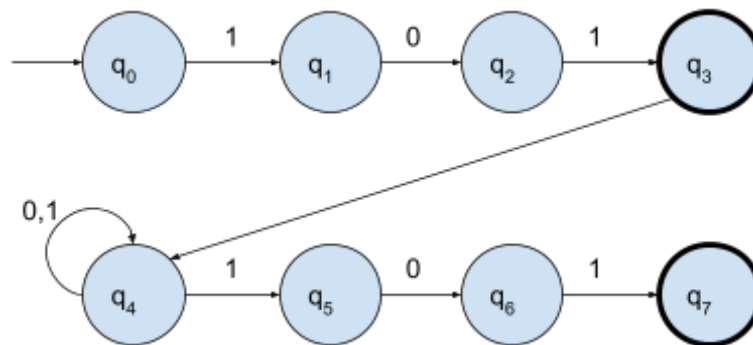
Claramente, qualquer entrada que não começa com 101 é rejeitada. Vamos analisar o processamento das cadeias 10110100 e 10110101.





Após a ocorrência do 101 inicial o estado q_4 torna-se ativo e funciona como o autômato que vimos anteriormente para aceitar cadeias que terminam com 101.

Há uma possível ambiguidade na definição textual da linguagem. A cadeia 101 deve fazer parte dessa linguagem ou não? Se você considerar que sim, o autômato deve ser modificado pois a estratégia adotada supõe que é possível separar a cadeia de entrada em duas partes, uma que começa com 101 e outra que termina com 101, mas são duas cadeias 101 distintas. O autômato abaixo aceita todas as cadeias que começam e terminam com 101, inclusive a cadeia que é apenas 101. A verificação da corretude do autômato fica por sua conta. O que fiz foi remover o laço do estado q_3 , assim ele desaparece para qualquer entrada de comprimento 4, e ele aceita a cadeia 101. O resto do autômato funciona como antes.



A descrição desse autômato é $D = (\{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7\}, \Sigma, \delta, q_0, \{q_7\})$. Com δ definido por:

δ	q_0	q_1	q_2	q_3	q_4	q_5	q_6	q_7
0	$\emptyset$	$\{q_2\}$	$\emptyset$	$\emptyset$	$\{q_4\}$	$\{q_6\}$	$\emptyset$	$\emptyset$
1	$\{q_1\}$	$\emptyset$	$\{q_3\}$	$\emptyset$	$\{q_4, q_5\}$	$\emptyset$	$\{q_7\}$	$\emptyset$

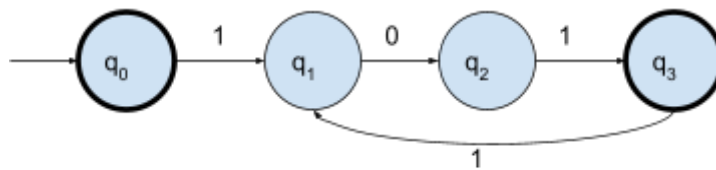
ε	$\emptyset$	$\emptyset$	$\emptyset$	$\{q_4\}$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
---------------	-------------	-------------	-------------	-----------	-------------	-------------	-------------	-------------

Exemplo**7:**

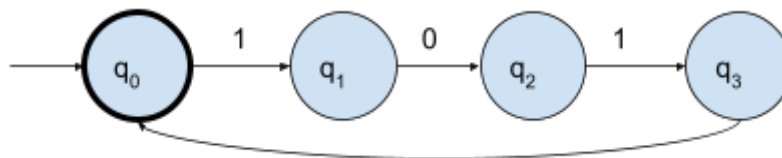
$L_7 = \{x \in \Sigma^* : x \text{ é formado por zero ou mais ocorrências não superpostas de } 101\}$

Esse exemplo também ilustra uma facilidade dos NFA. Perceba que queremos que o autômato que aceita apenas 101 aceite repetidamente, a cada ocorrência de 101. Para isso basta que o estado final aponte de volta para o estado q_1 quando ocorrer o próximo símbolo

1. O estado q_1 indica que o primeiro símbolo da sequência 101 foi entrado corretamente. Como também queremos que ε seja aceita, o estado inicial deve ser de aceitação.



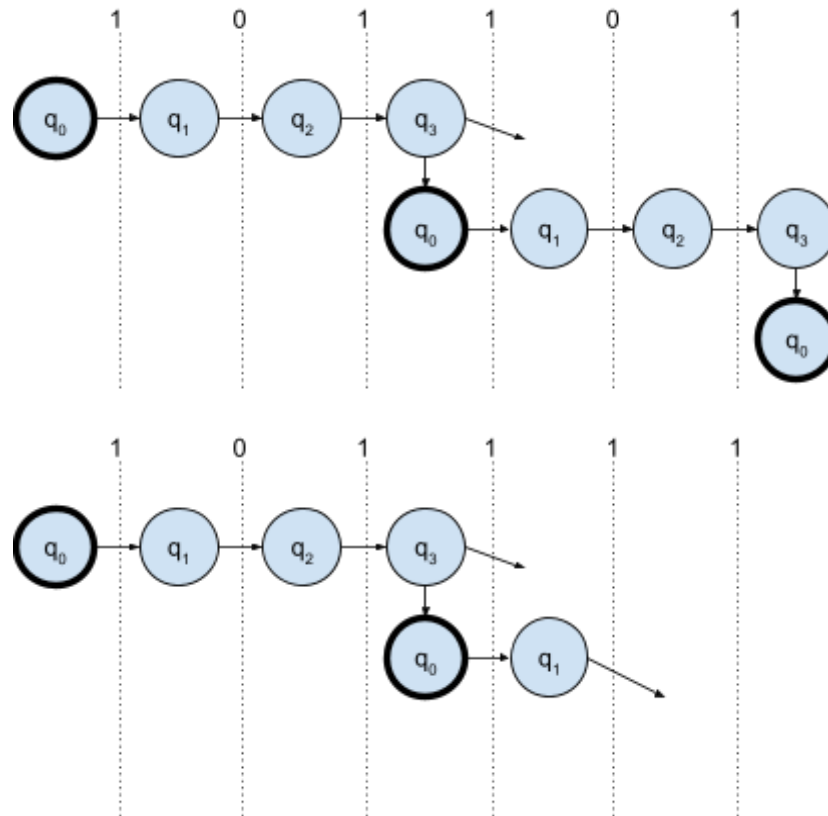
Esse autômato também poderia ser construído utilizando transições vazias. Toda vez que atingimos o estado q_3 a sequência 101 foi entrada, então basta retornar deste estado para o estado inicial. Nesse caso q_3 não precisa ser de aceitação, visto que há uma transição vazia para q_0 , então toda vez que o NFA está em q_3 também está em q_0 , que é de aceitação. O raciocínio aqui é análogo ao do exemplo anterior, mas ao invés de dividir em 2 partes, estamos dividindo em qualquer quantidade de partes tais que cada uma delas é aceita pelo autômato.



A descrição desse autômato é $D = (\{q_0, q_1, q_2, q_3\}, \Sigma, \delta, q_0, \{q_0\})$. Com δ definido por:

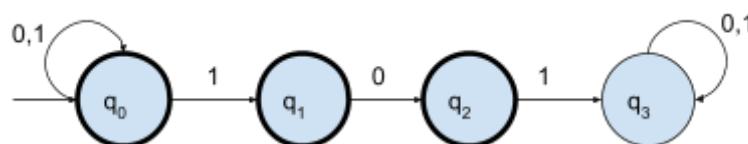
δ	q_0	q_1	q_2	q_3
0	$\emptyset$	$\{q_2\}$	$\emptyset$	$\emptyset$
1	$\{q_1\}$	$\emptyset$	$\{q_3\}$	$\emptyset$
ε	$\emptyset$	$\emptyset$	$\emptyset$	$\{q_0\}$

Mais uma vez, vamos analisar o processamento de algumas entradas. Considere as entradas 101101 e 101111.

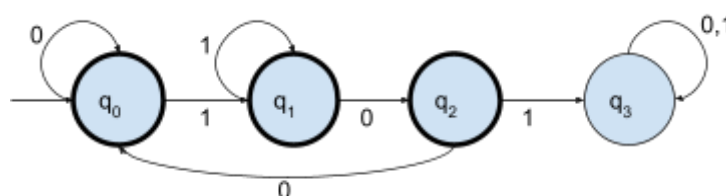


Exemplo 8: $L_8 = \{x \in \Sigma^* : x \text{ não contém } 101\}$

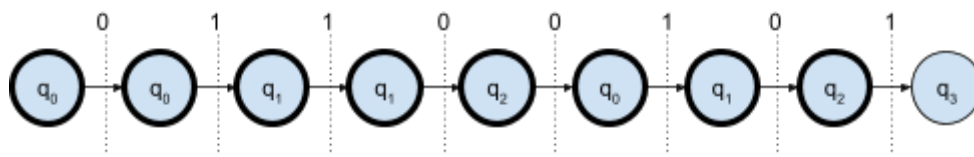
Note que, diferentemente dos DFA, a inversão dos estados de rejeição e aceitação não nos dá um autômato que aceita o complemento da linguagem.



Neste NFA, q_3 só é atingido quando a sequência 101 ocorre, mas nunca saímos de q_0 , então este autômato aceitaria qualquer entrada. Precisamos de um autômato que, após detectar a sequência, não aceite mais nada.



Este autômato é um NFA e também um DFA, não há transições vazias e nunca se está em mais de um estado simultaneamente. Além disso, todas as transições de cada estado estão definidas. Apesar de ser um DFA, vou fazer o detalhamento da computação de uma cadeia que é rejeitada pelo autômato.

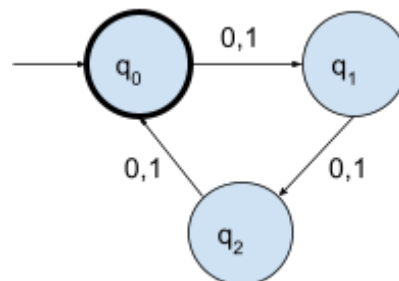


A descrição desse autômato é $D = (\{q_0, q_1, q_2, q_3\}, \Sigma, \delta, q_0, \{q_0, q_1, q_2\})$. Com δ definido por:

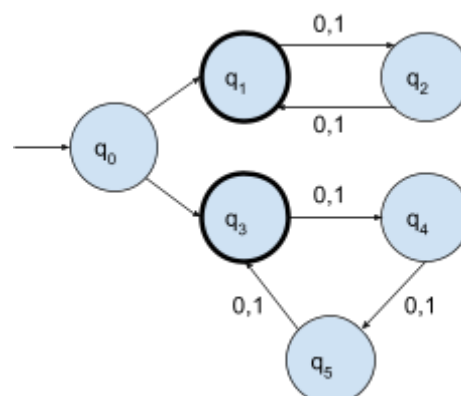
δ	q_0	q_1	q_2	q_3
0	$\{q_0\}$	$\{q_2\}$	$\{q_0\}$	$\{q_3\}$
1	$\{q_1\}$	$\{q_1\}$	$\{q_3\}$	$\{q_3\}$
ε	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Exemplo 9: $L_9 = \{x \in \Sigma^* : |x| \text{ é múltiplo de 2 ou 3}\}$

Vamos utilizar a capacidade de opção do NFA. Além disso, um problema de ter um comprimento múltiplo de 3 já foi resolvido nos exemplos de DFA.



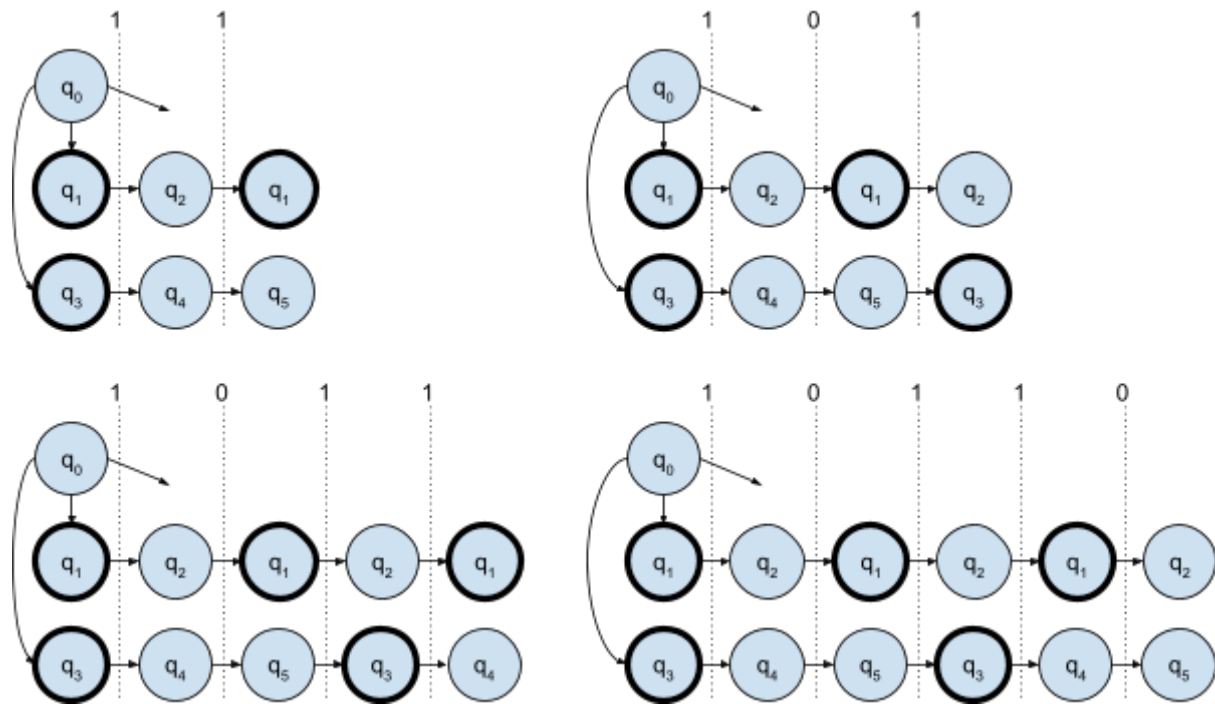
A idéia é combinar este verificador de comprimento múltiplo de 3 com um que verifica múltiplos de 2 através de uma estrutura de opção, como a do exemplo 5.



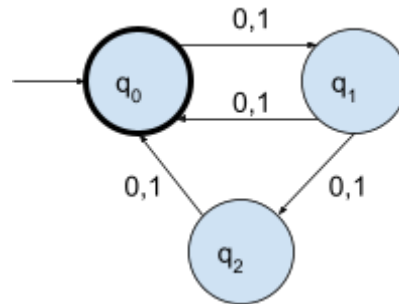
A descrição desse autômato é $D = (\{q_0, q_1, q_2, q_3, q_4, q_5\}, \Sigma, \delta, q_0, \{q_1, q_3\})$. Com δ definido por:

δ	q_0	q_1	q_2	q_3	q_4	q_5
0	$\emptyset$	$\{q_2\}$	$\{q_1\}$	$\{q_4\}$	$\{q_5\}$	$\{q_3\}$
1	$\emptyset$	$\{q_2\}$	$\{q_1\}$	$\{q_4\}$	$\{q_5\}$	$\{q_3\}$
ε	$\{q_1, q_3\}$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Mais uma vez, vamos detalhar a computação de duas algumas cadeias, Escolhi cadeias de comprimento 2 a 5. Então apenas a de comprimento 5 deve ser rejeitada.



A primeira vista pode parecer que a conversão dos estados q_0 , q_1 e q_3 em um único estado inicial de aceitação geraria um autômato que reconhece a mesma linguagem, mas isso não é verdade. Nesta configuração hipotética, cada vez que o verificador de múltiplos de dois voltasse ao estado inicial iniciaria um novo ciclo no verificador de múltiplos de 3, de tal modo que toda cadeia com comprimento maior que 1 seria aceita. Verifique isso por conta própria no autômato abaixo.



Exemplo 10: $L_{10} = \{x \in \Sigma^* : x = 0^a 1^b, a, b \in \mathbb{N}\}$

A descrição da linguagem diz que a cadeia deve ser formada por zero ou mais símbolos 0 seguidos de zero ou mais símbolos 1. Note que a cadeia vazia deve ser aceita e as cadeias formadas apenas por 0 ou apenas por 1, também. Primeiro vamos fazer apenas um NFA que aceita zero ou mais símbolos 0 e outro que aceita zero ou mais símbolos 1.



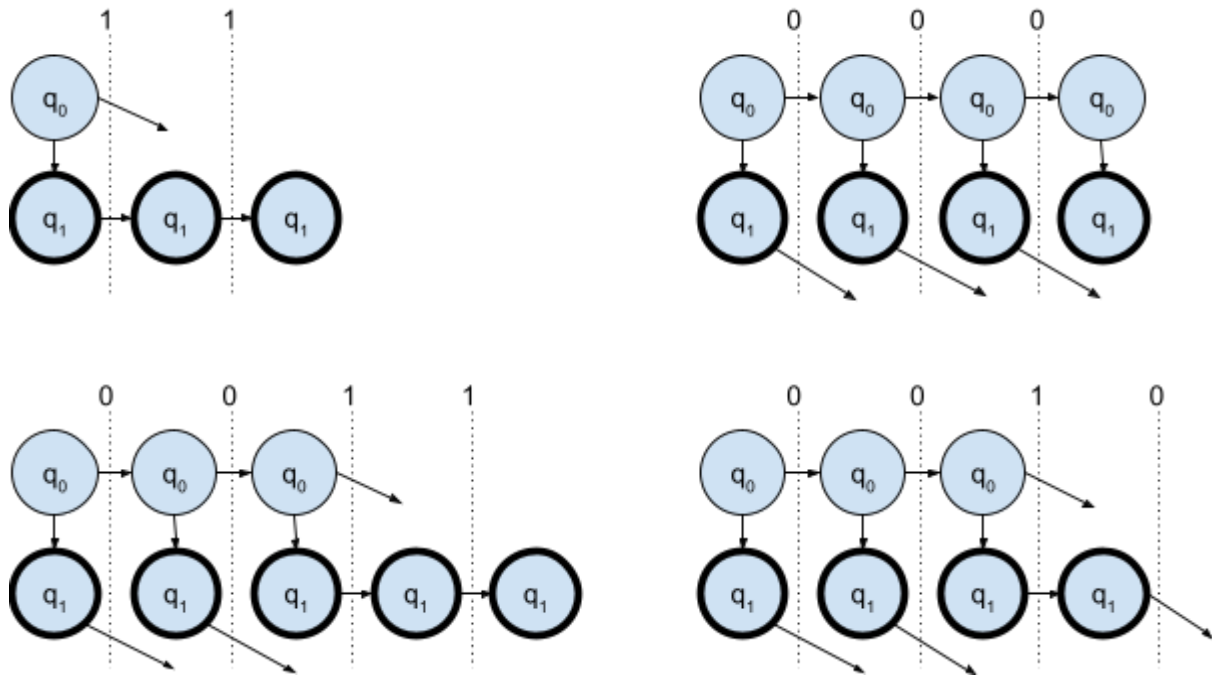
Podemos usar o sequenciamento de autômatos, como no exemplo 6.



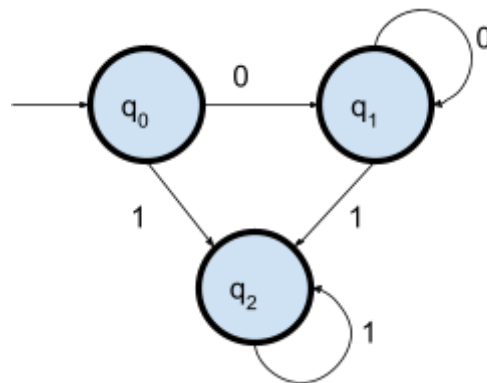
A descrição desse autômato é $D = (\{q_0, q_1\}, \Sigma, \delta, q_0, \{q_1\})$. Com δ definido por:

δ	q_0	q_1
0	$\{q_0\}$	$\emptyset$
1	$\emptyset$	$\{q_1\}$
ε	$\emptyset$	$\emptyset$

Vamos detalhar a computação deste autômato para algumas entradas. Perceba que enquanto lemos 0, os dois estados permanecem ativos. Ao ler o primeiro 1 o estado q_0 desaparece e ao ler o primeiro 0 depois disso, o estado q_1 também desaparece e nada mais é aceito.



Esta não é a única forma de fazer este autômato. O estado inicial deve ser de aceitação. Do estado inicial, se um 0 for lido, vamos para um novo estado que vai ficar ativo enquanto mais 0s forem lidos. Ao ler o símbolo 1, de qualquer um dos estados já existentes vamos para um novo estado que permanece ativo enquanto ler símbolos 1. Depois de ler o símbolo 1, se algum 0 for lido o autômato deve rejeitar.



Este autômato é um NFA mas também pode ser pensado como um DFA se assumirmos que q_2 leva para um estado de rejeição permanente (oculto no diagrama) quando lê um símbolo 0. Uma terceira forma de atacar o problema é que o autômato deve rejeitar qualquer cadeia que contenha 10.

DFA vs. NFA

No início da seção anterior eu disse que íamos tentar dar superpoderes aos Autômatos Finitos quebrando o determinismo. De fato os NFA parecem ser mais poderosos, afinal de contas eles podem estar em vários lugares (estados) ao mesmo tempo. Mas será que é isso mesmo?

Felizmente, tudo foi precisamente definido e podemos comparar essas máquinas. Para decidir se alguém é mais poderoso, vamos fazer dois testes:

1. Verificar se um NFA pode fazer tudo que um DFA faz. Ou seja, se toda linguagem reconhecida por um DFA também pode ser reconhecida por um NFA. Se isso for verdade, o NFA é igual ou mais poderoso que um DFA. Se for falso o DFA é mais poderoso.
2. Verificar se um DFA pode reconhecer todas as linguagens que um NFA reconhece. Se isso também for verdade os dois são iguais, se for falso o NFA é mais poderoso.

O primeiro teste é *simular* um DFA usando um NFA e o segundo é simular um NFA usando um DFA. Dizemos que duas máquinas são equivalentes se elas reconhecem a mesma linguagem. Pense nos autômatos como funções que recebem como entrada uma cadeia de Σ^* e retornam *aceita* ou *rejeita* como saída. Duas funções são equivalentes se produzem as mesmas saídas para as mesmas entradas, ou seja, dadas $f: X \rightarrow Y$ e $g: X \rightarrow Y$, f e g são equivalentes se $f(x) = g(x), \forall x \in X$.

Simular um DFA usando um NFA é bem simples, a própria definição já nos dá uma intuição de que um DFA pode ser visto como um caso particular de um NFA. Informalmente, um DFA é um NFA sem transições vazia e, dados um estado e um símbolo, há exatamente um estado de destino.

Teorema

Todo DFA tem um NFA equivalente.

Prova

Seja $D = (Q, \Sigma, \delta, q_0, F)$ um DFA, queremos construir um NFA $N = (S, \Gamma, \sigma, s_0, A)$ tal que $\mathcal{L}(D) = \mathcal{L}(N)$. Faça $S = Q$, $\Gamma = \Sigma$, $s_0 = q_0$ e $A = F$. A função de transição precisa ser modificada porque no DFA temos $\delta: Q \times \Sigma \rightarrow Q$ e no NFA temos $\sigma: Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$. Basta definir, para todo $r \in Q$ e todo $a \in \Sigma_\epsilon$:

- $\sigma(r, a) = \emptyset$ se $a = \epsilon$, ou seja, não há transições vazias; e
- $\sigma(r, a) = \{\delta(r, a)\}$ se $a \neq \epsilon$, transformando o estado $\delta(r, a)$ em um conjunto unitário para satisfazer a definição do NFA.

Com isso o primeiro teste foi verdadeiro e sabemos que o NFA é igual ou mais poderoso que o DFA, como esperado. A simulação de um NFA usando um DFA é o verdadeiro desafio, a única certeza que temos é que o alfabeto deve ser o mesmo, já que ambos devem reconhecer a mesma linguagem. Antes de tentar provar a equivalência, vamos lembrar das diferenças entre um DFA e um NFA e tentar superá-las.

O DFA “lembra-se” apenas do estado que está atualmente, que é único. Já o NFA lembra-se de todos os estados em que está atualmente, podendo qualquer subconjunto de seus estados. Portanto, se o NFA tem n estados existem 2^n situações possíveis em que ele

pode estar. Daí, se queremos simular um NFA usando um DFA, este último deverá ter 2^n estados, cada um dos estados do DFA representando um subconjunto dos estados do NFA.

Em ambos os casos somente um estado inicial é permitido, mas já vimos que é possível emular múltiplos estados iniciais em um NFA usando as transições vazias. Se o estado inicial do NFA é s_0 e este estado consegue alcançar os estados s_1 e s_2 através de uma ou mais transições vazias, os estados iniciais, na prática, são s_0 , s_1 e s_2 . Portanto o estado inicial do DFA deve ser aquele que representa o subconjunto de estados do NFA que é alcançável a partir do seu estado inicial através de zero ou mais transições vazias.

O NFA aceita a cadeia se qualquer um dos seus ramos de computação terminar em um estado de aceitação. Portanto, qualquer subconjunto de estados do NFA que contém pelo menos um estado de aceitação deve ser um estado de aceitação no DFA.

Quatro dos elementos que definem o DFA já foram resolvidos, falta apenas a função de transição. Como já sabemos que cada estado do DFA vai representar um subconjunto de estados do NFA, basta seguir a computação e verificar como se dá a transição entre subconjuntos. Por exemplo, suponha que o NFA está nos estados s_1 e s_2 e lê o símbolo a da entrada, suponha ainda que na função de transição do NFA define-se $\sigma(s_1, a) = \emptyset$ e $\sigma(s_2, a) = \{s_3\}$. Neste exemplo a transição se dá do subconjunto $\{s_1, s_2\}$ para o subconjunto $\{s_3\}$. Esse procedimento pode ser repetido para cada subconjunto e cada símbolo de entrada.

Note que a sugestão ignora a existência de transições vazias. No exemplo do parágrafo anterior, se s_3 alcança s_5 e s_7 através de transições vazias, o subconjunto efetivamente alcançado a partir de $\{s_1, s_2\}$ com entrada a seria $\{s_3, s_5, s_7\}$, já que as transições vazias se dão de forma imediata, sem leitura da entrada.

Teorema

Todo NFA tem um DFA equivalente.

Prova

Seja $N = (S, \Gamma, \sigma, s_0, A)$ um NFA, queremos construir um DFA $D = (Q, \Sigma, \delta, q_0, F)$ tal que $\mathcal{L}(D) = \mathcal{L}(N)$. Faça $\Sigma = \Gamma$, $Q = \mathcal{P}(S)$, cada estado do DFA é um subconjunto dos estados do NFA, e $F = \{R \in Q: R \cap A \neq \emptyset\}$, todo subconjunto que contém pelo menos um estado de aceitação do NFA é um estado de aceitação do DFA.

Podemos definir $q_0 = C_\varepsilon(s_0)$, ou seja, o estado inicial do DFA é o subconjunto de estados alcançados a partir do estado inicial do NFA após as transições vazias.

Finalmente, temos que $\delta(R, a) = \bigcup_{r \in R} C_\varepsilon(\sigma(r, a))$. Para simular o funcionamento do NFA, verificamos para cada estado do NFA no subconjunto atual, qual a transição dada pela função de transição do NFA seguida pelas transições vazias. Este é o procedimento de transição do NFA, cada um dos estados ativos realiza suas transições de forma

independente gerando um novo conjunto de estados ativos, e antes de fazer a próxima leitura da entrada todas as transições vazias são efetuadas.

Perceba que esta é a definição de transição de conjuntos de estados usada na definição alternativa da computação no NFA.

Esses dois teoremas nos dizem que DFA e NFA são modelos de computação equivalentes, ou seja, toda linguagem reconhecível por algum NFA também é reconhecível por algum DFA e vice-versa. Os NFA muitas vezes nos permitem criar diagramas de estado mais simples, mas tudo que é feito com um NFA também pode ser feito com um DFA. Logo, nossa tentativa de dar superpoderes aos autômatos finitos através da quebra do determinismo foi frustrada.

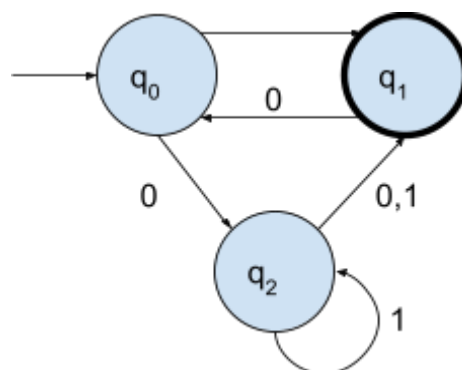
Uma consequência dessa equivalência é o corolário que segue, uma definição alternativa das linguagens regulares

Corolário

Uma linguagem é regular se, e somente se, algum NFA a reconhece.

Exemplo

Converta o seguinte NFA em um DFA equivalente.

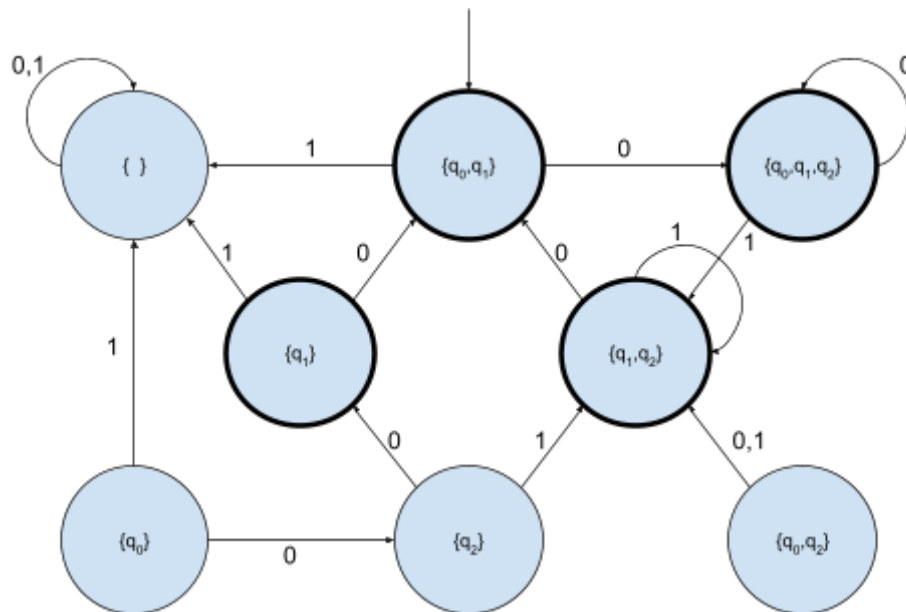


Utilizando o método descrito na prova temos $D = (S, \Sigma, \delta, s_0, F)$ tal que:

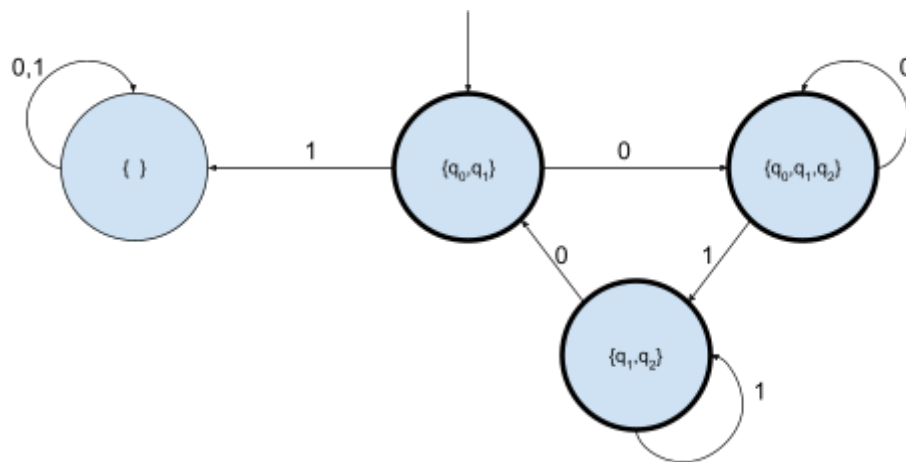
- $S = \{\emptyset, \{q_0\}, \{q_1\}, \{q_2\}, \{q_0, q_1\}, \{q_0, q_2\}, \{q_1, q_2\}, \{q_0, q_1, q_2\}\}$
- $\Sigma = \{0, 1\}$
- $F = \{\{q_1\}, \{q_0, q_1\}, \{q_1, q_2\}, \{q_0, q_1, q_2\}\}$
- $s_0 = \{q_0, q_1\}$, porque q_1 é alcançável a partir de q_0 com uma transição vazia.

δ	$\emptyset$	$\{q_0\}$	$\{q_1\}$	$\{q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$	$\{q_1, q_2\}$	$\{q_0, q_1, q_2\}$
0	$\emptyset$	$\{q_2\}$	$\{q_0, q_1\}$	$\{q_1\}$	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_1, q_2\}$
1	$\emptyset$	$\emptyset$	$\emptyset$	$\{q_1, q_2\}$	$\emptyset$	$\{q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_1, q_2\}$

O diagrama de estados desse DFA é dado a seguir



Note que os estados $\{q_0\}$, $\{q_1\}$, $\{q_2\}$ e $\{q_0, q_2\}$ podem ser removidos pois não são alcançáveis a partir do estado inicial. Portanto o seguinte DFA é equivalente ao NFA do início.



Tente determinar a linguagem reconhecida por esse autômato. Provavelmente é mais fácil no DFA equivalente que no NFA original.

Nem sempre todos os subconjuntos são necessários no DFA equivalente. Para evitar a criação de tais estados desnecessários basta fazer o processo de conversão passo a passo, inicialmente o conjunto de estados, Q , tem apenas o subconjunto inicial. Escolha um estado de Q que ainda não tem as transições definidas e, para cada símbolos do alfabeto, verifique o subconjunto alcançável e inclua-o em Q . Repita até que todos os estados em Q tenham suas transições definidas.

Operações Regulares

Nesta seção introduzimos três operações sobre linguagens. Essas operações são muito similares às operações já definidas para conjuntos cadeias. Sejam A e B duas linguagens sobre o mesmo alfabeto.

1. A *união* das linguagens A e B é uma nova linguagem definida como $A \cup B = \{x: x \in A \text{ ou } x \in B\}$. Note que eu escolhi usar o símbolo “ $\cup$ ” para a união. Alguns livros usam outros símbolos como por exemplo “ $\cup$ ” ou “ $+$ ”.
2. A *concatenação* das linguagens A e B é uma nova linguagem definida como $AB = \{xy: x \in A \text{ e } y \in B\}$. Alguns livros usam um símbolo entre as duas linguagens para indicar a concatenação, escolhas comuns são os símbolos “ $\cdot$ ” ou “ $\circ$ ”. Perceba como essa definição é consistente com a potência de alfabetos, por exemplo $AA = A^2$. Note que AB é diferente de BA , ou seja, não é uma operação comutativa.
3. A *estrela* da linguagem A é uma nova linguagem definida como $A^* = \{x_1 x_2 \dots x_k: k \geq 0 \text{ e } x_i \in A, i = 1, 2, \dots, k\}$. Equivalentemente, a estrela pode ser definida como $A^* = \bigcup_{k=0}^{\infty} A^k$. Note que é a mesma definição do fecho de Kleene para alfabetos. Lembre que $A^0 = \{\epsilon\}$.

Por que essas operações são chamadas de regulares? Qual a importância delas? Paciência jovem padawan. No resto desta seção e na seção seguinte, isso veremos.

Teorema

As linguagens regulares são fechadas sob a operação de união.

Prova 1

Primeiro vamos ter certeza que entendemos o que está sendo dito. Dizer que as linguagens regulares são fechadas sob uma operação quer dizer que o resultado da operação também é uma linguagem regular. Já vimos que para uma linguagem ser regular deve existir um autômato finito, determinístico ou não, que reconhece a linguagem. Portanto podemos provar que $A \cup B$ é regular mostrando que, dados um autômato M_1 que reconhece A e um autômato M_2 que reconhece B , podemos construir um autômato M que reconhece $A \cup B$.

Nesta primeira prova vamos utilizar apenas DFAs. O autômato M deve aceitar sempre que M_1 ou M_2 aceitarem. Podemos utilizar aqui uma técnica análoga àquela usada na conversão de um NFA em DFA, precisamos simular a execução de dois DFAs em paralelo. Sejam $M_1 = \{Q_1, \Sigma, \delta_1, q_1, F_1\}$ e $M_2 = \{Q_2, \Sigma, \delta_2, q_2, F_2\}$. Defina $M = \{Q, \Sigma, \delta, q_0, F\}$ tal que:

- $Q = Q_1 \times Q_2$, ou seja, todo par de estados onde o primeiro pertence a Q_1 e o segundo a Q_2 . Podemos assumir, sem perda de generalidade, que $Q_1 \cap Q_2 = \emptyset$, se não for o caso basta renomear os estados de um dos autômatos;
- $q_0 = (q_1, q_2)$, o estado inicial é o par que indica os estados iniciais de cada um dos dois autômatos;
- $F = (F_1 \times Q_2) \cup (F_2 \times Q_1) = \{(x, y): x \in F_1 \text{ ou } y \in F_2\}$. Ou seja, qualquer par de estados em que pelo menos um dos elementos está nos conjuntos de aceitação;
- $\delta((x, y), a) = (\delta(x, a), \delta(y, a))$, o novo estado é o par resultante da transição de cada um dos estados ativos.

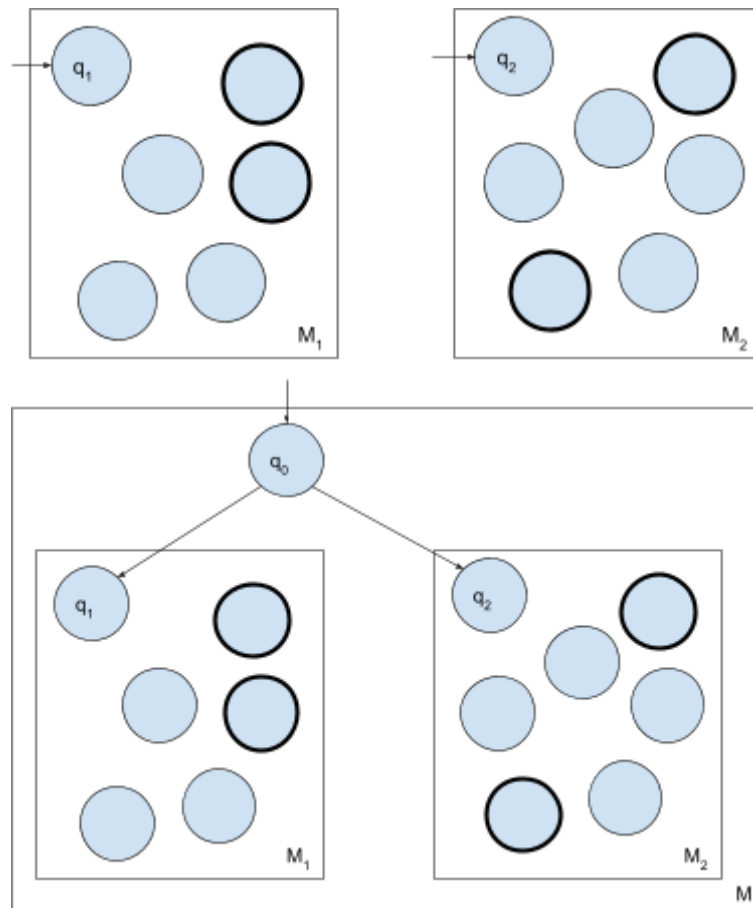
Prova 2

Agora vamos fazer a mesma prova usando NFAs. Sejam $M_1 = \{Q_1, \Sigma, \delta_1, q_1, F_1\}$ e $M_2 = \{Q_2, \Sigma, \delta_2, q_2, F_2\}$ NFAs tais que $\mathcal{L}(M_1) = A$ e $\mathcal{L}(M_2) = B$. Defina $M = \{Q, \Sigma, \delta, q_0, F\}$ tal que:

- $Q = \{q_0\} \cup Q_1 \cup Q_2$, como na prova anterior, assumimos que não há estados em comum entre os dois autômatos;
- $F = F_1 \cup F_2$, qualquer um dos estados de aceitação dos autômatos M_1 e M_2 são de aceitação na combinação dos dois;
- $\delta(q, a) =$
 - $\delta_1(q, a)$, se $q \in Q_1$;
 - $\delta_2(q, a)$, se $q \in Q_2$;
 - $\{q_1, q_2\}$, se $q = q_0$ e $a = \varepsilon$;
 - $\emptyset$, se $q = q_0$ e $a \neq \varepsilon$.

Deste modo temos que $\mathcal{L}(M) = A|B$.

Basicamente, o autômato resultante da prova utilizando NFA tem um novo estado inicial com transições vazias para os estados iniciais de M_1 e M_2 . Como o NFA já faz o processamento em paralelo, se qualquer um dos autômatos aceitar, M também aceita. A ilustração abaixo pode ajudar a entender o processo (que já foi usado no exemplo 5). Os autômatos foram desenhados sem as transições para evidenciar as transições acrescentadas.



Quanto à prova utilizando DFA, perceba que se trocássemos o conjunto de aceitação de $F = \{(x, y): x \in F_1 \text{ ou } y \in F_2\}$ para $F = \{(x, y): x \in F_1 \text{ e } y \in F_2\} = F_1 \times F_2$, M aceitaria somente se ambos, M_1 e M_2 , aceitam. Essa é a definição da operação de *interseção* entre as linguagens A e B , $A \cap B = \{x: x \in A \text{ e } x \in B\}$. Com esta modificação temos a prova de que as linguagens regulares são fechadas sob a operação de interseção.

Teorema

As linguagens regulares são fechadas sob a operação de concatenação.

Prova

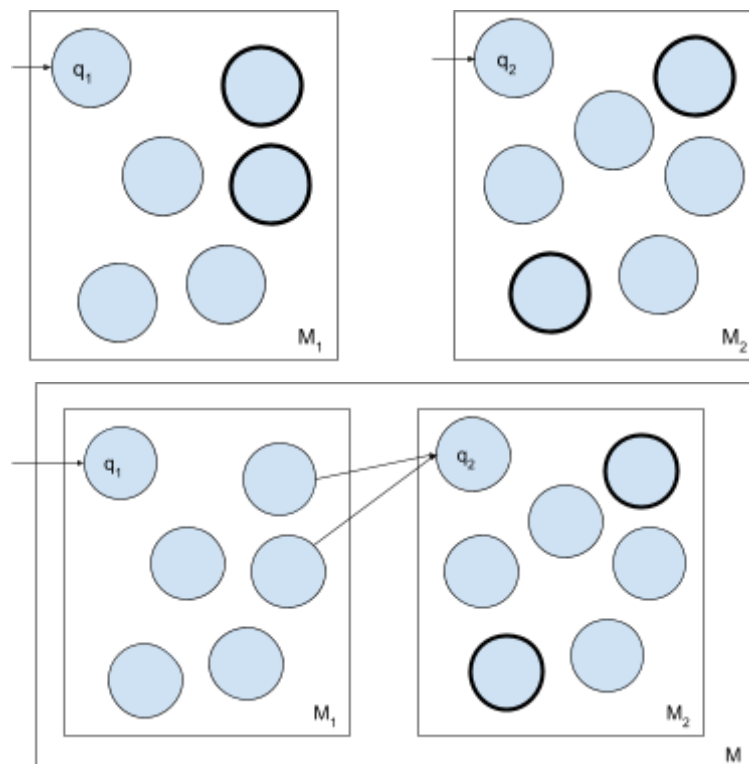
Vamos fazer a prova utilizando apenas NFAs. Sejam $M_1 = \{Q_1, \Sigma, \delta_1, q_1, F_1\}$ e $M_2 = \{Q_2, \Sigma, \delta_2, q_2, F_2\}$ NFAs tais que $\mathcal{L}(M_1) = A$ e $\mathcal{L}(M_2) = B$. Defina $M = \{Q, \Sigma, \delta, q_0, F\}$ tal que:

- $Q = Q_1 \cup Q_2$, como na prova anterior, assumimos que não há estados em comum entre os dois autômatos e não há nenhum estado extra nesse novo autômato;
- $q_0 = q_1$, o processamento inicia pelo início de M_1 ;
- $F = F_2$, e termina no fim de M_2 ;

- $\delta(q, a) =$
 - $\delta_1(q, a)$, se $q \in Q_1$ e $q \notin F_1$;
 - $\delta_1(q, a)$, se $q \in F_1$ e $a \neq \varepsilon$;
 - $\delta_1(q, a) \cup \{q_2\}$, se $q \in F_1$ e $a = \varepsilon$;
 - $\delta_2(q, a)$, se $q \in Q_2$.

Deste modo temos que $\mathcal{L}(M) = AB$.

Nesta construção, o autômato resultante inicia no estado inicial de M_1 e só aceita se M_2 aceitar. Os estados finais de M_1 têm transições vazias para o estado inicial de M_2 , ou seja, só é iniciado o processamento de M_2 depois que sabemos que M_1 aceita. O autômato como um todo só vai aceitar se a parte inicial da cadeia é aceita por M_1 e a parte final por M_2 . O ponto que separa “inicial” de “final” é arbitrariamente definido pela transição vazia. A ilustração abaixo pode ajudar a entender o processo (que já foi usado no exemplo 6). Os autômatos foram desenhados sem as transições para evidenciar as transições acrescentadas.



Teorema

As linguagens regulares são fechadas sob a operação estrela.

Prova

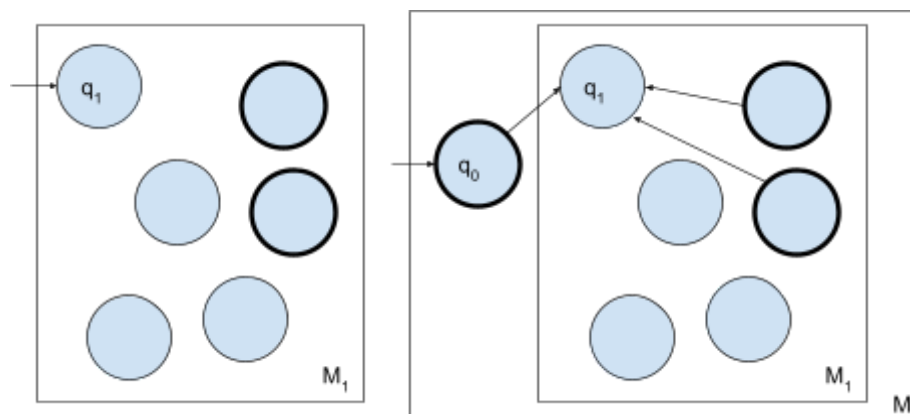
Vamos fazer a prova utilizando apenas NFAs. Seja $M_1 = \{Q_1, \Sigma, \delta_1, q_1, F_1\}$ um NFA tal que $\mathcal{L}(M_1) = A$. Defina $M = \{Q, \Sigma, \delta, q_0, F\}$ tal que:

- $Q = Q_1 \cup \{q_0\}$;
- $F = F_1 \cup \{q_0\}$, o novo estado inicial também é de aceitação;
- $\delta(q, a) =$
 - $\delta_1(q, a)$, se $q \in Q_1$ e $q \notin F_1$;
 - $\delta_1(q, a)$, se $q \in F_1$ e $a \neq \varepsilon$;
 - $\delta_1(q, a) \cup \{q_1\}$, se $q \in F_1$ e $a = \varepsilon$;
 - $\{q_1\}$, se $q = q_0$ e $a = \varepsilon$;
 - $\emptyset$, se $q = q_0$ e $a \neq \varepsilon$.

Deste modo temos que $\mathcal{L}(M) = A^*$.

Perceba que a operação estrela é uma sequência de concatenações. Então utilizamos uma idéia similar àquela na prova de que a concatenação é regular, os estados finais retornam ao estado inicial de M_1 . Além disso, não sabemos se M_1 aceita a cadeia vazia, que deve ser aceita, por isso incluímos um novo estado inicial que é de aceitação.

Se o novo estado inicial não fosse inserido teríamos a prova de que as linguagens regulares são fechadas sob a operação *cruz* ou *fecho positivo* definida como $A^+ = \bigcup_{k=1}^{\infty} A^k$. A ilustração abaixo pode ajudar a entender o processo (que já foi usado no exemplo 7). Os autômatos foram desenhados sem as transições para evidenciar as transições acrescentadas.



Uma operação entre linguagens é dita regular se as linguagens regulares são fechadas sob a operação. Existem outras operações regulares além daquelas enunciadas no início desta seção. Pelo comentário após a prova da união, já sabemos que a operação de interseção é regular. Também sabemos que a operação cruz é regular pelos comentários após a prova da estrela. As operações de negação e reversão também são operações regulares.

A *negação* ou *complemento* de uma linguagem A é definida como $\neg A = \{x \in \Sigma^* : x \notin A\}$, alguns textos utilizam uma barra sobre o nome do conjunto para indicar o complemento. A *reversão* de uma linguagem A é definida como $A^R = \{x^R : x \in A\}$, ou seja, é a reversão de cada uma das cadeias de A . A prova fica por sua conta, para uma ideia acerca da negação veja o exemplo 10 dos DFA.

Expressões Regulares (RE)

As expressões regulares (RE - *Regular Expression*) são uma forma compacta e eficiente para descrever linguagens. Assim como as fórmulas atômicas (símbolo proposicional e símbolo de verdade) em conjunto com as operações Booleanas definem as fórmulas lógicas válidas e os números e variáveis em conjunto com as operações aritméticas definem as expressões aritméticas, as linguagens elementares em conjunto com as operações regulares definem uma linguagem.

Você provavelmente já teve algum contato com expressões regulares, elas estão presentes em diversas linguagens de programação e ferramentas de processamento de texto como uma forma de descrever o padrão que deseja ser encontrado. Além de estarem disponíveis como ferramentas para o usuário final, internamente as expressões regulares são fundamentais em compiladores (analisador léxico) e editores de texto (realce de sintaxe).

Definição: Expressão Regular (RE)

Seja Σ um alfabeto não vazio.

1. $\emptyset$ é uma expressão regular;
2. ε é uma expressão regular;
3. Para cada $a \in \Sigma$, a é uma expressão regular;
4. Se R_1 e R_2 são expressões regulares, então $R_1 | R_2$ é uma expressão regular;
5. Se R_1 e R_2 são expressões regulares, então $R_1 R_2$ é uma expressão regular;
6. Se R é uma expressão regular, então R^* é uma expressão regular.

Perceba que os três primeiros casos são as expressões atômicas, o primeiro é a linguagem vazia, o segundo e o terceiro são as linguagens unitárias, com a cadeia vazia ou com um único símbolo, respectivamente. As três regras seguintes mostram como construir indutivamente expressões regulares a partir de outras expressões já conhecidas. Note que usamos apenas as três operações que provamos ser regulares na seção anterior, por isso a importância dessas três operações em particular.

Definição: Linguagem descrita por uma RE

A linguagem descrita por uma RE R , $\mathcal{L}(R)$, é o conjunto de todas as cadeias que podem ser escritas usando os símbolos presentes em R e as operações de união, concatenação e estrela lá descritos.

Às vezes dizemos que as expressões regulares geram cadeias. Na motivação da disciplina dissemos que iríamos estudar as gramáticas, que são geradores de cadeias. Nesse sentido, podemos considerar as RE como gramáticas, mas não são gramáticas no sentido de Chomsky, estas veremos em uma seção futura.

Similarmente aos autômatos, dizemos que duas expressões regulares são equivalentes se descrevem a mesma linguagem. Por conveniência, podemos usar potências e fecho positivo para simplificar a expressão escrita. Além disso, perceba que a operação de união pode ser representada por um conjunto, ou seja, $a|b|c = \{a, b, c\}$, daí, podemos também utilizar conjuntos (como o alfabeto) nas expressões regulares.

Mais uma vez por convenção, para evitar o excesso de parênteses nas expressões, vamos definir uma ordem de precedência das operações regulares, primeiro serão avaliadas as estrelas, depois as concatenações e por último as uniões. Os parênteses têm prioridade sobre todas as operações, sendo avaliados do mais interno para o mais externo.

A seguir são listadas algumas identidades que podem ser úteis.

Sejam R , R_1 e R_2 expressões regulares:

- | | |
|---|---|
| 1. $R\emptyset = \emptyset R = \emptyset$; | 9. $\emptyset^* = \varepsilon$; |
| 2. $R\varepsilon = \varepsilon R = R$; | 10. $\varepsilon^* = \varepsilon$; |
| 3. $R \emptyset = \emptyset R = R$; | 11. $(\varepsilon R)^* = R^*$; |
| 4. $R R = R$; | 12. $(\varepsilon R)(\varepsilon R)^* = R^*$; |
| 5. $R_1 R_2 = R_2 R_1$; | 13. $R^*(\varepsilon R) = (\varepsilon R)R^* = R^*$; |
| 6. $R(R_1 R_2) = RR_1 RR_2$; | 14. $R_1^*R_2 R_2 = R_1^*R_2$; |
| 7. $(R_1 R_2)R = R_1R R_2R$; | 15. $R_1(R_2R_1)^* = (R_1R_2)^*R_1$; |
| 8. $R(R_1R_2) = (RR_1)R_2$; | 16. $(R_1 R_2)^* = (R_1^*R_2^*)^*R_1^* = (R_2^*R_1^*)^*R_2^*$ |

Tente entender cada uma das identidades e, se não se convencer, faça uma prova. É importante perceber a diferença entre $\emptyset$ e ε . A linguagem vazia é o elemento nulo da concatenação e elemento neutro da união (como o 0 no produto e na soma da aritmética ou o falso na conjunção e na disjunção da lógica). A cadeia vazia é o elemento neutro da concatenação (como o 1 no produto da aritmética ou o verdadeiro na conjunção da lógica).

Teorema: Teorema de Arden

Se R , Q e P são linguagens regulares tais que $R = Q|RP$ e $\varepsilon \notin P$, então $R = QP^*$.

Prova

Vamos separar a prova em duas partes, primeiro vamos mostrar que $R \subseteq QP^*$ e depois que $QP^* \subseteq R$.

Vamos provar por indução em $|w|$ que se $w \in R$ então $w \in QP^*$. Se $|w| = 0$, ou seja $w = \varepsilon$, temos que $\varepsilon = Q|\varepsilon P$. Como $\varepsilon \notin P$, concluímos que $\varepsilon \in Q$ e, portanto, $R \subseteq Q \subseteq QP^*$. Vamos agora assumir agora que para toda cadeia $w \in R$ tal que $|w| < n$, $w \in QP^*$. Seja $x \in R$ uma cadeia com comprimento n . Temos que $x \in Q$ ou $x \in RP$. No primeiro caso claramente $x \in QP^*$, ou seja, $x \subseteq QP^*$. No segundo caso, $x = ab$ com $a \in R$ e $b \in P$. Como $\varepsilon \notin P$, temos que $|a| < |x| = n$ e, pela hipótese de indução, $a \in QP^*$. Finalmente, $x \in QP^*P$, ou seja $R \subseteq QP^*P \subseteq QP^*$.

Agora vamos provar por indução em i que, $QP^i \subseteq R$ para todo $i \geq 0$. O caso base, $Q \subseteq R$, é claro a partir de $R = Q|RP$. Agora suponha que $QP^k \subseteq R$. Então $QP^{k+1} = QP^kP$ e pela hipótese de indução $QP^{k+1} \subseteq RP \subseteq R$.

Esse teorema não é apresentado no livro do Sipser (até onde eu me lembro), mas pode ser utilizado em um procedimento importante que apresentaremos futuramente.

Exemplos de Expressões Regulares

Para todos os exemplos que seguem é importante que você tente fazer tanto a ida quanto a volta. Ou seja, da expressão para a descrição e da descrição para a expressão. Todos os exemplos são sobre o alfabeto $\Sigma = \{0, 1\}$.

Exemplo 1

$$(0|\varepsilon)(1|\varepsilon) = \{\varepsilon, 0, 1, 01\}$$

Exemplo 2

$$1^*(01^+)^* = 1^*(011^*)^* = \{x \in \Sigma^* : \text{todo } 0 \text{ em } x \text{ é seguido de pelo menos um } 1\}$$

Exemplo 3

$$(\Sigma\Sigma\Sigma)^* = ((0|1)(0|1)(0|1))^* = \{x \in \Sigma^* : |x| \text{ é múltiplo de } 3\}$$

Exemplo 4

$$(00|01|10|11)^* = (\Sigma\Sigma)^* = \{x \in \Sigma^* : |x| \text{ é par}\}$$

Exemplo 5

$$\Sigma 01 = (0|1)01 = \{001, 101\}$$

Exemplo 6

$$(00)^* (11)^* 1$$

$= \{x \in \Sigma^* : x \text{ é formado por uma quantidade par de 0s seguido de uma quantidade ímpar de 1s}\}$

Exemplo 7

$$(0|\varepsilon)1^* = \{x \in \Sigma^* : x \text{ é uma cadeia de 1s ou um 0 seguido por uma cadeia de 1s}\}$$

Exemplo 8

$$101(0|1)^* 101 = \{x \in \Sigma^* : x \text{ começa e termina com 101}\}$$

Exemplo 9

$$(0|1)^* 101(0|1)^* = \{x \in \Sigma^* : x \text{ contém 101}\}$$

Exemplo 10

$$(0|10)^* (1|\varepsilon)(1|01)^* (0|\varepsilon)$$

$= \{x \in \Sigma^* : \text{todos os pares de 0 adjacentes aparecem antes de qualquer par de 1 adjacentes}\}$

Essa linguagem pode ser complicada de descrever. Vou tentar descrever o procedimento de criação da expressão regular a partir da descrição da linguagem. Primeiro vamos criar uma expressão regular para toda cadeia que não tem 1s adjacentes. $(0|10)^*$ é uma expressão regular que descreve cadeias sem 1s adjacentes mas que terminam em 0. Para permitir que também termine em 1, vamos fazer $(0|10)^* (1|\varepsilon)$.

O resto da cadeia não pode conter 0s adjacentes, então basta inverter os 0s e 1s na expressão anterior, obtendo $(1|01)^* (0|\varepsilon)$ para descrever tais cadeias. Finalmente, a concatenação das duas expressões nos dá a linguagem desejada.

RE vs. NFA

Já discutimos que as expressões regulares descrevem linguagens, mas que linguagens são essas que podem ser descritas por essas construções? Como já é de se esperar, visto que estamos no capítulo de linguagens regulares, as linguagens regulares são aquelas descritas por expressões regulares. O objetivo desta seção é demonstrar o seguinte teorema.

Teorema

Uma linguagem é regular se, e somente se, é descrita por uma expressão regular.

Prova

A prova deste teorema é dividida em duas partes, que serão realizadas ao longo desta seção. Primeiro vamos mostrar que toda linguagem descrita por uma RE é regular e depois vamos mostrar que toda linguagem regular pode ser descrita por uma RE.

As provas serão feitas utilizando NFAs. Para mostrar que toda linguagem descrita por um RE é regular, vamos mostrar que, dado uma RE, R , podemos criar um NFA, N , tal que $\mathcal{L}(R) = \mathcal{L}(N)$. Na verdade, a própria definição das REs já indicam isso, temos três linguagens elementares (vazia e unitárias) que são claramente regulares. Além disso a combinação das linguagens se dá através de operações regulares, que geram linguagens regulares se os operandos são regulares. No entanto, vamos fazer o procedimento de prova pois assim temos um método para a conversão entre RE e NFA.

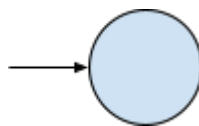
Teorema

Se uma linguagem é descrita por uma expressão regular, então ela é regular.

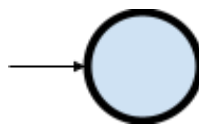
Prova

Seja R uma expressão regular, vamos converter R e um NFA, N . Vamos considerar cada um dos casos que definem uma RE.

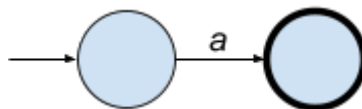
1. $R = \emptyset$. Então $\mathcal{L}(R) = \emptyset$ e o NFA abaixo reconhece $\mathcal{L}(R)$. Formalmente $N = (\{q\}, \Sigma, \delta, q, \emptyset)$, onde $\delta(s, x) = \emptyset$ para qualquer estado e entrada.



2. $R = \varepsilon$. Então $\mathcal{L}(R) = \{\varepsilon\}$ e o NFA abaixo reconhece $\mathcal{L}(R)$. Formalmente $N = (\{q\}, \Sigma, \delta, q, q)$, onde $\delta(s, x) = \emptyset$ para qualquer estado e entrada.



3. $R = a, a \in \Sigma$. Então $\mathcal{L}(R) = \{a\}$ e o NFA abaixo reconhece $\mathcal{L}(R)$. Formalmente $N = (\{q_1, q_2\}, \Sigma, \delta, q_1, \{q_2\})$, onde $\delta(q_1, a) = \{q_2\}$ e $\delta(s, x) = \emptyset$ para $s \neq q_1$ ou $x \neq a$.



4. $R = R_1 | R_2$.

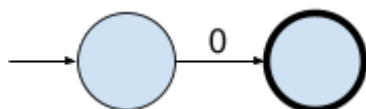
5. $R = R_1 R_2$

6. $R = R_1^*$

A construção de NFAs para os três últimos casos já foram mostradas nas provas das operações regulares.

Exemplo

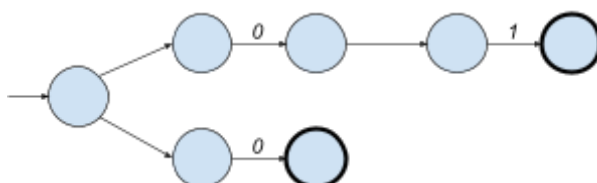
Converta a expressão regular $(01|0)^*$ em um NFA. Primeiro, os NFAs para 0 e 1.



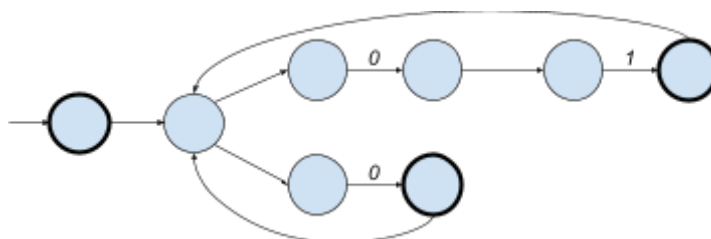
Utilizando a regra de concatenação, vamos criar um NFA para 01.



Agora usamos a regra da união para criar um NFA para $(01|0)$



Finalmente, usando a regra da estrela criamos o NFA desejado



Perceba que este NFA poderia ser mais simples, apenas aplicamos as regra diretamente.

Agora precisamos fazer a segunda etapa da prova, mas, para isso, vamos definir uma nova forma de autômato finito. Os Autômatos Finitos Não-Determinísticos Generalizados (GNFA - *Generalized Nondeterministic Finite Automata*) são NFA cujas transições podem ser representadas por expressões regulares.

Definição: Autômato Finito Não-Determinístico Generalizado (GNFA)

Um autômato finito não-determinístico generalizado é uma 5-upla $G = (Q, \Sigma, \delta, q_0, q_F)$, onde:

- Q , é o conjunto de estados;
- Σ , é o alfabeto;
- $\delta: (Q \setminus \{q_F\}) \times (Q \setminus \{q_0\}) \rightarrow \mathcal{R}$, é a função de transição onde $\mathcal{R}$ é o conjunto de todas as expressões regulares;
- q_0 , é o estado inicial;
- q_F , é o estado final.

Perceba que o GNFA é quase idêntico ao NFA, com exceção da função de transição. A transição $\delta(r, s) = 0^* 1$, por exemplo, diz que o autômato transita do estado r para o estado s se a entrada é qualquer cadeia descrita por $0^* 1$. A definição também impõe algumas limitações:

1. Só há um estado inicial e nenhuma transição aponta para ele (veja a definição da função de transição);
2. Só há um estado de aceitação e nenhuma transição parte dele. Perceba que qualquer NFA pode ser descrito com um único estado final, basta criar um novo estado final no NFA e usar transições vazias de todos os estados de aceitação antigos para esse novo estado.

A computação nessas máquinas se dá de forma similar aos NFA, com exceção de que as transições podem ler múltiplos valores da entrada.

Definição: Computação em um GNFA

Seja $G = (Q, \Sigma, \delta, q_0, q_F)$ um GNFA e seja $\omega = w_1 w_2 \cdots w_n$ uma cadeia sobre Σ onde cada $w_i \in \Sigma^*$. Defina a sequência $s_0 s_1 \cdots s_n$ de estados da seguinte forma:

- $s_0 = q_0$;
- Para i de 1 a n , s_i é um estado alcançável a partir de s_{i-1} em uma transição. Ou seja, s_i é tal que $\delta(s_{i-1}, s_i) = R_i$ e $w_i \in \mathcal{L}(R_i)$.
Daí, se existe tal sequência de estados de modo que:
- $s_n = q_F$, diz-se que M aceita ω ;

- $s_n \neq q_F$, diz-se que M *rejeita* ω .

Essencialmente, qualquer GNFA pode ser representado com apenas dois estados, o inicial e o final, e uma transição com a expressão regular que descreve a linguagem desejada. Um GNFA com função de transição $\delta(q_0, q_F) = (0|1)^* 101(0|1)^*$ é um autômato que reconhece a linguagem $\{x \in \{0, 1\}^* : x \text{ contém a cadeia } 101\}$.

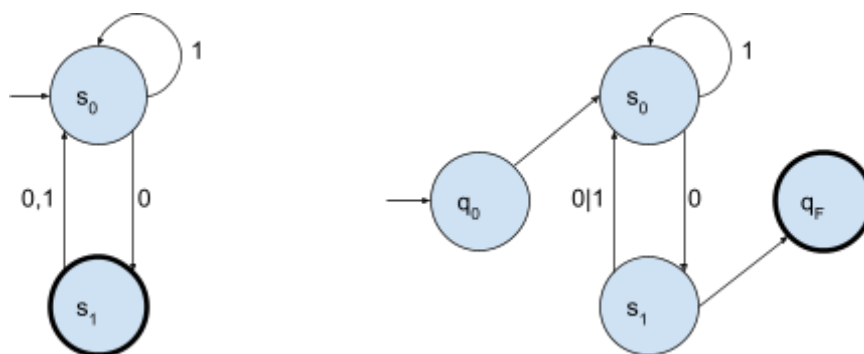
Definição: Linguagem Reconhecida por um GNFA

Seja $G = (Q, \Sigma, \delta, q_0, q_F)$ um GNFA. A *linguagem reconhecida* por G , denotada por $\mathcal{L}(G)$, é o conjunto de todas as cadeias sobre Σ^* aceitas por G : $\mathcal{L}(G) = \{x : x \in \Sigma^* \text{ e } G \text{ aceita } x\}$.

Além disso, qualquer DFA pode ser convertido em um GNFA acrescentando um novo estado inicial com transição vazia para o estado inicial do DFA e um novo estado final com transições vazias partindo de cada um dos estados de aceitação do DFA (que não serão mais de aceitação no GNFA).

Exemplo

Converta o DFA em um GNFA.



Agora vamos à prova que desejamos.

Teorema

Se uma linguagem é regular, então ela pode ser descrita por uma expressão regular.

Prova

De toda a discussão até agora, sabemos que se a linguagem é regular pode ser reconhecida por um DFA e, conseqüentemente, por um GNFA. A prova consiste em mostrar que qualquer GNFA pode ser reduzido a dois estados, e expressão regular na transição resultante é a expressão regular da linguagem reconhecida pelo DFA.

Primeiramente, vamos definir o procedimento de remover um estado do GNFA sem alterar sua linguagem. Seja $G = (Q, \Sigma, \delta, q_0, q_F)$ um GNFA com $k > 2$ estados. Defina o procedimento $\text{REDUZ}(G)$ como segue. O procedimento vai receber como entrada um GNFA G e produzir como saída um GNFA $G' = (Q', \Sigma, \delta', q_0, q_F)$. O novo conjunto de estados é obtido escolhendo aleatoriamente qualquer estado de Q , com exceção de q_0 e q_F , e removendo-o. Chamando esse estado de q_r temos então $Q' = Q \setminus \{q_r\}$.

Para definir a nova função de transição vamos reparar as arestas da seguinte forma: para qualquer $q_i \in Q' \setminus \{q_F\}$ e $q_j \in Q' \setminus \{q_0\}$ defina a nova função de transição como $\delta'(q_i, q_j) = \delta(q_i, q_r) \delta(q_r, q_j)^* \delta(q_r, q_j)$. Ou seja, a nova função de transição considera além da transição que já existia entre q_i e q_j , todas as transições entre q_i e q_j que passavam por q_r . O termo $\delta(q_r, q_r)$ aparece com estrela pois você pode repetir essa transição quantas vezes quiser antes de chegar em q_j .

Usando o procedimento $\text{REDUZ}(G)$ definimos o procedimento $\text{CONVERTE}(G)$ que recebe como entrada um GNFA com $k \geq 2$ estados e retorna uma expressão regular que descreve a linguagem reconhecida pelo autômato. $\text{CONVERTE}(G)$ é definida da seguinte forma: se $|Q| = 2$, retorne $\delta(q_0, q_F)$, se não, retorne $\text{CONVERTE}(\text{REDUZ}(G))$.

Agora vamos provar que a expressão regular retornada por essa função é equivalente a G . Vamos fazer por indução sobre o número de estados.

Base: Se $|Q| = 2$, então o GNFA tem uma única transição. A expressão regular que define essa transição descreve todas as cadeias que levam ao estado de aceitação, então essa expressão é equivalente a G .

Passo: Assuma que $|Q| = k$ e a hipótese vale para uma máquina com $k - 1$ estados. Para isso temos que provar que $\text{REDUZ}(G)$ produz um autômato G' equivalente a G . Suponha que G aceita a entrada w . Então em um ramo de aceitação na computação, G passa pelos estados $q_0 q_1 q_2 \dots q_F$. Se nenhum desses estados é o estado removido, claramente G' aceita w . Se q_r está na sequência, os estados q_i que vem imediatamente antes de q_r e q_j que vem imediatamente após q_r estão em G' e têm como transição uma expressão regular que descreve todas as cadeias que levam de q_i para q_j via q_r , então G' deve aceitar w .

No sentido contrário, vamos mostrar que se G' aceita w , G também aceita. Cada transição em G' descreve uma transição que já existia em G ou uma transição via q_r em G , então G deve aceitar w . Concluímos daí a equivalência entre os autômatos de entrada e saída do procedimento REDUZ .

A hipótese indutiva nos diz que quando a função é chamada recursivamente sobre $\text{REDUZ}(G)$ o resultado é uma linguagem regular equivalente a $\text{REDUZ}(G)$ porque este último tem $k - 1$ estados.

Perceba que a recursão não tem risco de entrar em um laço infinito pois $|Q|$ reduz de uma unidade a cada chamada recursiva e o critério de parada é $|Q| = 2$, que é o menor GNFA possível.

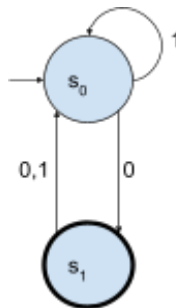
Este teorema finaliza a prova de que uma linguagem é regular se, e somente se, uma expressão regular a descreve e nos dá um procedimento para obter uma expressão regular a partir de um autômato.

Também podemos fazer essa conversão de forma algébrica. Considere que L_{q_i} é o conjunto de cadeias que levam um autômato do estado inicial até o estado q_i . Podemos escrever expressões para cada estado da seguinte forma $L_{q_i} = \bigcup_{q_k \in V(q_i)} L(q_k) a_{ik}$, onde $V(q_i)$ é o conjunto de estados que têm transição para q_i e a_{ik} é o conjunto de símbolos para os quais a transição $\delta(q_k, a_{ik}) = q_i$ é definida.

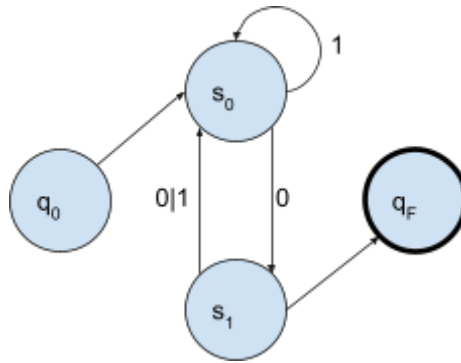
Feito isso, teremos um sistema de equações que precisa ser resolvido para definir L_{q_F} . A resolução desse sistema pode ser feita utilizando as identidades das expressões regulares e o teorema no fim daquela seção. A rigor seria necessário dar uma prova de que todo autômato pode ser convertido assim, ou seja, que o sistema sempre tem solução. Se tiver tempo eu escrevo a prova aqui algum dia.

Exemplo

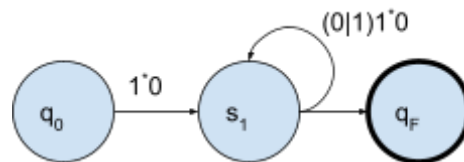
Converta o seguinte autômato para uma expressão regular



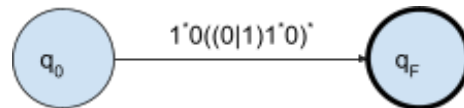
Primeiro vamos fazer através do procedimento de eliminação de estados. Primeiro vamos converter em um GNFA (isso já tinha sido feito no exemplo anterior).



Removendo o estado s_0 , precisamos adicionar uma aresta de q_0 para s_1 que passava por s_0 com expressão $(\epsilon)(1)^*(0)$ e uma transição de s_1 para ele mesmo, que consiste em sair de s_1 para s_0 ficar em s_0 e depois retornar para s_1 cuja expressão é $(0|1)(1)^*(0)$.



Finalmente a remoção de s_1 é compensada pela concatenação da expressão de q_0 para s_1 , seguida da estrela da expressão de s_1 para ele mesmo, seguida da expressão de s_1 para q_F . Essa é a expressão que representa a linguagem do autômato.

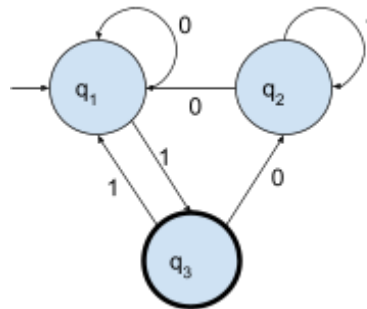


Agora vamos fazer de forma algébrica. Temos apenas duas equações $L_{s_1} = L_{s_0} 0$ e $L_{s_0} = \epsilon | L_{s_0} 1 | L_{s_1} (0|1) = (\epsilon | L_{s_1} (0|1)) | L_{s_0} 1$, note que ϵ aparece na expressão do estado inicial. O teorema nos diz que se $R = Q | R P$ e $\epsilon \notin P$, então $R = Q P^*$. Daí, fazendo $R = L_{s_0}$, $P = 1$ e $Q = (\epsilon | L_{s_1} (0|1))$, temos que $L_{s_0} = (\epsilon | L_{s_1} (0|1)) 1^*$.

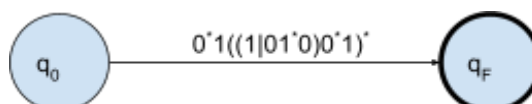
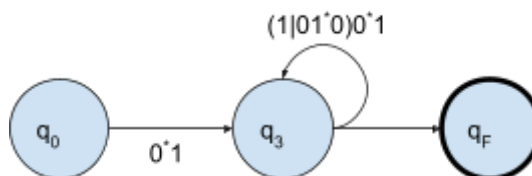
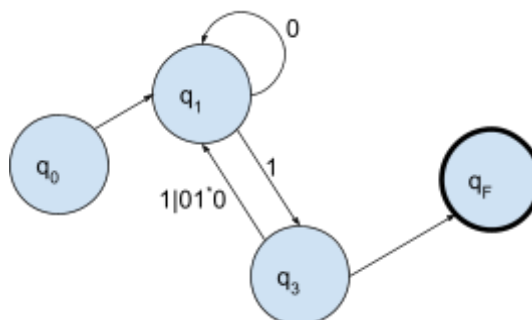
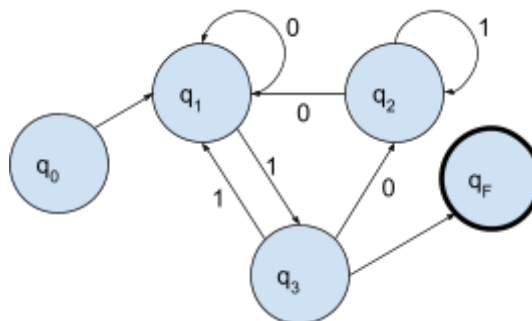
Substituindo na primeira expressão temos $L_{s_1} = (\epsilon | L_{s_1} (0|1)) 1^* 0 = 1^* 0 | L_{s_1} (0|1) 1^* 0$, nesta expressão, fazendo $R = L_{s_1}$, $P = (0|1) 1^* 0$ e $Q = 1^* 0$, temos que $L_{s_1} = 1^* 0 ((0|1) 1^* 0)^*$. Que é precisamente a expressão que obtivemos através do procedimento de eliminação de estados. A expressão obtida pelos dois procedimentos não precisa ser idêntica, basta que sejam equivalentes.

Exemplo

Converta o seguinte DFA em uma expressão regular.



Os passos serão realizados sem explicações adicionais.



De forma algébrica:

$$1. \ L_{q_1} = (L_{q_2} 0 | L_{q_3} 1 | \epsilon) | L_{q_1} 0$$

$$2. L_{q_2} = L_{q_3} 0 | L_{q_2} 1$$

$$3. L_{q_3} = L_{q_1} 1$$

Substituindo a expressão 3 nas expressões 1 e 2 temos:

$$4. L_{q_1} = (L_{q_2} 0 | \varepsilon) | L_{q_1} (0 | 11)$$

$$5. L_{q_2} = L_{q_1} 10 | L_{q_2} 1$$

Do teorema de Arden na expressão 5 temos:

$$6. L_{q_2} = (L_{q_1} 10)^* 1$$

Substituindo a expressão 6 na expressão 4:

$$7. L_{q_1} = \varepsilon | L_{q_1} (0 | 11 | 101^* 0)$$

E do teorema de Arden na expressão 7:

$$8. L_{q_1} = (0 | 11 | 101^* 0)^*$$

Finalmente, substituindo a expressão 8 na 3, obtemos a expressão desejada:

$$9. L_{q_3} = (0 | 11 | 101^* 0)^* 1$$

Que é equivalente à expressão do método de eliminação de estados.

Perceba que, da identidade 15, fazendo $R_1 = 0^* 1$ e $R_2 = (1 | 01^* 0)$, podemos escrever

$$0^* 1 ((1 | 01^* 0) 0^* 1)^* = R_1 (R_2 R_1)^* = (R_1 R_2)^* R_1 = (0^* 1 (1 | 01^* 0))^* 0^* 1$$

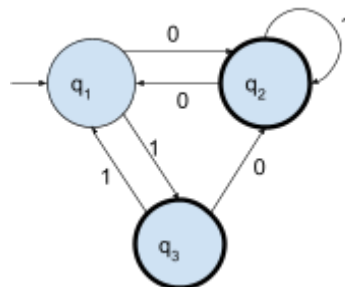
Da identidade 6, distributividade da concatenação sobre a união, podemos escrever $(0^* 1 (1 | 01^* 0))^* 0^* 1 = (0^* (11 | 101^* 0))^* 0^* 1$

Da identidade 16, com $R_1 = 0$ e $R_2 = 11 | 101^* 0$ podemos escrever

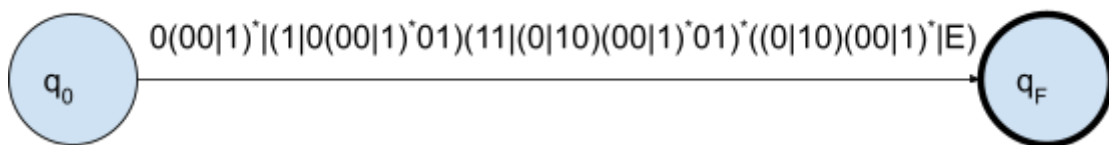
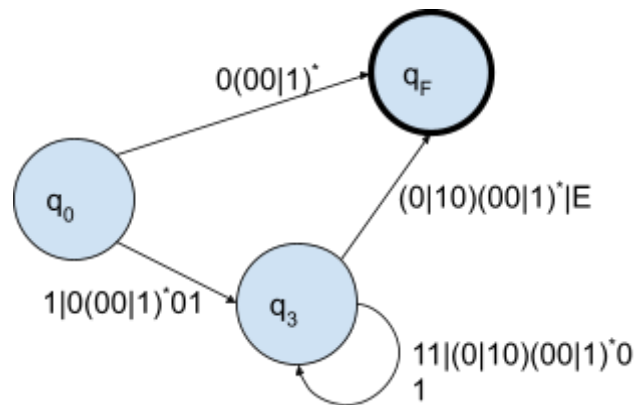
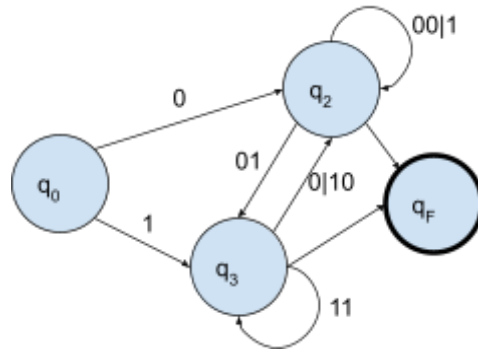
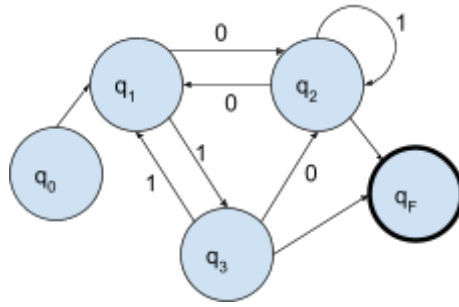
$$(0^* (11 | 101^* 0))^* 0^* 1 = (R_1^* R_2)^* R_1^* 1 = (R_1 | R_2)^* 1 = (0 | 11 | 101^* 0)^* 1$$

Exemplo

Converta o seguinte DFA em uma expressão regular.



Os passos serão realizados sem explicações adicionais.



De forma algébrica:

1. $L_{q_1} = \varepsilon | L_{q_2} 0 | L_{q_3} 1$
2. $L_{q_2} = (L_{q_1} 0 | L_{q_3} 0) | L_{q_2} 1$
3. $L_{q_3} = L_{q_1} 1$

Substituindo expressão 1 nas expressões 2 e 3:

4. $L_{q_2} = ((\varepsilon | L_{q_2} 0 | L_{q_3} 1) 0 | L_{q_3} 0) | L_{q_2} 1 = (0 | L_{q_3} (10 | 0)) | L_{q_2} (00 | 1)$
5. $L_{q_3} = (\varepsilon | L_{q_2} 0 | L_{q_3} 1) 1 = (1 | L_{q_2} 01) | L_{q_3} 11$

Usando o teorema de Arden na expressão 5 temos que:

$$6. L_{q_3} = (1|L_{q_2}01)(11)^*$$

Substituindo a expressão 6 na expressão 4:

$$7. L_{q_2} = (0|1(11)^*10|1(11)^*0)|L_{q_2}(00|1|01(11)^*10|01(11)^*0)$$

E do teorema de Arden na expressão 7:

$$8. L_{q_2} = (0|1(11)^*10|1(11)^*0)(00|1|01(11)^*10|01(11)^*0)^*$$

Substituindo a expressão 8 na expressão 6:

$$9. L_{q_3} = (1|(0|1(11)^*10|1(11)^*0)(00|1|01(11)^*10|01(11)^*0)^*01)(11)^*$$

A expressão final é $L_{q_2} | L_{q_3}$ já que ambos são estados de aceitação.

Neste último exemplo as expressões obtidas são distintas entre os dois métodos, como mostrar que elas são equivalentes? Depois voltaremos nesse problema (talvez só na disciplina de compiladores), mas adianto que uma forma possível é através de muito esforço algébrico tentar tornar as expressões idênticas, como fizemos no segundo exemplo.

Gramáticas Regulares (RG)

As gramáticas formais são conjuntos de regras que definem como criar cadeias de símbolos. Portanto, assim como as expressões regulares, as gramáticas geram uma linguagem. As regras gramaticais são chamadas de regras de produção e são, basicamente, regras de substituição de variáveis. Neste texto as variáveis iniciarão com letras maiúsculas. A regra *Frase* → *Sujeito Predicado* diz que a variável *Frase* pode ser substituída pela concatenação das variáveis *Sujeito* e *Predicado*. As regras *Artigo* → *um* e *Artigo* → *o* dizem que a variável *Artigo* pode ser substituída pela cadeia *um* ou pela cadeia *o*, essas duas regras podem ser combinadas em um única regra usando o operador de união *Artigo* → *um|o*

A derivação de uma cadeia consiste na aplicação repetida das regras de produção até que não hajam mais variáveis. Assim como os autômatos têm estado inicial, as gramáticas têm variável inicial.

Exemplo

Considere a seguinte gramática simplificada do português, que tem *Frase* como variável inicial.

Frase → *Sujeito Predicado*

Sujeito → *Artigo Substantivo*

Predicado → *Verbo*

Artigo → *o|um*

Substantivo → *cachorro|gato*

Verbo → *corre*

Verbo → *anda*

Um exemplo de derivação usando essa gramática é:

Frase $\Rightarrow$ *Sujeito Predicado* $\Rightarrow$ *Artigo Substantivo Predicado* $\Rightarrow$ *Artigo Substantivo Verbo*
 $\Rightarrow$ *o Substantivo Verbo* $\Rightarrow$ *o gato Verbo* $\Rightarrow$ *o gato anda*.

A linguagem gerada por essa gramática é

*{o cachorro corre, o cachorro anda, o gato corre, o gato anda,
 um cachorro corre, um cachorro anda, um gato corre, um gato anda}*

Talvez você lembre de algo similar na lógica de primeira ordem (resolução DPLL). O fato é que a noção é bem intuitiva. Vamos definir formalmente a gramática e como as cadeias são geradas.

Definição: Gramática Formal

Uma gramática formal é uma 4-upla $G = (V, \Sigma, R, S)$ onde:

- V é um conjunto finito chamado cujos elementos são chamados de *variáveis*;
- Σ é um conjunto disjunto de V , cujos elementos são chamados de *terminais*;
- R é um conjunto finito de *regras*, cada regra é do tipo
 - $(\Sigma \cup V)^* V (\Sigma \cup V)^* \rightarrow (\Sigma \cup V)^*$;
- $S \in V$, é chamada de *variável inicial*.

Perceba que as regras gramaticais transformam uma sequência de terminais e variáveis em outra, a única restrição é que deve haver pelo menos uma variável do lado esquerdo da regra. Neste texto, se a variável inicial não for explicitamente definida, S será a primeira variável do lado esquerdo da primeira regra de produção listada.

Definição: Derivação em um Passo

Seja $G = (V, \Sigma, R, S)$ uma gramática formal. Dizemos que a sequência y pode ser derivada em um passo a partir da sequência x , denotado por $x \Rightarrow y$, se existem $u, v, p, q \in (\Sigma \cup V)^*$ tais que $x = upv$, $y = uqv$ e a regra $p \rightarrow q$ pertence a R .

Definição: Derivação

Seja $G = (V, \Sigma, R, S)$ uma gramática formal e $x, y \in (\Sigma \cup V)^*$. Dizemos que a sequência y pode ser derivada da sequência x , denotado por $x \Rightarrow^* y$, se

- $x = y$, ou
- Existe inteiro $k \geq 2$ e uma sequência $s_1, s_2, \dots, s_k$ de cadeias em $(\Sigma \cup V)^*$ tal que
 - $s_1 = x$;
 - $s_k = y$; e
 - $s_1 \Rightarrow s_2 \Rightarrow \dots \Rightarrow s_k$

Perceba que a derivação, $\Rightarrow^*$, é o fecho transitivo da operação de derivação em um passo, $\Rightarrow$. Finalmente, temos a definição da linguagem de uma gramática.

Definição: Linguagem Gerada por uma Gramática

Seja $G = (V, \Sigma, R, S)$ uma gramática formal. A linguagem gerada por G , denotada por $\mathcal{L}(G)$, é o conjunto de todas as cadeias em Σ^* que podem ser derivadas a partir de S .
 $\mathcal{L}(G) = \{x \in \Sigma^* : S \Rightarrow^* x\}$.

Estas são definições gerais de gramáticas formais, mas precisamos definir as gramáticas regulares. Uma gramática regular é uma gramática formal com restrições sobre a forma das regras de produção.

Definição: Gramática Linear (LG)

Uma gramática formal é dita linear (LG - *Linear Grammar*) se tem uma única variável do lado esquerdo e no máximo uma variável do lado direito das regras de produção.

As gramáticas lineares também têm subclasses, dentre elas destacamos duas.

Definição: Gramática Linear à Direita

Uma gramática $G = (V, \Sigma, R, S)$ é linear à direita se suas regras de produção são do tipo

- $A \rightarrow x$, onde $A \in V$ e $x \in \Sigma^*$; ou
- $A \rightarrow xB$, onde $A, B \in V$ e $x \in \Sigma^*$.

Definição: Gramática Linear à Esquerda

Uma gramática $G = (V, \Sigma, R, S)$ é linear à esquerda se suas regras de produção são do tipo

- $A \rightarrow x$, onde $A \in V$ e $x \in \Sigma^*$; ou
- $A \rightarrow Bx$, onde $A, B \in V$ e $x \in \Sigma^*$.

E a partir daí temos a definição das gramáticas regulares (RG - *Regular Grammar*).

Definição: Gramática Regular (RG)

Uma gramática $G = (V, \Sigma, R, S)$ é regular se é linear à direita ou linear à esquerda.

Exemplo 1

Considere a gramática regular $G = (\{S\}, \{0, 1\}, R, S)$, com as seguintes regras

$S \rightarrow 01S$

$S \rightarrow 0$

Alguns exemplos de derivação nessa linguagem são:

$S \Rightarrow 0$

$S \Rightarrow 01S \Rightarrow 010$

$S \Rightarrow 01S \Rightarrow 0101S \Rightarrow 01010$

A linguagem gerada por esta gramática é $(01)^*0$, perceba que a primeira regra insere 01 e repete a variável S a segunda regra encerra a derivação substituindo a variável pelo terminal 0. Então podemos ter 01 zero ou mais vezes seguido de 0.

Exemplo 2

Considere a gramática regular $G = (\{S, A, B\}, \{0, 1\}, R, S)$, com as seguintes regras

$S \rightarrow A01$

$A \rightarrow A01|B$

$B \rightarrow 0$

Alguns exemplos de derivação nessa linguagem são:

$S \Rightarrow A01 \Rightarrow B01 \Rightarrow 001$

$S \Rightarrow A01 \Rightarrow A0101 \Rightarrow B0101 \Rightarrow 00101$

$S \Rightarrow A01 \Rightarrow A0101 \Rightarrow A0101 \Rightarrow A010101 \Rightarrow B010101 \Rightarrow 0010101$

A linguagem gerada por esta gramática é $0(01)^*01$. Perceba que a primeira regra insere 01 precedido pela variável A , então toda cadeia terá pelo menos um 01. A variável A pode repetir o processo inserindo várias cópias da cadeia 01 ou encerrar a derivação através de B . Como B aparece sempre na primeira posição, deve haver um 0 no início.

Esta linguagem poderia ser gerada por outras regras, como por exemplo:

$S \rightarrow A01$

$A \rightarrow A01|0$

RG vs. NFA

Assim como nas expressões regulares, existe uma relação estreita entre as gramáticas regulares e as linguagens regulares. O objetivo desta seção é mostrar isso.

Teorema

Uma linguagem é regular se, e somente se, é gerada por uma gramática regular.

Prova

A prova deste teorema é dividida em duas partes, que serão realizadas ao longo desta seção. Primeiro vamos mostrar que toda linguagem gerada por uma RG é regular e depois vamos mostrar que toda linguagem regular pode ser gerada por uma RG.

Vamos iniciar o nosso processo mostrando que, para linguagens regulares à direita, podemos construir um NFA equivalente à gramática.

Teorema

Se uma linguagem é gerada por uma gramática regular à direita, então ela é regular.

Prova

Seja $G = (V, \Sigma, R, S)$ uma gramática regular. Vamos construir um GNFA $N = (Q, \Gamma, \delta, q_0, q_F)$, tal que

- $Q = V \cup \{q_0, q_F\}$, cada variável é um estado do autômato além de um estado de aceitação e um inicial;
- $\Gamma = \Sigma$, o alfabeto do autômato é o conjunto de terminais da gramática, uma vez que a linguagem da gramática é definida sobre Σ^* ;
- $\delta(q_A, q_B) = x$ para toda regra do tipo $A \rightarrow xB$, $\delta(q_A, q_F) = x$ para toda regra do tipo $A \rightarrow x$ e $\delta(q_0, q_S) = \varepsilon$, essa regra simplesmente aponta o novo estado inicial para o estado que representa a variável inicial da gramática.

Perceba que o processamento do autômato é análogo ao da gramática. A regra $A \rightarrow xB$ nos diz que a variável A , escreve x e vai para a variável B , o autômato está no estado que representa a variável A , lê x da entrada e vai para o estado que representa a variável B .

Como a gramática é linear, a qualquer momento da derivação existe apenas uma variável na cadeia, e por isso a analogia funciona. Além disso, como só há uma variável, regras do tipo $A \rightarrow x$ encerram a derivação, que no caso do autômato é representado por uma transição para o estado de aceitação. A linguagem reconhecida por esse GNFA é idêntica à linguagem gerada pela gramática. Como já vimos que um GNFA pode ser descrito por uma expressão regular e as expressões regulares definem linguagens regulares, a linguagem gerada por uma gramática regular à direita é regular.

Perceba que essa prova determina um algoritmo para a construção de um NFA a partir de uma gramática regular. A segunda parte da prova consiste em fazer a volta.

Teorema

Se uma linguagem é regular, ela é gerada por uma gramática regular à direita.

Prova

Similar ao caso anterior, vamos partir de um NFA e descrever uma forma de construir uma gramática regular à direita equivalente. Como a linguagem é regular, existe um NFA $N = (Q, \Gamma, \delta, q_0, F)$ que reconhece a linguagem. Vamos construir uma gramática

$G = (V, \Sigma, R, S)$ tal que

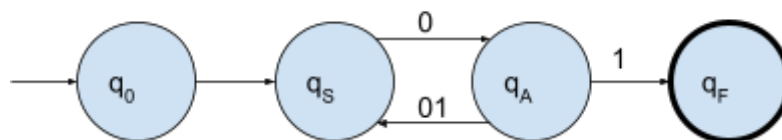
- $V = Q$, cada estado do autômato será uma variável na gramática;
- $S = q_0$, o processamento da gramática inicia onde o autômato inicia;
- $\Sigma = \Gamma$, os alfabetos são os mesmos, para definir a mesma linguagem;
- Para cada transição $\delta(q_i, x) = K$, onde $x \in \Sigma_\epsilon$ e K é um conjunto de estados, crie uma regra $q_i \rightarrow xq_k$, para cada $q_k \in K$. Para cada estado $q_{F_i} \in F$ adicione uma regra $q_{F_i} \rightarrow \epsilon$.

Se o autômato lê x e vai de q_i para q_j , a gramática escreve x e substitui a variável q_i pela variável q_j . Quando um estado de aceitação é alcançado, o autômato aceita, então a gramática deve ser capaz de parar de escrever nesses estados.

Os dois teoremas anteriores tratam apenas das gramáticas regulares à direita, mas o teorema inicial não fazia restrições sobre a gramática ser à direita ou à esquerda. Para uma gramática regular à esquerda que gera a linguagem L , se revertermos o lado direito de cada uma das regras, tal que as regras do tipo $A \rightarrow Bx$ viram $A \rightarrow x^R B$ e as do tipo $A \rightarrow x$ tornam-se $A \rightarrow x^R$, obtemos uma linguagem regular à direita que gera L^R . Daí concluímos que L^R é regular. Como a reversão é uma operação regular e $(L^R)^R = L$, L é uma linguagem regular.

Exemplo 1

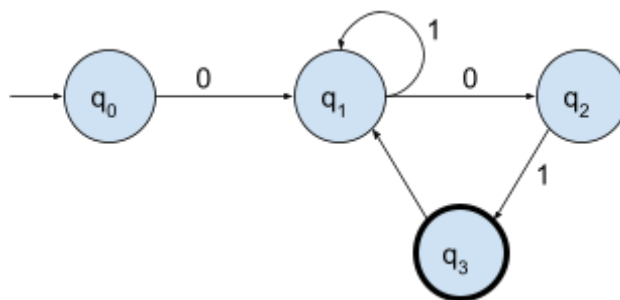
Dada a gramática $G = (\{S, A\}, \{0, 1\}, R, S)$ com regras $S \rightarrow 0A$ e $A \rightarrow 01S|1$, construa um autômato equivalente.



Que é equivalente à expressão regular $(001)^* 01$.

Exemplo 2

Dado o seguinte NFA, converta-o em uma gramática regular



A gramática equivalente é $G = (\{Q_0, Q_1, Q_2, Q_3\}, \{0, 1\}, R, Q_0)$ com as regras

$Q_0 \rightarrow 0Q_1, Q_1 \rightarrow 1Q_1, Q_1 \rightarrow 0Q_2, Q_2 \rightarrow 1Q_3, Q_3 \rightarrow Q_1, Q_3 \rightarrow \varepsilon$

Que é equivalente à expressão regular $01^*01(1^*01)^*$.

Lema do Bombeamento

Até agora já vimos que as linguagens regulares podem ser definidas de várias formas e que têm a propriedade de serem fechadas sob as operações regulares. Além destas, existe uma outra propriedade que é compartilhada por todas as linguagens regulares. Antes de enunciar formalmente qual é essa propriedade, vamos tentar intuir a idéia expressa por ela.

A quantidade de memória necessária para determinar se uma cadeia está na linguagem ou não, ou seja, se aceita ou não, é finita independentemente do comprimento da cadeia. Se uma linguagem é formada por uma quantidade infinita de cadeias e a memória é finita, deve haver uma estrutura repetitiva que forma essas cadeias (lembre da operação estrela nas expressões regulares).

Intuitivamente, se uma linguagem não segue esses padrões ela não deve ser regular. A linguagem $\{0^n1^n : n \geq 0\}$, por exemplo, não deve ser regular, parece improvável que um autômato consiga se lembrar de quantos 0 ele viu quando chegar a hora de contar os 1. Isso porque o valor de n é indefinido, se o n fosse fixo seria possível criar tal autômato. Além disso, chamo a atenção de que apesar de $\{0^n : n \geq 0\}$ ser equivalente a 0^* , a linguagem proposta não é equivalente a 0^*1^* , esta última pode ser lida como qualquer quantidade de símbolos 0 seguida de qualquer quantidade de símbolos 1, enquanto $\{0^n1^n : n \geq 0\}$ é uma quantidade de símbolos 0 seguida da mesma quantidade de 1.

A linguagem $\{1^n : n \text{ é um número primo}\}$ também não deve ser regular, já que os números primos não têm, até onde se sabe, uma estrutura repetitiva que possa ser aproveitada pelo autômato para determinar quem deve ou não ser aceito.

Agora vamos definir formalmente essa propriedade conhecida como *bombeamento*.

Teorema: Lema do Bombeamento

Se A é uma linguagem regular infinita, então existe um inteiro $p \geq 1$, chamado de comprimento de bombeamento, tal que toda cadeia s de A com $|s| \geq p$ pode ser escrita como $s = xyz$, onde

- $|y| > 0$,
- $|xy| \leq p$, e
- $xy^i z \in A$, para todo $i \geq 0$.

Prova

Seja Σ o alfabeto de A . Como A é regular, existe um DFA $D = (Q, \Sigma, \delta, q_0, F)$ que reconhece A . Considere $p = |Q|$. Seja $s = s_1 s_2 \dots s_n$ uma cadeia de A com $n \geq p$. Defina $r_1 = q_0$, $r_2 = \delta(r_1, s_1)$, $r_3 = \delta(r_2, s_2)$, ..., $r_{n+1} = \delta(r_n, s_n)$. Então quando o autômato lê a cadeia s , ele passa pela sequência de estados $r_1 r_2 \dots r_{n+1}$. Como s é uma cadeia de A , sabemos que $r_{n+1} \in F$.

Considere os primeiros $p + 1$ estados $r_1 r_2 \dots r_{p+1}$. Como o número de estados de D é p , pelo princípio da casa de pombos temos que pelo menos um estado ocorre duas vezes nesta sequência. Isto é, existem índices a e b tais que $1 \leq a < b \leq p + 1$ e $r_a = r_b$.

Defina $x = s_1 s_2 \dots s_{a-1}$, $y = s_a s_{a+1} \dots s_{b-1}$ e $z = s_b s_{b+1} \dots s_n$. Como $a < b$, temos $|y| \geq 1$, e está provada a primeira afirmação. Como $b \leq p + 1$, temos $|xy| = b - 1 \leq p$, provando a segunda afirmação. Para provar a terceira afirmação, lembre que $s = xyz$ é aceita por D . Quando lê x , o autômato move-se de $r_1 = q_0$ até r_a . Quando lê y , o autômato move-se de r_a até $r_b = r_a$, isto é, depois de ler y o autômato continua no mesmo estado. Quando lê z , o autômato vai de r_b a r_{n+1} . Portanto, a subcadeia y pode ser repetida qualquer quantidade $i \geq 0$ de vezes e a cadeia correspondente $xy^i z$ continua sendo aceita por D . Daí segue que $xy^i z \in A$ para todo $i \geq 0$.

É muito importante prestar atenção que essa propriedade é uma consequência da linguagem ser regular, ou seja é uma condição necessária para regularidade mas não é suficiente. Isto quer dizer que se uma linguagem não tem a propriedade de bombeamento ela não é regular, mas se ela tem não podemos dizer nada sobre ela. Outro ponto importante é que ele só vale para linguagens infinitas, já que o bombeamento diz que uma quantidade infinita de cadeias podem ser geradas pelo bombeamento, além de toda linguagem finita ser regular.

Vamos agora utilizar a propriedade de bombeamento das linguagens regulares para verificar algumas linguagens.

Exemplo 1: $A = \{0^n 1^n : n \geq 0\}$

Considere a linguagem que citamos anteriormente e que parecia não ser regular. Vamos agora provar por contradição que essa linguagem de fato não é regular.

Vamos assumir que A é regular. Seja $p \geq 1$ o comprimento de bombeamento. Considere a cadeia $s = 0^p 1^p$. Claramente $s \in A$ e $|s| = 2p \geq p$. Então, pelo lema do bombeamento, s pode ser escrito como $s = xyz$, onde $y \neq \varepsilon$, $|xy| \leq p$ e $xy^i z \in A$ para todo $i \geq 0$.

Observe que, como $|xy| \leq p$, y é formada apenas por símbolos 0, já que os primeiros p símbolos de s são 0. Qualquer cadeia $xy^i z$ com $i \neq 1$ não pertence a A pois as quantidades de 0 e 1 serão distintas. O que é uma contradição com o lema do bombeamento e concluímos que A não é uma linguagem regular.

Exemplo 2: $A = \{x \in \{0, 1\}^* : \text{as quantidades de 0 e 1 em } x \text{ são iguais}\}$

Essa linguagem não é idêntica à do exemplo anterior porque não há exigência de que os 0 estejam no início e os 1 no final. Vamos assumir que essa linguagem é regular e verificar se atende ao lema do bombeamento.

Novamente, perceba que a cadeia $0^p 1^p$ faz parte desta linguagem, e pela mesma argumentação usada no exemplo anterior podemos concluir que esta linguagem também não é regular.

Exemplo 3: $A = \{xx : x \in \{0, 1\}^*\}$

Essa linguagem é formada por cadeias que podem ser divididas ao meio de tal modo que a primeira metade é igual à segunda. Vamos assumir que A é regular e que p é o comprimento de bombeamento. Considere a cadeia $s = 0^p 10^p 1$. Então $s \in A$ e $|s| = 2p + 2 \geq p$.

Mais uma vez, a subcadeia y só pode ser formada por símbolos 0. Qualquer cadeia $xy^i z$ com $i \neq 1$ não pertence a A pois a quantidade de 0 na primeira parte será distinta da quantidade na segunda parte.

Perceba que a escolha da cadeia é fundamental, a cadeia $s = 0^{2p}$ faz parte da linguagem e pode ser bombeada, basta que a subcadeia y tenha comprimento par. O teorema diz que a propriedade vale para toda cadeia de comprimento maior ou igual a p , então basta que uma dessas cadeia não satisfaça a propriedade para que a linguagem não seja regular.

Exemplo 4: $A = \{0^m 1^n : m > n \geq 0\}$

As cadeias desta linguagem são formadas por uma cadeia de 0 seguida de uma cadeia de 1 e a quantidade 0 é maior que a quantidade de 1. Considere a cadeia $s = 0^{p+1} 1^p$. Então $s \in A$ e $|s| = 2p + 1 \geq p$.

Como a subcadeia y vai ser formada apenas por símbolos 0. A quantidade de 1 em xy^0z é p e a quantidade de 0 é no máximo p , daí, como $m \leq n$, xy^0z não pertence à linguagem. Logo, temos uma contradição e a linguagem não é regular.

Exemplo 5: $A = \{1^{n^2} : n \geq 0\}$

Nesta linguagem as cadeias são formadas apenas por símbolos 1 e têm comprimento que é um quadrado perfeito (1, 4, 9, 16, 25, ...). Considere a cadeia $s = 1^{p^2}$. Então $s \in A$ e $|s| = p^2 \geq p$.

Observe que $|s| = |xyz| = p^2$ e $|xy^2z| = |xyyz| = |xyz| + |y| = p^2 + |y|$. Como $|xy| \leq p$, temos que $|y| \leq p$. E sabemos que $|y| \geq 1$. Daí segue que $p^2 < |xy^2z| \leq p^2 + p < (p+1)^2$. Então o comprimento de xy^2z está entre dois quadrados consecutivos e concluímos que $xy^2z \notin A$. Daí temos uma contradição e a linguagem não é regular.

Exemplo 6: $A = \{1^n : n \text{ é um número primo}\}$

Argumentamos no início desta seção que esta linguagem não parece ser regular. Vamos tentar provar isso agora. As cadeias desta linguagem são formadas apenas por símbolos 1 e têm comprimento que é um número primo. Considere a cadeia $s = 1^k$, onde $k \geq p$ é um número primo.

Temos que

$$|xy^i z| = |x| + |z| + i|y| = |x| + |y| + |z| + (i-1)|y| = k + (i-1)|y|. \quad \text{Daí, para}$$

$i = k+1$ temos $|xy^{k+1} z| = k + (k+1-1)|y| = k + k|y| = k(1+|y|)$. Como $|y| \geq 1$, temos que $(1+|y|) \geq 2$, portanto o comprimento de $xy^{k+1} z$ não é um número primo, já que é expresso como o produto de dois inteiros maiores que 1. Daí temos uma contradição e a linguagem não é regular.

Exemplo 7: $A = \{x \in \{0,1\}^* : x \text{ é a representação binária de um número primo}\}$

Vamos assumir que o *bit* mais à esquerda é sempre 1, ou seja, não vamos representar os zeros à esquerda, então $A = \{10, 11, 101, 111, 1011, 1101, 10001, \dots\}$. Assuma que $N > 2^p$ é um número primo e seja s a sua representação binária. Observe que $|s| \geq p+1$ e sabemos que o primeiro e o último símbolo são 1, o primeiro por definição e o último porque os números binários que terminam com 0 são pares e somente 10 é um primo par.

Como $|s| \geq p+1 > p$ podemos escrever $s = xyz$ atendendo às propriedades do lema do bombeamento. Como $|xy| \leq p$ e $|s| \geq p+1$, temos $|z| \geq 1$. Defina A , B e C como sendo inteiros cujas representações binárias são x , y e z , respectivamente. Nesse caso, como são subcadeias de s , a imposição de que não há zeros à esquerda não é garantida,

de fato y pode ser formada somente por zeros, e nesse caso $B = 0$. Como o último símbolo é 1, temos que $C \geq 1$. Observe que $N = A \cdot 2^{|z|+|y|} + B \cdot 2^{|z|} + C$.

Considere que M é o número cuja representação binária é $xy^N z$. Daí, podemos escrever

$$M = A \cdot 2^{|z|+N|y|} + \sum_{k=0}^{N-1} B \cdot 2^{|z|+k|y|} + C = A \cdot 2^{|z|+N|y|} + B \cdot 2^{|z|} \sum_{k=0}^{N-1} 2^{k|y|} + C.$$

Vamos chamar o somatório de T . Então $(2^{|y|} - 1)T = 2^{N|y|} - 1$ (lembre da soma de uma PG).

Pelo teorema de Fermat, temos que $2^N \equiv 2 \pmod{N}$, implicando que $2^{N|y|} - 1 = (2^N)^{|y|} - 1 \equiv 2^{|y|} - 1 \pmod{N}$. E daí concluímos que $(2^{|y|} - 1)T \equiv 2^{|y|} - 1 \pmod{N}$. Observe que $2^{|y|} \leq 2^p < N$, porque $|y| \leq |xy| \leq p$. E também $2^{|y|} \geq 2$ porque $|y| \geq 1$. Daí segue que $1 \leq 2^{|y|} - 1 < N$, implicando que $2^{|y|} - 1$ não é um múltiplo de N . Disso tudo temos que $T \equiv 1 \pmod{N}$.

Como $M = A \cdot 2^{|z|+N|y|} + B \cdot 2^{|z|} \cdot T + C$, temos $M \equiv A \cdot 2^{|z|+|y|} + B \cdot 2^{|z|} + C \pmod{N}$. Isso, junto com a expressão de N , nos diz que $M \equiv 0 \pmod{N}$, ou seja, N divide M e daí concluímos que M não é primo, o que é uma contradição e a linguagem não é regular.

Esse exemplo foi feito para mostrar que nem sempre a utilização do lema do bombeamento é tão direta quanto nos exemplos anteriores.

Exemplo 8: $A = \{x \in \{0, 1\}^* : \text{a quantidade de 01 em } x \text{ é igual à quantidade de 10}\}$

Perceba que essa linguagem parece muito com a linguagem do exemplo 2, mas ao contrário daquela, vamos provar que essa linguagem é regular.

Não vamos utilizar o lema do bombeamento porque ele não serve para provar que linguagens são regulares, só servem para provar que não são. Ao invés disso vamos construir um DFA que reconhece a linguagem.

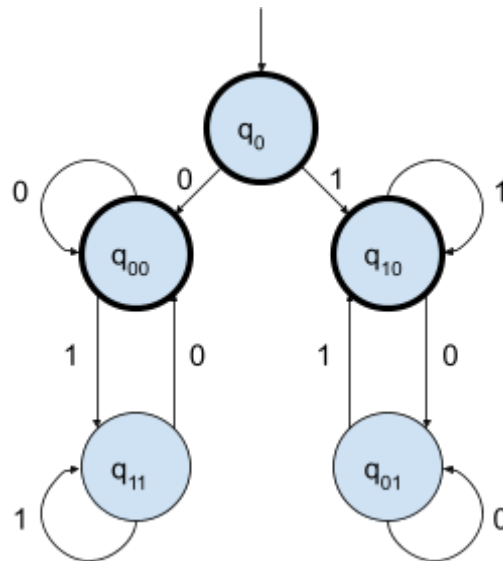
A grande diferença entre esta linguagem e aquela do exemplo 2 é que no exemplo anterior era necessário uma quantidade indefinida de memória para lembrar quantos 0 foram lidos. No caso da linguagem neste exemplo as sequência 01 e 10 sempre ocorrem de forma intercalada, ou seja, entre dois 01 consecutivos deve haver um 10 e entre dois 10 consecutivos deve haver um 01. Como há a alternância, a diferença entre as quantidade de 01 e 10 é no máximo 1.

Nosso DFA terá cinco estados que representam:

- q_0 é o estado inicial, nenhuma entrada foi lida e deve ser de aceitação, pois as quantidades de 01 e 10 são iguais (a zero);
- q_{00} indica que o último símbolo lido foi zero e a diferença entre as quantidades de 10 e 01 é 0, portanto deve ser um estado de aceitação;
- q_{11} indica que o último símbolo lido foi um e a diferença entre as quantidades de 10 e 01 é 1;
- q_{10} indica que o último símbolo lido foi um e a diferença entre as quantidades de 10 e 01 é 0, portanto deve ser um estado de aceitação;

- q_{01} indica que o último símbolo lido foi zero e a diferença entre as quantidades de 10 e 01 é 1.

A seguir segue o diagrama de estados do DFA que aceita essa linguagem



Este autômato é mesmo que foi utilizado para reconhecer a linguagem formada por cadeias que começam e terminam com o mesmo símbolo. Pense um pouco e você vai perceber que, por causa da propriedade discutida anteriormente da alternância entre 01 e 10, essas duas linguagens são idênticas.

Agora que já conhecemos algumas linguagens não-regulares, também podemos usar as operações regulares para provas de não-regularidade. Considere novamente a linguagem $A = \{x \in \{0, 1\}^* : \text{as quantidades de 0 e 1 em } x \text{ são iguais}\}$, a interseção $A \cap 0^*1^* = \{0^n1^n : n \geq 0\}$, sabemos que 0^*1^* é regular e, do exemplo 1, $\{0^n1^n : n \geq 0\}$ não é regular. Como a interseção é uma operação regular, A não pode ser uma linguagem regular, pois nesse caso a interseção também seria uma linguagem regular.

No entanto, nem sempre a operação entre uma linguagem não-regular e uma regular vai gerar uma linguagem não regular. Considere nos seguintes exemplos que L é uma linguagem não-regular sobre o alfabeto Σ . $L|\emptyset = L$ que não é regular, mas $L|\Sigma^* = \Sigma^*$ que é regular. $L \cap \Sigma^* = L$ que não é regular, mas $L \cap \emptyset = \emptyset$ que é regular. $L\{\varepsilon\} = L$ que não é regular, mas $L\emptyset = \emptyset$ que é regular.

Teorema de Myhill-Nerode

Na seção anterior vimos uma propriedade das linguagens regulares que é uma condição necessária para a regularidade. Nesta seção vamos ver uma condição necessária e suficiente para a regularidade. Além de servir para a demonstração de regularidade, também podemos utilizar uma consequência desta propriedade para obter autômatos com uma quantidade mínima de estados.

Antes de prosseguir com a definição da propriedade vamos estudar duas relações de equivalências úteis. A primeira relação de equivalência é definida sobre qualquer linguagem. Seja $A \subseteq \Sigma^*$ uma linguagem, dizemos que x e y são indistinguíveis em A , denotado por $x \equiv_A y$, se, e somente se, para toda cadeia $z \in \Sigma^*$ ou xz e yz estão ambos em A ou ambos não estão em A . Considere, por exemplo, a linguagem descrita por $01^*|00$. As cadeias 01 e 011 são indistinguíveis nessa linguagem porque $01z$ só pertence a A se $z \in 1^*$, do mesmo modo $011z$ só pertence a A e $z \in 1^*$. Então $01z$ e $011z$ estão ambos em A ou ambos fora de A .

A relação $\equiv_A$ é uma relação de equivalência, então particionar a linguagem A em classes de equivalência. As classes de equivalência são subconjuntos onde todos os elementos são equivalentes segundo a relação. Para a linguagem descrita por $01^*|00$, definida sobre o alfabeto $\Sigma = \{0, 1\}$, generalizando o exemplo anterior, as cadeias 01^k , $k > 0$, são indistinguíveis entre si. Note que 0 não é indistinguível de 01 , já que 00 está em A e 010 não está. Por um raciocínio semelhante, 0 é distinguível de 00 . ε é distinguível de 0 pois $\varepsilon 1 \notin A$ mas $01 \in A$. Por fim, todas as demais cadeias que não estão em A são indistinguíveis entre si, pois qualquer uma delas estão fora de A independentemente do que seja acrescido. Então existem cinco classes de equivalência em A , que são $\{\varepsilon\}$, $\{0\}$, $\{00\}$, $\{01^k, k > 0\}$, $\Sigma^* \setminus A$.

Outra relação de equivalência, desta vez definida sobre um DFA, é que duas cadeias x e y , são indistinguíveis com relação a um DFA $D = (Q, \Sigma, \delta, q_0, F)$, denotado por $x \equiv_D y$, se $\bar{\delta}(q_0, x) = \bar{\delta}(q_0, y)$. Ou seja, se o autômato termina no mesmo estado quando parte do estado inicial e lê x ou y (já utilizamos essa classe de equivalência, antes de defini-la, na conversão algébrica de um autômato em uma expressão regular). Como a quantidade de estados em um DFA é finita, a quantidade de classes de equivalências definidas pela relação $\equiv_D$ também é finita.

Teorema: Teorema de Myhill-Nerode

Uma linguagem A é regular se, e somente se $\equiv_A$ tem uma quantidade finita de classes de equivalência. Além disso, existe um DFA D que reconhece A que tem precisamente um estado para cada classe de equivalência.

Prova

Vamos dividir a prova desse teorema, os dois teoremas seguintes são as provas de cada uma das partes desse teorema.

Primeiro vamos mostrar que se uma linguagem é regular, $\equiv_A$ tem uma quantidade finita de classes de equivalência. Qual a importância disso? Esse é um método alternativo para mostrar que uma linguagem não é regular.

Teorema

Se A é uma linguagem regular, então $\equiv_A$ tem uma quantidade finita de classes de equivalências.

Prova

Se A é uma linguagem regular, existe um DFA $D = (Q, \Sigma, \delta, q_0, F)$ tal que $\mathcal{L}(D) = A$. Então, para uma cadeia ω , temos que $\omega \in A$ se, e somente se, $\bar{\delta}(q_0, \omega) \in F$. Suponha que $x \equiv_D y$, então $\bar{\delta}(q_0, x) = \bar{\delta}(q_0, y)$.

Seja $z \in \Sigma^*$. Temos então que $\bar{\delta}(q_0, xz) = \bar{\delta}(q_0, yz)$, já que tanto x quanto y levam ao mesmo estado, e a partir daí z levará ao mesmo destino. Sabemos que $xz \in A$ se, e somente se, $\bar{\delta}(q_0, xz) \in F$ e, similarmente, $yz \in A$ se, e somente se, $\bar{\delta}(q_0, yz) \in F$. Daí concluímos que $xz \in A$ se, e somente se, $yz \in A$, ou seja, $x \equiv_A y$.

Isto quer dizer que, sempre que duas cadeias levam ao mesmo estado elas estão na mesma classe de equivalência de $\equiv_A$. Ou seja, a quantidade de classes de equivalência de $\equiv_A$ é no máximo a quantidade de classes de equivalência de $\equiv_D$. Como já sabemos que a quantidade de classes de equivalência de $\equiv_D$ é finita, concluímos que a quantidade de classes de equivalência de $\equiv_A$ também é.

Note que o número de classes de equivalências de $\equiv_A$ não é necessariamente igual ao número de classes de equivalência de $\equiv_D$. É possível que duas ou mais classes de equivalência de $\equiv_D$ correspondam a uma única classe de $\equiv_A$. A seguir veremos um exemplo de como usar este teorema para provar que uma linguagem não é regular.

Exemplo

Considere a linguagem $A = \{0^n 1^n : n \geq 0\}$. Assuma que A é regular, então, pelo teorema anterior, ela tem uma quantidade finita de classes de equivalência. Considere as cadeias 0^a e 0^b com $a \neq b$, perceba que $0^a 1^a \in A$ mas $0^b 1^a \notin A$, então elas não são indistinguíveis. Como existem infinitas cadeias do tipo 0^k , $k > 0$ e cada uma delas está numa classe de equivalência distinta, a quantidade de classes de equivalência desta linguagem é infinito, o que é uma contradição. Logo, A não é regular.

Já tínhamos o lema do bombeamento, que também era uma condição necessária para a regularidade. A grande diferença deste método é que ele é suficiente, ou seja, se uma linguagem tem uma quantidade finita de classes de equivalência ela é regular. Vamos fazer a prova da segunda parte do teorema.

Teorema

Se $\equiv_A$ tem uma quantidade finita de classes de equivalência, então A é regular.

Prova

Considere que $A_1, A_2, \dots, A_n$ são as classes de equivalência de $\equiv_A$. Sabemos que os A_i são disjuntos entre si e $\bigcup_{i=1}^n A_i = \Sigma^*$. Defina um DFA $D = (Q, \Sigma, \delta, q_0, F)$ com um estado, $q_i, 1 \leq i \leq n$, para cada classe de equivalência. Vamos definir δ, q_0 e F tais que $\bar{\delta}(q_0, x) = q_i$ se, e somente se, $x \in A_i$.

Faça $q_0 = q_i$ tal que $\varepsilon \in A_i$. Ou seja, vamos partir da classe de equivalência que contém a cadeia vazia.

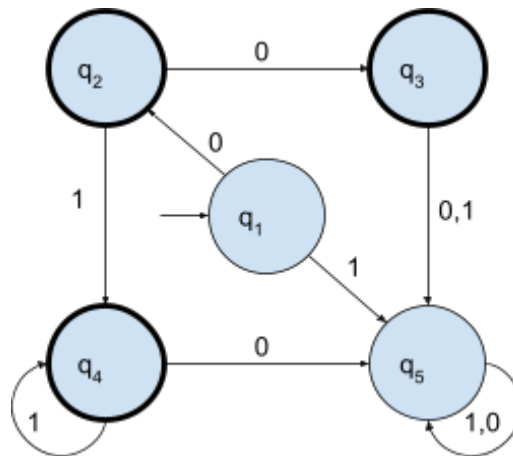
Note que para um A_j e qualquer $a \in \Sigma$, para todo $x, y \in A_j$, xa e ya estão ambos na mesma classe de equivalência de $\equiv_A$. Então, para determinar $\delta(q_j, a)$, tome algum $x \in A_j$ e, se $xa \in A_k$, faça $\delta(q_j, a) = q_k$.

Os estados finais são simplesmente $F = \{q_j : A_j \cap A \neq \emptyset\}$. Ou seja, todos os estados com exceção daquele que representa a classe de equivalência das cadeias que não estão em A .

O autômato construído através do procedimento descrito na prova anterior tem o menor número de estados possível. Sabemos que as classes de equivalência de $\equiv_A$ são formadas por uniões das classes de equivalência de $\equiv_D$. Então o menor número possível de classes de equivalência de $\equiv_D$ é o número de classes de equivalência de $\equiv_A$. Como cada classe de equivalência de $\equiv_D$ corresponde a um estado, o menor número de estados de um DFA que reconhece A é igual ao número de classes de equivalência de $\equiv_A$.

Exemplo

Já vimos que a linguagem descrita por $01^*|00$ tem cinco classes de equivalência: $A_1 = \{\varepsilon\}$, $A_2 = \{0\}$, $A_3 = \{00\}$, $A_4 = \{01^k, k > 0\}$, $A_5 = \Sigma^* \setminus A$. Vamos usar o método descrito na prova anterior para construir um DFA que reconhece essa linguagem. Teremos $Q = \{q_1, q_2, q_3, q_4, q_5\}$, com cada estado correspondendo a uma classe de equivalência. Como $\varepsilon \in A_1$, o estado inicial será q_1 . Os estados q_1 e q_5 não serão de aceitação, pois representam cadeias fora da linguagem.



Esse é o menor autômato possível para esta linguagem, já que cada estado corresponde a uma classe de equivalência de $\equiv_A$.

Mas e se não tivermos uma boa descrição da linguagem? Precisamos de um método para identificar as classes de equivalência de $\equiv_A$ a partir de um autômato que reconhece a linguagem A .

Uma abordagem possível seria olhar os pares de estados e verificar se eles são equivalentes ou não sob $\equiv_A$. Mas vamos tentar uma abordagem distinta, definindo uma relação de equivalência entre estados. Para cada $i \geq 0$, defina a relação de equivalência $\equiv_i$ sobre o conjunto de estados Q tal que $p \equiv_i q$ se, e somente se, para toda cadeia z de comprimento menor ou igual a i , $\bar{\delta}(p, z) \in F$ se e somente se $\bar{\delta}(q, z) \in F$.

A idéia básica é que se essa relação vale para todo i , então as entradas que alcançam p e q devem estar no mesmo estado do autômato mínimo. Vamos formalizar isso. Seja $A = L(D)$ a linguagem do autômato $D = (Q, \Sigma, \delta, q_0, F)$. Sejam $p, q \in Q$ e suponha que todo estado de D é alcançável a partir de q_0 (se não for pode removê-los). Então $p \equiv_i q$ para todo $i \geq 0$ se, e somente se, as classes de equivalência de $\equiv_D$ correspondentes a p e q estão na mesma classe de $\equiv_A$.

Teorema

As classes de equivalência de $\equiv_\infty$ são as mesmas de $\equiv_A$.

Prova

Primeiramente, se $p \equiv_i q$ para todo $i \geq 0$, então para qualquer $z \in \Sigma^*$, se $\bar{\delta}(q_0, x) = q$ e $\bar{\delta}(q_0, y) = p$, então sabemos que $\bar{\delta}(q_0, xz) \in F$ se, e somente se,

$\bar{\delta}(q_0, yz) \in F$. Isso significa que $xz \in A$ se, e somente se, $yz \in A$. Portanto duas cadeias quaisquer que alcançam os estados p e q estão na mesma classe de equivalência de $\equiv_A$.

Suponha agora que p e q não são equivalentes sob $\equiv_i$ para algum i . Então existe algum z de comprimento menor ou igual a i tal que apenas um dentre $\bar{\delta}(p, z)$ e $\bar{\delta}(q, z)$ está em F . Como p e q são alcançáveis a partir de q_0 , existem cadeias x e y tais que $\bar{\delta}(q_0, x) = p$ e $\bar{\delta}(q_0, y) = q$. Portanto apenas um dentre $\bar{\delta}(q_0, xz)$ e $\bar{\delta}(q_0, yz)$ está em F . O que significa que apenas um dentre xz e yz está em A . Concluimos então que x e y estão em classes de equivalência distintas de $\equiv_A$.

Minimização de DFAs

O teorema anterior vale para $\equiv_\infty$, mas não é muito prático verificar todos os $\equiv_i$. Vamos mostrar agora que basta verificar para uma quantidade finita de relações para chegar às classes de equivalência de $\equiv_A$, para ser mais exato $|Q| - 1$ destas relações.

Teorema

As classes de equivalência de $\equiv_{|Q|-1}$ são as mesmas de $\equiv_\infty$.

Prova

Considere um DFA $M = (Q, \Sigma, \delta, q_0, F)$. Primeiramente, vamos definir quais estados são equivalentes sob $\equiv_0$. p e q são equivalentes sob $\equiv_0$ se, e somente se, para $z = \varepsilon$, $\bar{\delta}(p, z) \in F$ se e somente se $\bar{\delta}(q, z)$. Isto quer dizer que p e q são equivalentes sob $\equiv_0$ se ambos estão em F ou ambos estão em $Q \setminus F$.

Agora vamos determinar as classes de equivalência de $\equiv_{i+1}$ dadas as classes de $\equiv_i$. Primeiro, perceba que os estados que são equivalentes sob $\equiv_{i+1}$ também são equivalentes sob $\equiv_i$ já que eles devem ter o mesmo comportamento para toda cadeia z de comprimento menor ou igual a i . Para verificar que elas também concordam para z' de comprimento $i + 1$, basta verificar se depois de ler o primeiro símbolo de z' , chegam em estados que são equivalentes sob $\equiv_i$. Em outras palavras, $p \equiv_{i+1} q$ se, e somente se, $p \equiv_i q$ e para todo $a \in \Sigma$, $\delta(p, a) \equiv_i \delta(q, a)$.

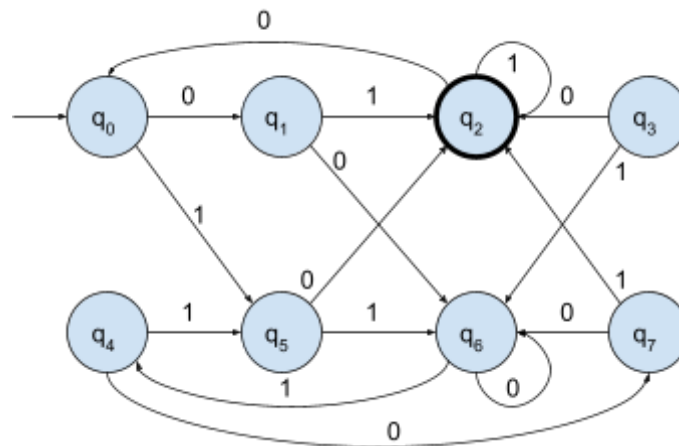
Note que se as classes de equivalência de $\equiv_i$ são as mesmas de $\equiv_{i+1}$, então as classes de equivalência de $\equiv_{i+2}$ também serão as mesmas. Ou seja, quando atinge-se um ponto onde não há alteração nas classes de equivalência, ela vale para todos os índices superiores.

A cada iteração a quantidade de classes de equivalência aumenta ou permanece igual. Se permanece igual, determinamos as classes de $\equiv_\infty$, se aumenta o limite superior de classes é $|Q|$, então teremos no máximo $|Q| - 1$ passos e daí, no pior caso as classes de equivalência de $\equiv_\infty$ são as mesmas de $\equiv_{|Q|-1}$.

A prova do teorema anterior determina um procedimento para encontrar as classes de equivalência de $\equiv_A$ dado um DFA. O procedimento para determinar um DFA mínimo é começar com as classes de equivalência de $\equiv_0$ e construir as de $\equiv_{i+1}$ a partir daquelas de $\equiv_i$, até que não haja alteração nas classes. Depois disso, todos os estados de uma mesma classes de equivalência podem ser transformados em um único estado.

Exemplo

Minimize o seguinte DFA



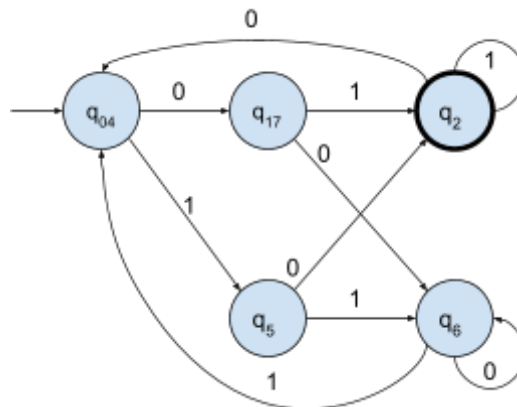
Antes de iniciar o procedimento, já podemos remover o estado q_3 pois ele não é alcançável, então vamos desconsiderá-lo. As classes de equivalência sob $\equiv_0$ são $\{q_0, q_1, q_4, q_5, q_6, q_7\}$ e $\{q_2\}$.

Agora devemos verificar se os estados das classe de equivalência de $\equiv_0$ continuam chegam na mesma classe após as transições. Quando o símbolo lido é 0 temos as seguintes transições: $q_0 \rightarrow q_1$, $q_1 \rightarrow q_6$, $q_4 \rightarrow q_7$, $q_5 \rightarrow q_2$, $q_6 \rightarrow q_6$ e $q_7 \rightarrow q_6$. Daí temos que q_5 é distinguível dos demais quanto à leitura do símbolo 0 porque ele aponta para uma classe de equivalência distinta. Quanto à leitura do símbolo 1, q_1 e q_7 são distinguíveis dos demais. Então as classes de equivalência sob $\equiv_1$ são: $\{q_0, q_4, q_6\}$, $\{q_1, q_7\}$, $\{q_5\}$ e $\{q_2\}$.

As classes que têm somente um estado não precisam mais ser avaliadas, pois não podem ser subdivididas. Vamos avaliar primeiro o par q_1 e q_7 , ambos transitam para um elemento de $\{q_0, q_4, q_6\}$ com entrada 0 e para q_2 com entrada 1, portanto são indistinguíveis. As transições de q_0 e q_4 transitam para q_6 com entrada 1 e para um elemento da classe

$\{q_1, q_7\}$ com entrada 0. Portanto, q_0 e q_4 permanecem indistinguíveis. Já q_6 é distinguível dos demais. Então as classes de equivalência sob $\equiv_2$ são: $\{q_6\}, \{q_0, q_4\}, \{q_1, q_7\}, \{q_5\}$ e $\{q_2\}$.

Pela mesma argumentação do parágrafo anterior, é fácil verificar que as classes de equivalência de $\equiv_3$ são as mesmas de $\equiv_2$. Portanto, chegamos no número mínimo de classes de equivalência. Abaixo está o autômato simplificado. Os estados q_{04} e q_{17} representam as classes de equivalência $\{q_0, q_4\}$ e $\{q_1, q_7\}$, respectivamente. q_3 foi removido porque não é alcançável.



O DFA mínimo é único, portanto uma forma eficiente de determinar equivalência entre DFAs é através da sua minimização.

Aplicações

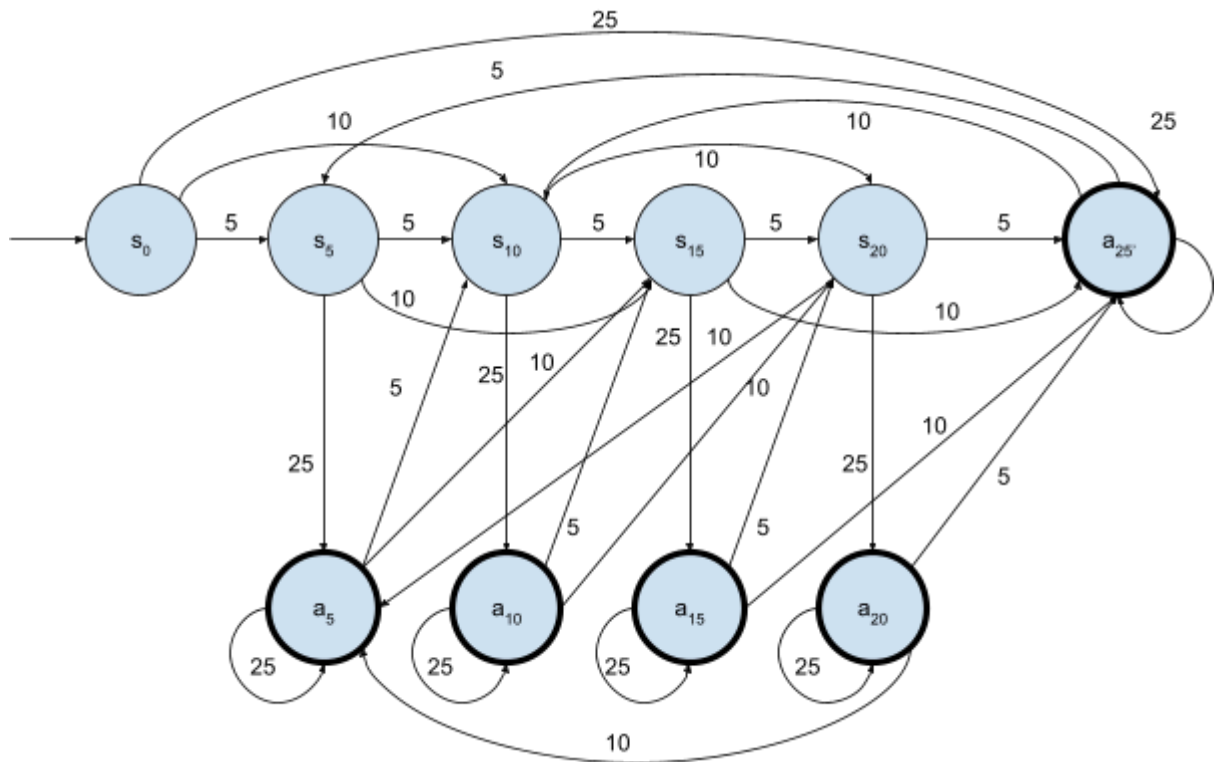
Em todo o texto consideramos as linguagens regulares de um ponto de vista puramente teórico, como uma representação de problemas de decisão e os autômatos como modelos extremamente simples de computador. No entanto, existem aplicações das linguagens regulares no mundo real. Decidi agrupar as aplicações em três classes, que são apresentadas nas subseções seguintes.

Modelagem

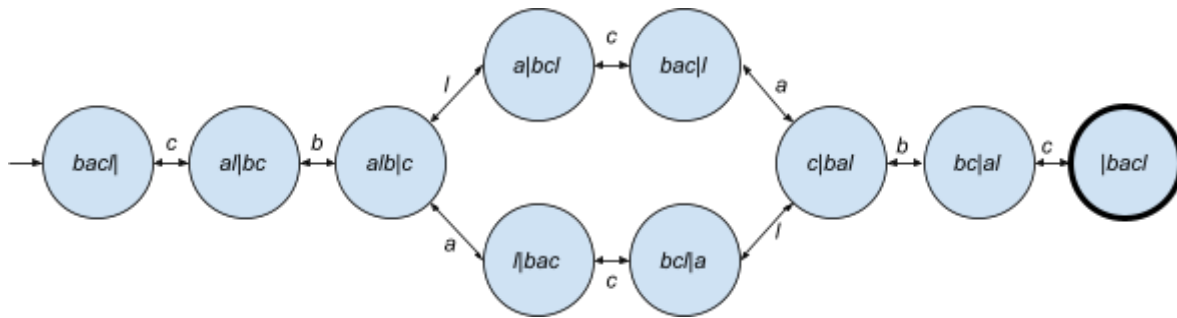
A primeira aplicação das linguagens regulares, dos autômatos finitos especialmente, é na modelagem de problemas em que o comportamento do agente que resolve o problema pode ser interpretado como uma transição entre uma quantidade finita de estados. Circuitos lógicos sequenciais são um exemplo clássico disso.

O primeiro exemplo, a máquina de pirulitos no início do texto, é um caso onde um problema “real” foi modelado como um autômato finito. Apesar de termos visto somente autômatos finitos decisores, que respondem com *aceita* ou *rejeita*. Também é possível associar uma ação à transição de estados, ou aos próprios estados. Considere o problema da máquina, mas agora a máquina guarda o saldo para a próxima compra, ou seja, se o cliente insere 3 moedas de 10 centavos a máquina dá um pirulito e fica com 5 centavos de saldo. O DFA abaixo representa o comportamento dessa máquina, e podemos atribuir o comportamento “liberar um pirulito” à entrada em um estado de aceitação. Então se entramos com 3 moedas de 10 e uma de 25 o autômato que parte de s_0 passa pelos

estados s_{10} , s_{20} , a_5 , a_{25} . Como entre duas vezes num estado de aceitação, dois pirulitos são liberados e o próximo cliente vai começar do estado a_5 , ou seja, com 5 centavos de crédito.



Isto também pode ser usado na modelagem de agentes simples que respondem deterministicamente, como personagens em jogos eletrônicos e robôs simples. Considere o clássico problema lógico onde um barqueiro precisa atravessar um lobo, um cordeiro e um alface de uma margem para outra do rio. O problema determina que o barqueiro só pode levar um elemento de cada vez e que, em nenhum momento o lobo pode ficar com o cordeiro ou o cordeiro com o alface sem a supervisão do barqueiro. Vamos criar um autômato que representa esse problema. Vamos assumir que todos os elementos estão na margem esquerda e o objetivo é levá-los à margem direita. O barqueiro pode atravessar o barco sozinho ou com um dos elementos, vamos usar a letra b para indicar a travessia do barqueiro, l para levar o lobo, c para levar o cordeiro e a para levar o alface. Vamos representar a situação atual através dos estados. $bc|al$, por exemplo, indica que o barqueiro e o cordeiro estão na margem esquerda enquanto o alface e o lobo estão na margem direita. Como temos quatro elementos que podem estar em duas situações (esquerda ou direita), temos um total de 16 estados possíveis, mas 6 deles não são permitidos pela regra do jogo ($acl|b$, $b|acl$, $ac|bl$, $bl|ac$, $cl|ba$, $ba|cl$). O NFA abaixo representa todas as situações possíveis, as transições são bidirecionais, para evitar desenhar duas transições e as transições não definidas são aquelas que violam as regras do jogo.



A partir daí podemos definir todas as sequências de transições que resolvem o problema, ou seja, levam da margem esquerda (estado inicial) para a margem direita (estado de aceitação). As soluções desse problema pode ser descrita por uma expressão regular grande demais para escrever aqui (ocupa pouco mais de duas páginas). Mas se estamos interessados apenas nas soluções sem repetição de estados, podemos ignorar todas as partes da expressão com estrela e ficamos apenas com $cb(lca|acl)bc = cblcabc|cbac|bc$, ou seja, duas soluções. A primeira solução pode ser lida como: transporta o cordeiro (c), atravessa sozinho (b), transporta o lobo (l), transporta o cordeiro (c), transporta o alface (a), atravessa sozinho (b) e transporta o cordeiro (c). As duas soluções são igualmente eficientes, em número de passos, pois as duas cadeias têm o mesmo comprimento.

Como um terceiro exemplo, imagine que você tem um problema que consiste em navegar através de um mapa discreto, ou seja, dividido em células cujas transições permitidas, em geral, são entre células adjacentes, como na maior parte dos jogos eletrônico. Se você modelar o mapa como um autômato, você pode encontrar todos os caminhos entre dois pontos marcando a origem como estado inicial e o destino como estado de aceitação. A expressão regular que descreve a linguagem desse autômato indica todas as sequências de movimento que levam do ponto de partida ao destino. Se trajetos com laços, ou seja, repetição de células, não te interessam, você pode remover todas as partes sob a operação estrela, como fizemos na solução do exemplo anterior.

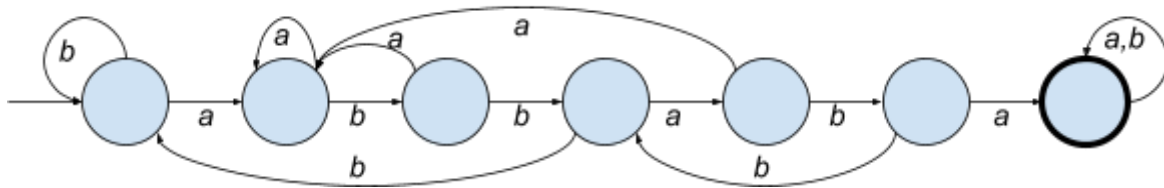
Outra aplicação interessante dos autômatos em modelagem de sistemas é quando substituímos as transições determinísticas por transições probabilísticas. Cada transição que sai de um estado tem uma probabilidade de ocorrer e pelo menos uma deve ocorrer. Este autômato é conhecido como Cadeia de Markov e tem aplicações nas mais diversas áreas da ciência, indo da física às ciências sociais. Para citar uma aplicação específica, o PageRank do Google, usado para determinar a “importância” das páginas, é definido como uma cadeia de Markov.

Casamento de Padrões

Considere o problema de determinar se a *string* $\alpha = abbaba$ está contida na *string* $\beta = abbabbabbabbaba$. Temos $|\alpha| = 6$ e $|\beta| = 15$. Uma solução simples seria comparar a cadeia α com cada subcadeia de comprimento 6 de β , iniciando do primeiro símbolo, do segundo, e assim por diante, até o décimo. De modo geral, teríamos que realizar o teste de igualdade $|\beta| - |\alpha| + 1$ vezes, no pior caso, onde α não ocorre em β ou ocorre somente no fim.

Pensando em termos de autômatos, podemos criar um autômato que aceita α e testar com as $|\beta| - |\alpha| + 1$ subcadeias de $|\beta|$. Como se estivéssemos lendo a entrada do usuário diversas vezes. Ora, já sabemos que é possível criar um autômato que reconhece

qualquer cadeia que contenha α e obter a resposta lendo uma única vez a entrada completa do usuário. Para o exemplo dado, o autômato dado abaixo aceita cadeias que contém *abbaba*, assumindo que o alfabeto é $\{a, b\}$.



Esta é a idéia implícita dos algoritmos de Knuth-Morris-Pratt para busca de *strings*. Na prática, como citamos na parte de modelagem, memórias ou funções auxiliares podem ser associadas para determinar onde ou quantas vezes ocorreu a cadeia procurada. Neste último caso precisaríamos de um autômato um pouco diferente, poderíamos remover o laço do estado de aceitação e fazer o processamento parar quando o mesmo fosse atingido. Daí, se uma nova ocorrência for desejada, volta ao estado inicial e refaz a verificação a partir de onde tinha parado anteriormente.

Perceba que o comportamento descrito anteriormente é justamente o que costuma-se observar nas ferramentas de busca em editores de texto. Uma ocorrência é detectada e então podemos partir para a próxima. Então a busca de subcadeias é uma aplicação óbvia das linguagens regulares.

Similarmente, se desejamos encontrar uma cadeia que tem certas propriedades, podemos descrever as propriedades através de expressões regulares, construir um autômato e processar a cadeia. Por exemplo, suponha que em uma entrada do usuário desejamos que ele digite um CPF, sabemos que o CPF é composto por 11 dígitos, com ponto após o terceiro e sexto e um hífen após o nono. Seja $D = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$. A seguinte expressão regular representa uma entrada de CPF sintaticamente válida, ou seja, que tem o formato adequado, $DDD . DDD . DDD - DD$.

As expressões regulares como uma forma de descrever padrões desejados em um texto estão disponíveis em diversas ferramentas de busca e linguagens de programação. A forma de escrever as expressões é um pouco diferente do que usamos neste texto, permitindo escrever expressões complexas de forma mais simples. Por exemplo a expressão regular $a|aa|aaa|aaaa|aaaaa$ descreve cadeias formadas pelo símbolo a de comprimento maior ou igual a 1 e menor ou igual a 5. Na maioria das expressões regulares usadas em programação isto poderia ser escrito simplesmente como $a\{1,5\}$ aqui o colchete não representa um conjunto, simplesmente uma notação para dizer que o símbolo a ocorre pelo menos 1 e no máximo 5 vezes.

Como anteriormente, muitas variações de um simples reconhecedor podem ser implementadas, permitindo a identificação de quantidade e/ou locais de ocorrências.

Diversas variantes desta idéia encontram aplicações em processamento de texto, como busca, realce de código e extração de informação em textos estruturados. Além disso, em antivírus esse procedimento pode ser usado para achar a “assinatura”, um certo padrão binário, de certos *Malwares*. Por fim, qualquer processamento de cadeias simbólicas pode aproveitar essas idéias, como a identificação de sequências de ácidos nucleicos ou proteínas na análise genética.

Análise Léxica

Este tópico poderia ser encaixado na seção anterior, mas decidi separá-lo por ser fundamental na construção de compiladores, que vocês verão numa disciplina futura.

Considere que alguém te dá um texto numa língua desconhecida e um dicionário dessa língua. Uma forma de proceder para tentar entender o texto é identificar as palavras e procurá-las uma a uma no dicionário. Perceba que o primeiro passo para fazer isso é ser capaz de identificar as palavras no texto. Você provavelmente vai ignorar a pontuação e os números quando fizer isso, porque você tem uma idéia de que as palavras são formadas por letras e separadas por espaços.

Este processo de identificar as palavras ou, como chamaremos daqui em diante, *tokens* da língua é a análise léxica. Quando um compilador recebe um programa escrito em uma certa linguagem de programação essa é a primeira etapa. Para determinar quais os *tokens* válidos, quase sempre uma linguagem regular é usada.

Uma ferramenta recebe a descrição dos *tokens*, através de expressão regular ou gramática regular, e cria um NFA que reconhece a linguagem formada por *tokens* válidos. Ao invés de simplesmente dizer sim ou não, um programa é associado aos estados de aceitação para efetuar alguma ação.

Aqui está uma descrição recursiva dos números em uma linguagem de programação hipotética.

Digito = 0|1|2|3|4|5|6|7|8|9

Sinal = + | - | ϵ

Inteiro = *Digito* ◦ *Digito**

InteiroComSinal = *Sinal* ◦ *Inteiro*

Decimal = *InteiroComSinal* ◦ ◦ *Inteiro* | *Sinal* ◦ ◦ *Inteiro* | *InteiroComSinal*

Expoente = *Decimal* | *Decimal* ◦ *E* ◦ *InteiroComSinal*

Real = *Decimal* | *Expoente*

Nas expressões, ◦ indica concatenação. O ponto (.) é um símbolo, assim como a letra *E* na definição do *Exponencial*. Muitas outras definições são necessárias para determinar os *tokens* de uma linguagem de programação, como palavras reservadas, identificadores e operadores.

Para determinar o tipo de uma determinada entrada são definidas regras de prioridade e de comprimento. Por exemplo, ao analisar a linha de código `int inteiro=10;` ao ler as três primeiras letras, `int`, o analisador não sabe se é a palavra reservada `int` ou parte do nome de uma variável, como acontece nas três letras seguintes. Os limites entre *tokens* são determinados pelos espaços e caracteres especiais. Pela regra de prioridade, `int` é identificado como uma palavra-chave na primeira ocorrência e a regra do comprimento mais longo determina que a segunda ocorrência de `int` faz parte do nome de uma variável pois é possível criar um *token* mais longo com os símbolos que seguem. Quando lê o símbolo = determina-se o *token* `inteiro` como um identificador de variável.

Apesar de ser capaz de identificar os *tokens*, as linguagens regulares não são capazes de efetuar outras tarefas relativamente simples, como determinar se todo parêntese aberto foi fechado. Para ser mais exato, não é sequer capaz de determinar se o número de parênteses abertos é igual ao de parênteses fechados, lembre que a linguagem $\{x \in \{0, 1\}^* : \text{as quantidades de 0 e 1 em } x \text{ são iguais}\}$ não é regular.

Considerações Finais

Em nossa exploração dos problemas de decisão através das linguagens formais, verificamos que existe um subconjunto dos problemas que são representados pelas linguagens regulares.

Vimos diversos formalismos equivalentes para descrever ou reconhecer esta linguagem, além de demonstrar como converter um formalismo em outro. As expressões e gramáticas regulares são um jeito fácil de dar uma descrição da linguagem enquanto os autômatos dão soluções para os problemas.

Como já foi dito, um problema de decisão pode ser interpretado como um problema de pertinência a uma linguagem, e os autômatos são capazes de decidir se uma cadeia pertence ou não a uma dada linguagem regular.

Através da conversão de linguagens e gramáticas em autômatos podemos facilmente determinar se uma dada cadeia pode ser gerada ou não por um desses formalismos.

Pensando nos autômatos como modelos primordiais de computadores, percebemos que a classe das linguagens regulares representam problemas que podem ser resolvidos com uma quantidade finita de memória. Mais que isso, a memória necessária independe do comprimento da cadeia de entrada, depende apenas da linguagem reconhecida pelo autômato. Desse modo, linguagens infinitas são possíveis quando há um padrão de repetição relativamente simples que pode ser realizado com memória constante.

Como já discutimos, a linguagem 0^n1^n , com $n \geq 0$, é um exemplo clássico de linguagem não regular. Apesar de ser simples para humanos, para reconhecer esta linguagem utilizando uma máquina seria necessário uma quantidade de memória proporcional a n , um contador, mas esta quantidade não é conhecida anteriormente ao processamento e por isso não é possível determinar um autômato que reconheça tal linguagem.

Isto posto, resta a pergunta, existe alguma máquina capaz de reconhecer as linguagens não regulares? Isso é o que veremos no restante do texto.

Linguagens Livres de Contexto

Tendo visto diversas formas equivalentes de caracterizar as linguagens formais, chegou a hora de partirmos para uma nova classe de linguagens. Desta vez iniciaremos com um formalismo gerador, uma gramática, que deixará evidente a origem do nome desta classe. Vamos começar com uma definição.

Definição: Linguagem Livre de Contexto

Uma linguagem A é dita *livre de contexto* se existe uma gramática livre de contexto G tal que $\mathcal{L}(G) = A$.

Já sabemos que $\mathcal{L}(G)$ denota a linguagem gerada por uma gramática, falta saber o que é uma gramática livre de contexto, que veremos agora.

Gramáticas Livre de Contexto (CFG)

Na seção sobre as Gramáticas Regulares já definimos o que é uma gramática formal e como as cadeias são geradas a partir das regras gramaticais. Além disso, foi definido que as linguagens regulares são gramáticas formais com restrições sobre a forma das suas regras de produção. A definição das linguagens regulares impõe restrições tanto ao lado esquerdo quando ao lado direito das regras de produção.

De forma similar, as Gramáticas Livres de Contexto (CFG - *Context Free Grammars*) também são definidas a partir de restrições no formato de suas regras de produção.

Definição: Gramática Livre de Contexto (CFG)

Uma gramática $G = (V, \Sigma, R, S)$ é livre de contexto se as suas regras de produção são do tipo

- $A \rightarrow x$, onde $A \in V$ e $x \in (\Sigma \cup V)^*$.

Essa definição nos diz que as CFG têm restrições apenas no lado esquerdo das regras de produção, e essa restrição é a mesma das Gramáticas Regulares. Podemos entender as CFG como uma versão menos restritiva das RG. Uma consequência direta desta definição é dada a seguir.

Teorema

As linguagens regulares são um subconjunto das linguagens livres de contexto.

Prova

A definição das CFG deixa claro que todas as Gramáticas Regulares são também CFG. Como as RG definem as linguagens regulares, toda linguagem regular pode ser expressa por uma CFG. Já que as linguagens livres de contexto são definidas pelas CFG, concluímos que toda linguagem regular também é livre de contexto.

Para determinar que as linguagens regulares são um subconjunto próprio das linguagens regulares, ou seja, que não são o mesmo conjunto, basta criar uma CFG que gere uma linguagem não regular. A seguir veremos alguns exemplos de CFG.

Antes que eu me esqueça, por que estas gramáticas são chamadas de livres de contexto? Lembre que na definição das gramáticas formais as regras poderiam ser do tipo $\alpha A \omega \rightarrow \alpha x \omega$, com $\alpha, \omega, x \in (\Sigma \cup V)^*$ e $A \in V$. Neste caso, a variável A só poderia ser substituída por x se estivesse precedida por α e sucedida por ω , ou seja, dentro deste contexto. No caso das CFG, a substituição das variáveis é feita independente do contexto.

Exemplos de CFG

Como no caso das linguagens regulares, projetar uma gramática que gera uma dada linguagem é um trabalho que envolve criatividade, tentativa e erro. Lembre que definir uma linguagem é equivalente a definir a solução para um problema de decisão. As restrições impostas pelo formalismo estudados nesta disciplina tornam o processo de projeto um pouco mais difícil no início pois a maioria de nós não está acostumada a pensar de acordo com estas regras. Tentar “pensar” como um autômato ou como uma gramática, ou seja, impor a você mesmo as restrições que estes modelos têm, pode ser um bom caminho para achar uma solução.

Nestes exemplos utilizaremos uma visualização bastante útil do processo de derivação chamado de árvore sintática (*parse tree*), que têm esse nome pois ajudam a visualizar a estrutura sintática da cadeia derivada.

Para simplificar a representação, vamos descrever as CFG apenas pelas regras de produção. Para determinar o conjunto de variáveis, basta observar os símbolos do lado esquerdo das regras. Os terminais são todos os símbolos do lado direito que não são variáveis. A variável é aquela da primeira regra.

Exemplo 1

A gramática simplificada do português, apresentada anteriormente, é uma CFG.

- *Frase* $\rightarrow$ *Sujeito Predicado*
- *Sujeito* $\rightarrow$ *Artigo Substantivo*
- *Predicado* $\rightarrow$ *Verbo*
- *Artigo* $\rightarrow$ *o|um*
- *Substantivo* $\rightarrow$ *cachorro|gato*
- *Verbo* $\rightarrow$ *corre*
- *Verbo* $\rightarrow$ *anda*

Já vimos que a linguagem gerada por essa gramática é

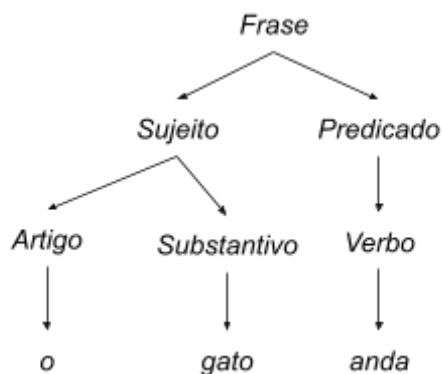
$\{o\ cachorro\ corre, o\ cachorro\ anda, o\ gato\ corre, o\ gato\ anda,$

um cachorro corre, um cachorro anda, um gato corre, um gato anda

Vamos agora fazer uma derivação da cadeia *o gato anda* e apresentar a árvore sintática associada:

Frase $\Rightarrow$ *Sujeito Predicado* $\Rightarrow$ *Artigo Substantivo Predicado* $\Rightarrow$ *Artigo Substantivo Verbo*
 $\Rightarrow$ *o Substantivo Verbo* $\Rightarrow$ *o gato Verbo* $\Rightarrow$ *o gato anda*.

Esta derivação pode ser representada da seguinte forma:



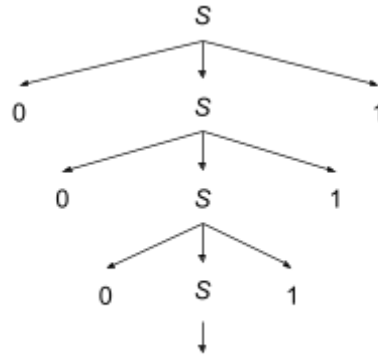
Perceba que a árvore sintática permite uma visualização clara da estrutura da frase. *o* é um artigo, *gato* um substantivo e *anda* um verbo. *o gato* é o sujeito e *anda* é o predicado. Na árvore sintática as cadeias nas folhas são formadas sobre Σ_ε e cada transição vem de alguma regra de produção.

Exemplo 2

Considere a linguagem não-regular $A = \{0^n 1^n : n \geq 0\}$. Vamos tentar projetar uma CFG G tal que $\mathcal{L}(G) = A$. Observe que as cadeias em A são formadas por um 0, seguido de uma cadeia qualquer de A , seguido por um 1. Além destas, a cadeia vazia também faz parte da linguagem. Com essa observação podemos fazer uma gramática bem simples, cuja única regra é $S \rightarrow \varepsilon | 0S1$.

Para derivar a cadeia $0^k 1^k$ a partir desta gramática, partindo de S , aplique a regra $S \rightarrow 0S1$ k vezes, depois aplique a regra $S \rightarrow \varepsilon$ uma vez para terminar a produção. Concluimos então que a linguagem A é livre de contexto.

A seguir damos um exemplo da derivação da cadeia 000111. A seta que não aponta para nada é a derivação da cadeia vazia.



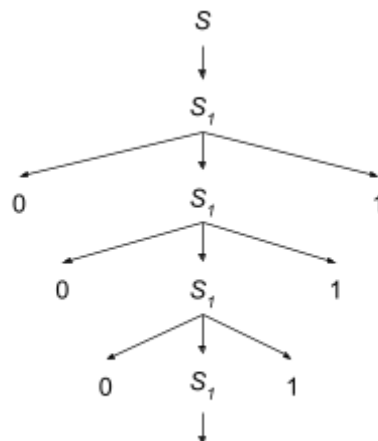
Exemplo 3

Crie uma CFG para a linguagem $A = \{0^n 1^n : n \geq 0\} \cup \{1^n 0^n : n \geq 0\}$. Já sabemos que a gramática $G_1 = S_1 \rightarrow \varepsilon | 0S_1 1$ gera a primeira parte da linguagem A . De forma análoga podemos fazer $G_2 = S_2 \rightarrow \varepsilon | 1S_2 0$ para gerar a segunda parte. A partir dessas duas gramáticas é simples criar uma gramática que gera a união das duas. Seja G uma CFG com as seguintes regras

- $S \rightarrow S_1 | S_2$
- $S_1 \rightarrow \varepsilon | 0S_1 1$
- $S_2 \rightarrow \varepsilon | 1S_2 0$

A linguagem de G é A , o novo estado inicial simplesmente cria uma opção de utilizar a gramática G_1 ou a gramática G_2 . De certa forma, esta construção é análoga à construção que fizemos com NFAs para demonstrar que as linguagens regulares são fechadas sob a operação de união.

A derivação da cadeia 000111 nesta gramática tem a seguinte árvore sintática.

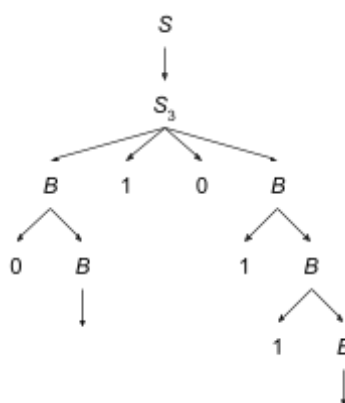


gera $\{0, 1\}^*$. A partir daí podemos criar todas as cadeias que contêm 10 através da regra $S_3 \rightarrow B10B$.

Por fim, a linguagem desejada é a união dessas três linguagens. Portanto a gramática que gera $\bar{A}$ é definida pelas regras

- $S \rightarrow S_1 | S_2 | S_3$
- $S_1 \rightarrow 1 | 0S_11 | S_11$
- $S_2 \rightarrow 0 | 0S_21 | 0S_2$
- $S_3 \rightarrow B10B$
- $B \rightarrow \varepsilon | 0B | 1B$

E a árvore sintática da cadeia 01011 nessa gramática é

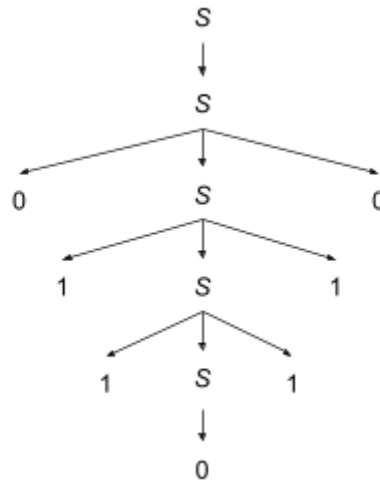


Exemplo 6

Considere a linguagem $A = \{w(0|1|\varepsilon)w^R : w \in \{0, 1\}^*\}$. Esta linguagem representa os palíndromos sobre o alfabeto $\Sigma = \{0, 1\}$. A gramática que gera esta linguagem tem as seguintes regras de produção:

- $S \rightarrow 0S0 | 1S1 | 0|1|\varepsilon$

As regras recursivas $S \rightarrow 0S0$ e $S \rightarrow 1S1$ inserem o mesmo elemento no fim da primeira metade e no início da segunda metade da cadeia gerada. A árvore sintática da cadeia 0110110 é



Exemplo 7

Considere a linguagem $A = \{a^m b^n c^{m+n} : m, n \in \mathbb{N}\}$. Esta linguagem representa o problema de verificar se uma soma está correta, a quantidade de a é o primeiro operando, a quantidade de b é o segundo operando e a quantidade de c é o total. Então verificar se $aabbcccc$ faz parte da linguagem é equivalente a verificar se $2 + 2 = 4$.

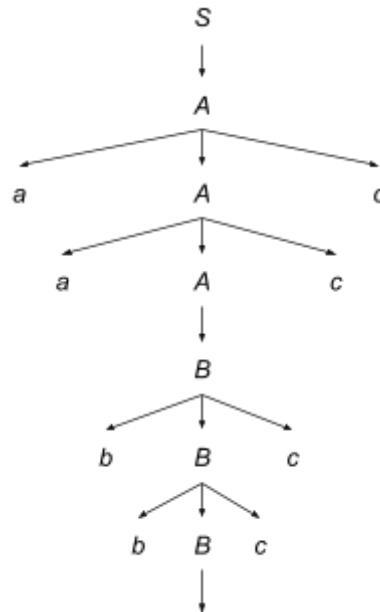
Primeiro, perceba que ε faz parte da linguagem, então a regra $S \rightarrow \varepsilon$ será uma das regras da gramática. Para as demais cadeias, basta criá-las sequencialmente, primeiro os a e depois os b . Cada a ou b inserido também deve inserir um c no final da cadeia, desse modo a quantidade de c será igual à soma de a e b .

A partir daí, a gramática desejada tem as seguintes regras de produção:

- $S \rightarrow \varepsilon | A$
- $A \rightarrow \varepsilon | aAc | B$
- $B \rightarrow \varepsilon | bBc$

O fato de que $A \Rightarrow^* a^m B c^m$, para todo $m \in \mathbb{N}$ e $B \Rightarrow^* b^n c^n$, para todo $n \in \mathbb{N}$ implica que, a partir de S temos $S \Rightarrow^* a^m B c^m \Rightarrow^* a^m b^n c^n c^m \Rightarrow^* a^m b^n c^{m+n}$, para todo $m, n \in \mathbb{N}$.

A árvore sintática para a cadeia $aabbcc$ é dada por



Essa gramática pode ser simplificada para as regras:

- $A' \rightarrow aA'c|B'$
- $B' \rightarrow \varepsilon|bB'c$

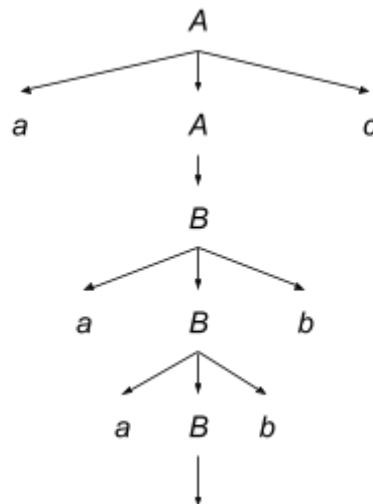
Exemplo 8

Vamos projetar uma CFG para a linguagem $A = \{a^m b^n c^{m-n} : m, n \in \mathbb{N} \text{ e } m \geq n\}$. Lembre que não precisamos executar a subtração de fato, basta encontrar um meio de escrever a cadeia com o formato desejado. Como $m \geq n$, para cada b existe um a . Então podemos dividir a cadeia como sendo $a^{m-n} a^n b^n c^{m-n}$, ou seja internamente temos a e b balanceados e externamente a e c balanceados, ou seja, para cada a em excesso temos um c .

Assim, podemos usar uma gramática similar à anterior com as regras:

- $A \rightarrow aAc|B$
- $B \rightarrow \varepsilon|aBb$

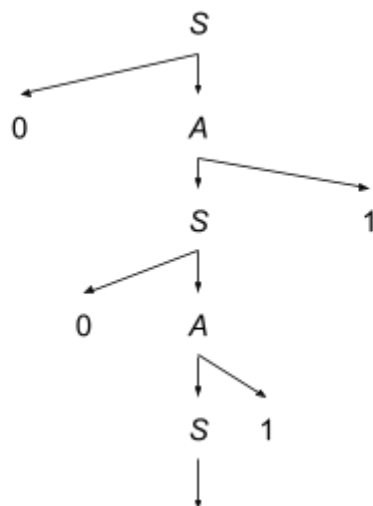
A derivação da cadeia $aaabbc$ é mostrada na árvore sintática que segue.

**Exemplo 9**

Considere a gramática com as seguintes regras:

- $S \rightarrow \varepsilon | 0A$
- $A \rightarrow S1$

Vamos verificar a produção da cadeia 0011 através da árvore sintática:



Percebe que apesar de parecer uma gramática regular, com regras de produção lineares, esta gramática mistura regras lineares à direita e à esquerda. Essa mistura permite a geração da linguagem $\{0^n 1^n : n \geq 0\}$, que não é regular. A gramática em questão insere um 0 à esquerda e depois obrigatoriamente insere um 1 à direita, só então a produção poder ser encerrada utilizando a regra $S \rightarrow \varepsilon$.

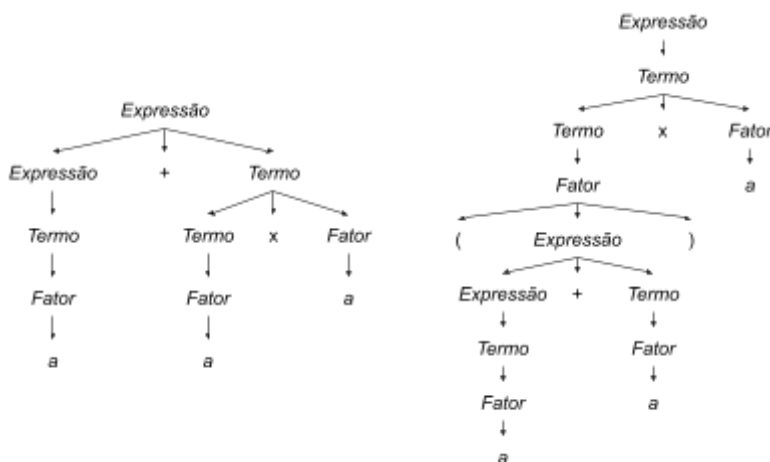
Exemplo 10

Considere a gramática com as seguintes regras:

- $Expressão \rightarrow Expressão + Termo \mid Termo$
- $Termo \rightarrow Termo \times Fator \mid Fator$
- $Fator \rightarrow (Expressão) \mid a$

A linguagem desta gramática são expressões aritméticas sintaticamente corretas envolvendo soma e multiplicação de a .

A seguir apresentamos as derivações de $a + a \times a$ e $(a + a) \times a$.



Perceba que a estrutura de precedência usual da avaliação de expressões aritméticas é capturada por esta gramática. Na árvore da esquerda é necessário calcular o produto antes de calcular a soma. Na árvore da direita é necessário calcular a soma, que está entre parênteses, para poder calcular o produto.

Ambiguidade

Algumas gramáticas podem gerar a mesma cadeia de diferentes formas, de modo que uma mesma cadeia pode ter árvores sintáticas distintas. Quando isso ocorre, dizemos que a gramática é ambígua.

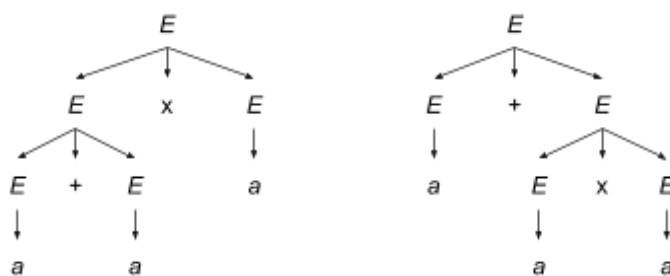
Perceba que não basta ter derivações distintas e sim árvores sintáticas distintas. Duas derivações podem ser distintas simplesmente pela ordem na qual escolhemos substituir as variáveis. Para focar na estrutura, vamos considerar derivações com uma ordem fixa de substituição de variáveis. A derivação de uma cadeia ω em uma gramática G é uma *derivação mais à esquerda* se a cada passo a variável mais à esquerda é substituída.

Definição: Gramática Ambígua

Uma cadeia ω é derivada de forma ambígua em uma gramática G se tem duas ou mais derivações mais à esquerda distintas. A gramática G é ambígua se gera alguma cadeia de forma ambígua.

Exemplo

Considere a CFG $E \rightarrow E + E \mid E \times E \mid (E) \mid a$. A linguagem desta gramática é a mesma da gramática do Exemplo 10. Mas vamos observar a árvore sintática da cadeia $a + a \times a$.



Agora temos duas árvores sintáticas distintas, uma em que o produto ocorre antes e outra em que a soma aparece antes. A gramática do Exemplo 10 foi projetada para capturar as precedências, eliminando a ambiguidade.

Infelizmente, não existe um método para tornar uma gramática não-ambígua. Pior ainda, algumas linguagens livres de contexto só podem ser geradas por gramáticas ambíguas. Tais linguagens são chamadas de *inerentemente ambíguas*.

Exemplo

Considere a linguagem $L = \{a^x b^y c^z : x = y \text{ ou } y = z\}$. Podemos pensar nessa linguagem como sendo a união de duas linguagens $L_1 = \{a^x b^y c^z : x = y\}$ e $L_2 = \{a^x b^y c^z : y = z\}$.

A linguagem L_1 têm cadeias com a e b balanceados e c livre. A gramática $S_1 \rightarrow AB$, $A \rightarrow aSb \mid \varepsilon$, $B \rightarrow cB \mid \varepsilon$ gera L_1 . Essa gramática faz o balanceamento e depois concatena quantos c forem desejados. Similarmente, L_2 é gerada pela gramática $S_2 \rightarrow CD$, $C \rightarrow aC \mid \varepsilon$, $D \rightarrow bDc \mid \varepsilon$, que insere uma quantidade arbitrária de a e depois faz o balanceamento de b e c .

As cadeias com $x = y = z$, a interseção de L_1 e L_2 , podem ser geradas por qualquer uma das gramáticas, sendo inerentemente ambíguas. Sempre que descrevemos uma linguagem pela união de linguagens não disjuntas a linguagem da interseção será gerada de forma ambígua.

E se tentássemos descrever L através de conjuntos disjuntos? A união de $L_3 = \{a^n b^n c^n : n \in \mathbb{N}\}$, $L_4 = \{a^n b^n c^m : m, n \in \mathbb{N} \text{ e } m \neq n\}$ e $L_5 = \{a^m b^n c^n : m, n \in \mathbb{N} \text{ e } m \neq n\}$, que são disjuntas, gera L . Mas, como veremos futuramente, não é possível criar CFGs para gerar estas linguagens.

Forma Normal de Chomsky

As regras em uma gramática livre de contexto, como definido na seção anterior, são do tipo $A \rightarrow x$, onde $A \in V$ e $x \in (\Sigma \cup V)^*$. Nesta seção vamos mostrar que uma CFG G pode ser transformada em uma CFG G' , tal que $\mathcal{L}(G) = \mathcal{L}(G')$ e as regras de G' são escritas em um formato restrito.

As regras de G' levam uma variável a duas outras ou uma variável a um terminal, além disso também é permitida a transição da variável inicial para a cadeia vazia, para permitir que a cadeia vazia seja aceita. Este formato especial é muito útil em algumas provas e algoritmos envolvendo CFGs.

Definição: Forma Normal de Chomsky (CNF)

Uma CFG $G = (V, \Sigma, R, S)$ é dita estar na *forma normal de Chomsky* (CNF - *Chomsky Normal Form*) se todas as regras em R estão em uma das seguintes formas:

- $A \rightarrow BC$, onde $A, B, C \in V$, $B \neq S$ e $C \neq S$;
- $A \rightarrow s$, onde $A \in V$ e $s \in \Sigma$;
- $S \rightarrow \varepsilon$.

Como dito anteriormente, uma CFG na forma normal de Chomsky tem a regra $S \rightarrow \varepsilon$ se, e somente se, $\varepsilon \in \mathcal{L}(G)$. Agora vamos formalizar e provar a afirmação desta seção.

Teorema

Seja Σ um alfabeto e $L \subseteq \Sigma^*$ uma linguagem livre de contexto. Existe uma CFG na forma normal de Chomsky que gera L .

Prova

Como L é uma linguagem livre de contexto, existe uma CFG $G = (V, \Sigma, R, S)$, tal que $\mathcal{L}(G) = L$. Vamos transformar G em uma gramática G' na CNF tal que $\mathcal{L}(G) = \mathcal{L}(G')$.

Passo 1: Eliminar a variável inicial do lado direito das regras. Seja $G_1 = (V_1, \Sigma, R_1, S_1)$, onde S_1 é uma nova variável inicial, $V_1 = V \cup \{S_1\}$ e $R_1 = R \cup \{S_1 \rightarrow S\}$. Esta nova gramática não tem S_1 do lado direito das regras, que é a nova variável inicial e $\mathcal{L}(G_1) = \mathcal{L}(G)$.

Passo 2: Uma regra ε é uma regra da forma $A \rightarrow \varepsilon$, onde A é uma variável que não é a variável inicial. Vamos eliminar todas as regras desse tipo. Vamos modificar a gramática G_1 , do passo anterior, da seguinte forma:

1. Remova a regra $A \rightarrow \varepsilon$ de R_1 .
2. Para cada regra da forma
 - a. $B \rightarrow A$, adicione a regra $B \rightarrow \varepsilon$ a R_1 , a menos que $B \rightarrow \varepsilon$ já tenha sido removida de R_1 . Isto transforma a derivação em dois passos $B \Rightarrow A \Rightarrow \varepsilon$ numa derivação de um passo $B \Rightarrow \varepsilon$;
 - b. $B \rightarrow vAw$, com $v, w \in (\Sigma \cup V_1)^*$ e $|vw| > 0$, adicione a regra $B \rightarrow vw$ a R_1 . Mais uma vez, uma derivação de dois passos $B \Rightarrow vAw \Rightarrow vw$ se transforma em uma derivação de um passo $B \Rightarrow vw$;
 - c. $B \rightarrow uAvAw$, onde $u, v, w \in (\Sigma \cup V_1)^*$, adicione as regras $B \rightarrow uAvw$, $B \rightarrow uvAw$ e $B \rightarrow uvw$ a R_1 . Se $u = v = w = \varepsilon$ e a regra $B \rightarrow \varepsilon$ já foi removida de R_1 , não adicione a regra $B \rightarrow uvw = B \rightarrow \varepsilon$. Nesse caso três regras são inseridas, uma para cada substituição de A por ε .
 - d. As regras nas quais A ocorre mais de duas vezes no lado direito são tratadas de forma similar ao caso anterior. Insira uma regra para cada uma das combinações de remoção de A , um a um, dois a dois, e assim por diante.

Esse processo é repetido até que todas as regras ε sejam removidas. Seja R_2 o conjunto de regras R_1 após o fim deste processo. A gramática $G_2 = (V_1, \Sigma, R_2, S_1)$ não tem S_1 do lado direito das regras, não tem regras ε (com exceção talvez de $S_1 \rightarrow \varepsilon$), e $\mathcal{L}(G_2) = \mathcal{L}(G_1) = \mathcal{L}(G)$, porque as substituições de regras feitas mantêm todas as derivações existentes anteriormente.

Passo 3: Uma regra unitária é do tipo $A \rightarrow B$, onde $A, B \in V_1$. Vamos eliminar as regras deste tipo. Vamos modificar a gramática G_2 , do passo anterior, da seguinte forma:

1. Remova a regra $A \rightarrow B$ de R_2 .
2. Para cada regra da forma $B \rightarrow x$, com $x \in (\Sigma \cup V_1)^*$, adicione a regra $A \rightarrow x$, a menos que seja uma regra unitária já removida. Mais uma vez, uma derivação de dois passos $A \Rightarrow B \Rightarrow x$ foi substituída por uma derivação em um passo $A \Rightarrow x$.

Esse processo é repetido até que todas as regras unitárias sejam removidas. Seja R_3 o conjunto de regras R_2 após o fim deste processo. A gramática $G_3 = (V_1, \Sigma, R_3, S_1)$ não tem S_1 do lado direito das regras, não tem regras ε (com exceção talvez de $S_1 \rightarrow \varepsilon$), não tem regras unitárias e $\mathcal{L}(G_3) = \mathcal{L}(G_2) = \mathcal{L}(G_1) = \mathcal{L}(G)$, porque as substituições de regras feitas mantêm todas as derivações existentes anteriormente.

Passo 4: Eliminar todas as regras que tem mais de dois símbolos do lado direito. Para cada regra em R_3 da forma $A \rightarrow x_1 x_2 \cdots x_k$, onde $x_i \in (V_3 \cup \Sigma)$:

1. Remova a regra $A \rightarrow x_1 x_2 \cdots x_k$.

2. Adicione as seguintes regras a R_3

- $A \rightarrow x_1 A_1$
- $A_1 \rightarrow x_2 A_2$
- $A_2 \rightarrow x_3 A_3$
- $\vdots$
- $A_{k-3} \rightarrow x_{k-2} A_{k-2}$
- $A_{k-2} \rightarrow x_{k-1} x_k$

Onde os A_i são variáveis novas. Perceba que nesse caso substituímos a derivação em um passo $A \Rightarrow x_1 x_2 \cdots x_k$ pela derivação em $(k - 1)$ passos $A \Rightarrow x_1 A_1 \Rightarrow x_1 x_2 A_2 \Rightarrow \cdots \Rightarrow x_1 x_2 \cdots x_{k-2} A_{k-2} \Rightarrow x_1 x_2 \cdots x_k$.

Seja R_4 o conjunto de regras R_3 após o fim deste processo e V_4 o conjunto V_1 acrescido das novas variáveis criadas neste passo. A gramática $G_4 = (V_4, \Sigma, R_4, S_1)$ não tem S_1 do lado direito das regras, não tem regras ε (com exceção talvez de $S_1 \rightarrow \varepsilon$), não tem regras unitárias, não tem nenhuma regra com mais de dois símbolos do lado direito e $\mathcal{L}(G_4) = \mathcal{L}(G_3) = \mathcal{L}(G_2) = \mathcal{L}(G_1) = \mathcal{L}(G)$, porque as substituições de regras feitas mantêm todas as derivações existentes anteriormente.

Passo 5: Eliminar todas as regras com dois símbolos do lado direito que não são ambos variáveis. Para toda regra de R_4 da forma $A \rightarrow x_1 x_2$ em que x_1 e x_2 não pertencem ambos a V_4 :

1. Remova a regra $A \rightarrow x_1 x_2$
2. Se
 - a. $x_1 \in \Sigma$ e $x_2 \in V_4$, adicione as regras $A \rightarrow X_1 x_2$ e $X_1 \rightarrow x_1$, onde X_1 é uma variável nova. Perceba que substituímos a derivação em um passo $A \Rightarrow x_1 x_2$ pela derivação em dois passos $A \Rightarrow X_1 x_2 \Rightarrow x_1 x_2$.
 - b. $x_1 \in V_4$ e $x_2 \in \Sigma$, adicione as regras $A \rightarrow x_1 X_2$ e $X_2 \rightarrow x_2$, onde X_2 é uma variável nova. Perceba que substituímos a derivação em um passo $A \Rightarrow x_1 x_2$ pela derivação em dois passos $A \Rightarrow x_1 X_2 \Rightarrow x_1 x_2$.
 - c. $x_1, x_2 \in \Sigma$ e $x_1 \neq x_2$, adicione as regras $A \rightarrow X_1 X_2$, $X_1 \rightarrow x_1$ e $X_2 \rightarrow x_2$, onde X_1 e X_2 são variáveis novas. Perceba que substituímos a derivação em um passo $A \Rightarrow x_1 x_2$ pela derivação em três passos $A \Rightarrow X_1 X_2 \Rightarrow x_1 X_2 \Rightarrow x_1 x_2$.
 - d. $x_1, x_2 \in \Sigma$ e $x_1 = x_2$, adicione as regras $A \rightarrow X_1 X_1$ e $X_1 \rightarrow x_1$, onde X_1 é uma variável nova. Perceba que substituímos a derivação em um passo $A \Rightarrow x_1 x_2$ pela derivação em três passos $A \Rightarrow X_1 X_1 \Rightarrow x_1 X_1 \Rightarrow x_1 x_1 = x_1 x_2$.

Seja R_5 o conjunto de regras R_4 após o fim deste processo e V_5 o conjunto V_4 acrescido das novas variáveis criadas neste passo. A gramática $G_5 = (V_5, \Sigma, R_5, S_1)$ não tem S_1 do lado direito das regras, não tem regras ε (com exceção talvez de $S_1 \rightarrow \varepsilon$), não

tem regras unitárias, não tem nenhuma regra com mais de dois símbolos do lado direito, não tem regras com dois símbolos do lado direito que não são ambos variáveis e $\mathcal{L}(G_5) = \mathcal{L}(G_4) = \mathcal{L}(G_3) = \mathcal{L}(G_2) = \mathcal{L}(G_1) = \mathcal{L}(G)$, porque as substituições de regras feitas mantêm todas as derivações existentes anteriormente.

Como G_5 está na forma normal de Chomsky e gera a mesma linguagem de G , a prova está completa.

Esta prova nos dá um procedimento para transformar qualquer CFG para a forma normal de Chomsky. Vamos ver dois exemplos de conversão.

Exemplo 1

Converte para a CNF a gramática $G = (\{A, B\}, \{0, 1\}, R, A)$ com regras R:

- $A \rightarrow BAB|B|\varepsilon$
- $B \rightarrow 00|\varepsilon$

Vamos aplicar o método da prova do teorema anterior passo a passo.

Passo 1: Eliminar a variável inicial no lado direito das regras. Para isso basta acrescentar uma nova variável inicial que gera a variável inicial antiga. Daí temos as regras

- $S \rightarrow A$
- $A \rightarrow BAB|B|\varepsilon$
- $B \rightarrow 00|\varepsilon$

Passo 2: Eliminar regras ε . Vamos eliminar primeiro a regra $A \rightarrow \varepsilon$. A aparece do lado direito em $S \rightarrow A$, então adicionamos a regra $S \rightarrow \varepsilon$, e também ocorre em $A \rightarrow BAB$ que gera a regra $A \rightarrow BB$. Temos agora:

- $S \rightarrow A|\varepsilon$
- $A \rightarrow BAB|B|BB$
- $B \rightarrow 00|\varepsilon$

A regra $B \rightarrow \varepsilon$ vai ser eliminada agora. A regra $A \rightarrow B$ geraria $A \rightarrow \varepsilon$, mas essa regra já foi removida. A regra $A \rightarrow BB$ geraria $A \rightarrow B$, que já faz parte da gramática e $A \rightarrow \varepsilon$, que já foi removida. Finalmente a regra $A \rightarrow BAB$ gera $A \rightarrow AB$, pela eliminação da primeira ocorrência de B , $A \rightarrow BA$, pela eliminação da segunda ocorrência e $A \rightarrow A$ pela eliminação de ambas. As regras agora são:

- $S \rightarrow A|\varepsilon$
- $A \rightarrow BAB|B|BB|AB|BA|A$
- $B \rightarrow 00$

A regra $S \rightarrow \varepsilon$ é permitida por ser a variável inicial.

Passo 3: Eliminar as regras unitárias. A regra $A \rightarrow A$ pode ser eliminada sem acrescentar nada. Temos neste momento:

- $S \rightarrow A|\varepsilon$
- $A \rightarrow BAB|B|BB|AB|BA$
- $B \rightarrow 00$

Para remover a regra $S \rightarrow A$, precisamos acrescentar as regras $S \rightarrow BAB|B|BB|AB|BA$, temos agora:

- $S \rightarrow \varepsilon|BAB|B|BB|AB|BA$
- $A \rightarrow BAB|B|BB|AB|BA$
- $B \rightarrow 00$

Agora vamos remover $S \rightarrow B$, acrescentando $S \rightarrow 00$ e remover $A \rightarrow B$, acrescentando $A \rightarrow 00$. As regras são agora:

- $S \rightarrow \varepsilon|BAB|BB|AB|BA|00$
- $A \rightarrow BAB|BB|AB|BA|00$
- $B \rightarrow 00$

Passo 4: Eliminar regras com mais de dois elementos do lado direito. Substitua a regra $S \rightarrow BAB$ pelas regras $S \rightarrow BA_1$ e $A_1 \rightarrow AB$. Similarmente, substitua $A \rightarrow BAB$ por $A \rightarrow BA_2$ e $A_2 \rightarrow AB$. O nosso conjunto de regras agora é:

- $S \rightarrow \varepsilon|BB|AB|BA|00|BA_1$
- $A \rightarrow BB|AB|BA|00|BA_2$
- $B \rightarrow 00$
- $A_1 \rightarrow AB$
- $A_2 \rightarrow AB$

Passo 5: Eliminar regras que não tem duas variáveis do lado direito. $S \rightarrow 00$ vai ser substituída por $S \rightarrow A_3A_3$ e $A_3 \rightarrow 0$, $A \rightarrow 00$ será substituída por $A \rightarrow A_4A_4$ e $A_4 \rightarrow 0$ e finalmente $B \rightarrow 00$ é removida e são acrescentadas $B \rightarrow A_5A_5$ e $A_5 \rightarrow 0$. Finalmente as regras na forma normal de Chomsky são:

- $S \rightarrow \varepsilon|BB|AB|BA|BA_1|A_3A_3$
- $A \rightarrow BB|AB|BA|BA_2|A_4A_4$
- $B \rightarrow A_5A_5$
- $A_1 \rightarrow AB$
- $A_2 \rightarrow AB$
- $A_3 \rightarrow 0$

- $A_4 \rightarrow 0$
- $A_5 \rightarrow 0$

Este é o resultado da aplicação direta das regras. Na prática percebe-se que A_1 e A_2 produzem a mesma cadeia AB , similarmente A_3 , A_4 e A_5 produzem apenas 0 . Então poderíamos simplificar este conjunto de regras e obter o seguinte conjunto equivalente:

- $S \rightarrow \varepsilon | BB | AB | BA | BX | ZZ$
- $A \rightarrow BB | AB | BA | BX | ZZ$
- $B \rightarrow ZZ$
- $X \rightarrow AB$
- $Z \rightarrow 0$

Onde as variáveis A_1 e A_2 foram identificadas com X e A_3 , A_4 e A_5 com Z .

Exemplo 2

Converta para a CNF a gramática $G = (\{S, A, B\}, \{0, 1\}, R, S)$ com R composto por:

- $S \rightarrow ASA | 0B$
- $A \rightarrow B | S$
- $B \rightarrow 1 | \varepsilon$

Dessa vez vou ser menos detalhista nas substituições, tente entender o que foi feito em cada um dos passos.

Passo 1: Insere novo estado inicial. • $S_0 \rightarrow S$ • $S \rightarrow ASA 0B$ • $A \rightarrow B S$ • $B \rightarrow 1 \varepsilon$	Passo 2.1: Remove $B \rightarrow \varepsilon$ • $S_0 \rightarrow S$ • $S \rightarrow ASA 0B 0$ • $A \rightarrow B S \varepsilon$ • $B \rightarrow 1$
Passo 2.2: Remove $A \rightarrow \varepsilon$ • $S_0 \rightarrow S$ • $S \rightarrow ASA 0B 0 SA AS S$ • $A \rightarrow B S$ • $B \rightarrow 1$	Passo 3.1: Remove $S \rightarrow S$ • $S_0 \rightarrow S$ • $S \rightarrow ASA 0B 0 SA AS$ • $A \rightarrow B S$ • $B \rightarrow 1$
Passo 3.2: Remove $S_0 \rightarrow S$	Passo 3.3: Remove $A \rightarrow B$ • $S_0 \rightarrow ASA 0B 0 SA AS$

• $S_0 \rightarrow ASA 0B 0 SA AS$ • $S \rightarrow ASA 0B 0 SA AS$ • $A \rightarrow B S$ • $B \rightarrow 1$	• $S \rightarrow ASA 0B 0 SA AS$ • $A \rightarrow S 1$ • $B \rightarrow 1$
Passo 3.4: Remove $A \rightarrow S$ • $S_0 \rightarrow ASA 0B 0 SA AS$ • $S \rightarrow ASA 0B 0 SA AS$ • $A \rightarrow 1 ASA 0B 0 SA AS$ • $B \rightarrow 1$	Passo 4.1: Remove $S_0 \rightarrow ASA$ • $S_0 \rightarrow AA_1 0B 0 SA AS$ • $S \rightarrow ASA 0B 0 SA AS$ • $A \rightarrow 1 ASA 0B 0 SA AS$ • $B \rightarrow 1$ • $A_1 \rightarrow SA$
Passo 4.2: Remove $S \rightarrow ASA$ • $S_0 \rightarrow AA_1 0B 0 SA AS$ • $S \rightarrow AA_2 0B 0 SA AS$ • $A \rightarrow 1 ASA 0B 0 SA AS$ • $B \rightarrow 1$ • $A_1 \rightarrow SA$ • $A_2 \rightarrow SA$	Passo 4.1: Remove $A \rightarrow ASA$ • $S_0 \rightarrow AA_1 0B 0 SA AS$ • $S \rightarrow AA_2 0B 0 SA AS$ • $A \rightarrow 1 AA_3 0B 0 SA AS$ • $B \rightarrow 1$ • $A_1 \rightarrow SA$ • $A_2 \rightarrow SA$ • $A_3 \rightarrow SA$
Passo 5.1: Remove $S_0 \rightarrow 0B$ • $S_0 \rightarrow AA_1 A_4B 0 SA AS$ • $S \rightarrow AA_2 0B 0 SA AS$ • $A \rightarrow 1 AA_3 0B 0 SA AS$ • $B \rightarrow 1$ • $A_1 \rightarrow SA$ • $A_2 \rightarrow SA$ • $A_3 \rightarrow SA$ • $A_4 \rightarrow 0$	Passo 5.1: Remove $S \rightarrow 0B$ • $S_0 \rightarrow AA_1 A_4B 0 SA AS$ • $S \rightarrow AA_2 A_5B 0 SA AS$ • $A \rightarrow 1 AA_3 0B 0 SA AS$ • $B \rightarrow 1$ • $A_1 \rightarrow SA$ • $A_2 \rightarrow SA$ • $A_3 \rightarrow SA$ • $A_4 \rightarrow 0$ • $A_5 \rightarrow 0$
Passo 5.1: Remove $S \rightarrow 0B$ • $S_0 \rightarrow AA_1 A_4B 0 SA AS$ • $S \rightarrow AA_2 A_5B 0 SA AS$ • $A \rightarrow 1 AA_3 A_6B 0 SA AS$	Removendo réplicas • $S_0 \rightarrow AX ZB 0 SA AS$ • $S \rightarrow AX ZB 0 SA AS$ • $A \rightarrow 1 AX ZB 0 SA AS$

• $B \rightarrow 1$ • $A_1 \rightarrow SA$ • $A_2 \rightarrow SA$ • $A_3 \rightarrow SA$ • $A_4 \rightarrow 0$ • $A_5 \rightarrow 0$ • $A_6 \rightarrow 0$	• $B \rightarrow 1$ • $X \rightarrow SA$ • $Z \rightarrow 0$
---	---

O processo é trabalhoso às vezes, mas pode ser automatizado. Perceba que a quantidade de regras aumenta bastante ao passar para a CNF. O comprimento do conjunto de regras também aumenta, sendo o comprimento a quantidade de símbolos (com repetição) que formam as regras. No exemplo que acabamos de encerrar, o comprimento da gramática original era 12 e passou para 38 na CNF.

Além da curiosidade teórica, de que é possível impor tais restrições sobre o lado direito das regras das CFG e continuar com a mesma linguagem, este processo parece sem sentido. Vou apresentar uma utilidade imediata para não dizer que o trabalho não compensa. Uma coisa muito simples de ser feita com autômatos pode ser bem complicada com uma CFG, como determinar se uma dada cadeia pode ou não ser gerada por uma gramática?

Determinar Pertinência de Cadeia à Linguagem de uma CFG

Para saber se uma cadeia pode ser gerada por uma gramática, devemos ser capazes de determinar uma sequência de derivações que parte da variável inicial e chega à cadeia desejada. Mas como fazer isso? Testando sistematicamente todas as derivações possíveis? Basicamente, sim, mas como fazer isso de forma inteligente?

Nesta seção apresento o algoritmo CYK (Cocke - Younger - Kasami) que decide se uma dada cadeia pertence ou não à linguagem de uma gramática livre de contexto dada na forma normal de Chomsky.

A idéia essencial é, como dito antes, testar todas as formas de criar cadeias, mas ao invés de partir da variável inicial, partimos da cadeia e vemos se é possível atingir a variável inicial.

Dado uma cadeia de comprimento k , primeiramente, o algoritmo verifica como as cadeias de comprimento unitário (terminais) podem ser derivadas. Depois, iterativamente, verifica como as cadeias unitárias podem ser combinadas para gerar subcadeias de comprimento dois, e assim sucessivamente até verificar como a cadeia de comprimento k pode ser derivada a partir das cadeias de comprimento menor que k . Se a variável inicial gera a cadeia, essa cadeia faz parte da linguagem

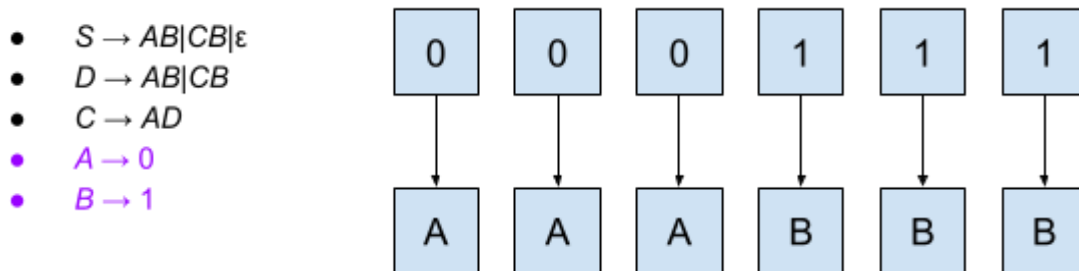
Para entender a intuição dada no parágrafo anterior, vamos verificar um exemplo.

Exemplo

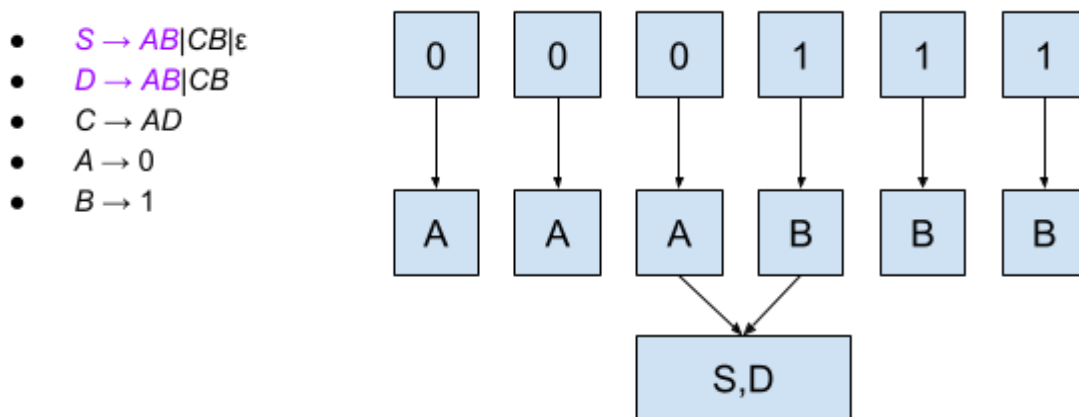
Considere a CFG com as seguintes regras na CNF:

- $S \rightarrow AB|CB|\epsilon$
- $D \rightarrow AB|CB$
- $C \rightarrow AD$
- $A \rightarrow 0$
- $B \rightarrow 1$

Vamos verificar se a cadeia 000111 pode ser gerada por essa gramática. Primeiro, verificamos como os terminais 0 e 1 podem ser produzidos:

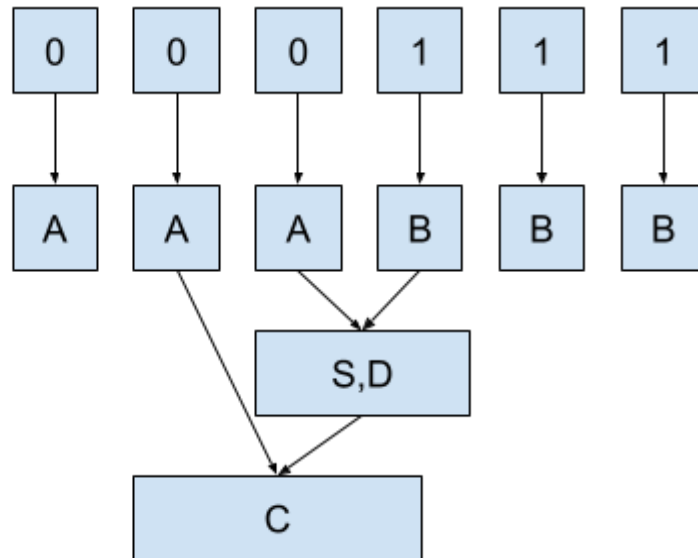


Agora vamos verificar todas as combinações de geradores adjacentes para ver qual pode ser produzido pelas regras dadas. Notamos que AB pode ser gerado por S ou D . As demais combinações, AA ou BB , não podem ser produzidas por nenhuma das regras.



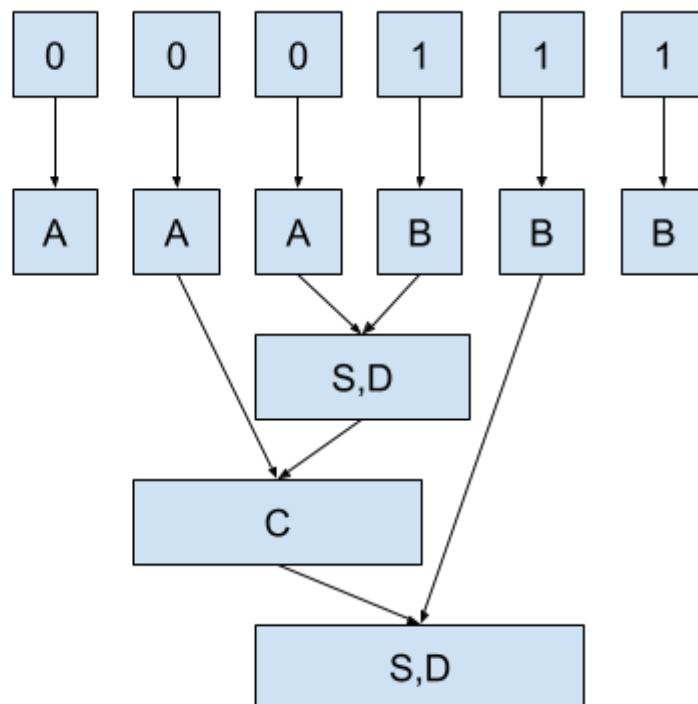
Agora sabemos que a cadeia de comprimento dois que inicia na terceira posição e termina na quarta (o 01 central) pode ser gerado por S ou D . Vamos verificar se é possível gerar uma combinação de um elemento unitário com esse novo elemento que representa uma cadeia de comprimento dois para assim gerar uma cadeia de comprimento 3. Notamos então que AD pode ser derivado a partir de C .

- $S \rightarrow AB|CB|\epsilon$
- $D \rightarrow AB|CB$
- $C \rightarrow AD$
- $A \rightarrow 0$
- $B \rightarrow 1$



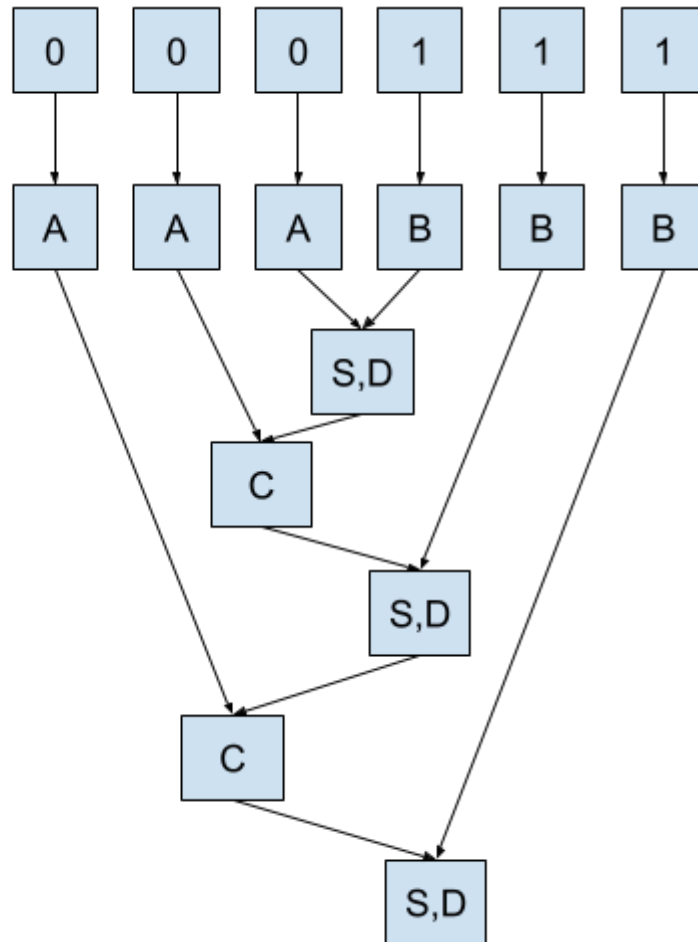
Seguindo o processo, percebemos que CB pode ser derivado de S ou D.

- $S \rightarrow AB|CB|\epsilon$
- $D \rightarrow AB|CB$
- $C \rightarrow AD$
- $A \rightarrow 0$
- $B \rightarrow 1$



Agora sabemos que a partir de S ou D existe uma sequência de derivações que gera a sequência 0011 central, de comprimento 4. Vamos fazer esse processo para o comprimento 5 e 6, que basicamente é uma repetição dos passos 3 e 4, para esta CFG.

- $S \rightarrow AB|CB|\epsilon$
- $D \rightarrow AB|CB$
- $C \rightarrow AD$
- $A \rightarrow 0$
- $B \rightarrow 1$



Finalmente, percebe-se que a variável inicial, S , pode gerar toda a cadeia. Portanto a cadeia 000111 faz parte da linguagem dessa CFG. De fato, a derivação desta cadeia é $S \Rightarrow CB \Rightarrow ADB \Rightarrow 0DB \Rightarrow 0CBB \Rightarrow 0ADBB \Rightarrow 00DBB \Rightarrow 00ABBB \Rightarrow 000BBB \Rightarrow 0001BB \Rightarrow 00011B \Rightarrow 000111$.

Se o último 1 não existisse, a variável do topo seria C , que não é inicial e portanto a cadeia 00011 não pode ser gerada por esta CFG. Nem sempre a análise é tão simples assim, esta gramática tem uma derivação quase única para qualquer cadeia da sua linguagem, o exemplo foi construído apenas para ilustrar a idéia.

Agora segue a descrição do algoritmo. Seja $\omega = w_1 w_2 \dots w_n$ a cadeia de comprimento n que desejamos saber se faz parte da linguagem da gramática $G = (V, \Sigma, R, S)$, dada na forma normal de Chomsky.

Algoritmo CYK

1. Se $\omega = \epsilon$ e $S \rightarrow \epsilon$ está em R , retorne SIM.
2. $D \leftarrow$ matriz $n \times n$ de conjuntos vazios.
3. Para i variando de 1 a n :
4. Para cada variável $A \in V$:

```

5.      Se  $A \rightarrow t$  é uma regra e  $t = \omega_i$  para algum  $i$ :
6.           $D(i, i) \leftarrow D(i, i) \cup \{A\}$ .
7.  Para  $c$  variando de 2 a  $n$ :
8.      Para  $i$  variando de 1 a  $n - c + 1$ :
9.           $j \leftarrow i + c - 1$ 
10.         Para  $k$  variando de  $i$  a  $j - 1$ :
11.             Para cada regra  $A \rightarrow BC$ :
12.                 Se  $B \in D(i, k)$  e  $C \in D(k + 1, j)$ :
13.                      $D(i, j) \leftarrow D(i, j) \cup \{A\}$ 
14. Se  $S \in D(1, n)$ :
15.     Retorne SIM.
16. Senão:
17.     Retorne NÃO.

```

O laço das linhas 3-6 verifica a geração dos símbolos terminais. O laço das linhas 7-13 faz as combinações das informações passadas. O elemento $D(x, y)$ da matriz é um conjunto que diz quais variáveis são capazes de derivar a subcadeia que inicia na posição x e tem comprimento $y - x + 1$. Apesar da matriz ter tamanho $n \times n$, apenas metade dela é preenchida. Essa descrição buscou apenas a clareza do entendimento e não eficiência.

Exemplo

Verifique se a cadeia 10010 faz parte da linguagem da gramática definida pelas regras

- $S \rightarrow AB|BC$
- $A \rightarrow BA|0$
- $B \rightarrow CC|1$
- $C \rightarrow AB|0$

Vamos utilizar o algoritmo descrito anteriormente. Primeiro devemos criar uma matriz de tamanho 5×5 cujos elementos são conjuntos vazios. O primeiro laço preenche a diagonal dessa matriz com as variáveis que derivam em um passo os terminais. Nesse caso, 0 pode ser derivado de $\{A, C\}$ e 1 de $\{B\}$.

$\{B\}$				
	$\{A, C\}$			
		$\{A, C\}$		
			$\{B\}$	
				$\{A, C\}$

Para $c = 2$, $i = 1$, $j = 2$, verificamos se é possível gerar a combinação de $D(1, 1)$ com $D(2, 2)$. As combinações são BA e BC , que podem ser geradas por A ou S , respectivamente. Este procedimento segue para a outras linhas até a quarta (observe como i varia no algoritmo) e obtemos a seguinte tabela:

$\{B\}$	$\{S, A\}$			
	$\{A, C\}$	$\{B\}$		
		$\{A, C\}$	$\{S, C\}$	
			$\{B\}$	$\{S, A\}$
				$\{A, C\}$

Para $c = 3$, $i = 1$, $j = 3$, verificamos as combinações de $D(1, 1)$ com $D(2, 3)$ e de $D(1, 2)$ com $D(3, 3)$. c é o comprimento da subcadeia que estamos gerando agora, $D(1, 3)$ que vai ser preenchido, é uma cadeia de comprimento 3 que inicia na posição 1. As combinações descritas são de uma cadeia de comprimento 1 na posição 1 seguida de uma cadeia de comprimento 2 que inicia na posição 2 ou uma cadeia de comprimento 2 que inicia na posição 1 seguida de uma cadeia de comprimento 1 que inicia na posição 3. Ou seja, todas as formas possíveis de se gerar a subcadeia desejada.

As combinações são BB , SA , SC , AA e AC , que não podem ser geradas por nenhuma regra e, por isso, $D(1, 3) = \emptyset$. Este procedimento segue para a outras linhas até a terceira e obtemos a seguinte tabela:

$\{B\}$	$\{S, A\}$	$\emptyset$		
	$\{A, C\}$	$\{B\}$	$\{B\}$	
		$\{A, C\}$	$\{S, C\}$	$\{B\}$
			$\{B\}$	$\{S, A\}$
				$\{A, C\}$

O processo continua para os outros valores de c e finalmente a tabela completa é (tente fazer e verifique o resultado):

$\{B\}$	$\{S, A\}$	$\emptyset$	$\emptyset$	$\{S, A, C\}$
	$\{A, C\}$	$\{B\}$	$\{B\}$	$\{S, A, C\}$
		$\{A, C\}$	$\{S, C\}$	$\{B\}$
			$\{B\}$	$\{S, A\}$
				$\{A, C\}$

Como $S \in D(1, 5) = \{S, A, C\}$, a cadeia 10010 pode ser gerada por esta CFG.

Voltaremos neste assunto na seção de aplicações das linguagens livres de contexto. Agora que já estudamos bastante o formalismo gerador das linguagens livres de contexto, vamos tentar criar um reconhecedor, ou seja, uma máquina que reconhece tais linguagens.

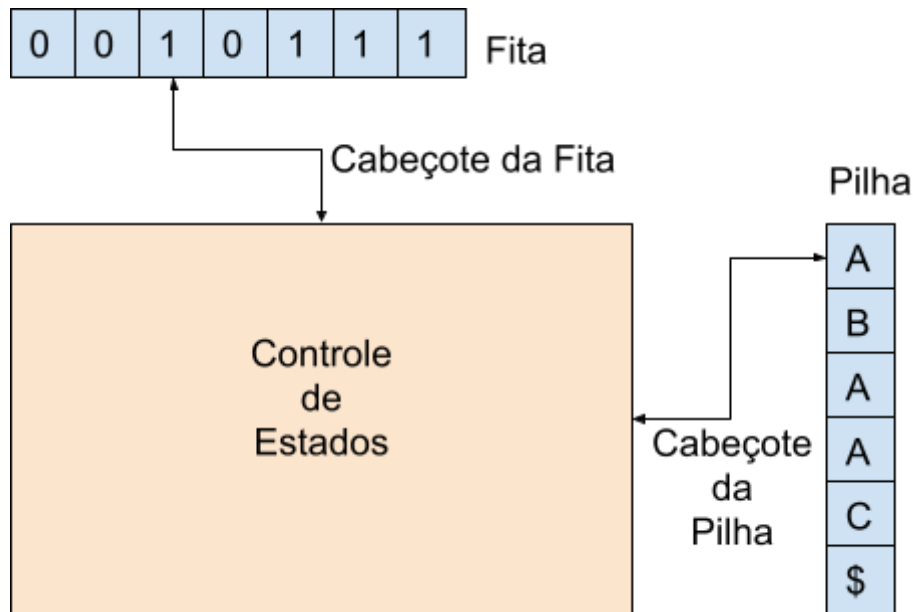
Autômatos com Pilha (PDA)

Nesta seção vamos estudar uma variação dos autômatos vistos no capítulo anterior, mas agora com mais poderes. A grande limitação dos autômatos finitos (determinísticos ou não) era a sua memória constante e limitada. As decisões tomadas pelos autômatos finitos são baseadas no estado atual e nos símbolos lidos da entrada. Como os estados são finitos e a entrada é lida uma única vez da esquerda para a direita, toda estrutura de repetição, presente nas linguagens infinitas, devem ser adequadamente representadas por repetições de estados. Tudo isso foi discutido anteriormente e o teorema de Myhill-Nerode evidencia este fato, visto que as linguagens regulares têm de ter uma quantidade finita de classes de equivalência.

Vamos superar este obstáculo adicionando aos autômatos finitos uma memória externa, uma pilha para ser mais exato. Muitos de vocês já devem estar familiarizados com o conceito de pilha, nem que seja com uma pilha de livros. A idéia é que os objetos em uma pilha são armazenados um “por cima” dos outros, de modo que tanto o armazenamento quanto o acesso são feitos sempre através do topo da pilha. Para não perder a mania de criar sigla para tudo, vamos chamar essa estrutura de dados de LIFO (*Last In First Out*) porque o último elemento inserido na pilha é o primeiro a ser removido, o topo da pilha.

Vamos agora dar uma descrição informal de um autômato com pilha (PDA - *PushDown Automata*). Os elementos que compõem um PDA são os seguintes:

1. Uma *fita* que é dividida em *células*. Cada célula armazena um símbolo que pertence a um conjunto finito Σ , chamado de *alfabeto da fita* (ou *alfabeto de entrada*). Esta fita é equivalente ao que chamamos de entrada nos autômatos finitos.
2. Um *cabeçote de fita* que se move ao longo da fita, uma célula por vez. Este cabeçote pode ler o conteúdo da célula na qual se encontra, efetuando um movimento para a célula imediatamente à direita. Isto equivale à afirmação nos autômatos finitos de que a entrada é lida da esquerda para a direita, sem retorno.
3. Uma *pilha* contendo símbolos de um conjunto finito Γ , chamado de *alfabeto da pilha*. Este conjunto contém um símbolo especial, $\$$, que indica o fundo da pilha. Perceba que o alfabeto da pilha e da fita não precisam ser o mesmo.
4. Um *cabeçote de pilha* que pode ler o símbolo no topo da pilha. Este cabeçote também insere elementos de Γ_ϵ no topo da pilha.
5. Um *controle de estados*, que pode estar em qualquer um de uma quantidade finita de *estados*. O conjunto de estados será chamado de Q . O conjunto Q tem um estado especial q_0 , chamado de *estado inicial* e um conjunto de *estados de aceitação*. O controle de estados transita entre os estados segundo as regras definidas por uma *função de transição*.



Com exceção da pilha e do seu cabeçote, todo o resto já existia em um autômato finito. A entrada para um autômato com pilha é uma cadeia de Σ^* . Essa entrada é escrita na fita, cujo cabeçote inicia no símbolo mais à esquerda da cadeia de entrada. Inicialmente, a pilha está vazia, e o autômato está no estado q_0 . Em um passo de computação o PDA faz o seguinte, assuma que o autômato está no estado q , a é um símbolo lido da fita e A é um símbolo lido da pilha:

1. O PDA transita para um novo estado r de Q , que pode ser igual a r ,
2. Se
 - a. $a = \varepsilon$, nada é lido da fita e o cabeçote da fita fica parado.
 - b. $a \neq \varepsilon$, um símbolo é lido e o cabeçote move-se para a direita.
3. Se
 - a. $A = \varepsilon$, nada é lido da pilha.
 - b. $A \neq \varepsilon$, um símbolo é lido e removido (*pop*) do topo pilha.
4. Uma cadeia $\omega \in \Gamma^*$ é escrita (*push*) na pilha. Para ser mais exato:
 - a. se $\omega = \varepsilon$, nada é escrito.
 - b. se $\omega = w_1 w_2 \dots w_k$, com $k \geq 1$ e $w_1, w_2, \dots, w_k \in \Gamma$, então os símbolos $w_1, w_2, \dots, w_k$ são inseridos em sequência reversa na pilha, de modo que w_1 é o novo elemento no topo da pilha.

Definição Formal de um Autômato com Pilha

Tendo em vista toda a discussão anterior, deve ficar fácil dar uma descrição formal dos PDA

Definição: Autômato com Pilha (PDA)

Um autômato com pilha (PDA) é uma 6-upla $N = (Q, \Sigma, \Gamma, \delta, q_0, F)$, onde:

- Q é um conjunto finito, cujos elementos são chamados de *estados*;
- Σ é um conjunto finito, chamado *alfabeto da fita*;
- Γ é um conjunto finito, chamado *alfabeto da pilha*;
- $\delta: Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow \mathcal{P}(Q \times \Gamma^*)$ é uma função, chamada de *função de transição*;
- $q_0 \in Q$, é chamado de *estado inicial*;
- $F \subseteq Q$, são os *estados de aceitação* (ou *estados finais*).

A computação em um PDA também pode ser descrita formalmente a partir da descrição anterior:

Definição: Computação em um PDA

Seja $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$ um NFA e seja $\omega = w_1 w_2 \dots w_n$ onde $w_i \in \Sigma_\epsilon$. Diz-se que P aceita ω se *existe* uma sequência $s_0 s_1 \dots s_n$ de estados e uma sequência $c_0 c_1 \dots c_n$ de cadeias de Γ^* tais que:

- $s_0 = q_0$ e $c_0 = \epsilon$;
- $(s_{i+1}, b) \in \delta(s_i, w_{i+1}, a)$, para $i = 0, 1, \dots, n-1$, onde $s_i = at$ e $s_{i+1} = bt$ com $a \in \Gamma_\epsilon$ e $b \in \Gamma^*$.
- $s_n \in F$.

Caso contrário, diz-se que P *rejeita* ω .

A linguagem do autômato continua sendo definida da mesma forma.

Definição: Linguagem Reconhecida por um PDA

Seja $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$ um PDA. A *linguagem reconhecida* por P , denotada por $\mathcal{L}(P)$, é o conjunto de todas as cadeias sobre Σ^* aceitas por P : $\mathcal{L}(P) = \{x \in \Sigma^* : P \text{ aceita } x\}$.

Perceba que o autômato com pilha descrito formalmente nesta seção é não-determinístico. Ao contrário dos autômatos finitos, os PDA determinísticos e não-determinísticos têm capacidades distintas, sendo os não-determinísticos mais poderosos. Não farei uma prova dessa afirmação, mas vou indicar em alguns exemplos que eles não podem ser reconhecidos por um PDA determinístico e por que não.

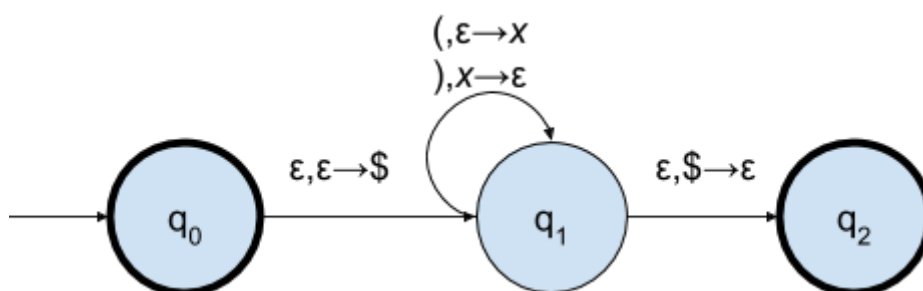
Exemplos de PDA

Vamos ver agora alguns exemplos de PDAs. Como são não-determinísticos, vamos apresentar diagramas e computações similares aos exemplos de NFA. O diagrama será levemente diferente porque as transições têm leitura tanto da entrada quanto da pilha, além

da escrita na pilha. Rotulamos as transições como “ $a, b \rightarrow c$ ” dizendo que o símbolo a foi lido da entrada, o símbolo b foi lido da pilha e a cadeia c foi escrita na pilha. Adicionalmente, quando há bifurcações na computação, cada bifurcação terá sua própria pilha, que será indicada como uma cadeia horizontal onde o símbolo mais à esquerda é o topo da pilha.

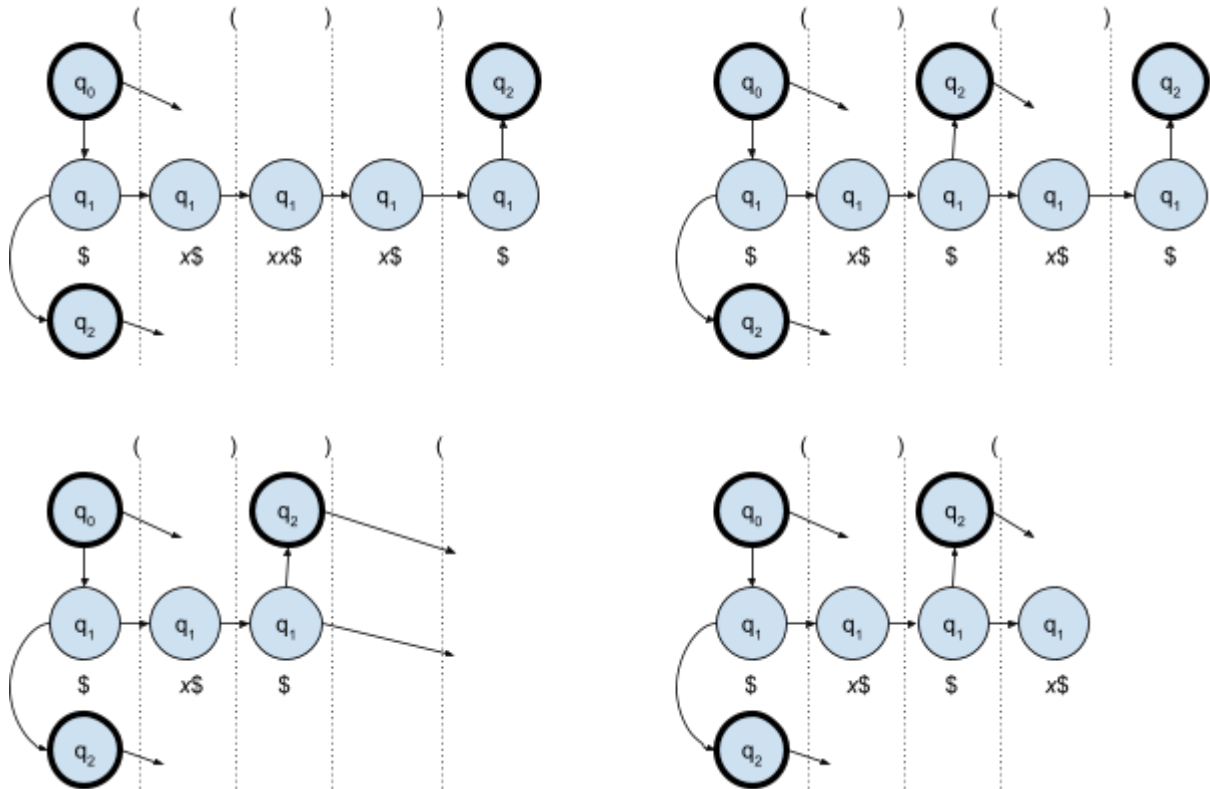
Exemplo 1: $L = \{x \in \{(),\}^* : x \text{ tem parênteses corretamente aninhados}\}$

Por simplicidade, nosso alfabeto é formado apenas de parênteses que devem estar corretamente aninhados, ou seja, não deve ser possível fechar um parêntese antes de abrí-lo. Em qualquer momento da leitura da cadeia de entrada, a quantidade de parênteses abertos deve ser maior ou igual a de parênteses fechados, além disso, ao fim da cadeia a quantidade deve ser igual.



O estado q_1 é quem faz o todo o trabalho. Quando um parêntese aberto é visto um novo símbolo é inserido na pilha, que funciona como um contador. Quando um parêntese fechado é visto, se o contador não estiver zerado, ele é decrementado.

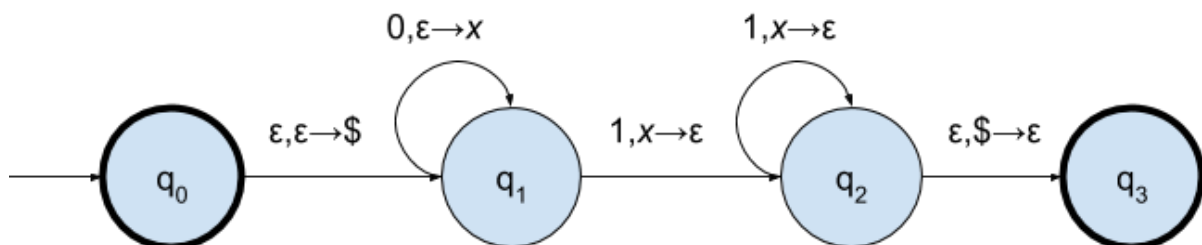
Vamos detalhar a computação deste PDA para as cadeias $()()$, $()()$, $()()()$ e $()()()$.



A descrição formal desse autômato é $P = (\{q_0, q_1, q_2\}, \{(\cdot)\}, \{\$, x\}, \delta, q_0, \{q_0, q_2\})$ com as seguintes transições: $\delta(q_0, \epsilon, \epsilon) = \{(q_1, \$)\}$, $\delta(q_1, (\cdot, \epsilon) = \{(q_1, x)\}$, $\delta(q_1, x) = \{(q_1, \epsilon)\}$ e $\delta(q_1, \epsilon, \$) = \{(q_2, \epsilon)\}$.

Exemplo 2: $L = \{0^n 1^n : n \geq 0\}$

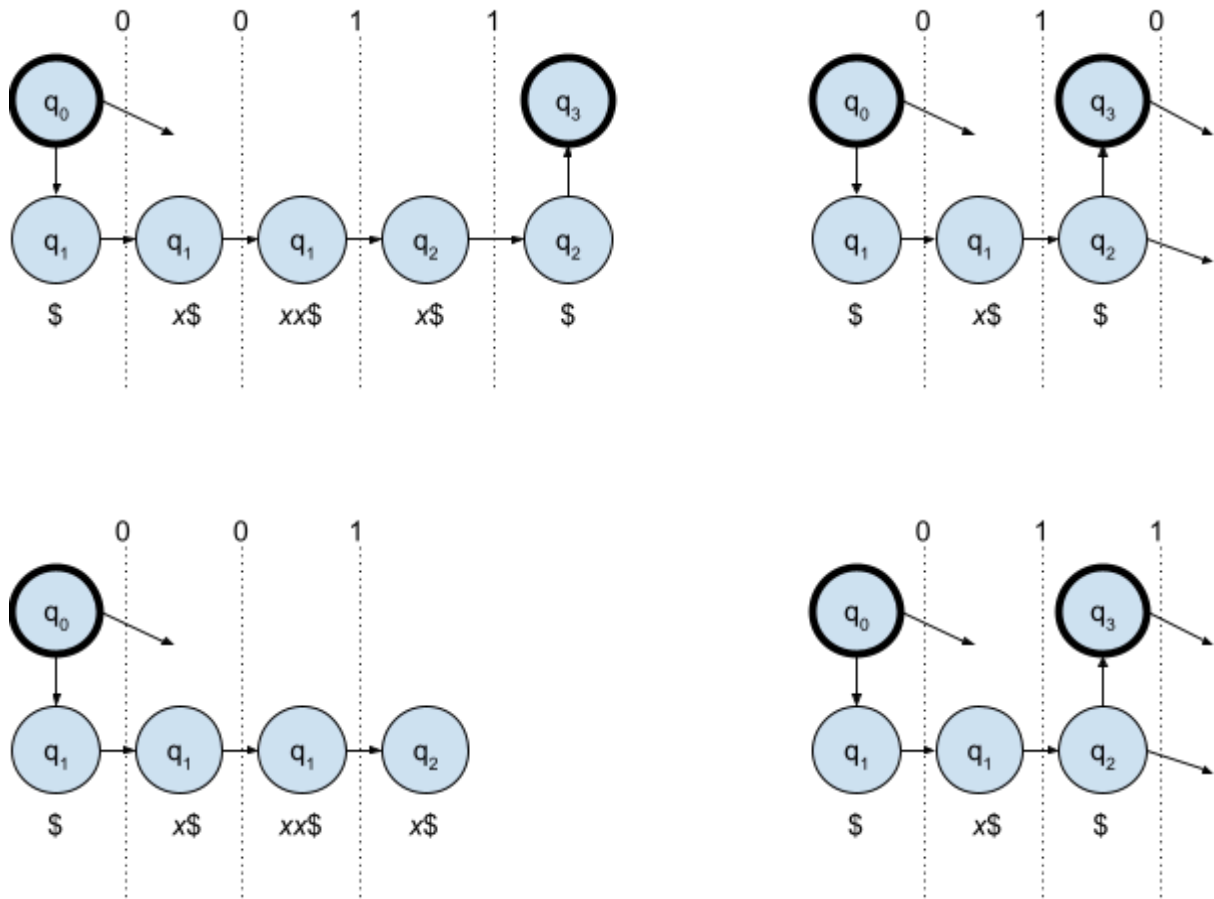
Para reconhecer essa linguagem precisamos contar quantos zeros ocorreram e, em seguida verificar se essa quantidade é igual à quantidade de uns. Podemos armazenar um símbolo na pilha para cada zero lido e em seguida remover da pilha para cada um lido. Se a pilha terminar vazia, as quantidades são iguais. Eis o diagrama de estados de um PDA que reconhece essa linguagem:



A transição de q_0 para q_1 apenas marca o fundo da pilha. O laço em q_1 insere um x na pilha para cada 0 lido da entrada. A transição de q_1 para q_2 , assim como o laço em q_2 ,

remove um x da pilha para cada 1 lido da entrada. Finalmente, a transição de q_2 para q_3 é realizada quando vemos o fundo da pilha.

Vamos ver a computação desse autômato para as cadeias 0011, 010, 001 e 011.



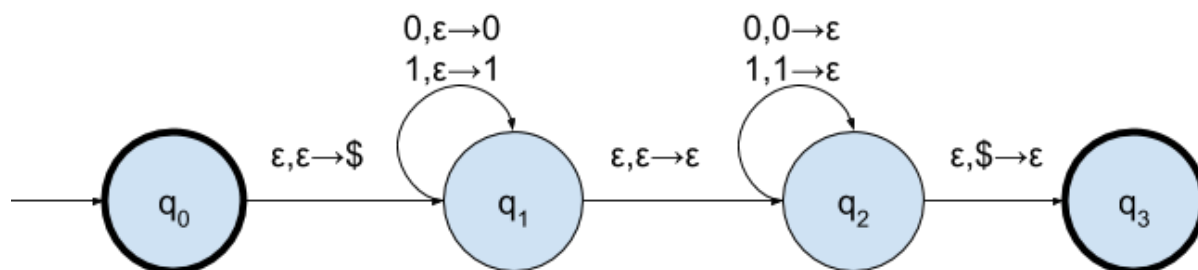
A descrição desse autômato é $P = (\{q_0, q_1, q_2, q_3\}, \{0, 1\}, \{\$, x\}, \delta, q_0, \{q_0, q_3\})$. Para evitar o excesso de espaço, vamos escrever apenas as transições definidas de δ , todas as demais levam para o conjunto vazio de estados: $\delta(q_0, \varepsilon, \varepsilon) = \{(q_1, \$)\}$, $\delta(q_1, 0, \varepsilon) = \{(q_1, x)\}$, $\delta(q_1, 1, x) = \{(q_2, \varepsilon)\}$, $\delta(q_2, 1, x) = \{(q_2, \varepsilon)\}$ e $\delta(q_2, \varepsilon, \$) = \{(q_3, \varepsilon)\}$.

Perceba que não há nenhum meio efetivo de testar se a pilha está vazia, por isso começamos inserindo um símbolo $\$$ para indicar o fundo da pilha. Similarmente, não há como verificar o fim da entrada, e só atingimos o estado de aceitação q_3 por causa da transição sem leitura da entrada. Há uma definição alternativa dos PDA na qual assume-se que há um símbolo especial para indicar o fim da entrada e também que a pilha já inicia com o símbolo $\$$ escrito, para um PDA determinístico estas condições são necessárias.

Exemplo 3: $L = \{ww^R : w \in \{0, 1\}^*\}$

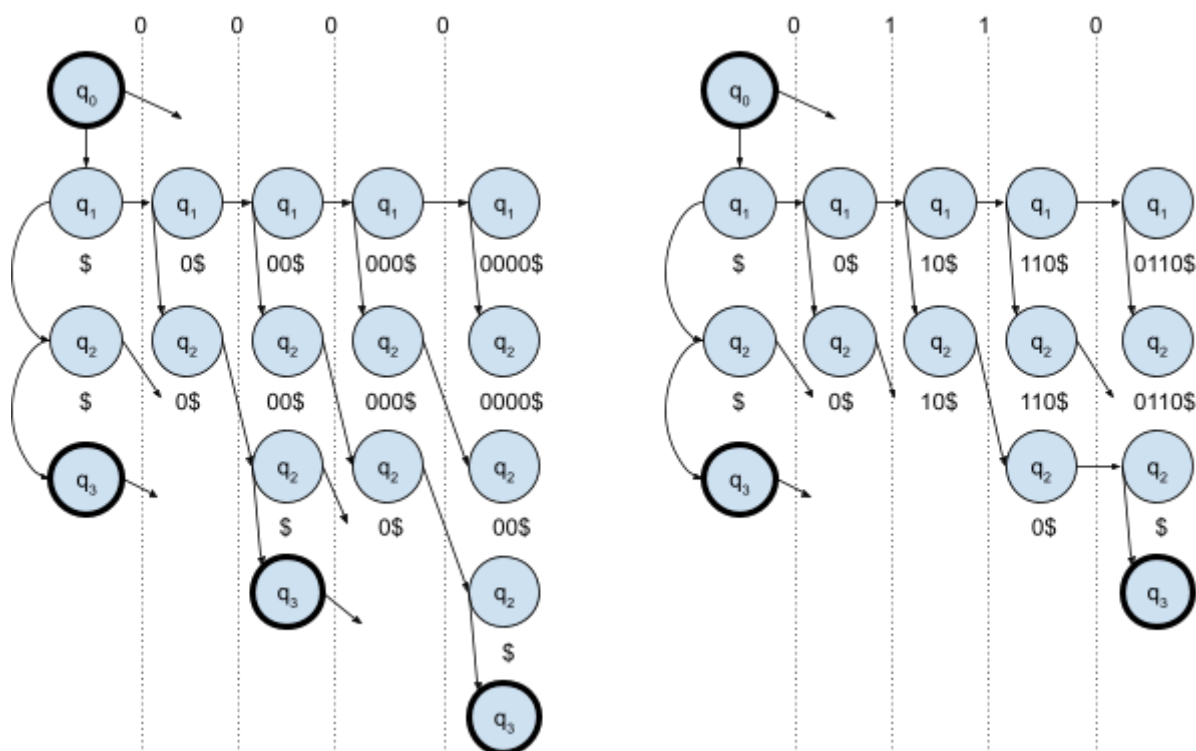
Vamos colocar os símbolos lidos na pilha e depois, quando atingir o meio, vamos desempilhar e obter o reverso da primeira metade. Perceba que não temos como saber

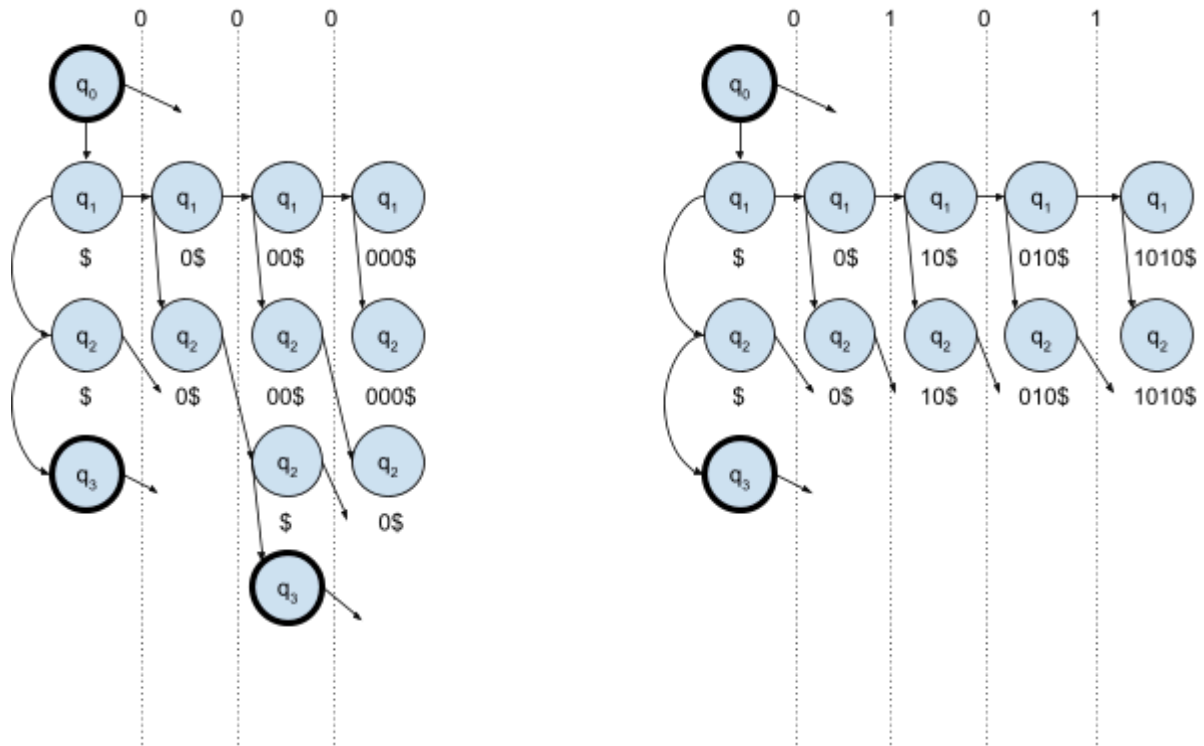
onde é o meio da cadeia de entrada, então o não-determinismo é essencial para “adivinhar” onde fica o meio.



O estado q_1 faz o armazenamento da entrada na pilha e o estado q_2 faz o desempilhamento conforme lê a entrada. Como não sabemos onde está a metade da cadeia de entrada, a transição vazia de q_1 para q_2 trata de fazer todos os testes possíveis, ou seja, sempre assumimos que estamos na metade.

Vamos verificar a computação deste PDA para as cadeias 0000, 0110, 000 e 0101.

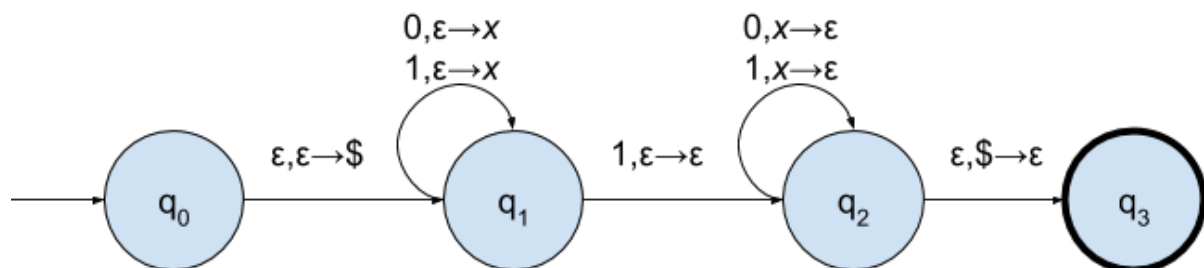




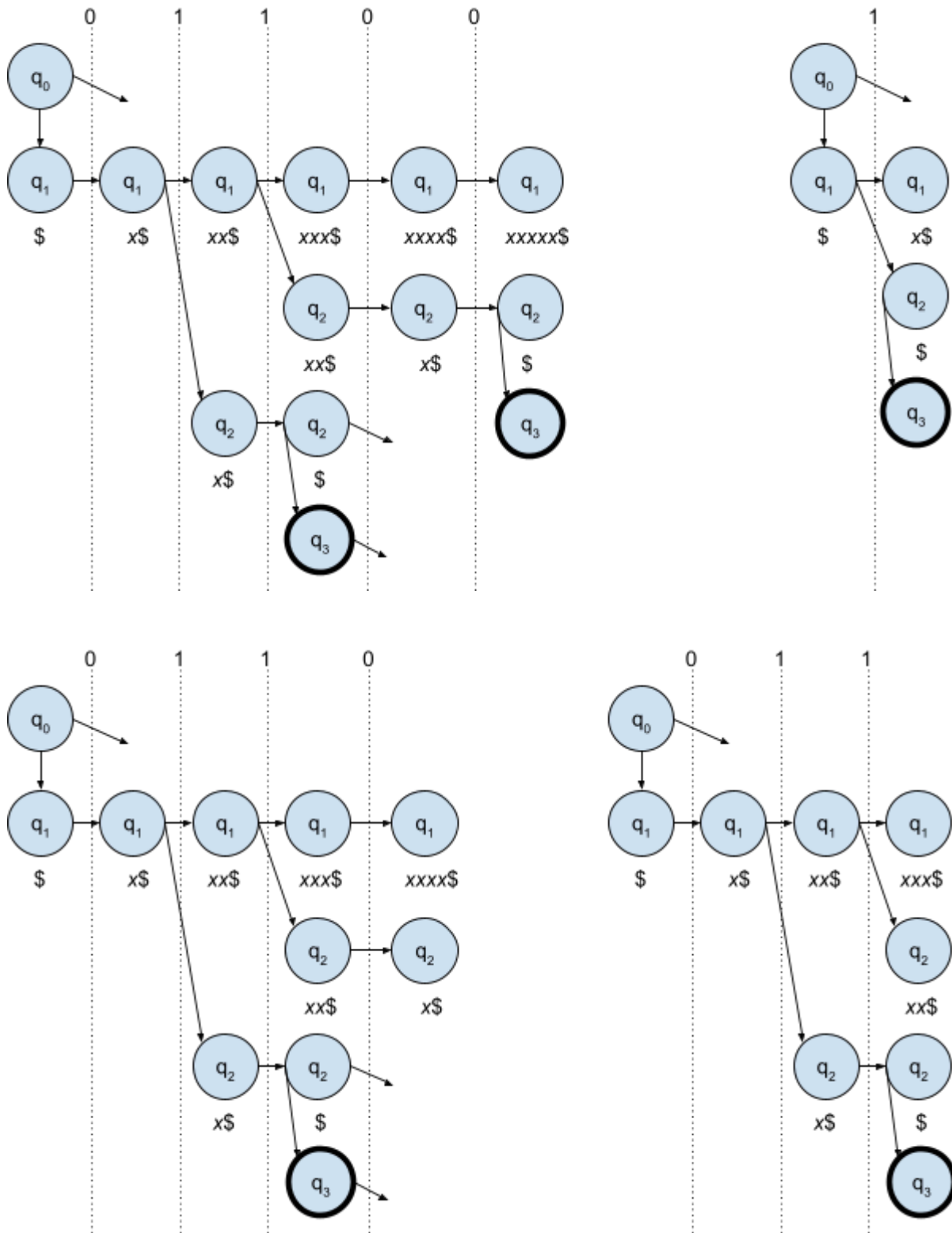
A descrição formal desse autômato é $P = (Q, \{0, 1\}, \{\$, 0, 1\}, \delta, q_0, \{q_0, q_3\})$ com $Q = \{q_0, q_1, q_2, q_3\}$ e as seguintes transições: $\delta(q_0, \epsilon, \epsilon) = \{(q_1, \$)\}$, $\delta(q_1, 0, \epsilon) = \{(q_1, 0)\}$, $\delta(q_1, 1, \epsilon) = \{(q_1, 1)\}$, $\delta(q_1, \epsilon, \epsilon) = \{(q_2, \epsilon)\}$, $\delta(q_2, 0, 0) = \{(q_2, \epsilon)\}$, $\delta(q_2, 1, 1) = \{(q_2, \epsilon)\}$ e $\delta(q_2, \epsilon, \$) = \{(q_3, \epsilon)\}$.

Exemplo 4: $L = \{a1b : a, b \in \{0, 1\}^*, |a| = |b|\}$

A linguagem descreve cadeias de comprimento ímpar pertencentes a $\{0, 1\}^*$ cujo símbolo central é 1. Por um motivo similar ao exemplo anterior, ao ler um símbolo 1 não sabemos se ele está no meio da cadeia ou não e o não-determinismo será essencial para resolver esse problema. Vamos contar a quantidade de símbolos lidos e, ao ler um símbolo 1, vamos “chutar” que ele está no meio da cadeia e começar a desempilhar ao ler os símbolos seguintes.



Vamos verificar a computação deste PDA para as cadeias 00110, 1, 0110, 011

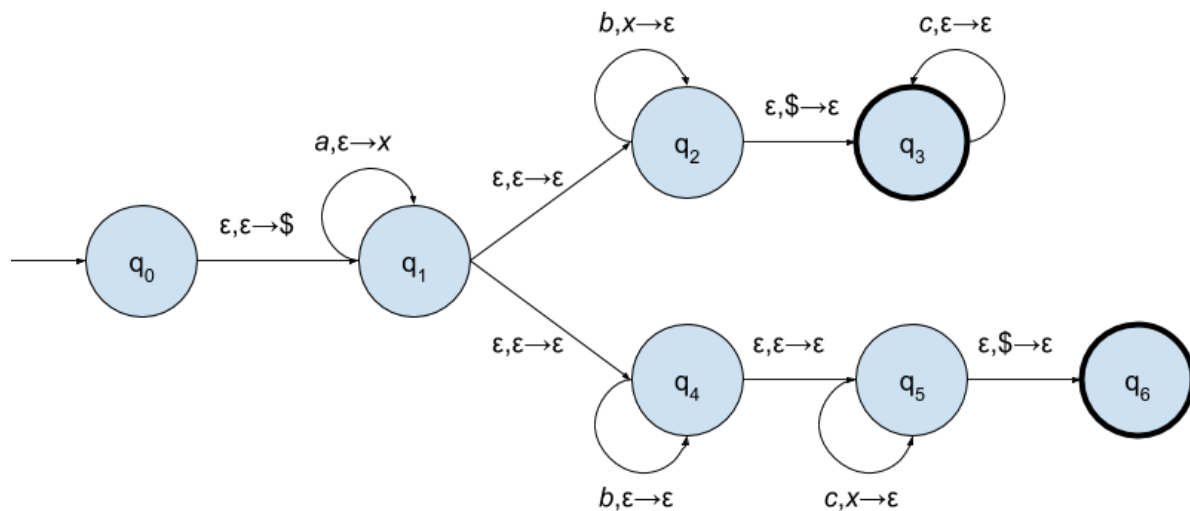


A descrição formal desse autômato é $P = (\{q_0, q_1, q_2, q_3\}, \{0, 1\}, \{\$, x\}, \delta, q_0, \{q_3\})$ com as seguintes transições: $\delta(q_0, \varepsilon, \varepsilon) = \{(q_1, \$)\}$, $\delta(q_1, 0, \varepsilon) = \{(q_1, x)\}$,

$$\begin{aligned} \delta(q_1, 1, \varepsilon) &= \{(q_1, x), (q_2, \varepsilon)\}, & \delta(q_2, 0, x) &= \{(q_2, \varepsilon)\}, & \delta(q_2, 1, x) &= \{(q_2, \varepsilon)\} & \text{e} \\ \delta(q_2, \varepsilon, \$) &= \{(q_3, \varepsilon)\}. \end{aligned}$$

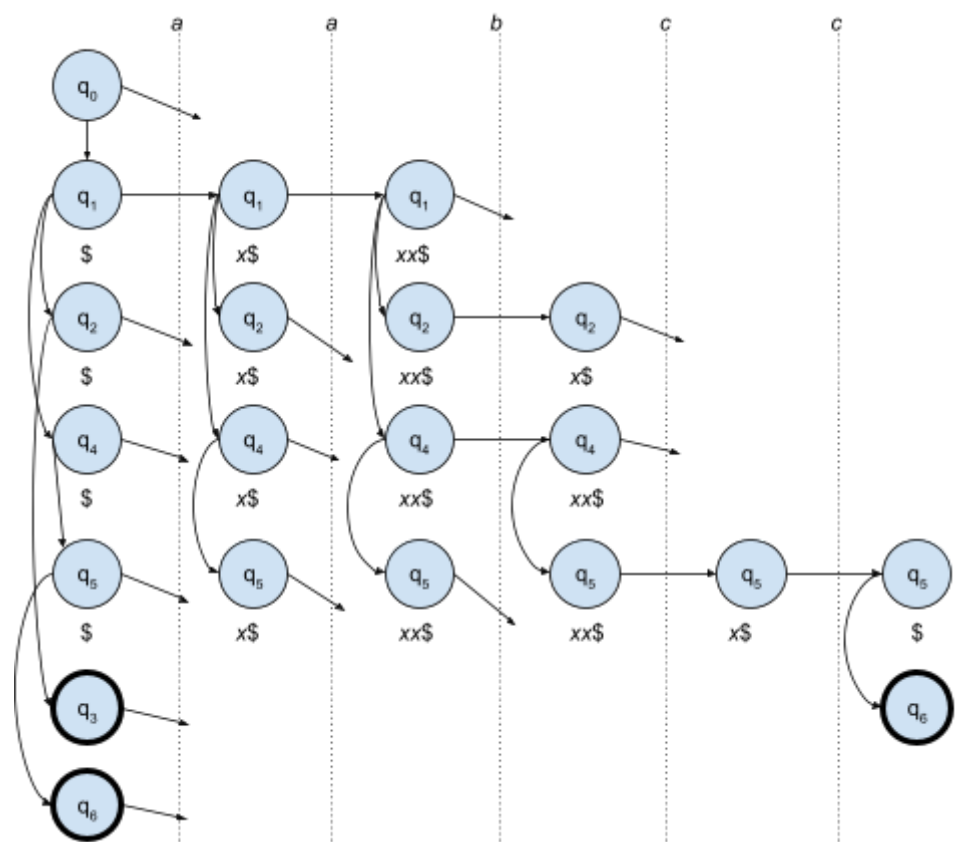
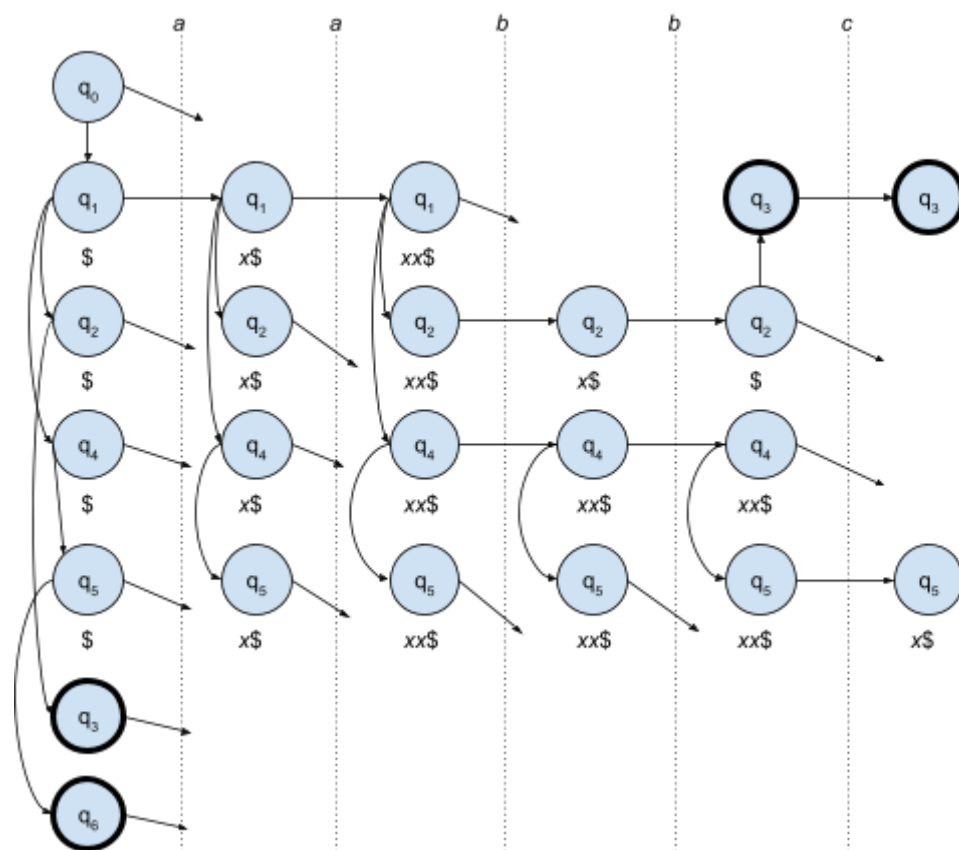
Exemplo 5: $L = \{a^x b^y c^z : x, y, z \geq 0, x = y \text{ ou } x = z\}$

Precisamos contar a quantidade de a e depois verificar se esta quantidade casa com a quantidade de b ou de c . O autômato que segue faz essa verificação.



O estado q_1 é o nosso contador, ele armazena a quantidade de a lido da entrada. Como não sabemos se os a devem ser confrontados com os b ou c , o *não-determinismo é fundamental nesse exemplo*. Temos transições vazias que ativam os dois ramos (superior e inferior) simultaneamente. O ramo superior confronta a quantidade de a com a de b e depois pode ler qualquer quantidade de c . O ramo inferior ignora os b e depois confronta a quantidade de a com a quantidade de c .

Vamos verificar a computação para as cadeias $aabbc$, $aabcc$, $aabbcc$ e $aabc$.



, $\delta(q_3, c, \varepsilon) = \{(q_3, \varepsilon)\}$, $\delta(q_4, b, \varepsilon) = \{(q_4, \varepsilon)\}$, $\delta(q_4, \varepsilon, \varepsilon) = \{(q_5, \varepsilon)\}$, $\delta(q_5, c, x) = \{(q_5, \varepsilon)\}$ e $\delta(q_5, \varepsilon, \$) = \{(q_6, \varepsilon)\}$.

PDA vs. CFG

Nos exemplos anteriores vimos que os PDAs são capazes de reconhecer diversas linguagens não-regulares. Vamos mostrar nesta seção que os PDAs são equivalentes, em termos de poder de representação, às gramáticas livres de contexto, ou seja, os PDAs reconhecem as linguagens livres de contexto.

Teorema

Uma linguagem é livre de contexto se, e somente se, é reconhecida por um autômato com pilha.

Prova

A prova deste teorema é dividida em duas partes, que serão realizadas ao longo desta seção. Primeiro vamos mostrar que toda linguagem livre de contexto pode ser reconhecida por um PDA e depois vamos mostrar que toda linguagem reconhecida por um PDA é livre de contexto.

De modo similar ao que fizemos em outras provas deste tipo, vamos fazer a prova através da conversão dos PDA em outro formalismo que define as linguagens livres de contexto, as CFGs, e vice-versa.

Teorema

Se uma linguagem é livre de contexto, então ela é reconhecida por um autômato com pilha.

Prova

Dado uma CFG $G = (V, \Sigma, R, S)$, crie um autômato com pilha $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$ tal que:

- $Q = \{q_0, q_L, q_F\}$,
- $F = \{q_F\}$,
- $\Gamma = V \cup \Sigma \cup \{\$\}$,
- A função de transição tem quatro casos:
 - a. $\delta(q_0, \varepsilon, \varepsilon) = \{(q_L, S\$)\}$,
 - b. $\delta(q_L, \varepsilon, A) = \{(q_L, w)\}$, para cada regra $A \rightarrow w$ da gramática
 - c. $\delta(q_L, a, a) = \{(q_L, \varepsilon)\}$,

$$d. \delta(q_L, \epsilon, \$) = \{(q_F, \epsilon)\}.$$

Esse autômato usa o não-determinismo para realizar as produções da gramática na pilha e verifica se a entrada dada coincide com alguma produção. Quando a entrada coincide a pilha esvazia e o autômato aceita.

Vamos discutir um pouco acerca do autômato descrito anteriormente. A primeira regra de transição sai do estado inicial para q_L e inicializa a pilha com S , lembrando que S é a variável inicial da gramática. As regras de transição do caso (b), que implementam as regras de produção da gramática, trabalham apenas sobre a pilha. Se o símbolo no topo da pilha é uma variável, ela é substituída segundo a regra da gramática. Como o autômato é não-determinístico, se há mais de uma substituição para a mesma variável, surgem múltiplas “cópias” do autômato, cada uma com uma substituição possível.

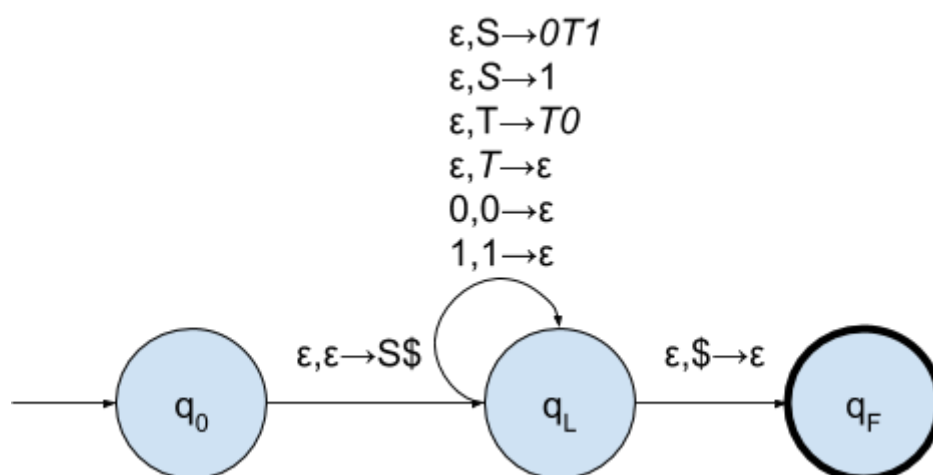
As regras do caso (c) lêem a entrada e verificam se ela coincide com o terminal no topo da pilha, neste caso o terminal lido da entrada é removido da pilha. Como o autômato gera todas as derivações possíveis, se a cadeia de entrada pode ser gerada pela gramática a pilha é esvaziada em algum ramo da computação e a regra do caso (d) leva o autômato ao estado de aceitação.

Exemplo

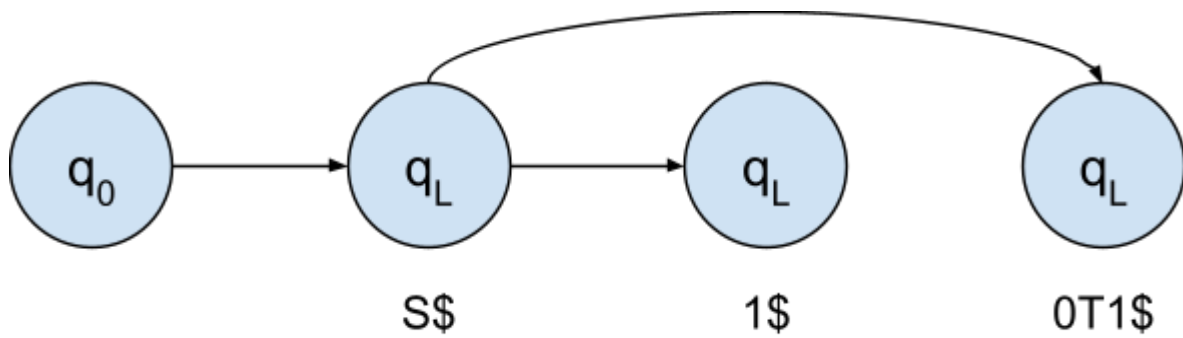
Converta a gramática $G = (\{S, T\}, \{0, 1\}, R, S)$ com regras

- $S \rightarrow 0T1|1$
- $T \rightarrow T0|\epsilon$

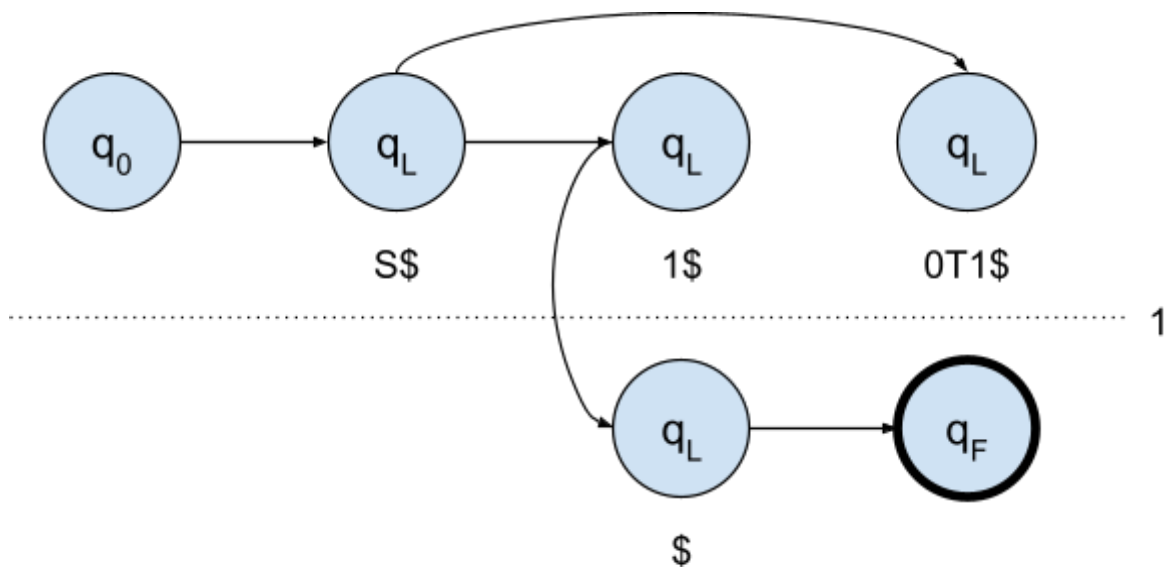
em um autômato com pilha. Seguindo a descrição dada na prova anterior, o autômato abaixo deve reconhecer a linguagem gerada por essa gramática.



Vamos detalhar o início da computação desse autômato, que ocorre antes mesmo de qualquer leitura da entrada.

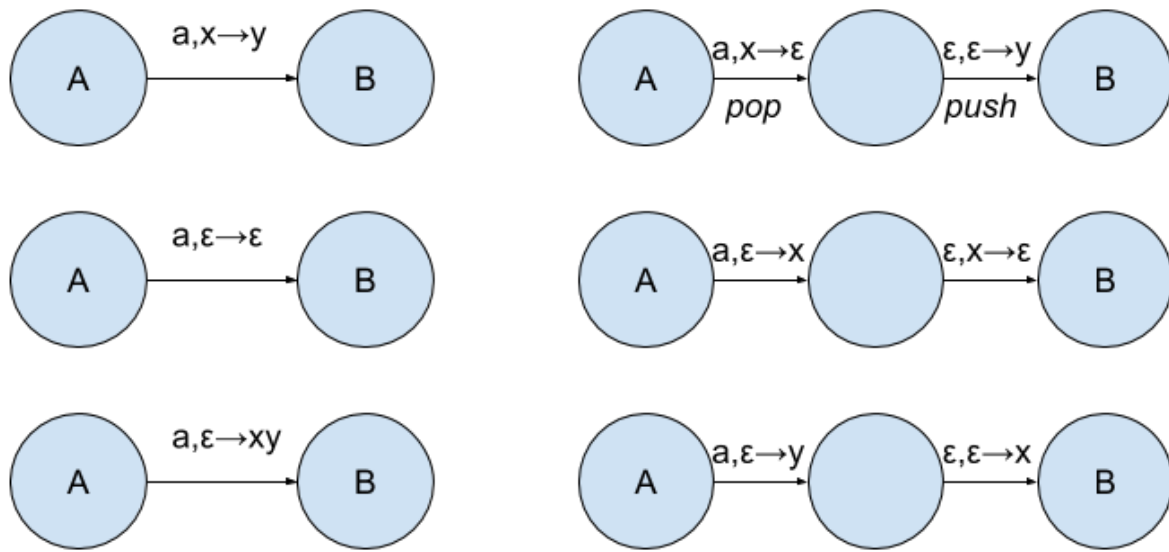


Se a cadeia de entrada inicia com 1 apenas o estado q_L com 1 no topo da pilha permanece ativo, e transita para o próprio estado q_L com apenas \$ na pilha. Neste momento uma transição vazia leva ao estado de aceitação, indicando que a cadeia 1 é aceita. De fato, essa é a única cadeia iniciada por 1 que é aceita por este autômato.



Se a cadeia de entrada inicia com 0 apenas o estado q_L com 0 no topo da pilha permanece ativo. Este estado transita para o estado q_L com T no topo da pilha, que, através das transições vazias, vai ativar infinitos estados simultâneos. Com pilhas terminando com $1\$$ e iniciando com toda quantidade de 0.

e removem o mesmo símbolo. Se a transição escreve mais de um símbolo na pilha, crie transições auxiliares para inserir os símbolos um a um.



Agora vamos criar uma gramática $G = (V, \Sigma, R, S)$ tal que:

- $V = \{A_{pq} : p, q \in Q\}$,
- $S = A_{q_0 q_f}$,
- Três tipos de regras serão criadas:
 - a. Para cada $p, q, r, s \in Q$, $t \in \Gamma$ e $a, b \in \Sigma_\epsilon$, se $(r, t) \in \delta(p, a, \epsilon)$ e $(q, \epsilon) \in \delta(s, b, t)$, crie uma regra $A_{pq} \rightarrow aA_{rs}b$.
 - b. Para cada $p, q, r \in Q$, crie uma regra $A_{pq} \rightarrow A_{pr}A_{rq}$.
 - c. Para cada $p \in Q$, crie uma regra $A_{pp} \rightarrow \epsilon$.

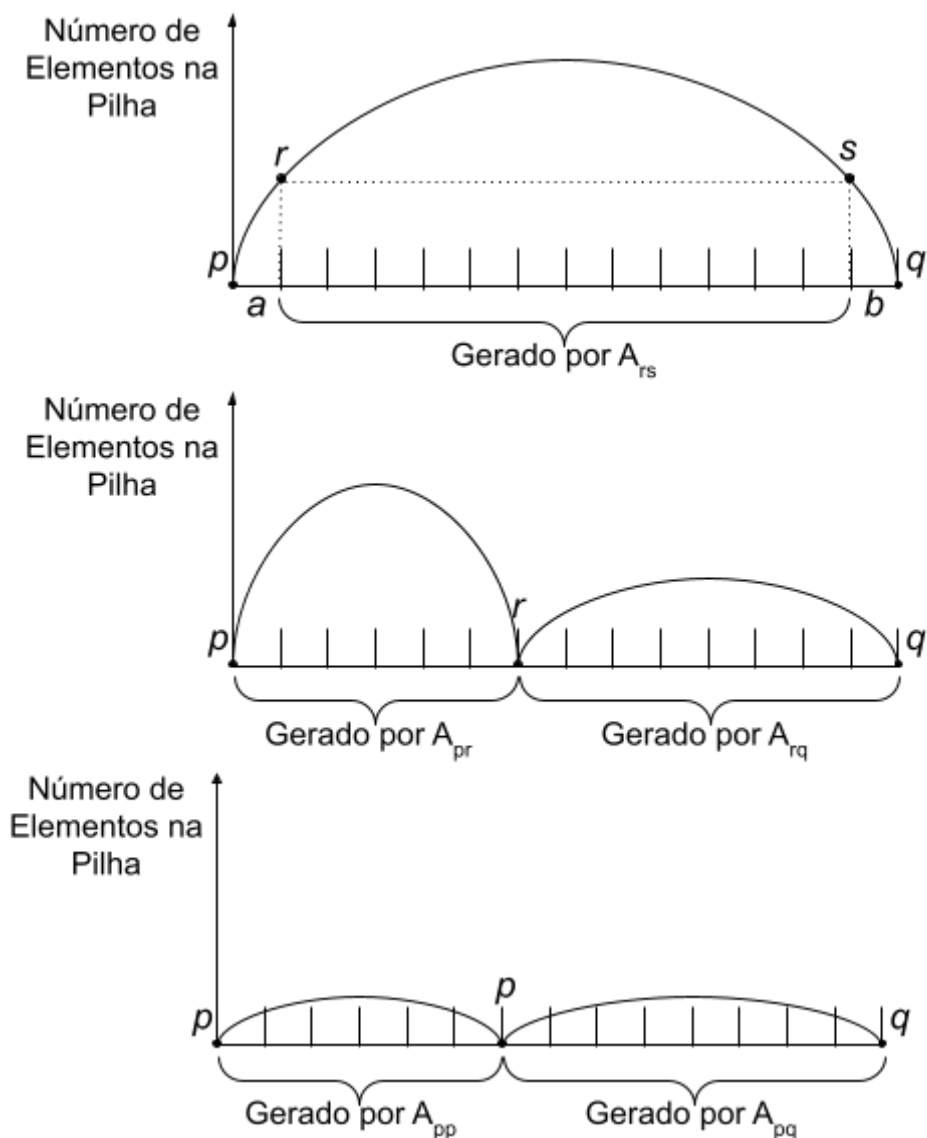
A linguagem gerada por G é a linguagem reconhecida por P .

A idéia desta construção é que a variável A_{pq} gera todas as cadeias que levam do estado p ao estado q , que mantêm a pilha inalterada.

As regras do tipo (a) consideram o caso em que a pilha só tem o mesmo conteúdo nos extremos entre p e q . Se o símbolo t é escrito na pilha ao ler a e transitar de p para r e é removido da pilha ao ler b e transitar de s para q , então as cadeias geradas por A_{rs} precedidas por a e seguidas por b levam do estado p para o estado q .

As regras do tipo (b) indicam que para ir de p a q com o mesmo conteúdo da pilha, é possível ir até um estado intermediário, r , que tem o mesmo conteúdo da pilha, e depois de r para q . Essa situação indica que em algum ponto no meio do processamento da cadeia a pilha se esvaziou.

As variáveis A_{pp} representam laços que não alteram a pilha, as regras do tipo (c) dizem que tais laços podem ser evitados.



Para convencê-lo de que essa construção de fato funciona, vamos apresentar uma prova de que A_{pq} gera x se, e somente se, x leva P do estado p ao estado q sem alterar o conteúdo da pilha.

Teorema

Dados o PDA P e a CFG G da prova do teorema anterior. Se A_{pq} gera x , então x leva P do estado p ao estado q sem alterar o conteúdo da pilha.

Prova

Vamos fazer uma prova por indução sobre o número de passos na derivação de x a partir de A_{pq} .

Base: A derivação tem 1 passo. Neste caso uma das regras $A_{pp} \rightarrow \varepsilon$ deve ser usada, o que quer dizer que $p = q$ e claramente o autômato não tem sua pilha alterada.

Passo: Assuma que a afirmação é verdadeira para derivações com menos de k passos.

Suponha que $A_{pq} \Rightarrow^* x$ com k passos. O primeiro passo dessa derivação é $A_{pq} \Rightarrow aA_{rs}b$ ou $A_{pq} \Rightarrow A_{pr}A_{rq}$.

No primeiro caso, vamos chamar de y a cadeia gerada por A_{rs} . Temos que $x = ayb$. Como $A_{rs} \Rightarrow^* y$ em $(k - 1)$ passos, pela hipótese de indução a pilha se mantém inalterada na transição de r para s através da leitura de y . Pela criação da gramática, sabemos que a existência da regra $A_{pq} \rightarrow aA_{rs}b$ implica que o autômato vai do estado p para o estado r lendo a da entrada e escrevendo t na pilha e vai do estado s para o estado q lendo b da entrada e removendo t da pilha. Então, ao transitar do estado p para q , a leitura de a insere um elemento t na pilha, a leitura de y mantém a pilha inalterada e a leitura de b remove o elemento t da pilha. Desse modo, antes a leitura de $x = ayb$ não altera o conteúdo da pilha.

No segundo caso, vamos chamar a cadeia gerada por A_{pr} de y e a cadeia gerada por A_{rq} de z . Temos então $x = yz$. Como $A_{pr} \Rightarrow^* y$ tem no máximo $(k - 1)$ passos e $A_{rq} \Rightarrow^* z$ também, pela hipótese de indução y leva o autômato do estado p ao estado r sem alterar a pilha e a leitura de z leva o autômato do estado r ao estado q sem alterar a pilha. Portanto, a leitura de $x = yz$ não altera o conteúdo da pilha.

Teorema

Dados o PDA P e a CFG G da prova do teorema anterior. Se x leva P do estado p ao estado q sem alterar o conteúdo da pilha, então A_{pq} gera x .

Prova

Vamos fazer a prova por indução sobre a quantidade de transições na computação de P ao ler a cadeia x e ir do estado p ao estado q sem alterar a pilha.

Base: A computação tem 0 passos. Neste caso o estado inicial e final é o mesmo, ou seja, $p = q$ e a cadeia lida só pode ser vazia, $x = \varepsilon$. Pela construção de G , existe a regra $A_{pp} \rightarrow \varepsilon$, portanto $A_{pp} \Rightarrow^* x$.

Passo: Assuma que a afirmação é verdadeira para computações com menos de k passos. Suponha que P tem uma computação que, ao ler a cadeia x , leva do estado p ao estado q sem alterar o conteúdo da pilha. Existem dois casos possíveis, ou a pilha só volta à condição inicial no fim da computação ou ela volta à condição inicial em algum ponto intermediário.

No primeiro caso, o símbolo, t , que é inserido na pilha no primeiro passo ao ler o símbolo a da entrada deve ser o mesmo que é removido no último passo ao ler o símbolo b da entrada. Considere que r é o estado alcançado após a leitura do primeiro símbolo da

entrada e que s é o estado anterior à leitura do último símbolo de x . Desse modo, pela criação da gramática, existe a regra $A_{pq} \rightarrow aA_{rs}b$. Seja $x = ayb$. Como a computação sobre a cadeia y tem $(k - 2)$ passos, pela hipótese de indução ela leva do estado r ao estado s sem alterar o conteúdo da pilha e $A_{rs} \Rightarrow^* y$. Portanto $A_{pq} \Rightarrow^* x$.

No segundo caso, seja r o estado onde a condição inicial da pilha é novamente alcançada. Seja y a subcadeia de x que leva do estado p ao estado r e z a subcadeia de x que leva do estado r ao estado q , de modo que $x = yz$. Pela hipótese de indução, temos que $A_{pr} \Rightarrow^* y$ e $A_{rq} \Rightarrow^* z$. Como a regra $A_{pq} \rightarrow A_{pr}A_{rs}$ faz parte da gramática, concluímos que $A_{pq} \Rightarrow^* x$.

Isto conclui a prova e temos então que as gramáticas livres de contexto e os autômatos com pilha representam o mesmo conjunto de linguagens, ou seja, as linguagens livres de contexto. Lembre que quando falamos de autômatos com pilha estamos nos referindo ao autômato não-determinístico. Já vimos, através dos exemplos, que a versão determinística não tem a mesma capacidade da versão não-determinística, ao contrário do que acontecia com os autômatos finitos. As linguagens reconhecidas por autômatos com pilha determinísticos é um subconjunto próprio das linguagens livres de contexto e são chamadas de linguagens livres de contexto determinísticas.

É possível provar que as linguagens livres de contexto determinísticas não são inerentemente ambíguas, ou seja, é sempre possível criar uma gramática não-ambígua para elas. Intuitivamente, isto vem do fato da ambiguidade surgir da necessidade de não-determinismo no PDA equivalente.

Lema do Bombeamento

As limitações dos autômatos finitos vinham do fato de sua memória ser limitada. Agora temos um autômato com uma memória auxiliar ilimitada, ainda existem linguagens que não podem ser reconhecidas por PDAs? A resposta é sim. Apesar da memória ser ilimitada, a forma de acesso a esta memória (LIFO) nos dá limitações sobre o que é possível fazer com ela.

Considere a linguagem $L_1 = \{\omega\omega : \omega \in \{0,1\}^*\}$. Já vimos que a linguagem $\{\omega\omega^R : \omega \in \{0,1\}^*\}$ pode ser reconhecida por um PDA no exemplo 3 dos PDAs, basta empilharmos os símbolos da entrada e depois ir desempilhando usando o não-determinismo para decidir onde é a metade da cadeia. Note que, ao desempilhar, a cadeia $\omega = w_1w_2 \dots w_k$ é verificada na ordem inversa do que foi lido pois o símbolo w_1 vai ser o primeiro a ser empilhado, ficando no fundo da pilha, e o símbolo w_k será o último a ser empilhado, ficando no topo da pilha. Não parece ter uma forma evidente de verificar a cadeia ω na ordem direta utilizando a pilha, portanto a linguagem L_1 parece não ser livre de contexto.

Considere a linguagem $L_2 = \{a^n b^n c^n : n \geq 0\}$. Esta linguagem tem quantidades iguais de a , b e c . Já vimos como utilizar a pilha como contador e até como comparar a quantidade de a com as quantidades de b ou c nos exemplos 2 e 5 dos PDAs. No entanto para garantir

que tanto os b quanto os c ocorrem na mesma quantidade dos a , seria necessário manter o conteúdo da pilha e, ao mesmo tempo, decrementá-la para comparar as quantidades. Uma “solução” possível seria considerar que o PDA só aceita se os dois estados de aceitação estão ativos ao fim da computação, mas isso altera a definição do autômato e não seria mais um PDA. Então, L_2 parece não ser livre de contexto porque a definição dos PDA só nos permite ter um contador e neste caso precisaríamos de 2.

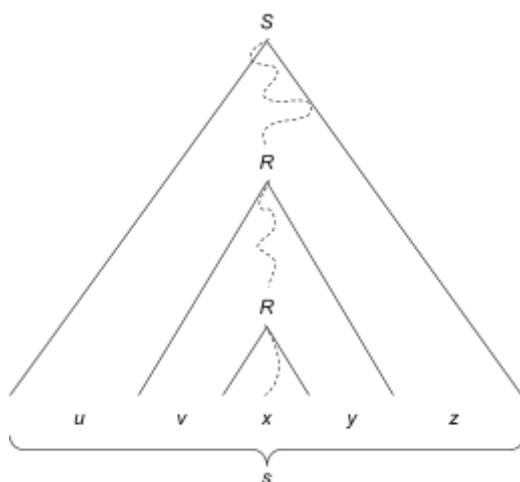
De modo similar ao que foi visto para as linguagens regulares, as linguagens livres de contexto também possuem uma versão da propriedade de *bombeamento*, ou seja, existem partes das cadeias que são maiores que um certo comprimento que podem ser repetidos e mantêm a nova cadeia na linguagem.

No caso das linguagens regulares, vimos que é impossível aceitar uma cadeia de comprimento maior ou igual ao número de estados sem que haja repetição de estados, e essa propriedade dá origem à propriedade de bombeamento. No caso das linguagens livres de contexto vamos considerar as CFGs.

Se a regra mais comprida da gramática tem b elementos do lado direito, isto quer dizer que a árvore sintática tem no máximo b descendentes do nível imediatamente inferior à variável inicial. No segundo nível temos no máximo b^2 descendentes e assim por diante, de modo que no nível n temos no máximo b^n ramos que partem da variável inicial.

Se uma gramática tem $|V|$ variáveis, o número máximo de níveis sem que ocorram repetições de variáveis é $|V|$. Se essa gramática gera alguma cadeia s de comprimento $b^{|V|} + 1$, a árvore sintática tem pelo menos $|V| + 1$ níveis, o que implica na repetição de pelo menos uma variável em algum ramo da árvore sintática, pelo princípio da casa dos pombos.

Na árvore sintática de s , os nós referentes à variável repetida, R , particionam os terminais que formam s em cinco partes. Como numa CFG não há restrição quanto ao lado direito das regras de produção, toda variável pode ter uma cadeia antes e/ou depois dela. Desse modo, a sequência de derivações que leva a variável inicial, S , até a primeira ocorrência da variável R gera a parte inicial e final da cadeia s . A sequência que leva R de volta a R gera a segunda e quarta parte da cadeia s . Finalmente, a derivação da primeira repetição de R até o fim da árvore sintática gera a parte central de s .



Algumas dessas partes podem ser vazias, a depender das regras da gramática, mas pelo menos v ou y têm comprimento positivo. Como estas partes são geradas pela sequência de derivações que produz novamente a variável R , elas podem ser repetidas indefinidamente no processo de derivação. Vamos agora enunciar e provar o lema do bombeamento para linguagens livres de contexto, que já deve estar claro a partir desta discussão.

Teorema

Se A é uma linguagem livre de contexto, existe um número p , chamado de comprimento de bombeamento, tal que toda cadeia s de A com $|s| \geq p$ pode ser escrita como $s = uvxyz$, onde

- $|vy| > 0$,
- $|vxy| \leq p$, e
- $uv^i xy^i z \in A$, para todo $i \geq 0$.

Prova

Seja G uma CFG que gera A . Pela discussão anterior, se b é o maior número de símbolos do lado direito de alguma regra de G , a derivação de qualquer cadeia s de comprimento maior ou igual a $b^{|V|} + 1$ deve ter ao menos uma variável R que se repete em algum ramo de sua árvore sintática.

Vamos considerar duas ocorrências de R em algum ramo. Abaixo da variável inicial temos toda a árvore sintática, que gera $s = uvxyz$. Abaixo da primeira ocorrência há uma subárvore que gera a cadeia vxy e Abaixo da segunda ocorrência há uma sub-árvore que gera a cadeia x .

Se v e y fossem ambos vazios, seria possível gerar a cadeia s sem a repetição de variáveis, o que é uma contradição. Portanto $|vy| > 0$.

Como vxy é gerado pela subárvore que inicia na primeira ocorrência de R , é sempre possível escolher esta ocorrência de R de modo que $|vxy| \leq p$, já que p é o comprimento que obriga a repetição.

As subárvore que geram x e vxy iniciam na mesma variável, R , portanto podemos substituir uma pela outra e ainda assim obter uma árvore sintática válida nesta gramática. Substituindo a menor na maior, obtemos a cadeia uxy e substituindo a maior na menor obtemos $uvvxyyz$, que por sua vez pode sofrer uma nova substituição para obtermos $uvvvxyyyz$. Seguindo este raciocínio, podemos obter árvores sintáticas válidas para qualquer cadeia da forma $uv^i xy^i z$, com $i \geq 0$. Daí concluímos que $uv^i xy^i z \in A$, para todo $i \geq 0$.

Este teorema, assim como o bombeamento nas linguagens regulares, indica uma condição necessária para a linguagem ser livre de contexto mas não suficiente. Vamos ver agora alguns exemplos da aplicação do teorema para provar que algumas linguagens não são livres de contexto.

Exemplo 1: $L_1 = \{\omega\omega : \omega \in \{0, 1\}^*\}$

Vamos assumir que essa linguagem é livre de contexto. Seja p o comprimento de bombeamento dado pelo teorema.

Considere a cadeia $s = 0^p 1^p 0^p 1^p$ de comprimento $4p$. Pelo teorema podemos dividir a cadeia em cinco partes, $s = uvxyz$, com $|vxy| \leq p$.

A subcadeia vxy não pode ficar na primeira metade de s porque o bombeamento ia alterar a primeira parte, que deixaria de ser $0^p 1^p$ que permanece intocado na segunda parte. Analogamente, não pode ficar na segunda metade de s porque alteraria a segunda parte, mantendo a subcadeia $0^p 1^p$ intocada na primeira parte.

Portanto, vxy só pode estar no meio da cadeia. No entanto, deste modo v será composto apenas por símbolos 1 e y apenas por símbolos 0. Assim, o bombeamento vai gerar cadeias onde a primeira parte tem uma quantidade de 1 diferente de 1^p , que permanece na segunda e a segunda parte terá uma quantidade de 0 diferente de 0^p , que permanece no início da cadeia.

Portanto, é impossível bombear esta cadeia de comprimento maior que p e L_1 não pode ser livre de contexto.

A escolha da cadeia é fundamental para a prova, se tivéssemos escolhido $s = 0^p 10^p 1$ seria possível bombear a cadeia fazendo com que x seja o primeiro 1 e v e y sejam formados por uma quantidade igual de zeros do primeiro e do segundo bloco 0^p , respectivamente.

Exemplo 2: $L_2 = \{a^n b^n c^n : n \geq 0\}$

Vamos assumir que L_2 é livre de contexto e p é o comprimento de bombeamento dado pelo teorema. Considere a cadeia $s = a^p b^p c^p$ de comprimento $3p$. Ao dividir a cadeia s , se as subcadeias v e y são formadas por um único tipo de símbolo, ao bombear podemos manter iguais as quantidades de no máximo dois dos 3 símbolos, e as cadeias bombeadas não farão parte da linguagem.

A outra possibilidade é que pelo menos um dos dois, v ou y , é formado por mais de um tipo de símbolo, neste caso o bombeamento fará com que os símbolos fiquem fora da ordem estipulada pela linguagem, e também não faz parte de L_2 .

Como estes são todos os casos possíveis, é impossível bombear esta cadeia de comprimento maior que p e concluímos que L_2 não é livre de contexto.

Exemplo 3: $L_3 = \{a^i b^j c^k : 0 \leq i \leq j \leq k\}$

Como no caso anterior, vamos assumir que L_3 é livre de contexto, p é o comprimento de bombeamento e $s = a^p b^p c^p$.

Se v ou y é formado por mais de um tipo de símbolo, o bombeamento fará com que os símbolos fiquem fora da ordem estipulada pela linguagem, e não faz parte de L_3 .

Se v e y são formados por um único tipo de símbolo temos três casos possíveis:

1. a não ocorre em v ou y . Ao remover v e y teremos menos b e c que a , violando a regra da linguagem.
2. b não ocorre em v ou y . Ao duplicar v e y teremos mais a que b , violando a linguagem.
3. c não ocorre em v ou y . Ao duplicar v e y teremos mais a que c , violando a linguagem.

Portanto é impossível bombear s e L_3 não é livre de contexto.

Exemplo 4: $L_4 = \{a^m b^n c^{mn} : m, n \in \mathbb{N}\}$

Esta linguagem representa o problema de verificar se um produto está correto, a quantidade de a é o primeiro operando, a quantidade de b é o segundo operando e a quantidade de c é o produto. Então verificar se $aabbcccc$ faz parte da linguagem é equivalente a verificar se $2 \times 2 = 4$. Já vimos que a linguagem que verifica a soma é livre de contexto (Exemplo 7 das CFGs), vamos provar que a verificação do produto não é.

Considere que L_4 é livre de contexto, p é o comprimento de bombeamento e a cadeia $s = a^p b^p c^{p^2}$ de comprimento $p^2 + 2p$.

Se v e y contêm apenas a e b ou se contêm apenas c , estamos alterando apenas os operandos ou apenas o produto, e claramente o resultado do bombeamento estará fora da linguagem. Além disso v e y só podem ser formados por um tipo de símbolo, senão o bombeamento criaria cadeias fora da ordem estipulada pela linguagem.

Nossa única chance de conseguir um bombeamento nesta linguagem é se v é formado apenas por símbolos b e y por símbolos c . Daí concluímos que $vy = b^j c^k$ e $uxz = a^p b^{p-j} c^{p^2-k}$, com $j > 0$ e $k > 0$. Como uxz deve fazer parte da linguagem, temos que $p(p-j) = p^2 - k$, que implica que $pj = k$. Portanto $|vxy| \geq |vy| = j + k = j + pj > p$. Mas, pelo lema do bombeamento, $|vxy| \leq p$.

Daí concluímos que não é possível bombear esta cadeia e L_4 não é livre de contexto.

Outra forma de pensar sobre a impossibilidade de bombeamento é que ao replicar v e y vamos somar (ou subtrair) uma quantidade j a um dos operandos e uma outra quantidade k ao produto. Temos então que achar valores para j e k tais que $p(p + (i-1)j) = p^2 + (i-1)k$ para todo $i \geq 0$. Chegamos à mesma conclusão que $k = jp$ e $|vy| = j + k = j(p+1) > p$.

Exemplo 5: $L_5 = \{1^n : n \text{ é um número primo}\}$

Já vimos que esta linguagem não é regular (exemplo 6 do lema do bombeamento para linguagens regulares). Vamos agora mostrar que ela também não é livre de contexto.

Assuma que L_5 é livre de contexto, p é o comprimento de bombeamento e a cadeia $s = 1^k$, onde $k \geq p$ é um número primo. Neste caso só há o símbolo 1, então a única coisa que sabemos é que $|vy| \geq 1$. Para mostrar que esta linguagem não é livre de contexto devemos achar um valor para i tal que $|uv^i xy^i z|$ não é primo.

Temos que $|uv^i xy^i z| = |s| + (i - 1)|vy| = k + (i - 1)|vy|$. Para $i = k + 1$ temos $|uv^i xy^i z| = k + (k + 1 - 1)|vy| = k(1 + |vy|)$, que não pode ser primo pois é expresso através do produto de dois números maiores que 1. Daí, concluímos que L_5 não é livre de contexto.

Exemplo 6: $L_6 = \{a^j b^k c^j d^k : j, k \geq 0\}$

Considere que L_6 é livre de contexto, p é o comprimento de bombeamento e a cadeia $s = a^p b^p c^p d^p$ de comprimento $4p$.

Como $|vxy| \leq p$, é impossível que v e y tenham, respectivamente, elementos de a e c ou elementos de b e d . Desse modo, qualquer tentativa de bombeamento vai desbalancear estas quantidades e a cadeia resultante estará fora da linguagem.

Deste modo, concluímos que L_6 não é livre de contexto.

Vimos no lema do bombeamento para linguagens regulares que as operações regulares podem nos ajudar nas provas de não-regularidade. Por isso, na seção seguinte vamos investigar as propriedades de fechamento do conjunto das linguagens livres de contexto. Além de auxiliar nas provas, as propriedades de fechamento podem auxiliar no projeto de CFGs.

Propriedades de Fechamento

Vamos considerar nesta seção algumas operações sob as quais as linguagens livres de contexto são fechadas ou não.

Teorema

As linguagens livres de contexto são fechadas sob a operação de união.

Prova

Dadas linguagens livres de contexto L_1 e L_2 , tais que L_1 é gerada por $G_1 = (V_1, \Sigma_1, R_1, S_1)$ e L_2 é gerada por $G_2 = (V_2, \Sigma_2, R_2, S_2)$. Faça V_2 disjunto de V_1 alterando

em V_2 os nomes de todas as variáveis que também ocorrem em V_1 e atualize as regras de R_2 para os novos nomes. A gramática $G = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, R, S)$, com $R = R_1 \cup R_2 \cup \{S \rightarrow S_1 S_2\}$ gera a união de L_1 e L_2 .

Teorema

As linguagens livres de contexto são fechadas sob a operação de concatenação.

Prova

Dadas linguagens livres de contexto L_1 e L_2 , tais que L_1 é gerada por $G_1 = (V_1, \Sigma_1, R_1, S_1)$ e L_2 é gerada por $G_2 = (V_2, \Sigma_2, R_2, S_2)$. Faça V_2 disjunto de V_1 alterando em V_2 os nomes de todas as variáveis que também ocorrem em V_1 e atualize as regras de R_2 para os novos nomes. A gramática $G = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, R, S)$, com $R = R_1 \cup R_2 \cup \{S \rightarrow S_1 S_2\}$ gera a concatenação de L_1 e L_2 .

Teorema

As linguagens livres de contexto são fechadas sob a operação estrela.

Prova

Dada uma linguagem livre de contexto L_1 tal que L_1 é gerada por $G_1 = (V_1, \Sigma_1, R_1, S_1)$. A gramática $G = (V_1 \cup \{S\}, \Sigma_1, R, S)$, com $R = R_1 \cup \{S \rightarrow S_1 S | \varepsilon\}$ gera a estrela de L_1 .

Teorema

As linguagens livres de contexto são fechadas sob a operação de reversão.

Prova

Dada uma linguagem livre de contexto L_1 tal que L_1 é gerada por $G_1 = (V_1, \Sigma_1, R_1, S_1)$. A gramática G , obtida a partir da reversão do lado direito de cada uma das regras de G_1 gera a linguagem L_1^R .

Do ponto de vista formal, estas provas não estão completas pois seria necessário provar, por indução na maior parte dos casos, que as construções propostas funcionam.

Além disso, existem outras operações sob as quais as linguagens livres de contexto são fechadas.

Vamos agora ver duas operações sob as quais as linguagens livres de contexto não são fechadas

Teorema

As linguagens livres de contexto não são fechadas sob a operação de interseção.

Prova

Vamos fazer uma prova por absurdo, dando um contra-exemplo. O que este teorema diz é que a interseção de duas linguagens livres de contexto pode não ser uma linguagem livre de contexto. Considere $L_1 = \{a^n b^n c^m : n, m \geq 0\}$ e $L_2 = \{a^n b^m c^n : n, m \geq 0\}$, já sabemos que ambas são livres de contexto, a primeira é uma linguagem com quantidades iguais de a e b e a outra é uma linguagem com quantidades iguais de a e c (ver exemplo 5 dos PDAs). A interseção destas linguagens é $L_1 \cap L_2 = \{a^n b^n c^n : n \geq 0\}$, que sabemos que não é livre de contexto (exemplo 2 do lema do bombeamento para linguagens livres de contexto). Portanto temos um contra-exemplo e concluímos que as linguagens livres de contexto não são fechadas sob a operação de interseção.

Teorema

As linguagens livres de contexto não são fechadas sob a operação de complemento.

Prova

Assuma que as linguagens livres de contexto são fechadas sob a operação de complemento. Sejam L_1 e L_2 linguagens livres de contexto. Pela hipótese, temos que $\neg L_1$ e $\neg L_2$ são livres de contexto, e daí $\neg L_1 \cup \neg L_2$ também é livre de contexto. Finalmente, temos que $\neg(\neg L_1 \cup \neg L_2)$ também é livre de contexto, mas, pelo teorema de DeMorgan, $\neg(\neg L_1 \cup \neg L_2) = L_1 \cap L_2$, que sabemos que não é necessariamente livre de contexto. Portanto, chegamos em um absurdo e provamos que as linguagens livres de contexto não são fechadas sob a operação de complemento.

Por fim, temos um caso particular da interseção.

Teorema

A interseção de uma linguagem regular com uma linguagem livre de contexto é uma linguagem livre de contexto.

Prova

Dadas uma linguagem livre de contexto L_1 , reconhecida pelo PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$, e uma linguagem regular L_2 , reconhecida pelo DFA $D = (S, \Sigma, \gamma, s_0, A)$ Vamos mostrar como construir um PDA que simula a execução simultânea das duas máquinas e só aceita se ambas aceitam. A construção será similar ao método usado para simular duas DFAs simultaneamente.

Crie um PDA $P' = (Q', \Sigma, \Gamma, \delta', q_0', F')$ tal que:

- $Q' = Q \times S$,
- $q_0' = (q_0, s_0)$,
- $F' = F \times A$,
- $\delta'((q_i, s_i), a, x)$ é formado por todos os pares $((q_j, s_j), y)$ tais que
 - $\gamma(s_i, a) = s_j$, e
 - $(q_j, y) \in \delta(q_i, a, x)$.

Cada estado de P' é uma par onde o primeiro estado representa um estado de P e um estado de D . O estado inicial de P' é o par que representa os estados iniciais de P e D . Os estados de aceitação de P são os pares com estados de aceitação tanto de P quanto de D , deste modo P' só aceita de ambos aceitam.

A função de transição de P' executa as transições de P e D simultaneamente. Como apenas P escreve na pilha, basta acrescentarmos o estado para o qual D transita ao estado para o qual P transita.

Particularmente, note que D não efetua transições vazias, portanto nestes casos s_j será igual a s_i , na função de transição de P' .

P' aceita a interseção das linguagens L_1 e L_2 , e como é um PDA, concluímos que esta interseção é uma linguagem livre de contexto.

Estas propriedades podem ajudar tanto na prova de que uma linguagem não é livre de contexto quanto não projeto de tais linguagens. Considere a linguagem $\{a^i b^j c^k : i \neq j \text{ ou } j \neq k\}$, por exemplo. Esta linguagem pode ser expressa como a união de $\{a^i b^j c^k : i < j\}$, $\{a^i b^j c^k : i > j\}$, $\{a^i b^j c^k : j < k\}$ e $\{a^i b^j c^k : j > k\}$, se conseguimos criar uma CFG para cada uma destas linguagens, realizar a união é bastante simples.

Aplicações

As linguagens livres de contexto foram tratadas de um ponto de vista puramente matemático, como um conjunto de subconjuntos de Σ^* . Originalmente as CFGs foram criadas por Chomsky para aplicações em linguística, na modelagem de linguagens naturais, mas depois foi verificado que nem todas as nuances das linguagens naturais podem ser capturadas pelas CFGs. Apesar disso, muitas estruturas das linguagens naturais são de fato representadas pelas CFGs e de fato este formalismo ainda é usado no processamento

de linguagem natural. Mas sem dúvida foi na matemática, particularmente na computação, onde as linguagens livres de contexto encontraram seu lugar.

Primeiramente, do ponto de vista da teoria da computação, os PDAs são máquinas mais poderosas que os autômatos finitos, possuem memória ilimitada mas a forma de acesso a essa memória ainda limita estas máquinas. Segundo, como as linguagens representam problemas, esta classe de linguagens representa mais problemas que a classe das linguagens regulares, que é um subconjunto próprio das linguagens livres de contexto. Isto quer dizer que os PDAs, ou as CFGs, podem resolver alguns problemas que os autômatos finitos não são capazes, e chegamos mais perto de responder a pergunta inicial desta disciplina, que é definir quais problemas podem ser resolvidos através de algoritmos. Sabemos agora que todos os problemas que podem ser representados por uma linguagem livre de contexto podem ser resolvidos algoritmicamente.

A aplicação real das linguagens livres de contexto são similares ao seu objetivo inicial. Apesar de não serem adequadas para representar todas as características das linguagens naturais, as CFGs são capazes de representar satisfatoriamente muitas linguagens artificiais, como linguagens de marcação, lógica e linguagens de programação. As regras de formação das expressões da lógica de primeira ordem, das expressões regulares e das fórmulas aritméticas encaixam-se nas definições das CFGs. Então, de modo geral, podemos dizer que as definições de muitas linguagens artificiais podem ser feitas através das CFGs. A DTD (*Document Type Definition*) das XML (*Extensible Markup Language*) é essencialmente uma CFG, por exemplo.

Vamos agora ver uma aplicação que será bastante importante no estudo de compiladores e também no processamento de linguagem natural.

Análise Sintática (*Parsing*)

Como discutimos, muitas linguagens (ou partes delas) podem ser descritas através de CFGs. A análise sintática consiste em, dado uma cadeia e uma gramática, definir a árvore sintática que deu origem àquela cadeia. Esta é a segunda etapa de um compilador, após a análise léxica discutida nas aplicações das linguagens regulares.

Pensando um pouco, é fácil perceber que muitos aspectos das linguagens de programação ou das linguagens de marcação funcionam de forma similar à idéia de parênteses balanceados. As *tags* em HTML (*HyperText Markup Language*) e blocos de código em linguagens de programação são exemplos disso. A gramática para expressões aritméticas apresentada no exemplo 10 das CFGs também pode ser útil em uma linguagem de programação.

E qual a necessidade da análise sintática? Esta análise facilita o entendimento da função de cada *token* na expressão. A árvore sintática das expressões aritméticas, citada anteriormente, permite a identificação da precedência e da ordem na qual as operações devem ser executadas para a obtenção do resultado correto. A árvore sintática de uma frase de alguma linguagem natural permite identificar quem é o sujeito, o predicado, o verbo e assim facilitar o entendimento (semântica) da sentença. A árvore sintática de um documento XML nos ajuda a identificar os componentes do documento, assim como a relação de aninhamento entre eles. Além disso, a análise sintática nos permite identificar erros sintáticos como parênteses abertos ou função sem operando.

O algoritmo CYK, discutido na seção acerca da forma normal de Chomsky, pode ser facilmente modificado para gerar a árvore sintática da cadeia ao determinar se ela pertence à linguagem da gramática. Também é possível usar analisadores sintáticos do tipo

top-down, ou seja, que fazem uma busca a partir da variável inicial pela sequência de substituições que geram a cadeia desejada.

Se a gramática é ambígua, não sabemos qual das árvores sintáticas será retornada pelos algoritmos. Isto é um problema? Depende da semântica, ou seja, depende do significado daquela gramática. Ambiguidade em uma gramática que define expressões aritméticas envolvendo soma e subtração não é relevante, já numa expressão envolvendo outras operações, produto por exemplo, pode levar a resultados errados.

Nas aplicações em processamento de linguagem natural é muito comum a utilização de uma versão probabilística das CFGs, e buscamos a árvore sintática mais provável para a sentença.

Considerações Finais

Neste capítulo vimos que o conjunto das linguagens livres de contexto é mais expressivo que o conjunto das linguagens regulares. De fato, as linguagens regulares são um subconjunto próprio das livres de contexto.

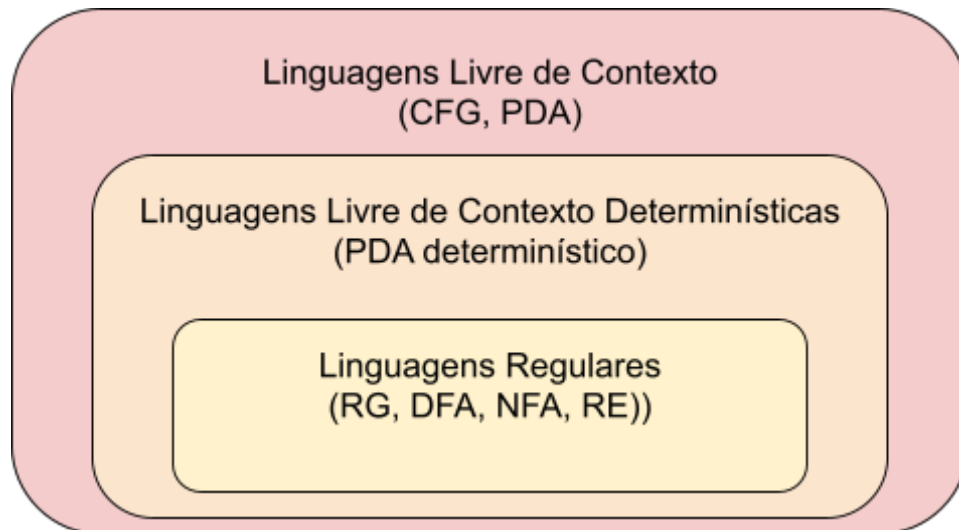
Vimos dois formalismos equivalentes para gerar ou reconhecer esta linguagem, além de demonstrar como converter um formalismo em outro. As gramáticas livres de contexto geram as linguagens enquanto os autômatos com pilha dão soluções para os problemas.

Os autômatos com pilha são capazes de decidir se uma cadeia pertence ou não a uma dada linguagem livre de contexto. De forma até mais simples, pois às vezes a simulação de um PDA pode ser quase impossível, o algoritmo CYK também é capaz de determinar se uma dada cadeia pertence ou não à linguagem de uma gramática. De fato, as aplicações reais das linguagens livres de contexto se dão através das CFGs e não dos PDAs.

Pensando nos PDAs como modelos de computadores, percebemos que a classe das linguagens livres de contexto representam problemas que podem ser resolvidos com uma quantidade infinita de memória, mas que é acessada de forma restrita (LIFO). Mais que isso, podemos pensar que são problemas que podem ser resolvidos usando um único contador auxiliar que pode ser incrementado ou decrementado e o único teste que conseguimos fazer é se ele é igual a zero ou não. Ou ainda que podem ser resolvidos pelo armazenamento do reverso da cadeia lida da entrada.

Como já discutimos, a linguagem $a^n b^n c^n$, com $n \geq 0$, é um exemplo clássico de linguagem que não é livre de contexto. Apesar de ser aparentemente simples, para reconhecer esta linguagem utilizando uma máquina seriam necessários dois contadores similares àquele descrito no parágrafo anterior. Também há uma estrutura de repetição que gera as linguagens infinitas, um pouco mais complexa que aquelas das linguagens regulares, mas simples o suficiente para ser descrita através da propriedade do bombeamento.

É importante lembrar que os autômatos com pilha discutidos são não-determinísticos e que, ao contrário do caso dos autômatos finitos, os autômatos com pilha determinísticos não são equivalentes aos não-determinísticos. As linguagens descritas por PDAs determinísticos são chamadas de linguagens livres de contexto determinísticas, que são um subconjunto das linguagens livres de contexto e que têm a propriedade de não serem inerentemente ambíguas. A figura abaixo mostra as classes de linguagens que conhecemos até o momento.



No próximo capítulo vamos nos livrar da limitação na forma de acesso à memória e verificar se é possível reconhecer linguagens além das livre de contexto.

Linguagens Recursivamente Enumeráveis

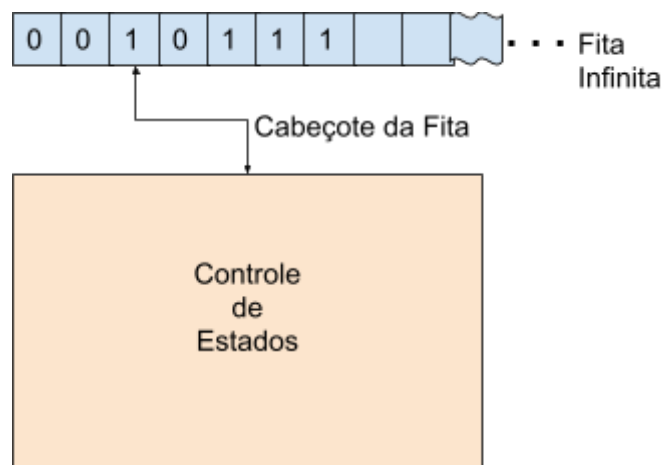
Vimos no capítulo anterior que os PDAs não são capazes de reconhecer a linguagem que envolve o balanceamento de 3 símbolos, $\{a^n b^n c^n : n \geq 0\}$, e hipotetizamos que seria possível se tivéssemos duas pilhas. Também levantamos a hipótese de que as limitações desses autômatos com pilha vêm da restrição na forma de acesso à memória.

A simples troca da pilha por outra forma de memória nos dá mais poderes? Se for necessário fazer o balanceamento de k símbolos vamos precisar de $(k - 1)$ pilhas? O acesso irrestrito a memória nos permite resolver qualquer problema? Tentaremos responder essas perguntas ao longo do resto deste texto.

Primeiramente, vamos apresentar uma nova máquina reconhecedora de linguagens e toda discussão partirá daí.

Máquinas de Turing (TM)

Este modelo de computação foi proposto por Alan Turing em 1936 e basicamente é um autômato finito com um acesso irrestrito à memória.



Ao contrário dos autômatos que foram discutidos até agora, o cabeçote da fita das máquinas de Turing (TM - *Turing Machines*) pode se mover em qualquer direção, permitindo que a entrada seja lida múltiplas vezes e em qualquer ordem. Além disso, a fita é infinita e funciona como a memória auxiliar da máquina, que por sua vez tem acesso irrestrito, já que o cabeçote funciona em ambas as direções.

Inicialmente a fita contém apenas a cadeia de entrada que pode ser editada pela máquina conforme a computação ocorre. A máquina lê algo da fita e então pode escrever, mover o cabeçote e mudar de estado conforme sua função de transição determinar.

Como a leitura da entrada pode ser feita várias vezes, a máquina continua computando até que entre em um estado de aceitação ou rejeição. Ao contrário dos autômatos, a máquina pára imediatamente ao entrar em um estado de aceitação, produzindo a saída *aceita*, ou de rejeição, produzindo a saída *rejeita*. Se a máquina não entrar em nenhum destes estados ela continuará computando para sempre, sem nunca parar ou produzir uma saída.

Definição Formal de uma Máquina de Turing

Como fizemos para as outras máquinas, vamos agora dar uma definição precisa das máquinas que discutimos brevemente nos parágrafos anteriores.

Definição: Máquina de Turing (TM)

Uma máquina de Turing (TM) é uma 7-upla $T = (Q, \Sigma, \Gamma, \delta, q_0, q_A, q_R)$, onde:

- Q é um conjunto finito, cujos elementos são chamados de *estados*;
- Σ é um conjunto finito, chamado *alfabeto de entrada*;
- Γ é um conjunto finito, chamado *alfabeto da fita*, tal que $\Sigma \subset \Gamma$ e $\sqcup \in \Gamma$
- $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{E, N, D\}$ é uma função, chamada de *função de transição*;
- $q_0 \in Q$, é chamado de *estado inicial*;
- $q_A \in Q$, é chamado de *estado de aceitação*;
- $q_R \in Q$, é chamado de *estado de rejeição*.

Como a entrada está inicialmente escrita na fita, devemos ter $\Sigma \subset \Gamma$. Além disso, Γ sempre tem um símbolo especial $\sqcup$, que indica que a célula está vazia. Como nos autômatos, a função de transição é o elemento essencial para definir o funcionamento desta máquina. Perceba que a função recebe um estado e um símbolo da fita, e retorna um estado, um símbolo da fita e um elemento do conjunto $\{E, N, D\}$. Este conjunto trata da movimentação do cabeçote da fita, E representa um movimento para a esquerda, N ficar parado e D para a direita. Portanto, $\delta(q_i, a) = (q_j, b, D)$ indica que se a máquina lê o símbolo a no estado q_i , ela transita para o estado q_j , escreve o símbolo b na fita, substituindo a , e move o cabeçote para a célula da direita.

O estado atual, o conteúdo da fita e a posição do cabeçote determinam uma *configuração* da máquina. Divida o conteúdo da fita em duas subcadeias, uma que vai do início da fita até a posição anterior ao cabeçote e outra que vai da posição do cabeçote ao fim do conteúdo da fita. Deste modo, podemos representar uma configuração como uqv , onde u é a primeira subcadeia, v é a segunda subcadeia e q é o estado atual. Podemos agora dar uma definição formal da computação em uma máquina de Turing.

Definição: Computação em uma TM

Seja $T = (Q, \Sigma, \Gamma, \delta, q_0, q_A, q_R)$ uma TM e $\omega \in \Sigma^*$. Sejam $a, b, c \in \Gamma$, $u, v \in \Gamma^*$ e $k > 0$. Dizemos que a configuração $uaq_i b v$ gera a configuração $uacq_j v$ se $\delta(q_i, b) = (q_j, c, D)$, $uaq_i b v$ gera a configuração $uq_j a c v$ se $\delta(q_i, b) = (q_j, c, E)$ e $uaq_i b v$ gera a configuração $uaq_j c v$ se $\delta(q_i, b) = (q_j, c, N)$. Como a fita é limitada à esquerda, vamos considerar que um movimento para a esquerda não tem efeito neste caso, ou seja, $q_i b v$ gera a configuração $q_j c v$ se $\delta(q_i, b) = (q_j, c, E)$. Ao processar a cadeia de entrada ω os seguintes passos ocorrem:

- $C_1 = q_0 \omega$;
- C_i gera C_{i+1} , as configurações são alteradas conforme as regras de transição.
- Se $C_k = xq_A y$, $x, y \in \Gamma^*$, a computação para T aceita ω .
- Se $C_k = xq_R y$, $x, y \in \Gamma^*$, a computação para T rejeita ω .

A definição da linguagem reconhecida pela máquina de Turing é a mesma que utilizamos para as outras máquinas

Definição: Linguagem Reconhecida por uma TM

Seja $T = (Q, \Sigma, \Gamma, \delta, q_0, q_A, q_R)$ uma TM. A *linguagem reconhecida* por T , denotada por $\mathcal{L}(T)$, é o conjunto de todas as cadeias sobre Σ^* aceitas por T : $\mathcal{L}(T) = \{x \in \Sigma^* : T \text{ aceita } x\}$.

A partir daí temos a definição das linguagens deste capítulo.

Definição: Linguagem Recursivamente Enumerável (Turing-reconhecível)

Uma linguagem A é dita *recursivamente enumerável (Turing-reconhecível)* se existe uma máquina de Turing T tal que $\mathcal{L}(T) = A$.

Ao processar uma cadeia usando uma máquina de Turing, além da aceitação e rejeição, existe a possibilidade da máquina nunca produzir uma saída, entrando num laço infinito. Às vezes, distinguir uma máquina em laço infinito de uma que está simplesmente demorando pode ser bem difícil. Por isso, as máquinas que sempre param (aceitando ou rejeitando) são preferíveis e são chamadas de *decisores*. Os autômatos que vimos anteriormente são exemplos de decisores. Dizemos que essas máquinas *decidem* a linguagem.

Se considerarmos apenas as máquinas de Turing que não entram em laço infinito, ou seja, as decisoras, temos um importante subconjunto das linguagens recursivamente enumeráveis.

Definição: Linguagem Recursiva (Turing-decidível)

Uma linguagem A é dita *recursiva (Turing-decidível)* se existe uma máquina de Turing T tal que $\mathcal{L}(T) = A$ e T nunca entra em laço infinito.

Exemplos de TM

A seguir seguem alguns exemplos de máquinas de Turing.

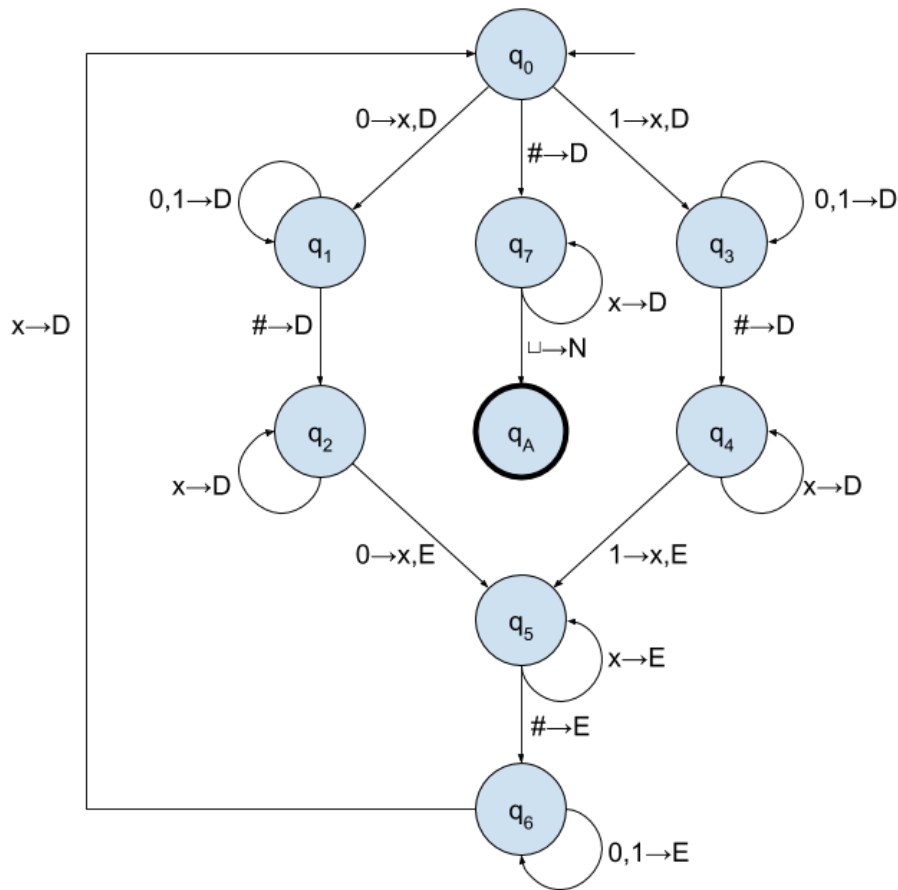
Exemplo 1: $A = \{\omega\#\omega : \omega \in \{0, 1\}^*\}$

Vamos projetar uma TM para decidir esta linguagem. A idéia é fazer um zigue-zague através da fita e verificar se os símbolos em posições correspondentes a partir do início da fita e a partir do # são iguais. Um procedimento mais detalhado, levando em conta as restrições de operação da TM seria.

1. Marque o primeiro símbolo da fita, mova o cabeçote para a direita até a célula imediatamente à direita de #.
 - a. Se não há #, rejeite.
 - b. Se o símbolo é diferente do que foi marcado anteriormente, rejeite.
 - c. Se o símbolo é igual ao que foi marcado, marque este símbolo também.
2. Retorne até o último símbolo marcado à esquerda de # e mova o cabeçote para a direita. Se o símbolo atual não é #, vá para o passo 3, senão, vá para o passo 4.
3. Marque o símbolo atual e mova o cabeçote até a célula imediatamente à direita do símbolo marcado mais à direita.
 - a. Se o símbolo é diferente do que foi marcado anteriormente, rejeite.
 - b. Se o símbolo é igual ao que foi marcado, marque este símbolo e volte para 2.
4. Vá até o fim do conteúdo da fita. Se há algum símbolo não marcado, rejeite.

O passo 1 verifica se há o #, como manda a linguagem e compara o primeiro símbolo. Os passos 2 e 3 formam um laço que compara os demais símbolos, um a um. Se a cadeia após # for menor que a cadeia anterior, ela será rejeitada pois algum símbolo da primeira cadeia será comparado com $\sqcup$. O passo 4 verifica se a cadeia após # é maior que a primeira cadeia.

Um diagrama de estados desta máquina é dado a seguir.



Antes de explicar o que cada estado está fazendo, preciso dar algumas explicações sobre os rótulos das transições. uma transição do tipo $a \rightarrow b, X$ indica que o símbolo a é lido, o símbolo b é escrito e o movimento X é efetuado. Portanto, $0 \rightarrow x, D$ indica que o símbolo 0 é lido, substituído por x e o cabeçote move-se para a direita. A transição $\# \rightarrow L$ indica que o símbolo $\#$ é lido e o cabeçote move-se para a esquerda, nenhuma escrita é efetuada. As transições que têm mais de um símbolo do lado esquerdo indicam que qualquer um dos símbolos lidos faz a mesma operação. $0, 1, x \rightarrow L$ indica que a transição ocorre se for lido 0, 1 ou x e o cabeçote moverá para a esquerda.

Outro ponto a se notar é que o estado de rejeição, q_R , não está no diagrama. Para evitar o excesso de transições, assume-se que todas as transições não presentes no diagrama levam para q_R . Em q_7 , por exemplo, qualquer leitura que não seja x ou $\sqcup$ levará para q_R . Agora já podemos explicar a máquina.

Os ramos $q_0 q_1 q_2 q_5$ verifica se o símbolo lido na primeira parte é 0 e o símbolo correspondente na segunda parte também é. O ramo $q_0 q_3 q_4 q_5$ faz o mesmo para o símbolo 1. O ramo $q_5 q_6 q_0$ faz o retorno ao primeiro símbolo não marcado. Juntos, estes estados executam os passos 1, 2 e 3 descritos anteriormente. Se o símbolo atual é $\#$, tudo antes de $\#$ já foi marcado e o autômato entra no ramo $q_0 q_7 q_A$, chegando em q_A somente se não há 0 ou 1 não marcado na fita.

A partir do diagrama fica claro que, na descrição formal, $Q = \{q_0, \dots, q_7, q_A, q_R\}$, $\Sigma = \{0, 1, \#\}$ e $\Gamma = \{0, 1, \#, x, \sqcup\}$. A função de transição é:

δ	q_0	q_1	q_2	q_3	q_4	q_5	q_6	q_7
0	(q_1, x, D)	$(q_1, 0, D)$	(q_5, x, E)	$(q_3, 0, D)$	$(q_R, 0, N)$	$(q_5, 0, E)$	$(q_6, 0, E)$	$(q_R, 0, N)$
1	(q_3, x, D)	$(q_1, 1, D)$	$(q_R, 1, N)$	$(q_3, 1, D)$	(q_5, x, E)	$(q_5, 1, E)$	$(q_6, 1, E)$	$(q_R, 1, N)$
#	$(q_7, \#, D)$	$(q_2, \#, D)$	$(q_R, \#, N)$	$(q_4, \#, D)$	$(q_R, \#, N)$	$(q_6, \#, E)$	$(q_R, \#, N)$	$(q_R, \#, N)$
x	(q_R, x, N)	(q_R, x, N)	(q_2, x, D)	(q_R, x, N)	(q_4, x, D)	(q_5, x, E)	(q_0, x, D)	(q_7, x, D)
$\sqcup$	$(q_R, \sqcup, N)$	$(q_R, \sqcup, N)$	$(q_R, \sqcup, N)$	$(q_R, \sqcup, N)$	$(q_R, \sqcup, N)$	$(q_R, \sqcup, N)$	$(q_R, \sqcup, N)$	$(q_A, \sqcup, N)$

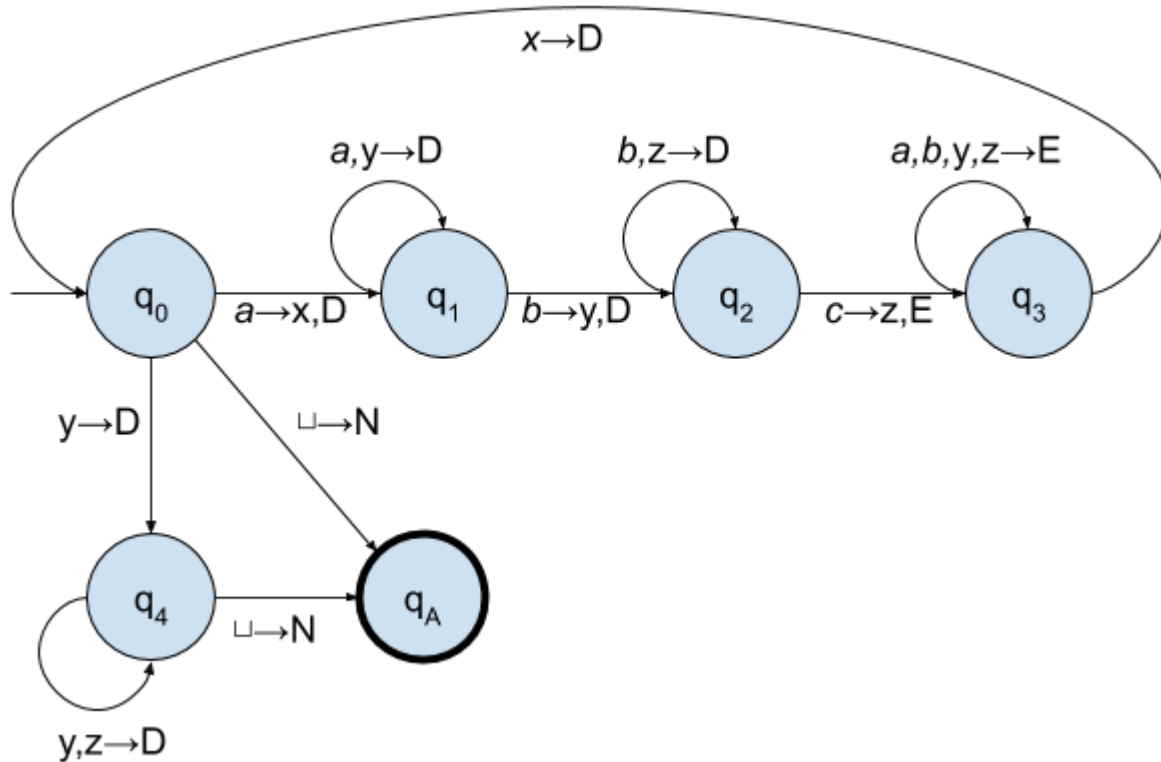
Ao contrário do diagrama, a função deve ser completamente definida para atender à definição formal. As transições sem escrita do diagrama são implementadas escrevendo de volta o mesmo símbolo lido. Ao entrar em q_R ou q_A é irrelevante o que é escrito ou o movimento realizado, pois a computação vai parar e teremos o retorno. Eu escolhi manter a fita inalterada e o cabeçote parado.

Para acompanhar a computação podemos usar uma sequência de configurações da máquina. Vamos verificar como se dá a computação para a cadeia 01#01.

$q_0 01\#01$	$xq_6 1\#x1$	$xx\#xq_4 1$	$xx\#q_7 xx$
$xq_1 1\#01$	$q_6 x1\#x1$	$xx\#q_5 xx$	$xx\#xq_7 x$
$x1q_1\#01$	$xq_0 1\#x1$	$xxq_5\#xx$	$xx\#xxq_7$
$x1\#q_2 01$	$xxq_3\#x1$	$xq_6 x\#xx$	$xx\#xxq_A$
$x1q_5\#x1$	$xx\#q_4 x1$	$xxq_0\#xx$	

Exemplo 2: $A = \{a^n b^n c^n : n \geq 0\}$

Essa linguagem pode ser decidida por uma máquina de Turing de forma similar à estratégia usada na abordagem anterior. A ideia é percorrer a fita diversas vezes, a cada vez um a , um b e um c são marcados. Caso uma das letras não esteja disponível para marcação, a cadeia é rejeitada.



O estado q_0 leva ao estado de aceitação se não há nada escrito na fita ($n = 0$). O ramo $q_0q_1q_2q_3$ marca um de cada um dos símbolos. O símbolo a , visto em q_0 é substituído por x e o cabeçote move-se para a direita até achar um símbolo b . O símbolo b , quando visto em q_1 é substituído por y e o cabeçote move-se para a direita até achar um símbolo c . Em q_2 o símbolo c é visto e substituído por z . Nesse momento o cabeçote retorna até encontrar um símbolo x (último a marcado) e o processo é reiniciado. Se todas as letras a foram marcadas, o estado q_0 verá um símbolo y e vai transitar para q_4 . Este estado verifica se todo o resto da cadeia foi marcado, aceitando a cadeia, ou se ainda há b ou c desmarcado, que é rejeitada.

Para acompanhar a computação podemos usar uma sequência de configurações da máquina. Vamos verificar como se dá a computação para a cadeia $aaabbbccc$:

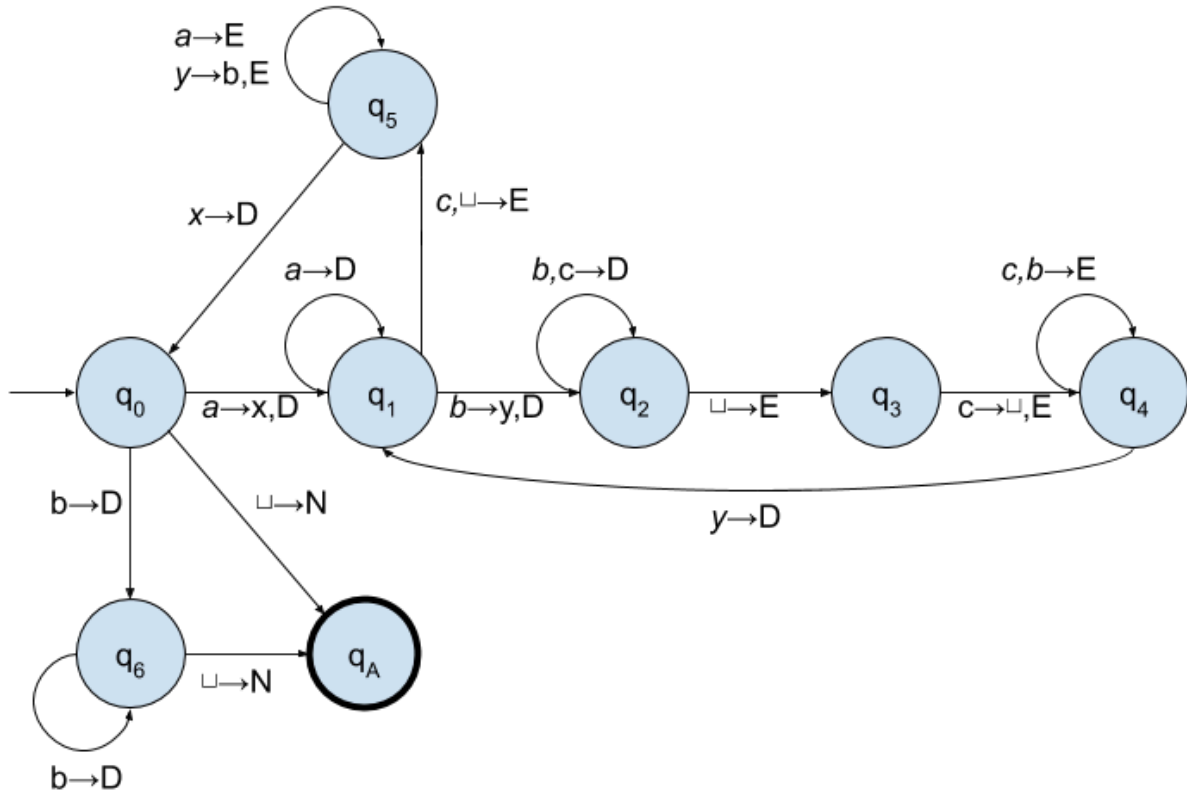
$q_0aaabbbccc$	$xaayq_3bbzcc$	$xxayq_1bbzcc$	$xxq_3aaybzzc$	$xxxyyyzzq_2c$	$xxxq_4yyzzz$
$xq_1aabbbccc$	$xaaq_3ybbzcc$	$xxayq_2bzcc$	$xq_3xaybzzc$	$xxxyyyq_3zz$	$xxxq_4yzzz$
$xaq_1abbbccc$	$xaq_3aybbzcc$	$xxayybq_2zcc$	$xxq_0aaybzzc$	$xxxyyyq_3zzz$	$xxxq_4yzzz$
$xaaq_1bbbccc$	$xq_3aaybbzcc$	$xxayybq_2cc$	$xxxq_1ybbzcc$	$xxxq_3yzzz$	$xxxq_4yzzz$
$xaayq_2bbccc$	$q_3xaaybbzcc$	$xxayybq_3zcc$	$xxxq_1ybbzcc$	$xxxq_3yzzz$	$xxxq_4yzzz$
$xaaybq_2bccc$	$xq_0aaybbzcc$	$xxayq_3bzcc$	$xxxq_1ybbzcc$	$xxxq_3yzzz$	$xxxq_4yzzz$
$xaaybbq_2ccc$	$xxq_1aybbzcc$	$xxayq_3ybzzc$	$xxxq_1ybbzcc$	$xxxq_3yzzz$	$xxxq_4yzzz$
$xaaybq_3bzzc$	$xxa_1ybbzcc$	$xxa_3ybbzcc$	$xxxq_1ybbzcc$	$xxxq_3yzzz$	$xxxq_4yzzz$

Exemplo 3: $A = \{a^m b^n c^{mn} : m, n \in \mathbb{N}\}$

Esta linguagem verifica o produto de dois números naturais. Para verificar o produto devemos ver se para cada a existe uma quantidade de c igual à quantidade de b . Além

disso, devemos aceitar cadeias formadas só por a , só por b ou a cadeia vazia, que correspondem respectivamente a $n = 0$, $m = 0$ e ambos iguais a zero.

A ideia é marcar um a , seguir até os b e, para cada b apagar um c . Para fazer isso vamos fazer um ziguezague entre os b e c , marcando um b e apagando um c . Depois disso restauramos todos os b , passamos para o próximo a e repetimos o processo. Depois que todos os a estiverem marcados, não deve existir nenhum c na cadeia e sabemos que o produto está correto.



A transição $q_0 q_A$ aceita a cadeia vazia, o ramo $q_0 q_6 q_A$ aceita cadeias formadas só por b . Nos demais casos, q_0 marca um a e passa para q_1 , que move o cabeçote para a direita enquanto houver a . Ao ver um b ele é marcado com y e passa para q_2 , que move o cabeçote para a direita até o fim da cadeia e volta uma posição, chegando em q_3 com o cabeçote no fim da cadeia. O estado q_3 apaga o último c e transita para q_4 , que retorna o cabeçote para a esquerda enquanto houver b ou c e transita para q_1 . Em q_1 , se ainda há b não marcado o processo é repetido, se não há mais b , o cabeçote é movido para a esquerda desmarcando os b até encontrar o último a marcado. Se ainda há a não marcados, o processo se repete. Se todos os a foram marcados a cadeia é aceita se não há c sobrando.

Para acompanhar a computação podemos usar uma sequência de configurações da máquina. Vamos verificar como se dá a computação para a cadeia $aabbcccc$:

$q_0 a a b b c c c c$	$x a y b c c c q_3 c$	$x a y y c c q_2 c$	$x q_5 a b b c c$	$x x y b q_4 c$	$x x y q_5 y$
$x q_1 a b b c c c c$	$x a y b c c c q_4 c$	$x a y y c c c q_2$	$q_5 x a b b c c$	$x x y q_4 b c$	$x x q_5 y b$
$x a q_1 b b c c c c$	$x a y b c q_4 c c$	$x a y y c c c q_3 c$	$x q_0 a b b c c$	$x x q_4 y b c$	$x q_5 x b b$
$x a y q_2 b c c c c$	$x a y b q_4 c c c$	$x a y y c q_4 c$	$x x q_1 b b c c$	$x x y q_1 b c$	$x x q_0 b b$
$x a y b q_2 c c c c$	$x a y q_4 b c c c$	$x a y y q_4 c c$	$x x y q_2 b c c$	$x x y y q_2 c$	$x x b q_6 b$
$x a y b c q_2 c c c$	$x a q_4 y b c c c$	$x a y q_4 y c c$	$x x y b q_2 c c$	$x x y y c q_2$	$x x b b q_6$

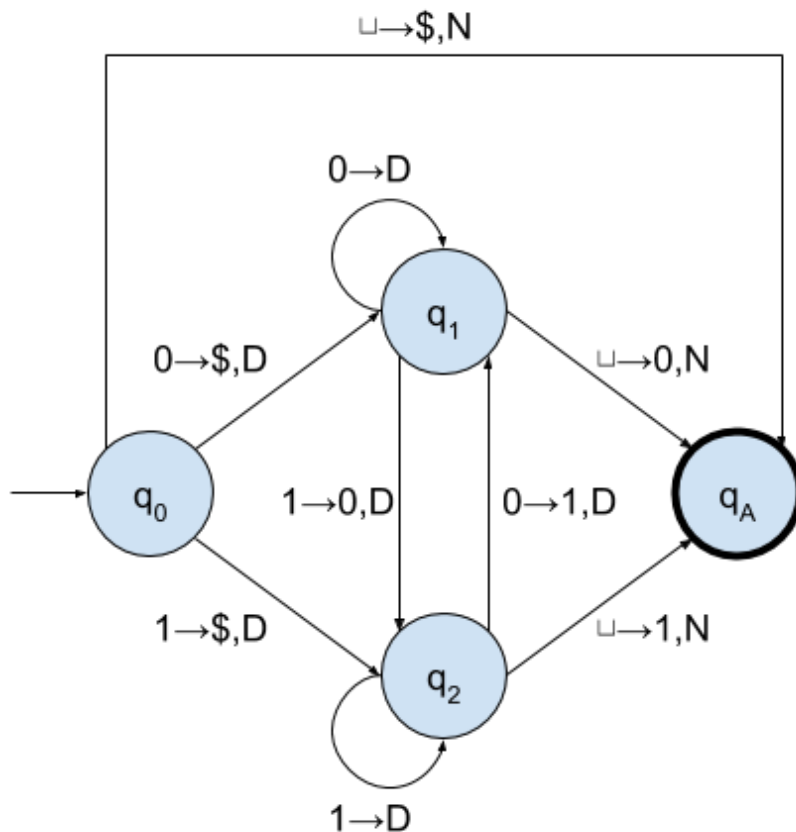
xaybccq ₂ cc	xayq ₁ bccc	xayyq ₁ cc	xxybcq ₂ c	xyyyq ₃ c	xxbbq _A
xaybcccq ₂ c	xayyq ₂ ccc	xayq ₅ ycc	xxybccq ₂	xxyq ₄ y	
xaybccccq ₂	xayycq ₂ cc	xaq ₅ ybcc	xxybcq ₃ c	xxyyq ₁	

Exemplo 4: $A = \{\omega\omega : \omega \in \{0, 1\}^*\}$

Perceba que essa linguagem é muito similar à linguagem do exemplo 1 mas não temos mais o símbolo # que demarcava o meio da cadeia. Então precisaremos primeiro determinar onde está o meio da cadeia.

Este exemplo será um pouco longo pois vou aproveitar para apresentar uma outra forma de encarar as máquinas de Turing. Até agora pensamos nas TM como máquinas que aceitam ou rejeitam uma dada cadeia de entrada e, nesse processo, o conteúdo da fita pode ser alterado. Outra forma de enxergar estas máquinas é que podemos projetá-las para aceitar somente quando um certo conteúdo estiver escrito na fita e o cabeçote estiver em uma determinada posição.

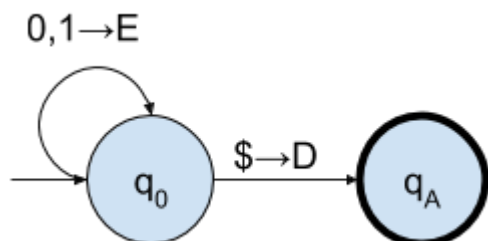
Um problema comum é tentar retornar ao início da fita, mas do jeito que a TM foi definida isso não é possível porque não tem nada que marque a primeira posição da fita. Se estamos na primeira posição e movemos para a esquerda, simplesmente ficamos no lugar mas não sabemos se estamos no primeiro símbolo da entrada. Para resolver esse problema, vamos criar uma máquina que, dada uma certa entrada, insere um símbolo novo (\$) no início da fita, deslocando todos os outros símbolos de uma casa para a direita.



Essa máquina, quando inicia com uma cadeia de $\{0,1\}^*$, lê o primeiro símbolo e substitui por \$. O estado q_1 lembra que o último símbolo lido foi 0 e o estado q_2 lembra que

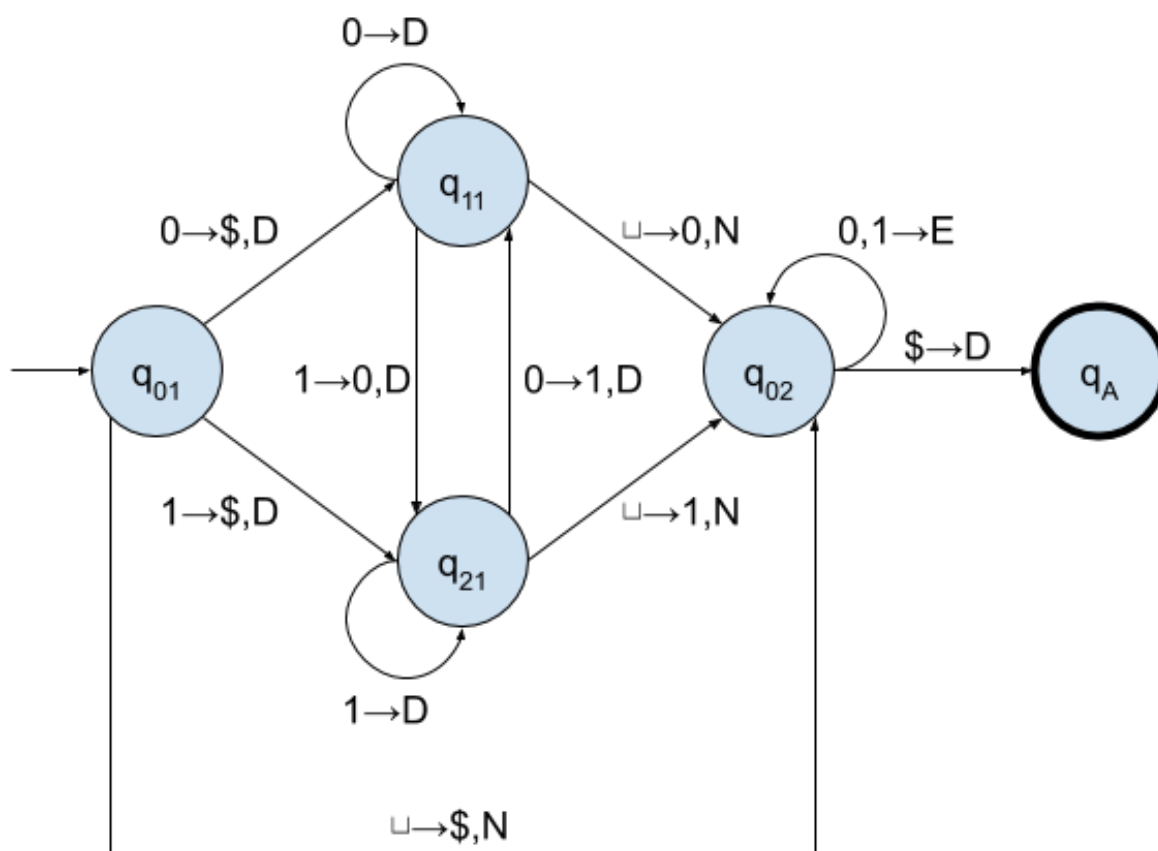
o último símbolo lido foi 1. Ao chegar no fim da cadeia é acrescentado o último símbolo lido e entra no estado de aceitação. Portanto, se a entrada é a cadeia 01101 ao fim do processamento a fita terá escrito \$01101 e o cabeçote estará na última posição da cadeia. Ou seja, a configuração inicial é q_001101 e a configuração final é $\$0110q_A1$.

Agora considere a seguinte máquina:



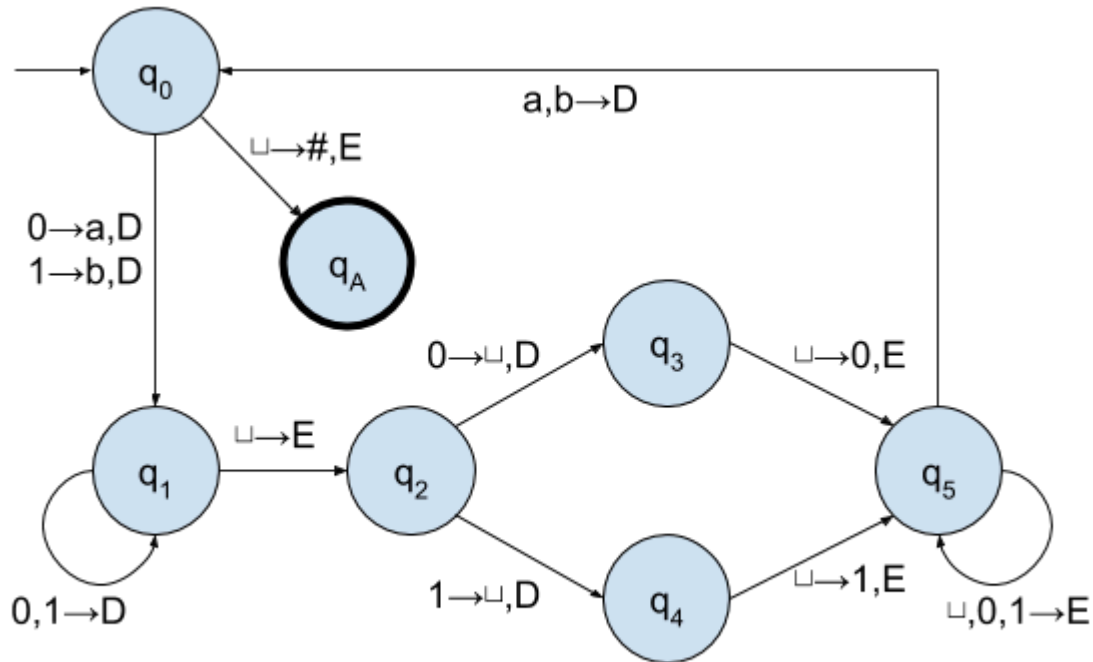
Essa máquina move o cabeçote para a esquerda até achar um \$ e aí move para a direita e aceita. Pensando de forma isolada, ela aceita qualquer cadeia que inicie com \$.

Agora, o que acontece se conectarmos estas duas máquinas em série, ou seja, substituiremos o q_A da primeira máquina pelo q_0 da segunda.



Segundo o que foi dito anteriormente, se a configuração inicial é $q_{01}01101$, ao entrar no estado q_{02} estamos na configuração $\$0110q_{02}1$. A partir daí o cabeçote move para a esquerda até encontrar um \$ (que agora marca o início da fita) e daí move-se para a direita, portanto a configuração final será $\$q_A01101$.

Desse modo, é como se a “saída” da primeira máquina (configuração final) fosse usada como entrada para a segunda máquina. Apresentada a idéia, vamos projetar uma forma de identificar o meio de uma cadeia de comprimento par.



Essa máquina inicia em q_0 , marcando o símbolo mais à esquerda (0 vira a e 1 vira b) e depois vai até o último símbolo da cadeia, chegando em q_2 . Nesse ponto o último símbolo é deslocado uma célula para a direita e o cabeçote retorna até o símbolo não marcado mais à esquerda. O processo se repete e, como marcamos um símbolo mais à esquerda e depois deslocamos o símbolo contíguo mais à direita, a cada iteração dois símbolos são afetados. Se a cadeia tem comprimento par, em algum momento o símbolo não marcado mais à esquerda será vazio e estaremos no meio da cadeia. Nesse momento vamos inserir o símbolo # e encerrar o processamento. Se a cadeia tem comprimento ímpar ela será rejeitada.

Para ilustrar, se a cadeia inicial é 011 o processamento se dá da seguinte forma:

$q_0 010$	$a1q_1 0$	$a1q_2 0$	$a1q_5 \sqcup 0$	$q_5 a1 \sqcup 0$	$abq_1 \sqcup 0$
$aq_1 10$	$a10q_1$	$a1 \sqcup q_3$	$aq_5 1 \sqcup 0$	$aq_0 1 \sqcup 0$	$aq_2 b \sqcup 0$
					(rejeita)

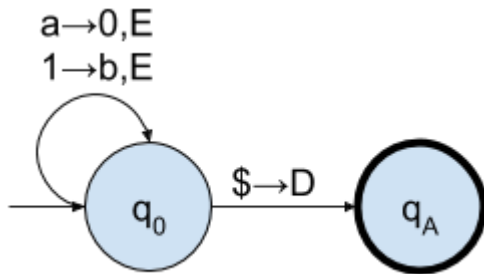
O processamento da cadeia 0110 se dá da seguinte forma:

$q_0 0110$	$a110q_1$	$a1q_5 1 \sqcup 0$	$abq_1 1 \sqcup 0$	$abq_5 \sqcup 10$
$aq_1 110$	$a11q_2 0$	$aq_5 11 \sqcup 0$	$ab1q_1 \sqcup 0$	$aq_5 b \sqcup 10$
$a1q_1 10$	$a11 \sqcup q_3$	$q_5 a11 \sqcup 0$	$abq_2 1 \sqcup 0$	$abq_0 \sqcup 10$
$a11q_1 0$	$a11q_5 \sqcup 0$	$aq_0 11 \sqcup 0$	$ab \sqcup q_4 \sqcup 0$	$aq_A b \# 10$

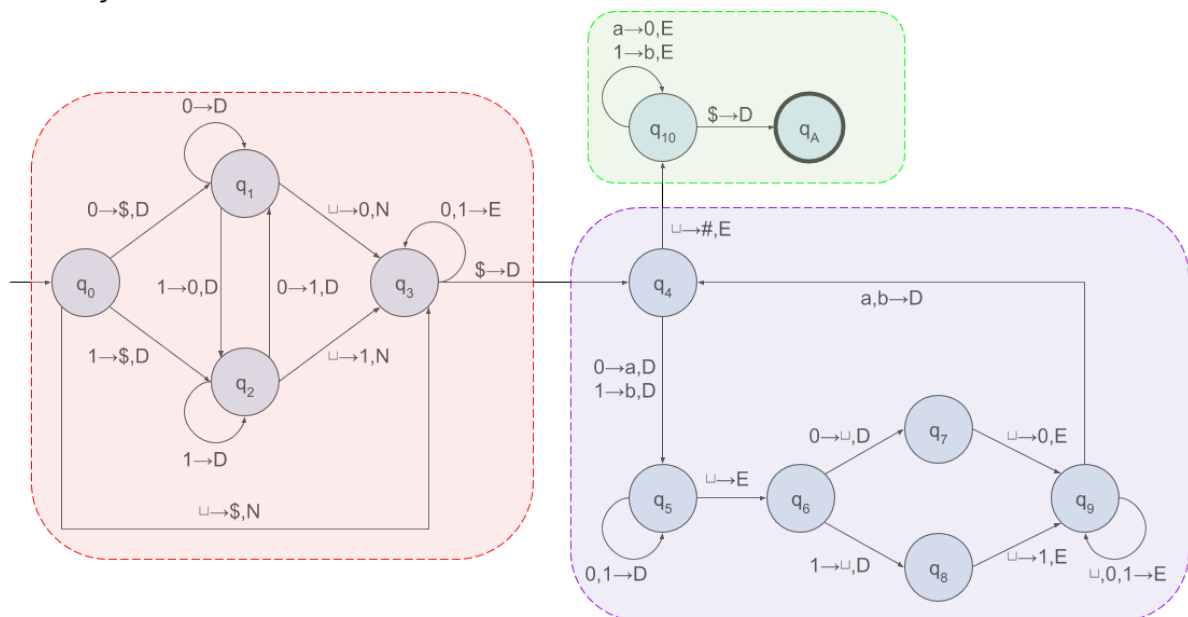
De modo geral, se a cadeia tem comprimento par e é formada por zeros e uns, ela será dividida em duas partes. A primeira parte terá os 0 substituídos por a e os 1 por b.

Além disso aparecerá o símbolo # no meio e o cabeçote estará no último símbolo da primeira metade. Toda outra cadeia é rejeitada.

Desse ponto, supondo que há um símbolo que indica o início da cadeia, é fácil retornar ao início restaurando os 0 e 1 que foram substituídos, como na máquina abaixo.



Finalmente, utilizando todas estas máquinas sequencialmente temos a seguinte construção.



A parte em vermelho é a máquina que marca o início da fita, a parte em roxo acha o meio da cadeia de entrada (se tiver comprimento par) e a parte em verde restaura os símbolos substituídos e retorna ao início da entrada. Supondo que a entrada é 01101001, a configuração inicial será $q_0 01101001$ e, ao entrar em q_4 pela primeira vez a configuração será $\$ q_4 01101001$. Ao entrar em q_{10} pela primeira vez a configuração será $\$ a b b q_{10} a \# 1001$ e, finalmente, é aceita na configuração $\$ q_A 0110 \# 1001$.

Então fizemos uma máquina que pega uma cadeia da linguagem $\{ww : w \in \{0,1\}^*\}$ e leva para uma cadeia equivalente de $\{w\#w : w \in \{0,1\}^*\}$. A partir deste ponto, basta usar a mesma idéia de sequenciamento de máquinas de Turing e aplicar a máquina utilizada no exemplo 1. Há um \$ extra no início da fita, mas perceba que o ponteiro está no símbolo imediatamente após \$, e a máquina do exemplo 1 funcionará normalmente, sem enxergar este símbolo extra.

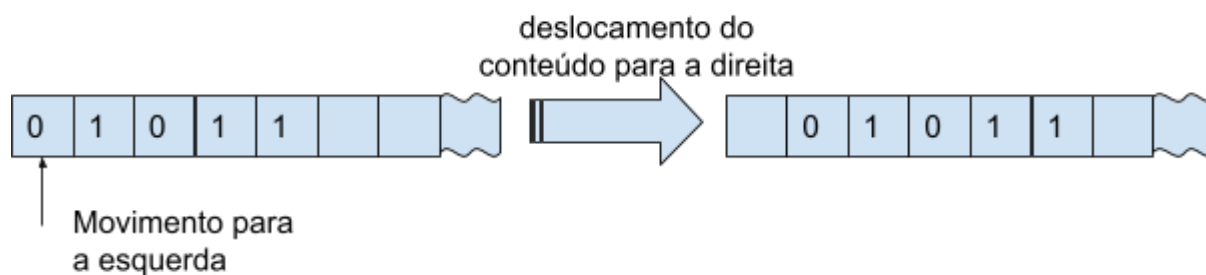
Existem soluções mais simples, mas eu achei essa mais didática porque fica fácil acompanhar passo a passo como a máquina foi construída. Alguns textos assumem que sempre existe um símbolo especial que marca o início da fita, nestes casos a parte em vermelho seria desnecessária.

Variações da Máquina de Turing

Na seção anterior descrevemos uma máquina determinística que tem acesso irrestrito a uma memória infinita, que se dá através do movimento do cabeçote de leitura/escrita sobre uma fita infinita à direita. Vamos explorar algumas variações dessa máquina para tentar torná-la mais poderosa.

Fita Duplamente Infinita

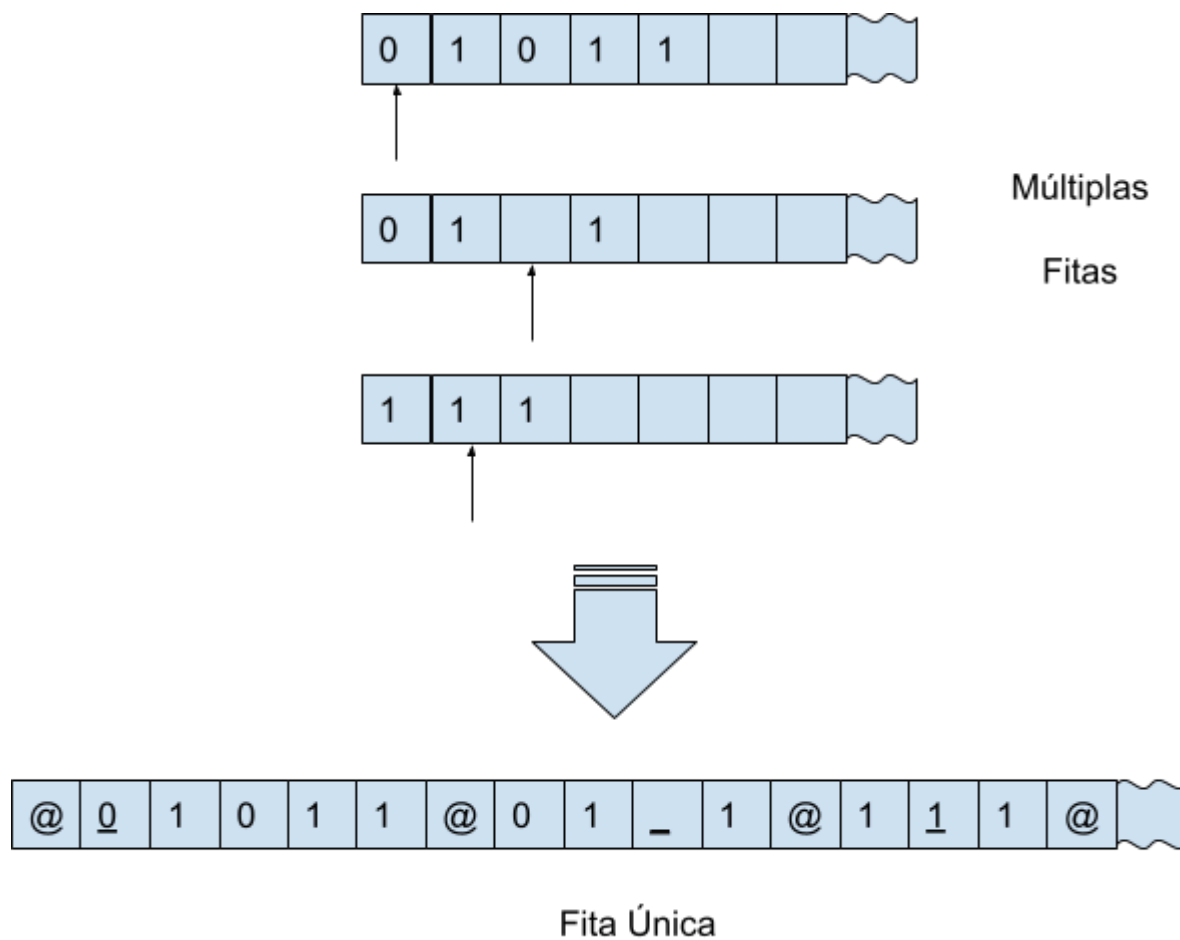
Primeiramente, considere que a máquina de Turing tem uma fita que é infinita nas duas direções. Claramente, a máquina de Turing com fita semi-infinita pode ser simulada por esta máquina, basta inserir um símbolo especial à esquerda da entrada e não permitir que movimentos à esquerda sejam feitos quando esse símbolo for visto. Também é possível simular esta máquina usando a fita semi-infinita, basta que toda vez que a máquina tentar mover à esquerda da primeira célula, todo o conteúdo da fita seja deslocado para a direita. Como a fita é infinita, sempre é possível fazer esse deslocamento, o que é equivalente a ter infinitas posições à esquerda. Portanto, essa máquina com fita duplamente infinita é equivalente à máquina original.



Lembre que os conjuntos $\mathbb{N}$ e $\mathbb{Z}$ têm a mesma cardinalidade, para tal basta que os números negativos de $\mathbb{Z}$ sejam mapeados para os números ímpares de $\mathbb{N}$ e os positivos de $\mathbb{Z}$ sejam mapeados para os números pares de $\mathbb{N}$. Desse modo, todo elemento de $\mathbb{Z}$ tem um correspondente em $\mathbb{N}$ e portanto os conjuntos têm a mesma cardinalidade. Ou seja, se $x \in \mathbb{Z}$, faça $y = -2x + 1$ se $x < 0$ ou $y = 2x$ se $x \geq 0$, desse modo $y \in \mathbb{N}$.

Múltiplas Fitas

Numa segunda tentativa, considere uma máquina de Turing com múltiplas fitas. Esta máquina tem k fitas, cada uma com seu próprio cabeçote. Para fazer a transição, são lidos os símbolos de todas as fitas e cada cabeçote escreve e se move de forma independente dos demais. Mais uma vez, é claro que esta máquina pode simular aquilo que é feito por uma máquina com uma única fita, basta ignorar o que ocorre nas demais fitas. Mas será que é possível simular essa máquina utilizando uma única fita? Primeiramente, perceba que é possível representar o conteúdo de múltiplas fitas em uma única fita, vamos usar um símbolo especial que não faz parte do alfabeto da máquina com múltiplas fitas (no exemplo, @) para separar os conteúdos das fitas e, para saber qual a posição atual do cabeçote, também vamos usar símbolos especiais. Cada símbolo do alfabeto da máquina com múltiplas fitas terá um símbolo especial correspondente (no exemplo que segue é o próprio símbolo sublinhado).



Para simular um passo, a máquina varre a fita a partir do primeiro @ até o último @ para determinar os símbolos marcados. Depois uma segunda varredura é feita para escrever e atualizar os símbolos marcados, conforme a função de transição da máquina com múltiplas fitas. Se a qualquer momento o símbolo marcado cair em uma das @, todo o conteúdo da fita deve ser deslocado de uma célula para a direita.

Portanto, a máquina com múltiplas fitas é equivalente à máquina com uma única fita em termos de poder de representação.

Não-determinismo

Finalmente, vamos verificar se o não-determinismo nos permite reconhecer novas linguagens. Do ponto de vista da computação, a principal diferença entre o não-determinismo e o determinismo é que no determinismo temos um único ramo de computação enquanto no não determinismo temos uma árvore de simulação onde, de uma dada configuração, podemos atingir várias outras, que são executadas simultaneamente. Existe uma quantidade máxima de filhos que uma dada configuração pode ter na árvore de computação de uma máquina não-determinística, no caso das máquinas de Turing, é a cardinalidade de $\mathcal{P}(Q \times \Gamma \times \{E, N, D\})$, que chamaremos de b . A função de transição de uma dada configuração dá como resultado um conjunto de configurações, que serão agora numeradas. Então, a partir da configuração inicial, podemos atribuir um endereço para cada configuração da árvore de computação indicando qual elemento da função de transição foi utilizado.

Essa árvore de computação hipotética mostra as transições efetuadas. Da configuração inicial, C_0 , existem 3 configurações que podem ser atingidas através do não determinismo. Perceba que cada configuração tem um rótulo associado, em vermelho, que indica como aquela configuração foi alcançada. O rótulo 212 indica que, a partir da configuração inicial, tomamos a segunda opção primeiro, depois a primeira e depois a segunda. Se os rótulos são visitados em ordem lexicográfica, os nós da árvore são visitados da esquerda para a direita, descendo um nível de cada vez.

Agora podemos usar uma máquina de Turing com três fitas para simular o não determinismo da seguinte forma:

1. Inicialmente a fita 1 contém a entrada e as fitas 2 e 3 estão vazias.
2. Escreva o símbolo 1 na fita 3 e volte o cabeçote da fita 3 para o início.
3. Copie o conteúdo da fita 1 para a fita 2.
4. Use a fita 2 para simular o não-determinismo. Leia o próximo símbolo da fita 3 para determinar qual das opções da função de transição não-determinística será utilizada. Se não há mais símbolos na fita 3, se a opção é inválida ou se uma configuração de rejeição é encontrada, vá para a etapa 5. Se uma configuração de aceitação da máquina não-determinística é encontrada, aceite.
5. Substitua o conteúdo da fita 3 com a próxima cadeia, em ordem lexicográfica e retorne o cabeçote para o início. Volte para o passo 3.

Então a máquina de Turing não-determinística também é equivalente à máquina de Turing determinística que definimos inicialmente.

Outros Modelos

E quanto a outros modelos? Será que existe algum outro modelo de computação mais poderoso que a Máquina de Turing? A resposta é: “aparentemente não”. Diversos outros modelos de computação foram apresentados ao longo da história (PDAs com múltiplas pilhas, cálculo lambda, gramáticas irrestritas) e todos eles foram mostrados ser equivalentes em poder de representação.

Todos estes modelos compartilham das mesmas características essenciais: acesso irrestrito a memória ilimitada.

Considere duas linguagens de programação distintas, como C e Haskell. Apesar de bem diferentes em suas estruturas, existe algum algoritmo que pode ser programado em uma linguagem e não na outra? Claro que não, podemos compilar Haskell em C e vice-versa, o que quer dizer que as duas linguagens descrevem a mesma classe de algoritmos (algumas tarefas podem ser mais fáceis de codificar em uma linguagem do que outra, mas essa não é a questão), assim como qualquer outra linguagem de programação razoável. A equivalência entre os modelos computacionais se dá pelo mesmo motivo, todas os modelos que compartilham certas características podem simular-se uns aos outros.

Definição de Algoritmo

Informalmente, um algoritmo é uma coleção de instruções simples para cumprir uma certa tarefa. Lidamos o tempo todo com algoritmos na vida e podemos achar registros bem antigos de algoritmos na matemática, como o algoritmo de Euclides para encontrar o maior divisor comum (300 A.C.), o dos Babilônios para achar raiz quadrada (1600 A.C.) ou até o

de multiplicação de dois números, encontrado em escritos egípcios (2000 A.C.). Mesmo antes disso já existiam algoritmos para outras áreas como rituais, receitas e agricultura.

Apesar dessa longa história, somente no século XX uma noção precisa de algoritmo foi criada, na tentativa de responder um enigma matemático. Em 1900, o matemático David Hilbert apresentou 23 problemas que seriam um desafio para os matemáticos do século que iniciava. O décimo problema de Hilbert pedia para projetar “um processo através do qual é possível determinar em uma quantidade finita de operações” se um dado polinômio tem raízes inteiras. Aparentemente, ele acreditava que tal algoritmo existia, bastava que alguém o descobrisse.

Hoje, sabemos que não há algoritmo que resolva tal problema. No entanto, como seria possível para os matemáticos da época provar que tal algoritmo não existe sem uma definição precisa do que vem a ser um algoritmo? A noção intuitiva serve para criar um algoritmo, mas para provar que um não existe é insuficiente.

Em 1928, Hilbert propôs um novo problema, conhecido como Problema da Decisão, que pedia um algoritmo para decidir se uma dada afirmação pode ser provada a partir de um conjunto de axiomas utilizando as regras da lógica de primeira-ordem.

Em 1936, Alonzo Church e Alan Turing mostram (independentemente, ou quase) que não há algoritmo para resolver os problemas de Hilbert, desde que se entenda por algoritmo o que pode ser computado por seus modelos. Church usou um sistema notacional chamado cálculo- λ para definir algoritmos (a programação funcional é fortemente influenciada por este sistema) e Turing propôs suas máquinas. Essas duas definições mostraram-se equivalentes e nasce daí o que hoje é conhecido como Hipótese de Church-Turing: tudo que pode ser computado de forma mecânica pode ser computado por uma Máquina de Turing (ou qualquer outro modelo equivalente).

Ou seja, as máquinas de Turing são a definição de algoritmo, tudo que os computadores atuais podem fazer pode ser feito por uma máquina de Turing e, além disso, todo algoritmo pode ser descrito por uma máquina de Turing.

Neste texto eu escolhi chamar de hipótese de Church-Turing e não de tese porque essa afirmação não pode ser provada. Ela é aceita como verdadeira porque, apesar de muitos esforços, nenhum modelo de computação mais poderoso foi encontrado até hoje.

Deste ponto em diante, usarei Máquina de Turing como sinônimo de algoritmo e não vou mais dar implementações das máquinas, apenas uma descrição do algoritmo a ser executado. O algoritmo deve conter passos simples o suficiente para convencer o leitor de que é possível implementá-lo com uma TM.

Decidibilidade

Já vimos que as máquinas de Turing são um modelo genérico de computação e que definem a noção de algoritmo. Vamos agora investigar o poder dos algoritmos para resolver problemas com o intuito de explorar os limites da computação.

Por que estudar insolubilidade? Uma utilidade fundamental em saber se um problema é algoritmicamente insolúvel é que assim você tem consciência de que o problema precisará ser simplificado para que uma solução algorítmica seja encontrada (essas simplificações envolvem, por exemplo, considerar apenas alguns casos ou aceitar soluções aproximadas).

Como um primeiro exemplo, vamos considerar o décimo problema de Hilbert, citado na seção anterior.

Exemplo 1: Dado um polinômio P , determine se ele tem raízes inteiras.

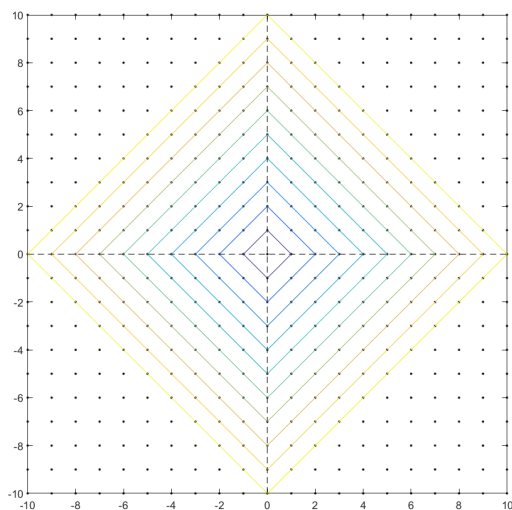
Primeiro, podemos descrever esse problema como a decisão da linguagem $H = \{\langle P \rangle \mid P \text{ é um polinômio com raízes inteiras}\}$. A notação $\langle P \rangle$ indica uma codificação do polinômio como uma cadeia de símbolos. A codificação específica que foi escolhida não é tão importante, porque sempre é possível projetar uma máquina de Turing para traduzir entre codificações (como fizemos no Exemplo 4 das TM).

Eis uma máquina de Turing que reconhece essa linguagem:

1. Conte a quantidade de variáveis de P (vamos chamar essa quantidade de n).
2. para cada n -upla de $\mathbb{Z}^n$:
 - a. Calcule o valor de P para a tupla atual. Se o valor é 0, aceite.

Se P tem uma raiz inteira, eventualmente ela será achada por esse algoritmo, mas se ele não tem, o algoritmo nunca produzirá um resultado.

Para garantir que todas as tuplas sejam visitadas, a ordem de exploração é importante. Imagine, para o caso bidimensional, que você começa com $(0,0)$ e resolve incrementar a cada iteração somente o segundo elemento e, quando chegar no final, você incrementa o primeiro elemento. Como os inteiros são infinitos, o primeiro elemento será sempre zero. Então o ideal é começar de $(0,0)$ e depois testar todas as tuplas com soma absoluta (normal L_1) 1 $\{(1,0), (-1,0), (0,1), (0,-1)\}$, depois de soma absoluta 2 $\{(0,2), (0,-2), (-2,0), (2,0), (1,1), (-1,1), (1,-1), (-1,-1)\}$, e assim sucessivamente. Isso garante que todas as direções serão exploradas igualmente. A figura abaixo ajuda a ilustrar essa ordem de exploração, as linhas coloridas unem pontos com a mesma norma L_1 , que incrementa conforme nos afastamos do centro. Existem outras estratégias possíveis de exploração, essa é apenas uma delas.



Portanto, chegamos à conclusão que essa máquina reconhece a linguagem mas não a decide, porque entra em um laço infinito quando a saída deveria ser negativa. No entanto, isso não prova se a linguagem é decidível ou não, apenas concluímos que o algoritmo proposto não resolve o problema para todas as instâncias.

Exemplo 2: Dado um polinômio P sobre x , determine se ele tem raízes inteiras.

Esse problema é uma versão particular do caso anterior onde P tem apenas uma variável. O algoritmo dado anteriormente também reconhece essa linguagem, já que ela está contida na linguagem anterior, mas não a decide.

No entanto, para polinômios de uma única variável, podemos mostrar que as suas raízes estão dentro do intervalo $\pm(k+1) c_v / c_o$, onde k é a ordem do polinômio, c_v é o coeficiente de maior valor absoluto e c_o é o coeficiente do termo de maior ordem. Então a seguinte máquina de Turing decide esse esta linguagem:

1. Determine a ordem do polinômio (chamaremos de k).
2. Determine o coeficiente com maior valor absoluto (chamaremos de c_v).
3. Determine o coeficiente do termo de maior ordem (chamaremos de c_o).
4. Calcule $(k+1)c_v / c_o$ (chamaremos de d).
5. Para cada x inteiro do intervalo $[-d, d]$:
 - a. Calcule o valor de P para o x atual. Se o valor é 0, pare e aceite.
6. Rejeite.

Concluimos então que esse subconjunto do problema anterior pode ser decidido por uma máquina de Turing, ou seja, existe um algoritmo que sempre pára e dá a resposta correta para a pergunta. Em 1970 foi mostrado por Matiyasevich que o problema do exemplo 1 é de fato indecidível.

Nos exemplos que seguem veremos alguns problemas relativos às linguagens que estudamos nesta disciplina.

Exemplo 3: Dados um autômato finito determinístico, D , e uma cadeia, x , determinar se x é aceita por D .

Esse problema pode ser descrito através da seguinte linguagem $A_{\text{DFA}} = \{\langle D, x \rangle : D \text{ é um DFA e } x \text{ é uma cadeia aceita por } D\}$.

Desejamos criar uma máquina de Turing que seja capaz de decidir essa linguagem, ou seja, se D é uma descrição válida de um DFA na codificação escolhida e x é uma cadeia aceita por D a máquina deve aceitar, caso contrário deve rejeitar, sem nunca entrar em um laço infinito. A seguinte máquina resolve esse problema:

P = para entrada $\langle D, x \rangle$ faça:

1. Determine se D é um DFA válido, se não for, rejeite.
2. Simule D com entrada x .
3. Se a simulação termina em um estado de aceitação, aceite. Se não, rejeite.

O primeiro passo pode ser verificado facilmente, basta verificar se todos os elementos da descrição formal estão presentes e se a função de transição está corretamente definida. Esse passo termina em tempo finito.

O segundo passo consiste em, a partir da descrição dada, simular a computação do DFA para a entrada x . Para isso, basta mantermos um registro do estado atual e da posição do cabeçote de leitura, ou seja, qual símbolo está sendo lido agora. De posse desses dois elementos, vamos à função de transição e verificamos qual transição deve ser efetuada. Depois disso atualizamos o estado atual e passamos ao próximo símbolo. Isso se repete até que toda a entrada seja lida ou uma transição não esteja definida.

O terceiro passo consiste em verificar qual foi o último estado da simulação, se ele pertence ao conjunto de aceitação, P deve aceitar, indicando que o autômato simulado aceita a cadeia x . Caso contrário, deve rejeitar.

Como todos os passos são finitos e executam as tarefas necessárias, a máquina P decide a linguagem A_{DFA} .

Exemplo 4: Dados um autômato finito não determinístico, N , e uma cadeia, x , determinar se x é aceita por N .

Esse problema pode ser descrito através da seguinte linguagem $A_{NFA} = \{\langle N, x \rangle : N \text{ é um NFA e } x \text{ é uma cadeia aceita por } N\}$.

No exemplo anterior criamos uma máquina que decide A_{DFA} , e já sabemos que é possível usar uma máquina de Turing dentro de outra, como uma espécie de sub-rotina. Vamos aproveitar o resultado anterior para criar uma máquina que decide A_{NFA} .

Q = para entrada $\langle N, x \rangle$, faça:

1. Verifique se N é um NFA válido, se não for, rejeite.
2. Converta N em um DFA equivalente, D .
3. Execute a máquina P do exemplo 3 com entrada $\langle D, x \rangle$.
4. Retorne o que P retornar.

O passo 2 permite aproveitar o resultado do exemplo anterior. Perceba que já temos um algoritmo para converter um NFA em um DFA (veja a seção DFA vs. NFA). Então é possível executar o passo 2 em tempo finito. E a máquina Q decide a linguagem A_{NFA} .

O mesmo raciocínio usado no exemplo 2 pode ser utilizado para mostrar que as linguagem $A_{RE} = \{\langle R, x \rangle : R \text{ é uma RE e } x \text{ é uma cadeia gerada por } R\}$ e $A_{RG} = \{\langle G, x \rangle : G \text{ é uma RG e } x \text{ é uma cadeia gerada por } G\}$ são decidíveis. Já temos algoritmos para converter expressões regulares ou gramáticas regulares em autômatos finitos.

Exemplo 5: Dado um DFA, D , determine se a linguagem de D é vazia.

Esse problema pode ser descrito através da seguinte linguagem $E_{DFA} = \{\langle D \rangle : D \text{ é um DFA e } \mathcal{L}(D) = \emptyset\}$.

Em uma primeira tentativa, poderíamos usar uma abordagem similar à dos exemplos anteriores e simular o autômato D para todas as cadeias possíveis em ordem lexicográfica.

H = para entrada $\langle D \rangle$, faça:

1. Verifique se D é um DFA válido, se não for, rejeite.
2. Para cada cadeia, x , de Σ^* , em ordem lexicográfica, faça:

- a. Execute a máquina P do exemplo 3 com entrada $\langle D, x \rangle$.
- b. Se P aceita, rejeita.
3. Aceite.

O passo 2.b rejeita sempre que o autômato D aceita alguma cadeia, ou seja, a linguagem de D não é vazia. O passo 3 só ocorre depois que todas as cadeias de Σ^* foram rejeitadas, mas esse conjunto é infinito e o passo 3 nunca vai ocorrer. Portanto, a máquina H , descrita acima, reconhece o complemento da linguagem E_{DFA} mas não decide a linguagem, pois não dá nenhum retorno para as cadeias que pertencem a E_{DFA} .

O fato desse algoritmo não decidir a linguagem não quer dizer que a linguagem é indecidível, indica simplesmente que esse algoritmo em particular não decide. Para mostrar que uma linguagem é indecidível seria necessário mostrar que não há algoritmo capaz de decidi-la.

Vamos tentar uma abordagem distinta. A linguagem de um DFA é vazia se é impossível alcançar os estados de aceitação a partir do estado inicial, ou se não há estado de aceitação. Então vamos usar esse raciocínio para criar uma máquina que decide tal linguagem.

R = para entrada $\langle D \rangle$, faça:

1. Verifique se D é um DFA válido, se não for, rejeite.
2. Marque o estado inicial.
3. Repita até que nenhum novo estado seja marcado:
 - a. Marque todo estado que tem uma transição proveniente de algum estado já marcado.
4. Se algum estado de aceitação for marcado, rejeite. Se não, aceite.

R só aceita se não é possível alcançar algum estado de aceitação, ou seja, se a linguagem de D é vazia. Em todos os outros casos ele rejeita. Perceba que a iteração do passo 3 é finita, uma vez que o número de estados do DFA é finito. Portanto, H é uma TM que decide E_{DFA} .

Exemplo 6: Dados dois DFA, D e E , determine se são equivalentes.

Esse problema pode ser descrito através da seguinte linguagem $EQ_{DFA} = \{\langle D, E \rangle : D \text{ e } E \text{ são DFAs e } \mathcal{L}(D) = \mathcal{L}(E)\}$.

Mais uma vez, uma abordagem por simulação seria inapropriada. Poderíamos simular os dois autômatos para todas as cadeias de Σ^* e rejeitar caso eles respondam de forma distinta. No entanto, só seríamos capazes de descobrir se os autômatos não são equivalentes. No caso de serem equivalentes nenhuma resposta seria produzida, pelo mesmo argumento usado no exemplo anterior.

Perceba que se dois autômatos têm a mesma linguagem a diferença simétrica das linguagens deve ser vazia, ou seja, $(\mathcal{L}(D) \cap \neg \mathcal{L}(E)) \cup (\mathcal{L}(E) \cap \neg \mathcal{L}(D)) = \emptyset$. Temos algoritmos para criar autômatos que reconhecem o complemento, a interseção e a união das linguagens de DFAs. Portanto podemos criar a seguinte máquina para decidir a equivalência de dois DFAs:

T = para entrada $\langle D, E \rangle$, faça:

1. Verifique se D e E são DFAs válidos, se pelo menos um deles não for, rejeite.
2. Crie um DFA C tal que $\mathcal{L}(C) = (\mathcal{L}(D) \cap \neg\mathcal{L}(E)) \cup (\mathcal{L}(E) \cap \neg\mathcal{L}(D))$.
3. Execute a TM R , do exemplo 5, com entrada $\langle C \rangle$.
4. Retorne o que R retornar.

Novamente, usamos uma solução anterior para resolver este problema. Perceba que a máquina R decide se um dado DFA tem linguagem vazia ou não. Se o DFA C , contruído no passo 2, tem linguagem vazia quer dizer que D e E têm a mesma linguagem. Como R é um decisor, T também é. T é um decisor para a linguagem EQ_{DFA} .

Todos os exemplos até agora envolvem linguagens regulares a linguagem A_{DFA} mostra que qualquer linguagem regular pode ser reconhecida por uma máquina de Turing, basta simular o DFA que reconhece a linguagem desejada.

Agora vamos ver alguns exemplos envolvendo linguagens livres de contexto.

Exemplo 7: Dados uma gramática livre de contexto, G , e uma cadeia, x , determinar se x é gerada por G .

Esse problema pode ser descrito através da seguinte linguagem $A_{CFG} = \{\langle G, x \rangle : G \text{ é uma CFG e } x \text{ é uma cadeia gerada por } G\}$.

Para este problema vamos mostrar dois decisores distintos, o primeiro deles usa um algoritmo apresentado nesse texto.

V_1 = para entrada $\langle G, x \rangle$, faça:

1. Verifique se G é uma CFG válida. Se não for, rejeite.
2. Transforme G em uma gramática equivalente G_{CNF} na forma normal de Chomsky.
3. Execute o algoritmo CYK com entrada $\langle G_{CNF}, x \rangle$.
4. Retorne o que CYK retornar.

Uma outra abordagem é perceber que a árvore sintática de uma gramática na CNF só tem duas formas de ramificação: uma variável transformando-se em duas outras ou uma variável transformando-se em um terminal. Em uma cadeia de comprimento n , ou seja, formada por n terminais, teremos n folhas na árvore sintática. Então teremos n substituições do tipo Variável $\rightarrow$ Terminal. Removendo essas transições da árvore sintática, sobra uma árvore binária completa (cada elemento tem exatamente dois descendentes ou é uma folha). Perceba que uma árvore deste tipo com uma folha não executou nenhuma substituição. Com duas folhas, uma substituição foi efetuada. Com três folhas uma das folhas anteriores deixou de ser folha e foi substituída por duas novas variáveis. De modo geral, se temos n folhas, $n-1$ substituições do tipo Variável $\rightarrow$ Variável Variável foram realizadas. Isso quer dizer que a produção de uma cadeia de comprimento n por uma CFG na CNF tem exatamente $n + n - 1 = 2n - 1$ passos (se a cadeia não é vazia). Podemos usar isso para decidir se uma gramática gera uma certa cadeia ou não.

V_2 = para entrada $\langle G, x \rangle$, faça:

1. Verifique se G é uma CFG válida. Se não for, rejeite.

2. Transforme G em uma gramática equivalente G_{CNF} na forma normal de Chomsky.
3. Se $|x| = 0$ e $S \rightarrow \varepsilon$ faz parte de G_{CNF} , aceite.
4. Liste todas as derivações com $2|x| - 1$ passos.
5. Se alguma das derivações é x , aceite. Se não, rejeite.

Tanto V_1 quanto V_2 são decisores da linguagem A_{CFG} . Como no caso anterior, podemos usar essas máquinas para decidir $A_{PDA} = \{\langle P, x \rangle : P \text{ é um PDA e } x \text{ é uma cadeia aceita por } P\}$, bastando para isso converter o PDA P em uma CFG equivalente.

Perceba que simular um PDA pode não ser uma boa idéia pois a simulação pode entrar em um laço infinito se há infinitas configurações simultâneas (ver exemplo na seção PDA vs. CFG). O PDA em si não está em um laço, ele está imediatamente em todas as configurações, só a simulação entra em uma laço infinito pois é determinística e precisa simular uma configuração por vez.

Exemplo 8: Dado uma CFG, G , determine se a linguagem de G é vazia.

Esse problema pode ser descrito através da seguinte linguagem $E_{CFG} = \{\langle G \rangle : G \text{ é uma CFG e } \mathcal{L}(G) = \emptyset\}$.

Podemos adotar uma estratégia similar àquela adotada para mostrar que E_{DFA} é decidível. Vamos verificar se é possível partir da variável inicial e chegar a alguma cadeia formada apenas por terminais. Para isso, vamos partir dos terminais e verificar se é possível produzi-los a partir das outras variáveis da gramática.

W = para entrada $\langle G \rangle$, faça:

1. Verifique se G é uma CFG válida. Se não for, rejeite.
2. Marque todos os símbolos terminais de G .
3. Repita até que nenhuma nova variável seja marcada:
 - a. Marque toda variável A tal que $A \rightarrow x$ é um regra de G e x é formada apenas por símbolos já marcados.
4. Se a variável inicial for marcada, rejeite. Se não, aceite.

Como no caso dos autômatos, o número de variáveis em uma CFG é finito, portanto o laço no passo 3 será eventualmente encerrado. A variável inicial só é marcada se é possível alcançar alguma cadeia formada apenas por terminais. Se isso não for possível, a linguagem desta gramática é vazia.

Exemplo 9: Dadas duas CFG, G e H , determine se G e H são equivalentes.

Esse problema pode ser descrito através da seguinte linguagem $EQ_{CFG} = \{\langle G, H \rangle : G \text{ e } H \text{ são CFGs e } \mathcal{L}(G) = \mathcal{L}(H)\}$.

Nesse problema não podemos usar uma abordagem similar à que usamos para EQ_{DFA} porque as linguagens livres de contexto não são fechadas sob as operações de interseção e complemento.

Poderíamos criar um reconhecedor para $\neg EQ_{CFG}$, verificando se G e H geram ou não cada uma das cadeias de Σ^* . Se os resultados são distintos para alguma cadeia,

verificamos que elas não são equivalentes, no entanto nunca será possível verificar se são de fato equivalentes, pois seria necessário testar todas as infinitas cadeias de Σ^* .

Vamos deixar este problema em aberto até um momento futuro quando seremos capazes de provar que esta linguagem é de fato indecidível.

Como A_{CFG} é decidível, podemos concluir que toda linguagem livre de contexto é decidível por uma máquina de Turing, bastando para isso simular a gramática que gera a linguagem desejada.

Definição: Linguagem co-Turing-reconhecível

Uma linguagem A é dita *co-Turing-reconhecível* se existe uma máquina de Turing T tal que $\mathcal{L}(T) = \neg A$.

Ou seja, A é *co-Turing-reconhecível* se o seu complemento é Turing-reconhecível.

A linguagem EQ_{CFG} , do exemplo anterior, é um exemplo dessa classe de linguagem.

Teorema

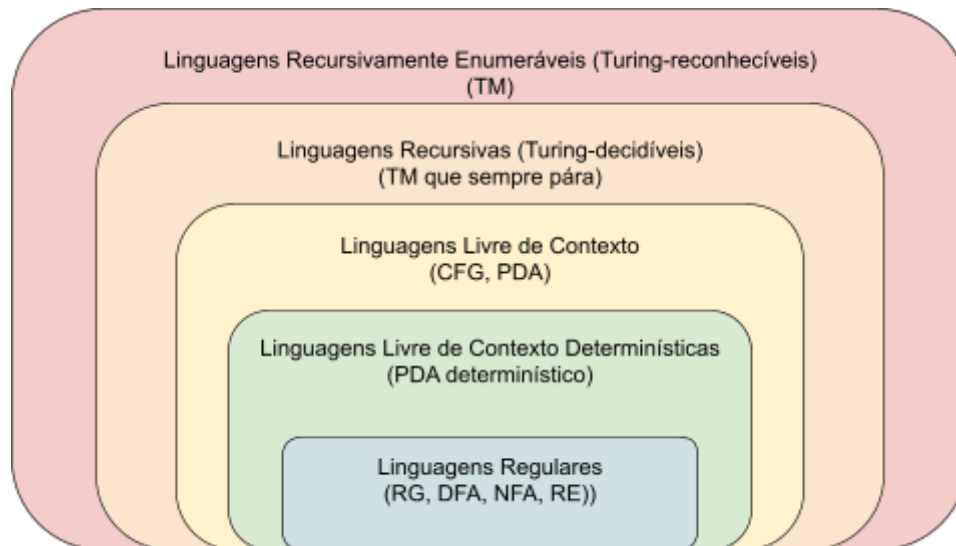
Uma linguagem é Turing-decidível se, e somente se, é Turing-reconhecível e co-Turing-reconhecível.

Prova

Se a linguagem é decidível, ela é obviamente Turing-reconhecível e co-Turing-reconhecível, pois existe uma TM, o decisor, que pára para qualquer entrada. Vamos provar o sentido inverso.

Se a linguagem é Turing-reconhecível, existe uma máquina M_1 que pára para todas as cadeias que pertencem à linguagem. Se é também co-Turing-reconhecível existe uma máquina M_2 que pára para toda cadeia que não pertence à linguagem. Podemos criar um decisor M executando M_1 e M_2 em paralelo. Se M_1 pára, M retorna o mesmo que M_1 . Se M_2 pára, M retorna o contrário de M_2 . Como uma das duas máquinas vai parar, M sempre pára.

Diante de tudo que foi exposto até o momento, podemos atualizar nossa hierarquia de linguagens.



Vamos continuar nossos exemplos com os problemas da aceitação, vacuidade e equivalência para as máquinas de Turing.

Exemplo 10: Dados uma máquina de Turing, M , e uma cadeia, x , determinar se x é aceita por M .

Esse problema pode ser descrito através da seguinte linguagem $A_{TM} = \{\langle M, x \rangle : M \text{ é uma TM e } x \text{ é uma cadeia aceita por } M\}$.

Uma primeira tentativa é usar o argumento de simulação, usado em A_{DFA} .

U = para entrada $\langle M, x \rangle$, faça:

1. Verifique se M é uma TM válida, Se não for, rejeite.
2. Simule M com entrada x .
3. Se a simulação aceita, aceite. Se a simulação rejeita, rejeite.

Como no caso dos DFAs, simular uma máquina de Turing requer apenas saber qual o estado atual e onde está o cabeçote de leitura/escrita. A partir daí, consulta a função de transição e atualiza o conteúdo da fita, a posição do cabeçote e o estado atual.

A máquina U, por si só, já é bastante interessante, ela é uma máquina de Turing que simula qualquer outra máquina de Turing a partir de suas descrições. Essa máquina é conhecida como máquina de Turing universal. Esse conceito é a base para os nossos computadores modernos, onde temos uma máquina (o computador) que simula outras máquinas (os programas) que por sua vez podem simular outras máquinas (compiladores/interpretadores) e assim por diante.

Apesar de interessante, a máquina U não resolve o nosso problema. Se a máquina M , que está sendo simulada, entra em um laço infinito, a máquina U também não produzirá nenhuma saída. Então, U é apenas um reconhecedor da linguagem A_{TM} e não um decisor.

Vamos tentar o caminho inverso. Assuma que a linguagem A_{TM} é decidível por uma máquina hipotética H. Ou seja, para entrada $\langle M, x \rangle$, H pára e aceita se M aceita x e pára e rejeita em todos os outros casos.

Considere agora a seguinte máquina

C = para entrada $\langle M \rangle$

1. Execute H com entrada $\langle M, \langle M \rangle \rangle$.
2. Se H aceita, rejeite. Se H rejeita, aceite.

A descrição de uma máquina é uma cadeia, então o que a máquina C faz é decidir se a máquina M rejeita sua própria descrição. Então, $D(\langle M \rangle) = \text{aceita}$, se M não aceita $\langle M \rangle$; e $D(\langle M \rangle) = \text{rejeita}$, se M aceita $\langle M \rangle$.

O que acontece se dermos a descrição de D como entrada para D ? Teremos $D(\langle D \rangle) = \text{aceita}$, se D não aceita $\langle D \rangle$; e $D(\langle D \rangle) = \text{rejeita}$, se D aceita $\langle D \rangle$. O que é, obviamente, uma contradição. Portanto, a hipótese inicial é falsa, ou seja, a máquina H não existe e A_{TM} é indecidível.

Perceba que essa foi a primeira linguagem que de fato provamos ser indecidível.

Antes de continuar com os nossos exemplos, é preciso focar em uma técnica que já estamos utilizando mesmo sem tê-la enunciado, a redutibilidade.

Redutibilidade

Dizemos que um problema A é redutível a um problema B se podemos transformar A de tal forma que podemos usar a solução de B para solucionar A . Esse é um conceito bastante utilizado em nossa vida cotidiana, muitas vezes de forma imperceptível.

Considere o problema de calcular o produto dos números romanos X e V . Até onde sabemos, não existem algoritmos para efetuar operações aritméticas com números romanos, o que podemos fazer é convertê-los para números arábicos e efetuar o produto de 10 e 5, obtendo 50 que pode ser convertido de volta para números romanos e podemos afirmar que o produto de X e V é L .

De modo mais específico, estamos utilizando, até agora, um tipo específico de redução chamada de redução por mapeamento.

Definição: Redução por Mapeamento

Uma linguagem A é redutível por mapeamento a uma linguagem B , denotado por $A \leq_m B$, se existe uma função computável $f: \Sigma^* \rightarrow \Sigma^*$ tal que, para todo x , $x \in A \Leftrightarrow f(x) \in B$. A função f é a redução de A para B .

Definição: Função Computável

Uma função $f: \Sigma^* \rightarrow \Sigma^*$ é computável se há uma máquina de Turing M tal que, para toda entrada x , aceita com apenas $f(x)$ escrito na fita.

Operações aritméticas, transformações de máquinas ou de cadeias, são exemplos de funções computáveis. No exemplo 4 da seção Máquinas de Turing utilizamos esse conceito para reduzir a linguagem $\{xx : x \in \{0,1\}^*\}$ à linguagem $\{x\#x : x \in \{0,1\}^*\}$. Várias funções foram utilizadas para identificar e marcar o meio da cadeia com o símbolo $\#$.

Também fizemos a redução de A_{NFA} para A_{DFA} , EQ_{DFA} para E_{DFA} e citamos que era possível fazer a redução de A_{PDA} para A_{CFG} .

Em todos os casos, como a linguagem destino era decidível, concluímos que a linguagem original também era. Vamos enunciar isso formalmente.

Teorema

Se $A \leq_m B$, e B é decidível, então A é decidível.

Prova

Seja M um decisor de B e f a redução de A para B . Eis um decisor para A :

N = para entrada x , faça:

1. Obtenha $f(x)$.
2. Execute M com entrada $f(x)$.
3. Retorne o que M retornar.

Como f é uma redução de A para B , se x pertence a A , $f(x)$ pertence a B . Então, M aceita $f(x)$ se x pertence a A . Portanto, N decide A .

Corolário

Se $A \leq_m B$, e A é indecidível, então B é indecidível.

Teorema

Se $A \leq_m B$, e B é Turing-reconhecível, então A é Turing-reconhecível.

Prova

Seja M uma TM que reconhece B e f a redução de A para B . Eis um decisor para A :

N = para entrada x , faça:

4. Obtenha $f(x)$.
5. Execute M com entrada $f(x)$.
6. Retorne o que M retornar.

Como f é uma redução de A para B , se x pertence a A , $f(x)$ pertence a B . Então, M aceita $f(x)$ se x pertence a A . Se M entra em laço infinito, N também entra. Portanto, N reconhece A .

Corolário

Se $A \leq_m B$, e A não é Turing-reconhecível, então B não é Turing-reconhecível.

Estes teoremas são nossas principais armas para provar que certas linguagens não são decidíveis ou reconhecíveis. Agora temos todas as ferramentas necessárias para continuar nossos exemplos.

Exemplo 11: Dado uma TM, M , determine se a linguagem de M é vazia.

Esse problema pode ser descrito através da seguinte linguagem $E_{TM} = \{\langle M \rangle : M \text{ é uma TM e } \mathcal{L}(M) = \emptyset\}$.

Vamos supor que E_{TM} é decidível por uma máquina H , reduzir A_{TM} a E_{TM} e provar, por contradição, que E_{TM} é indecidível. O ponto crucial dessa prova é construir a redução, devemos definir uma máquina S que usa H como subrotina e através disso, consegue decidir A_{TM} . Uma idéia é rodar H sobre $\langle M \rangle$, se H aceita, a linguagem de M é vazia e sabemos que M rejeita a cadeia x , o que resolve parte do nosso problema. Mas, se H rejeita, sabemos que M aceita algo. Como saber se M aceita uma cadeia x específica? A solução é criar uma máquina auxiliar, N , que sabemos que rejeita tudo com exceção de x , nesse caso ela funciona como M . Desse modo, se H rejeita quando a entrada é $\langle N \rangle$, sabemos que M aceita x . A máquina S abaixo faz a redução de A_{TM} para E_{TM} .

S = para entrada $\langle M, x \rangle$, faça:

1. Usando $\langle M \rangle$ e x construa a máquina N da seguinte forma:

N = para entrada w , faça:

- a. Se $w \neq x$, rejeite.
- b. Se $w = x$, execute M com entrada x .
- c. Retorne o que M retornar.
2. Execute H com entrada $\langle N \rangle$.
3. Se H aceita, rejeite. Se H rejeita, aceite.

Pela discussão anterior, se H aceita, N rejeita tudo, e M rejeita x . Se H rejeita, N aceita x , mas N só pode aceitar x se M aceita x . Portanto S é um decisor de A_{TM} . Como sabemos que A_{TM} é indecidível, concluímos que E_{TM} também é indecidível e a máquina H não pode existir.

Pela definição dada, A_{TM} não é redutível a E_{TM} por mapeamento. Suponha, que $A_{TM} \leq_m E_{TM}$, isso implica que $\neg A_{TM} \leq_m \neg E_{TM}$ usando a mesma função de redução. O problema é que $\neg E_{TM}$ é Turing-reconhecível enquanto $\neg A_{TM}$ não é, isso contradiz os teoremas apresentados. Logo, não pode haver função f que reduza A_{TM} a E_{TM} por mapeamento. A redução utilizada no exemplo anterior foi $A_{TM} \leq_m \neg E_{TM}$. A função computável que faz esta redução é a máquina N .

Exemplo 12: Dadas duas TM, M e N , determine se são equivalentes.

Esse problema pode ser descrito através da seguinte linguagem $EQ_{TM} = \{\langle M, N \rangle : M \text{ e } N \text{ são TMs e } \mathcal{L}(M) = \mathcal{L}(N)\}$.

Vamos mostrar que EQ_{TM} é indecidível fazendo uma redução de E_{TM} para EQ_{TM} . Apesar de E_{TM} ser indecidível, é fácil criar uma máquina de Turing X que rejeita tudo, basta que o estado inicial leve para o estado de rejeição para qualquer símbolo lido. Se X é

equivalente a uma certa máquina M , sabemos que a linguagem de M é vazia. Se H é um decisor de EQ_{TM} , a máquina S descrita abaixo é um decisor de E_{TM} .

S = para entrada $\langle M \rangle$, faça:

1. Construa uma TM X que rejeita tudo.
2. Execute H com entrada $\langle M, X \rangle$.
3. Retorne o que H retornar.

Como E_{TM} é indecidível e S efetua uma redução de E_{TM} para EQ_{TM} , concluímos que EQ_{TM} também é indecidível.

Já sabemos que as máquinas de Turing são capazes de decidir qualquer linguagem regular. É possível determinar se a linguagem reconhecida por uma dada máquina de Turing é regular ou não? É o que veremos no exemplo que segue.

Exemplo 13: Dada uma TM, M , determine se a linguagem reconhecida por M é regular.

Esse problema pode ser descrito através da seguinte linguagem $R_{TM} = \{\langle M \rangle : M \text{ é uma TM e } \mathcal{L}(M) \text{ é regular}\}$.

Vamos fazer uma redução de A_{TM} para R_{TM} . A idéia é criar uma máquina auxiliar, N , que aceita uma linguagem regular se M aceita x e uma linguagem não-regular se M rejeita x . Sabemos que Σ^* é uma linguagem regular. Vamos fazer uma máquina que aceita qualquer entrada, ou seja, Σ^* , se M aceita x e aceita somente entradas da forma $0^n 1^n$, que não é regular, caso contrário. Suponha que existe uma máquina H que decide R_{TM} .

S = para entrada $\langle M, x \rangle$, faça:

1. Usando $\langle M \rangle$ e x construa a máquina N da seguinte forma:

N = para entrada w , faça:

 - d. Se w é da forma $0^n 1^n$, aceite.
 - e. Se não, execute M com entrada x .
 - f. Retorne o que M retornar.
2. Execute H com entrada $\langle N \rangle$.
3. Se H aceita, aceite. Se H rejeita, rejeite.

S é um decisor de A_{TM} . A linguagem de N é regular se M aceita x e não-regular caso contrário. Portanto, se H aceita N , a linguagem de N é regular e M aceita x . Caso contrário, M não aceita x . Como A_{TM} é indecidível, R_{TM} também é.

Exemplo 14: Dada uma TM, M , determine se a linguagem reconhecida por M é finita.

Esse problema pode ser descrito através da seguinte linguagem $F_{TM} = \{\langle M \rangle : M \text{ é uma TM e } |\mathcal{L}(M)| < \infty\}$.

Esse é um problema aparentemente mais simples que o anterior, pois as linguagens finitas são um subconjunto das linguagens regulares. Mas, ainda assim, a linguagem é indecidível. Vamos reduzir, mais uma vez, A_{TM} a F_{TM} . Como no caso anterior, vamos supor que há uma máquina H que decide F_{TM} e usá-la para resolver A_{TM} . Vamos criar uma

máquina auxiliar, N , que aceita qualquer cadeia se M aceita x e rejeita qualquer cadeia se M rejeita x .

S = para entrada $\langle M, x \rangle$, faça:

1. Usando $\langle M \rangle$ e x construa a máquina N da seguinte forma:

N = para entrada w , faça:

- g. Execute M com entrada x .
- h. Retorne o que M retornar.
2. Execute H com entrada $\langle N \rangle$.
3. Se H aceita, rejeite. Se H rejeita, aceite.

S é um decisor de A_{TM} . Perceba que a linguagem de N é Σ^* , uma linguagem infinita, se M aceita x e é $\emptyset$, uma linguagem finita, se M não aceita x . Portanto, se H aceita N , concluímos que M rejeita x e se H rejeita N , M aceita x .

Toda propriedade não-trivial de uma linguagem reconhecida por uma TM é indecidível, isto é enunciado no teorema que segue e poderia ser utilizado para provar mais facilmente que as linguagens dos dois exemplos anteriores são indecidíveis.

Teorema: Teorema de Rice (1)

Seja P a linguagem formada por todas as TMs que satisfazem uma certa condição p . Considere que p não é trivial, ou seja, algumas linguagens têm a propriedade p , mas não todas. P é indecidível.

Prova

Suponha que P é decidido por uma máquina H . Sem perda de generalidade, assumamos que as máquinas com linguagem vazia não fazem parte de P , se não for o caso, podemos considerar o complemento de P . Como P não é trivial, existe uma máquina N pertencente a P . Considere a seguinte máquina:

S = para entrada $\langle M, x \rangle$, faça:

1. Construa uma TM M_x da seguinte forma:
 - M_x = para entrada w , faça:
 1. Se M aceita x e N aceita w , aceite.
 2. Se não, rejeite.
2. Execute H com entrada $\langle M_x \rangle$.
3. Se H aceita, aceite. Se H rejeita, rejeite.

Se M aceita x , então $\mathcal{L}(M_x) = \mathcal{L}(N)$ e M_x deve ser aceita por H , pois sabemos que N faz parte de P . Se M não aceita x , então $\mathcal{L}(M_x) = \emptyset$ e M_x deve ser rejeitado por H , pois sabemos que as máquinas com linguagem vazia não fazem parte de P . Como A_{TM} não é decidível, P também não pode ser.

A seguir damos um exemplo da aplicação deste teorema para provar que uma certa propriedade é indecidível.

Exemplo 15: $L = \{\langle M \rangle : M \text{ é uma TM e } |\mathcal{L}(M)| \geq 10\}$

Σ^* é recursivamente enumerável e a propriedade $|\Sigma^*| \geq 10$ é verdadeira. $\emptyset$ é recursivamente enumerável e a propriedade $|\emptyset| \geq 10$ é falsa. Portanto, pelo teorema de Rice, L é indecidível.

Vamos agora verificar uma versão restrita das máquinas de Turing, os autômatos linearmente limitados (LBA - *Linear Bounded Automata*). Um LBA pode ser entendido como uma máquina de Turing cuja fita é limitada ao comprimento da entrada.

Apesar de ter memória finita, como os DFA, a memória dos LBA não é constante para qualquer entrada, ela depende do comprimento da entrada apresentada o que os tornam mais poderosos que os DFA. De fato, os LBA definem uma classe de linguagem intermediária entre as linguagens livres de contexto e as linguagens recursivas, chamada de linguagens sensíveis ao contexto.

Do ponto de vista das gramáticas formais, a diferença entre uma gramática irrestrita, que é o formalismo gerador equivalente às máquinas de Turing e as gramáticas sensíveis ao contexto, equivalente aos LBA, é bem sutil. Gramática formais $G = (V, \Sigma, R, S)$ são irrestritas se as regras são do tipo $xVy \rightarrow x(\Sigma \cup V)^*y$, com $x, y \in (\Sigma \cup V)^*$, enquanto para gramáticas sensíveis ao contexto as regras são do tipo $xVy \rightarrow x(\Sigma \cup V)^+y$. Ou seja, não é permitido a substituição de uma variável pela cadeia vazia. A única exceção a este formato é a substituição $S \rightarrow \varepsilon$.

Vamos considerar o problema da aceitação para os LBA.

Exemplo 16: Dados um autômato linearmente limitado, L , e uma cadeia, x , determinar se x é aceita por L .

Esse problema pode ser descrito através da seguinte linguagem $A_{\text{LBA}} = \{\langle L, x \rangle : L \text{ é um LBA e } x \text{ é uma cadeia aceita por } L\}$.

O motivo pelo qual A_{TM} era reconhecível, mas não decidível por simulação é que existia a possibilidade da máquina entrar em um laço infinito. Se observarmos as configurações pelas quais a máquina passa em sua computação, existem duas possibilidades em um laço infinito: seu histórico de computação nunca repete nenhuma configuração, ou, como a máquina é determinística, ao visitar um mesmo estado pela segunda vez toda a sequência entre a primeira visita e a repetição se repetirá indefinidamente.

O número total de configurações possíveis em um LBA é finito. Uma configuração é definida pelo estado atual, pela posição do cabeçote e pelo conteúdo da fita. Como a fita tem comprimento limitado ao tamanho da entrada, existem apenas $|x|$ valores possíveis para a posição do cabeçote e os possíveis conteúdos da fita são $|\Gamma|^{|x|}$, onde Γ é o alfabeto da fita. Portanto o número total de configurações possíveis é $|Q| \times |x| \times |\Gamma|^{|x|}$, onde $|Q|$ é o número de estados, $|\Gamma|$ é o número de símbolos no alfabeto da fita e $|x|$ é o comprimento da entrada e, consequentemente, da fita.

Como o número de configurações é finito, um LBA só pode entrar em um laço infinito se repetir uma configuração pelo qual já passou. Considere a seguinte máquina:

S = para entrada $\langle L, x \rangle$, faça:

1. Verifique se L é um LBA válido. Se não for, rejeite.
2. Simule L com entrada x por $|Q| \times |x| \times |\Gamma|^{|x|}$ passos, ou até parar.
3. Se L parar, retorne o mesmo que L .
4. Se L não parou, rejeite.

Como o número de configurações é finito, qualquer LBA que executa por mais que $|Q| \times |x| \times |\Gamma|^{|x|}$ passos está repetindo configurações e, portanto, em um laço infinito e por isso é rejeitado. Portanto, a máquina S é um decisor de A_{LBA} .

Mais uma vez, como A_{LBA} é Turing-decidível, podemos concluir que toda linguagem sensível ao contexto pode ser decidida por uma TM, bastando para isso emular o comportamento da LBA que aceita a linguagem desejada. Podemos atualizar mais uma vez nossa hierarquia de linguagens para incluir essa linguagem intermediária.



Exemplo 17: Dados um autômato linearmente limitado, L , determine se a linguagem de L é vazia.

Esse problema pode ser descrito através da seguinte linguagem $E_{LBA} = \{\langle L \rangle : L \text{ é um LBA e } \mathcal{L}(L) = \emptyset\}$.

Novamente, a prova de indecidibilidade se dá através da redução de A_{TM} para E_{LBA} . Dados uma TM M e uma cadeia x , precisamos criar um LBA, B , tal que se M aceita x a linguagem de B não é vazia e, caso contrário, a linguagem não é vazia. Vamos supor que existe um decisor H para E_{LBA} .

Para tal, vamos construir B de tal modo que ele aceite um histórico de computação de aceitação de M para a cadeia x . Se M aceita x , B aceita uma cadeia e se M não aceita x ,

B não aceita nada. Um histórico de computação de aceitação é uma sequência de configurações $c_1, c_2, \dots, c_n$, que inicia da configuração inicial, utiliza apenas transições válidas (segundo a função de transição) e encerra em uma configuração de aceitação, ou seja, que contém o estado de aceitação. É a sequência de configurações pelas quais a máquina passa ao aceitar a cadeia x .

Vamos assumir que o histórico de computação é apresentado como uma única cadeia, com as configurações separadas umas das outras por um símbolo que não é utilizado pela TM, digamos que esse símbolo seja $\#$. Então ela se apresenta como $\#c_1\#c_2\#\dots\#c_n\#$.

O LBA B funciona da seguinte forma. Quando recebe uma entrada w , B deve aceitar se w é uma sequência de computação de aceitação de M em x .

B = para entrada w , faça:

1. Verifique se a subcadeia antes do primeiro $\#$ é a configuração inicial de M com entrada x . Se não for, rejeite.
2. Varra a fita da esquerda para a direita e faça:
 - a. Verifique se a subcadeia entre o i -ésimo e o $(i+1)$ -ésimo $\#$ é uma configuração que pode ser alcançada a partir da configuração presente na subcadeia entre o $(i-1)$ -ésimo e o i -ésimo $\#$. Se não for, rejeite.
3. Verifique se o estado de aceitação está presente na última configuração. Se estiver, aceite. Se não, rejeite.

Agora considere a seguinte TM:

S = para entrada $\langle M, x \rangle$, faça:

1. Construa o LBA B conforme descrito acima.
2. Execute H com entrada $\langle B \rangle$.
3. Se H aceita, rejeite. Se H rejeita, aceite.

A linguagem de B só não é vazia se M aceita x . Portanto, se H rejeita B , a linguagem de B é vazia, e S aceita, indicando que M aceita x . Caso contrário, S rejeita. Desse modo, S é um decisor de A_{TM} . Como sabemos que A_{TM} é indecidível, concluímos que H não existe e E_{TM} é indecidível.

A redução do exemplo anterior utiliza uma técnica conhecida como redução por histórico de computação. Os históricos de computação são finitos e as máquinas determinísticas têm no máximo um histórico de computação para uma dada cadeia.

Exemplo 18: Dada uma gramática livre de contexto, G , determine se a G gera todas as cadeias possíveis.

Esse problema pode ser descrito através da seguinte linguagem $T_{CFG} = \{\langle G \rangle : G \text{ é uma CFG e } \mathcal{L}(L) = \Sigma^*\}$.

Vamos usar novamente o método dos históricos de computação para reduzir A_{TM} a T_{CFG} . Vamos supor que existe uma máquina H que decide T_{CFG} . A idéia é criar uma CFG que

produz todas as cadeias ou não produz todas as cadeias a depende se M aceita ou não uma certa cadeia x .

Já sabemos que se x é aceita por M , existe um único histórico de aceitação de M com x . Usando M e x vamos criar uma CFG que gera todas as cadeias com exceção da do histórico de computação de aceitação de M com x , vamos chamar essa cadeia de s_{aMx} . Portanto, se M rejeita x , G deve gerar Σ^* e, se M aceita x , G gera $\Sigma^* - s_{aMx}$.

Para simplificar a explicação, vamos criar um PDA ao invés de uma CFG. Já sabemos que é possível converter um PDA em uma CFG então não há problemas nisso. Nosso PDA deve aceitar tudo menos s_{aMx} . Já vimos no exemplo anterior que há 3 condições que devem ser satisfeitas para que um histórico de computação válido e de aceitação:

1. A primeira configuração deve ser a configuração inicial de M com entrada x .
2. A configuração c_{i+1} deve ser alcançada a partir de c_i utilizando a função de transição de M .
3. A configuração final deve conter o estado de aceitação.

O PDA P , construído utilizando M e x , deve aceitar se qualquer uma das três condições for violada.

P = para entrada w , faça:

1. Não deterministicamente, vá para 3 ramos.
2. O primeiro ramo verifica se a primeira configuração presente em w é uma é a configuração inicial de M com entrada x . Se não for, aceite.
3. O segundo ramo verifica se a última configuração presente em w contém o estado de aceitação de M . Se não tem, aceite.
4. O terceiro ramo decide, não-deterministicamente que chegou na configuração c_i . Então ele coloca c_i na pilha até encontrar o símbolo $\#$. A partir daí ele desempilha o conteúdo e compara com c_{i+1} . Se não é uma configuração válida, segundo a função de transição de M , aceite.
5. Rejeite.

A função de transição de M é implementada em P utilizando estados e transições. Perceba que P é um PDA criado especificamente para uma máquina M , ele não simula qualquer máquina. O único problema dessa descrição é que assumimos que estamos usando a mesma representação de histórico de computação do exemplo anterior $\#c_1\#c_2\#\dots\#c_n\#$. No entanto, ao empilhar c_i não podemos desempilhar e comparar diretamente com c_{i+1} porque ao desempilhar c_i será visto em ordem reversa.

Para resolver esse problema basta assumirmos que o histórico de computação é escrito como $\#c_1\#c_2^R\#c_3\#c_4^R\#\dots\#c_n\#$, ou seja, as configurações nas posições pares da lista são escritas em ordem reversa. Desse modo, ao desempilhar, as cadeias estão na ordem adequada para comparação.

Não há problemas em fazer isso pois o histórico de computação é representado da forma que decidimos, não existe nenhuma obrigação de escrevê-la da forma que assumimos no exemplo anterior.

Finalmente, considere a seguinte máquina de Turing:

S = para entrada $\langle M, x \rangle$, faça:

1. Construa o PDA P conforme descrito acima.
2. Converta P em uma CFG equivalente G .

3. Execute H com entrada $\langle G \rangle$.
4. Se H aceita, rejeite. Se H rejeita, aceite.

Se H aceita G , quer dizer que G gera todas as cadeias. Projetamos P , e por consequência G , para gerar todas as cadeias somente se M não aceita x , portanto S rejeita. Por outro lado, se H rejeita G , há alguma cadeia que G não gera e sabemos, por construção, que essa cadeia é o histórico de computação de M com x , portanto S aceita.

S é um decisor de A_{TM} . Como A_{TM} é indecidível, T_{CFG} também é.

Finalmente temos uma forma fácil de mostrar que a linguagem EQ_{CFG} , do exemplo 9, é de fato indecidível.

Exemplo 19: Dadas duas gramáticas livres de contexto, G e H , determine se G e H são equivalentes.

Esse problema pode ser descrito através da seguinte linguagem $EQ_{CFG} = \{\langle G, H \rangle : G \text{ e } H \text{ são CFGs e } \mathcal{L}(G) = \mathcal{L}(H)\}$.

Vamos reduzir T_{CFG} a EQ_{CFG} . Assuma que existe M que decide EQ_{CFG} . Apesar de T_{CFG} ser indecidível, é muito fácil projetar uma CFG que gera Σ^* . Se S é a variável inicial, basta que a gramática tenha as regras $S \rightarrow \epsilon$, $S \rightarrow AS$, e, para cada $a \in \Sigma$, $A \rightarrow a$. Considere a seguinte TM:

S = para entrada $\langle G \rangle$, faça:

1. Crie uma gramática H que gera Σ^* .
2. Execute M com entrada $\langle G, H \rangle$.
3. Retorne o mesmo que M .

Se M aceita, então G é equivalente a H , ou seja, $\mathcal{L}(G) = \Sigma^*$. Se M rejeita, então G não é equivalente a H , ou seja $\mathcal{L}(G) \neq \Sigma^*$. Portanto, S é um decisor de T_{CFG} . Como T_{CFG} é indecidível, concluímos que EQ_{CFG} também é.

Como um último exemplo, vamos abordar um pequeno quebra-cabeças.

Exemplo 20: O problema da correspondência de Post (PCP - *Post Correspondence Problem*), pode ser definido da seguinte forma. Dado um conjunto, P , de pares (p_i, t_i) , uma solução do problema é uma sequência $S = s_1, s_2, \dots, s_n$ tal que $p = p_{s_1} p_{s_2} \dots p_{s_n} = t_{s_1} t_{s_2} \dots t_{s_n} = t$, $n > 0$ e não necessariamente $s_i \neq s_j$. Ou seja, a concatenação de uma quantidade não nula dos primeiros elementos dos pares, na ordem dada por S e possivelmente com repetição, é igual à concatenação dos segundos elementos dos pares nas mesmas condições.

Por exemplo, se $P = \{(a, ab), (abc, c), (b, ca), (ca, a)\}$, podemos construir uma solução da seguinte forma. A única tupla em que os dois elementos começam com o mesmo símbolo é a primeira, então temos $s_1 = 1$. Depois disso, temos $p = a$ e $t = ab$, para que sejam iguais, é necessário ocorrer um b em p . A única tupla cujo primeiro elemento começa com b é a terceira, então temos $s_2 = 3$. Até o momento temos $p = ab$ e $t = abca$.

Seguindo esse raciocínio, chegamos à conclusão que a sequência $S = 1, 3, 4, 1, 2$ é uma solução, gerando a cadeia $p = t = abcaaabc$.

Suponha agora que $P = \{(a, ab), (a, bc), (b, ca), (c, aa)\}$. Esse conjunto claramente não tem solução, pois o segundo elemento de todas as tuplas é maior que o primeiro elemento. Então t vai ser sempre maior que p .

O problema da correspondência de Post é descrito pela seguinte linguagem PCP = $\{\langle P \rangle : P \text{ é uma instância válida do PCP e } P \text{ tem solução}\}$.

Vamos mostrar que esse problema é indecidível fazendo uma redução de A_{TM} a PCP. Vamos assumir que H decide PCP e vamos construir uma instância do PCP que só tem solução se M aceita x .

Primeiro, vamos considerar uma versão modificada do PCP onde s_1 é, obrigatoriamente, 1. Ou seja $MPCP = \{\langle P \rangle : P \text{ é uma instância válida do PCP e } P \text{ tem solução e a solução inicia com a primeira tupla de } P\}$.

Esta restrição extra facilita a criação da instância desejada, futuramente veremos como nos livrar dessa restrição. Seja $M = (Q, \Sigma, \Gamma, \sqcup, q_0, q_A, q_R)$ e $x = x_1x_2...x_m \in \Sigma^*$. Vamos construir uma instância P_M de MPCP que só tem solução se M aceita x . A solução do MPCP consiste na concatenação das tuplas até que $p = t$. A idéia é fazer com que as concatenações escrevam em p e t as configurações de M ao processar a cadeia x . p estará sempre um estado à frente de t . t só vai alcançar p , momento em que teremos uma solução do MPCP, se M aceita x .

Seguiremos passo a passo a construção dessa instância do PCP para facilitar a compreensão do seu funcionamento.

A primeira tupla de P_M será $(\#q_0x_1x_2...x_m\#, \#)$. Pela exigência do MPCP a solução precisa começar com essa tupla. p_1 corresponde à configuração inicial de M com entrada x e t_1 tem apenas o marcador $\#$.

Para obter uma solução, a cadeia t precisa alcançar a cadeia p . Além disso, queremos que p e t representem as configurações de M ao processar x . Vamos forçar a simulação de um passo de M adicionando tuplas para simular a função de transição. Nesta construção vamos assumir que a máquina M não tenta mover o cabeçote para a esquerda quando está na primeira célula da fita, se não for o caso, podemos alterar M para que funcione dessa forma. Também assumimos que a TM não deixa o cabeçote parado.

As tuplas que simulam a função de transição são as seguintes:

- Para todo $a, b \in \Gamma$ e todo $q, r \in Q$ onde $q \neq q_R$, se $\sqcup(q, a) = (r, b, D)$, adicione a tupla (br, qa) a P_M .
- Para todo $a, b, c \in \Gamma$ e todo $q, r \in Q$ onde $q \neq q_R$, se $\sqcup(q, a) = (r, b, E)$, adicione a tupla (rcb, cqa) a P_M .
- Para todo $a \in \Gamma$, adicione a tupla (a, a) a P_M .
- Insira as tuplas $(\#, \#)$ e $(\sqcup\#, \#)$ a P_M .

As tuplas do item (a) tratam do movimento para a direita do cabeçote, as do item (b) tratam do movimento para a esquerda do cabeçote, as do item (c) simplesmente copiam os símbolos que não estão adjacentes ao cabeçote. As tuplas do item (d) permitem inserir o símbolo que separa as configurações inclusive inserindo o símbolo $\sqcup$ no fim da configuração, para simular a fita infinita.

Apenas para verificar como a solução está sendo construída até o momento, considere que $\Gamma = \{0, 1, 2, \sqcup\}$, $\sqcup(q_0, 0) = (q_1, 2, D)$, $\sqcup(q_1, 1) = (q_2, 0, D)$, $\sqcup(q_1, 2) = (q_A, 0, D)$, $\sqcup(q_2, 0) = (q_0, 2, E)$ e $x = 0100$. Até esse momento, P_M tem as seguintes tuplas:

- $(\#q_00100\#, \#)$
- $(0, 0), (1, 1), (2, 2)$ e $(\sqcup, \sqcup)$
- $(2q_1, q_00), (0q_2, q_11)$ e $(0q_A, q_12)$
- $(q_002, 0q_20), (q_012, 1q_20), (q_022, 2q_20), (q_0\sqcup2, \sqcup q_20)$
- $(\#, \#)$ e $(\sqcup\#, \#)$

A solução começa com 1, que faz $p = \#q_00100\#$ e $t = \#$. O segundo elemento da solução deve ser alguma peça que começa com q_0 no segundo elemento. A única possibilidade é 6 e aí temos $p = \#q_00100\#2q_1$ e $t = \#q_00$. Depois usamos 3, 2 e 2 e teremos $p = \#q_00100\#2q_1100$ e $t = \#q_00100$. Em seguida é necessário inserir $\#$, usando a tupla 13. Temos agora $p = \#q_00100\#2q_1100\#$ e $t = \#q_00100\#$, perceba que t tem agora a configuração inicial e p tem a próxima configuração a partir da inicial. Seguindo este raciocínio podemos chegar até o seguinte ponto:

$$p = \#q_00100\#2q_1100\#20q_200\#2q_0020\#22q_120\#220q_A0\#220$$

$$t = \#q_00100\#2q_1100\#20q_200\#2q_0020\#22q_120\#220$$

Note que conforme a solução é construída, somos obrigados a efetuar a simulação de M . Sempre p terá comprimento maior que t , pois todas as tuplas adicionadas têm $|p_i| \geq |t_i|$. O processo vai continuar até que um estado de parada seja alcançado.

O que vamos fazer agora é acrescentar tuplas para que as duas cadeias fiquem iguais caso um estado de aceitação ocorra:

e) Para todo $a \in \Gamma$, acrescente (q_A, aq_A) e (q_A, q_Aa) a P_M .

Estas regras vão eliminar os símbolos presentes na configuração um a um, restando apenas q_A . Continuando nosso exemplo, foram acrescentadas as tuplas

- $(q_A, 0q_A), (q_A, 1q_A), (q_A, 2q_A), (q_A, \sqcup q_A), (q_A, q_A0), (q_A, q_A1), (q_A, q_A2)$ e $(q_A, q_A\sqcup)$

Seguindo com a construção da solução chegamos à seguinte situação:

$$p = \#q_00100\#2q_1100\#20q_200\#2q_0020\#22q_120\#220q_A0\#220q_A\#22q_A\#2q_A\#q_A\#$$

$$t = \#q_00100\#2q_1100\#20q_200\#2q_0020\#22q_120\#220q_A0\#220q_A\#22q_A\#2q_A\#$$

E finalmente, para fechar o casamento das cadeias, acrescente a tupla $(\#, q_A\#\#)$ a P_M . Daí obtemos:

$$p = \#q_00100\#2q_1100\#20q_200\#2q_0020\#22q_120\#220q_A0\#220q_A\#22q_A\#2q_A\#q_A\#\#$$

$$t = \#q_00100\#2q_1100\#20q_200\#2q_0020\#22q_120\#220q_A0\#220q_A\#22q_A\#2q_A\#q_A\#\#$$

Isso conclui a construção de P_M , que só tem solução se M aceita x . Bom, agora falta criar uma instância de PCP com a mesma propriedade. A única diferença entre PCP e MPCP é a exigência de iniciar a solução com a primeira tupla na versão modificada. Se simplesmente considerarmos que P_M é uma instância de PCP, ela obviamente tem solução, independentemente de M aceitar x ou não, basta usar uma das tuplas (a, a) como solução.

Como no caso da linguagem T_{CFG} , vamos modificar a forma que representamos o histórico de computação. Seja $w = w_1w_2\dots w_n$ qualquer cadeia. Defina as seguintes operações:

- $\star w$ adiciona o símbolo $\star$ antes de cada símbolo de w .
- $w\star$ adiciona o símbolo $\star$ depois de cada símbolo de w .
- $\star w\star$ adicione o símbolo $\star$ antes do primeiro símbolo de w , depois do último símbolo de w e entre os símbolos de w .

Para converter P_M em uma instância P de PCP que simula a TM M com entrada x , faça o seguinte. Suponha que $P_M = \{(p_1, t_1), (p_2, t_2), (p_3, t_3), \dots, (p_k, t_k)\}$ então P será $P = \{(\star p_1, \star t_1 \star), (\star p_2, \star t_2 \star), (\star p_3, \star t_3 \star), \dots, (\star p_k, \star t_k \star), (\star \bullet, \bullet)\}$. A única tupla que pode iniciar uma solução de P é a primeira, por causa da inserção dos símbolos $\star$. Além de forçar que a solução comece com a primeira tupla, a presença dos $\star$ não afeta as possíveis combinações, porque eles simplesmente se alternam com os símbolos originais. Os símbolos originais agora acontecem apenas nas posições pares da solução. A tupla $(\star \bullet, \bullet)$ está presente para que possa aparecer o $\star$ que faltava no final de p .

De posse dessa instância P do PCP, podemos criar uma decisor para A_{TM} :

S = para entrada $\langle M, x \rangle$, faça:

1. Usando M e x , crie a instância P_M do MPCP, como descrito acima.
2. Converta P_M em P , conforme a discussão anterior.
3. Execute H com entrada $\langle P \rangle$.
4. Retorne o mesmo que H .

Lembre que H é o decisor de PCP. Da construção dada acima, P só tem solução se M aceita x . Portanto, a máquina S é uma redução de A_{TM} a PCP. Portanto, concluímos que PCP não é decidível.

Vimos vários exemplos de linguagens decidíveis e indecidíveis. Perceba que na maior parte dos casos as provas foram feitas através de reduções ou do teorema de Rice. Vamos tratar agora das linguagens irreconhecíveis.

Linguagens Irreconhecíveis

Até agora não discutimos as linguagens irreconhecíveis, ou seja, que não têm uma máquina de Turing que as reconheça. Já vimos um teorema que diz que uma linguagem é decidível se, e somente se, é Turing-reconhecível e co-Turing-reconhecível.

Por consequência, dos exemplos que vimos até agora, podemos concluir que $\neg A_{TM}$ é irreconhecível, pois A_{TM} é reconhecível e indecidível. Pelos teoremas vistos até agora também podemos concluir que se $\neg A_{TM}$ é redutível a alguma linguagem L , L também não é Turing-reconhecível.

Existem muitas linguagens que não são Turing-reconhecíveis? Podemos mostrar que a maioria das linguagens não é Turing-reconhecível. Para isso, vamos começar investigando um pouco o tamanho dos conjuntos infinitos.

Conjuntos Contáveis

Um conjunto C é contável se a sua cardinalidade é menor ou igual à cardinalidade dos naturais. Se C é finito, isso é óbvio. Mas e se C é infinito? Como comparar o seu tamanho aos naturais, que também é infinito?

Dizemos que dois conjuntos, A e B , têm o mesmo tamanho se é possível definir uma função f tal que $f(a) \neq f(b)$ sempre que $a \neq b$ e para todo elemento b de B existe um elemento a de A tal que $f(a) = b$, essa função é uma correspondência. Em uma correspondência, cada elemento de A mapeia para um único elemento de B e todo

elemento de B tem um único elemento de A que mapeia para ele. É simplesmente uma forma de parear os elementos de A com os elementos de B.

Exemplo: $|\mathbb{N}| = |\mathbb{Z}|$

O conjunto dos inteiros é infinito nas duas direções, positivo e negativo, enquanto o conjunto dos naturais é infinito em apenas uma direção. Aparentemente, podemos acreditar que os inteiros têm o dobro do tamanho dos naturais, no entanto isso não é verdade.

Seja $f(x) = x/2$, se x é par e $f(x) = -(x + 1)/2$, se x é ímpar. Essa função mapeia cada número natural em um número inteiro, os números pares são mapeados para os inteiros positivos e os números ímpares para os inteiros negativos. Similarmente, $g(x) = 2x$, se $x \geq 0$ e $g(x) = -(2x + 1)$, se $x < 0$, mapeia cada número inteiro em um número natural, os positivos são mapeados nos naturais pares enquanto os negativos são mapeados nos ímpares.

Exemplo: $|\mathbb{N}| = |\mathbb{N}^2|$

Mais uma vez, temos um conjunto que aparenta ser muito maior que outro. No entanto, usando um argumento similar àquele usado no exemplo 1 da seção de decidibilidade, podemos mostrar que estes dois conjuntos são de mesma cardinalidade.

Os elementos de $\mathbb{N}^2$ são tuplas onde cada elemento pertence aos naturais. Se tentarmos enumerar as tuplas mantendo o primeiro elemento fixo e incrementando o segundo, ou vice-versa, nunca incrementamos o primeiro elemento, pois nunca chegamos ao fim das possibilidades para o segundo elemento. Como no exemplo anterior, o segredo é enumerar os elementos de forma alternada em todas as direções de crescimento.

No caso dos números inteiros, o crescimento se dá no sentido dos positivos e negativos. Para enumerá-los demos um passo em cada direção de crescimento, ou, alternativamente, listamos todos os números em ordem crescente de valor absoluto.

No caso de $\mathbb{N}^2$, vamos listar as tuplas em ordem crescente da soma dos seus valores. Para todas as tuplas que têm a mesma soma, vamos listá-las em ordem crescente do primeiro elemento. Portanto, as dez primeiras tuplas na enumeração serão (0, 0), (0, 1), (1, 0), (0, 2), (1, 1), (2, 0), (0, 3), (1, 2), (2, 1) e (3, 0). Para cada valor de soma existe uma quantidade finita de elementos. Para ser mais exato, neste caso temos $(s + 1)$ elementos

com soma s . Portanto, existem exatamente $\sum_{i=1}^s i = s(s + 1)/2$ elementos com comprimento menor que s , onde $s = x + y$.

Agora precisamos de uma correspondência para associar cada uma destas tuplas a um número natural distinto. A função $f(x, y) = (x + y)(x + y + 1)/2 + x$ resolve esse problema. O primeiro termo é o total de termos com soma menor que $(x+y)$ o segundo termo simplesmente soma x , que foi a ordem em que decidimos listar as tuplas de mesmo comprimento.

Também é possível definir um mapeamento de $\mathbb{N}$ em $\mathbb{N}^2$ com a função $g(n) = (x, y)$, onde $x = n - k(k+1)/2$, $y = k - x$ e $k = \lfloor ((8n + 1)^{1/2} - 1)/2 \rfloor$. Aqui k foi determinado como a soma $(x+y)$ associada à tupla de índice n . $\lfloor \bullet \rfloor$ é o operador *ínfimo* (arredondar para baixo).

A tabela abaixo mostra o resultado destas funções para as 10 tuplas listadas anteriormente conforme a correspondência proposta.

n	(x, y)	$f(x, y)$	$g(n)$
0	(0, 0)	$(0 + 0)(0 + 0 + 1)/2 + 0 = 0$	$k = 0; x = 0 - 0(0+1)/2 = 0; y = 0$
1	(0, 1)	$(0 + 1)(0 + 1 + 1)/2 + 0 = 1$	$k = 1; x = 1 - 1(1+1)/2 = 0; y = 1$
2	(1, 0)	$(1 + 0)(1 + 0 + 1)/2 + 1 = 2$	$k = 1; x = 2 - 1(1+1)/2 = 1; y = 0$
3	(0, 2)	$(0 + 2)(0 + 2 + 1)/2 + 0 = 3$	$k = 2; x = 3 - 2(2+1)/2 = 0; y = 2$
4	(1, 1)	$(1 + 1)(1 + 1 + 1)/2 + 1 = 4$	$k = 2; x = 4 - 2(2+1)/2 = 1; y = 1$
5	(2, 0)	$(2 + 0)(2 + 0 + 1)/2 + 2 = 5$	$k = 2; x = 5 - 2(2+1)/2 = 2; y = 0$
6	(0, 3)	$(0 + 3)(0 + 3 + 1)/2 + 0 = 6$	$k = 3; x = 6 - 3(3+1)/2 = 0; y = 3$
7	(1, 2)	$(1 + 2)(1 + 2 + 1)/2 + 1 = 7$	$k = 3; x = 7 - 3(3+1)/2 = 1; y = 2$
8	(2, 1)	$(2 + 1)(2 + 1 + 1)/2 + 2 = 8$	$k = 3; x = 8 - 3(3+1)/2 = 2; y = 1$
9	(3, 0)	$(3 + 0)(3 + 0 + 1)/2 + 3 = 9$	$k = 3; x = 9 - 3(3+1)/2 = 3; y = 0$

Utilizando uma argumentação similar, podemos mostrar que o conjunto de todas as cadeias sobre um alfabeto Σ , ou seja, Σ^* também é contável. Perceba que para um dado comprimento n existe um número finito, $|\Sigma|^n$, de cadeias sobre o alfabeto. Se listarmos estas cadeias em ordem lexicográfica, podemos associar um número natural para cada cadeia.

Se o seu alfabeto é simbólico, sem ordem clara, associe um número para cada símbolo e construa as cadeias em ordem crescente do rótulo associado ao símbolo. Esse é um resultado importante: $|\Sigma^*| = |\mathbb{N}|$.

Conjuntos Não Contáveis

Resta uma questão: existem conjuntos maiores que os naturais? Sim, os conjuntos não contáveis.

Exemplo: $|\mathbb{R}| > |\mathbb{N}|$

Para mostrar que $\mathbb{R}$ é incontável, precisamos mostrar que não existe correspondência entre $\mathbb{N}$ e $\mathbb{R}$. Essa prova será por contradição. Assuma que existe uma correspondência f entre $\mathbb{N}$ e $\mathbb{R}$. Portanto, f associa cada elemento de $\mathbb{N}$ a cada elemento de $\mathbb{R}$. Vamos achar um elemento de $\mathbb{R}$ que não foi associado a nenhum elemento de $\mathbb{N}$, que será nossa contradição.

Construa um número real x de tal modo que cada dígito de x difere, em pelo menos um dígito, de cada número real que foi pareado. Para ilustrar a construção de x , assumamos que a tabela abaixo mostra alguns dos valores da hipotética correspondência entre $\mathbb{N}$ e $\mathbb{R}$.

n	$f(n)$
0	0.00000...

1	5.12345...
2	2.45213...
3	3.12121...
4	8.94023...
$\vdots$	$\vdots$

Sem perda de generalidade, vamos assumir que a parte inteira de x é 0. O primeiro dígito decimal de x vai ser escolhido para diferir do primeiro dígito decimal de $f(1)$, o segundo dígito decimal deve ser diferente daquele de $f(2)$, e assim sucessivamente, de tal modo que o i -ésimo dígito decimal de x difere do i -ésimo dígito decimal de $f(i)$.

Não vamos permitir que os dígitos decimais de x sejam 0 ou 9 para evitar um pequeno problema técnico que números como 0.199999... e 2.000000... são idênticos apesar da representação decimal ser distinta.

Pela tabela dada podemos ter $x = 0.13334...$, de modo que x é diferente de todos os números presentes na tabela. Portanto, a correspondência f não mapeou todos os reais e concluímos que é impossível existir tal correspondência. Portanto, $\mathbb{R}$ não é contável.

Utilizando um argumento similar, podemos mostrar que o conjunto de todas as cadeias binárias infinitas também é incontável. Basta assumir que a correspondência existe e criar uma nova cadeia binária que difere em pelo menos um dígito de cada uma das cadeias que foram mapeadas.

Como tudo isso pode nos ajudar a mostrar que existem muitas linguagens que não são Turing-reconhecíveis?

Já sabemos que as representações de máquinas de Turing podem ser escritas como cadeias sobre um alfabeto Σ . Portanto, o conjunto de todas as máquinas de Turing é tão grande quanto Σ^* , ou seja, é um conjunto contável.

Uma linguagem nada mais é que um subconjunto de Σ^* . Podemos representar uma linguagem através de uma cadeia binária infinita, para tal, associe cada dígito da cadeia binária a uma cadeia de Σ^* e preencha o dígito com 1 se a cadeia faz parte da linguagem ou com 0 se ela não faz parte. Portanto, o conjunto de todas as linguagens sobre um alfabeto Σ é o conjunto de todos os subconjuntos de Σ^* que, por sua vez, é o mesmo que o conjunto de todas as cadeias binárias infinitas. Já argumentamos anteriormente que este conjunto não é contável.

Daí concluímos que o conjunto de todas as linguagens é maior que o conjunto de todas as máquinas de Turing. De fato, o conjunto das linguagens é infinitamente maior que o conjunto das máquinas de Turing, portanto, concluímos que a maioria das linguagens não é Turing-reconhecível.

Apesar desse resultado teórico, é importante salientar que, na prática, muitas destas linguagens não têm interesse. Algumas delas são impossíveis até de descrever, como as linguagens formadas por cadeias selecionadas aleatoriamente.

Por fim, vamos dar um exemplo que não tínhamos discutido até agora. Sabemos que EQ_{TM} não é decidível, o que quer dizer que EQ_{TM} não é Turing-reconhecível ou $\neg\text{EQ}_{\text{TM}}$ não é Turing-reconhecível (o “ou” nessa frase é o ou lógico).

Exemplo: EQ_{TM} não é Turing-reconhecível nem co-Turing-reconhecível.

Primeiro, vamos mostrar que A_{TM} é redutível a $\neg EQ_{TM}$. A função de redução é:

F = Para entrada $\langle M, x \rangle$, faça:

1. Construa M_1 tal que M_1 rejeita tudo.
2. Construa M_2 tal que M_2 aceita tudo se M aceita x e nada, caso contrário.
3. Retorne $\langle M_1, M_2 \rangle$

Portanto, se M aceita x , M_1 e M_2 não são equivalentes, e se M não aceita x , M_1 e M_2 são equivalentes. Então F reduz A_{TM} a $\neg EQ_{TM}$, como desejado. Como é uma redução por mapeamento, $A_{TM} \leq_m \neg EQ_{TM} \Leftrightarrow \neg A_{TM} \leq_m EQ_{TM}$. Sabendo que $\neg A_{TM}$ não é Turing-Reconhecível, concluímos que EQ_{TM} também não é.

Agora vamos mostrar que A_{TM} é redutível a EQ_{TM} . A função de redução é:

F = Para entrada $\langle M, x \rangle$, faça:

1. Construa M_1 tal que M_1 aceita tudo.
2. Construa M_2 tal que M_2 aceita tudo se M aceita x e nada, caso contrário.
3. Retorne $\langle M_1, M_2 \rangle$

Se M aceita x , M_1 e M_2 são equivalentes e não são se M não aceita x . Então G reduz A_{TM} a EQ_{TM} . Como no caso anterior temos $A_{TM} \leq_m EQ_{TM} \Leftrightarrow \neg A_{TM} \leq_m \neg EQ_{TM}$. Portanto, $\neg EQ_{TM}$ não é Turing-reconhecível.

O teorema de Rice também trata da irreconhecibilidade de algumas linguagens, como enunciado a seguir.

Teorema: Teorema de Rice (2)

Qualquer propriedade não-monotônica da linguagem reconhecida por uma máquina de Turing não é Turing-reconhecível.

Para uma propriedade ser não-monotônica devem haver pelo menos duas linguagens Turing-reconhecíveis tais que a propriedade vale para uma e não para a outra. Além disso, a linguagem que satisfaz a propriedade deve ser um subconjunto próprio da linguagem que não satisfaz.

Exemplo: $F_{TM} = \{ \langle M \rangle : M \text{ é uma TM e } |\mathcal{L}(M)| < \infty \}$.

Já vimos que essa linguagem não é decidível. A linguagem $\emptyset$ é Turing-reconhecível e finita. A linguagem Σ^* é Turing-reconhecível e infinita. Além disso, $\emptyset \subset \Sigma^*$. Portanto, pelo teorema de Rice, esta linguagem não é Turing-reconhecível.

Exemplo: $R_{TM} = \{ \langle M \rangle : M \text{ é uma TM e } \mathcal{L}(M) \text{ é regular} \}$.

Já vimos que essa linguagem não é decidível. A linguagem $\emptyset$ é Turing-reconhecível e regular. A linguagem $\{0^n 1^n : n \geq 0\}$ é Turing-reconhecível e não é regular. Além disso, $\emptyset \subset \{0^n 1^n : n \geq 0\}$. Daí, temos que R_{TM} não é Turing-reconhecível.

Propriedades de Fechamento

Assim como fizemos para as outras linguagens, vamos enunciar algumas operações sob as quais as linguagens Turing-decidíveis e Turing-reconhecíveis são fechadas.

Teorema

As linguagens Turing-decidíveis e Turing-reconhecíveis são fechadas sob a operação de união.

Prova

Seja M_1 uma TM que reconhece a linguagem A e M_2 uma TM que reconhece a linguagem B. Considere a seguinte TM:

S = para entrada x, faça:

1. Simule M_1 e M_2 simultaneamente com entrada x.
2. Se M_1 ou M_2 aceitam, aceite.
3. Se M_1 e M_2 rejeitam, rejeite.

Como as duas máquinas são simuladas simultaneamente, dando um passo de simulação em cada uma, S aceita se qualquer uma das máquinas aceitar, mesmo que uma delas entre em laço infinito. Portanto S reconhece $A \cup B$.

Se M_1 e M_2 são decisores, S é um decisor de $A \cup B$. Isso porque a única forma de S entrar em um laço infinito é se M_1 e M_2 estão em laço infinito, se são decisores isso nunca ocorre.

Teorema

As linguagens Turing-decidíveis e Turing-reconhecíveis são fechadas sob a operação de concatenação.

Prova

Seja M_1 uma TM que reconhece a linguagem A e M_2 uma TM que reconhece a linguagem B. Considere a seguinte TM:

S = para entrada x, faça:

1. Não-deterministicamente, divida x em duas partes, $x=vw$.
Em cada ramo não-determinístico de computação:
2. Simule simultaneamente M_1 com entrada v e M_2 com entrada w.

3. Se M_1 e M_2 aceitam, aceite.
4. Se M_1 ou M_2 rejeitam, rejeite.

Neste caso usamos o não-determinismo apenas para simplificar a descrição da máquina, mas já sabemos que toda TM não determinística tem um equivalente determinístico. Supondo que a cadeia de entrada tem comprimento n , existem $(n + 1)$ pares $v_i w_i$ possíveis. Poderíamos computá-los e simular simultaneamente M_1 em todos os v_i e M_2 em todos os w_i . Se ambas aceitam para um mesmo índice i , aceite.

S aceita se M_1 aceita a primeira parte de x e M_2 aceita a segunda parte. Portanto S reconhece AB.

Se M_1 e M_2 são decisores, S é um decisor de AB. Isso porque a única forma de S entrar em um laço infinito é se M_1 ou M_2 estão em laço infinito, se são decisores isso nunca ocorre.

Teorema

As linguagens Turing-decidíveis e Turing-reconhecíveis são fechadas sob a operação estrela.

Prova

Seja M uma TM que reconhece a linguagem A. Considere a seguinte TM:

S = para entrada x , faça:

1. Não-deterministicamente, divida x em partes, $x = p_1 p_2 \dots p_n$.
Em cada ramo não-determinístico de computação:
2. Simule simultaneamente M com cada uma das partes.
3. Se todas as simulações aceitam, aceite.
4. Se pelo menos uma simulação rejeita, rejeite.

Novamente usamos o não-determinismo para simplificar a descrição da máquina. Supondo que a cadeia de entrada tem comprimento n , existe uma quantidade finita de formas que podemos dividi-la em um certo número de partes variando de 1 a n . Se considerarmos que as partes não podem ser vazias, existem 2^{n-1} formas de dividir a cadeia em partes. Poderíamos computá-las e simular simultaneamente M em todas as partes. Se todas as simulações aceitam para uma mesma divisão, aceite.

S aceita se M se M aceita todas as partes de x para uma dada divisão. Portanto S reconhece A^* .

Se M é um decisor, S é um decisor de A^* . Isso porque a única forma de S entrar em um laço infinito é se M está em laço infinito, e isso nunca ocorre.

Teorema

As linguagens Turing-decidíveis e Turing-reconhecíveis são fechadas sob a operação de união.

Prova

Seja M_1 uma TM que reconhece a linguagem A e M_2 uma TM que reconhece a linguagem B. Considere a seguinte TM:

S = para entrada x , faça:

1. Simule M_1 e M_2 simultaneamente com entrada x .
2. Se M_1 e M_2 aceitam, aceite.
3. Se M_1 ou M_2 rejeitam, rejeite.

S aceita se ambas as máquinas aceitam. Portanto S reconhece $A \cap B$.

Se M_1 e M_2 são decisores, S é um decisor de $A \cap B$. Isso porque a única forma de S entrar em um laço infinito é se M_1 e M_2 estão em laço infinito, se são decisores isso nunca ocorre.

Teorema

As linguagens Turing-decidíveis são fechadas sob a operação de complementação.

Prova

Seja M uma TM que decide a linguagem A. Considere a seguinte TM:

S = para entrada x , faça:

1. Simule M com entrada x .
2. Se M aceita, rejeite.
3. Se M rejeita, aceite.

S aceita se M rejeita e vice-versa. Portanto S decide $\neg A$.

Atenção, o último teorema só vale para linguagens Turing-decidíveis. Já vimos que as linguagens Turing-reconhecíveis não podem ser fechadas sob complementação. Se fossem, elas seriam também co-Turing-reconhecíveis e, consequentemente, Turing-decidíveis.

O subconjunto das linguagens Turing-reconhecíveis que é fechado sob complementação é justamente o conjunto das linguagens Turing-decidíveis.

Considerações Finais

Neste texto exploramos os limites teóricos da computação através do estudo das linguagens e das máquinas que as reconhecem. Vimos modelos de computação cada vez mais poderosos, em termos de capacidade de representação, até chegarmos às máquinas de Turing, que é o modelo mais poderoso que se conhece. Basicamente, as máquinas de

Turing são equivalentes a qualquer modelo de computação que permita acesso irrestrito à memória infinita.

Estes modelos foram propostos na década de 30, antes da criação das máquinas eletro-mecânicas que hoje chamamos de “computador”. A máquina de Turing é a base da arquitetura de von Neumann, criada em 1952, e que ainda é a base dos computadores digitais até hoje. O conceito da máquina de Turing universal, uma máquina capaz de simular qualquer outra máquina, é fundamental na arquitetura. Apesar de ser o modelo base, é importante lembrar que a TM é um modelo teórico, na prática não existe tal máquina já que não dá para implementar memória infinita. O que temos, na prática, é uma quantidade de memória tão grande que pode ser considerada infinita em diversos casos.

Como as linguagens determinam o que pode ou não ser feito por um computador? Lembre que todo problema de decisão, que tem resposta binária, pode ser associado à pertinência ou não a uma linguagem. Mas, por que estudar apenas os problemas de decisão? Há dois motivos para isso, um mais fraco, que é histórico e outro mais forte que é teórico. Lembre que o que motivou a criação das máquinas de Turing foram os problemas de decisão de Hilbert, este é o motivo histórico. Do ponto de vista teórico, os problemas de cômputo de função (dados uma função e seus argumentos, determinar o valor da saída), busca (dada a saída de uma função, determinar valores para seus argumentos) e até otimização (buscar valores para os argumentos de uma função, possivelmente sujeitos a condições, de modo a maximizar ou minimizar sua saída) podem ser convertidos em problemas de decisão. Portanto, estudar os problemas de decisão é suficiente para determinar os limites teóricos da computação.

Dizemos que um problema tem solução algorítmica se a linguagem que o representa é Turing-decidível. Os problemas Turing-reconhecíveis podem ser considerados parcialmente solúveis, já que há o risco de não obter resultado para certas entradas. É aí onde está a determinação do limite da computação, alguns problemas não podem ser resolvidos por um computador mesmo que tenha tempo e memória infinitos. Vale lembrar que dizemos que um problema não é solúvel se há pelo menos uma instância para a qual não conseguimos solução. Uma consequência disso é que, na prática, muitos problemas insolúveis podem ser resolvidos se considerarmos algumas restrições extras, ou seja, se olharmos apenas para um subconjunto das instâncias do problema. O 10º problema de Hilbert é um exemplo disso, ele não é decidível no caso geral mas é decidível se considerarmos apenas polinômios de uma variável.

Ao longo do texto fiz questão de dizer que estamos estudando os limites *teóricos* da computação, que um modelo é mais poderoso que o outro *em termos de capacidade de representação* e em momento algum foi discutido a eficiência dessas máquinas. As máquinas de Turing não-determinísticas, por exemplo, têm a mesma capacidade de representação da versão determinística mas é potencialmente mais eficiente, já que pode efetuar diversos ramos de computação em paralelo.

Enquanto estudamos aqui o limite do que é teoricamente possível, considerando que tempo e memória são recursos infinitos, há um ramo da teoria da computação que estuda a “dificuldade” em resolver os problemas, que é a Teoria da Complexidade. Dificuldade é um termo vago e, dentro deste ramo, ela é definida em termos dos custos dos recursos de tempo (quantidade de operações) e espaço (memória), que são expressos através de funções do tamanho da entrada. Os problemas são divididos em classes de dificuldade que são relacionadas às máquinas de Turing. Talvez você já tenha ouvido falar das classes P e NP de complexidade temporal, fazem parte da classe P todos os problemas para os quais há uma máquina de Turing determinística, com uma fita, que o resolve e tem uma função de

custo temporal que é um polinômio ou alguma função que cresce mais lentamente que um polinômio. A classe NP tem uma definição similar, bastando trocar o determinístico por não-determinístico, vindo daí o N antes do P. Outra definição possível é dizer que um problema é da classe NP se o custo para verificar se uma solução é válida tem custo polinomial usando uma máquina determinística. Claramente, $P \subset NP$, mas se NP é igual ou não a P é uma questão aberta na teoria da complexidade.

Existem muitas outras classes de complexidade e elas nos ajudam a entender o quão difícil é resolver um dado problema. Não vou me alongar nesse assunto, ficando para um próximo texto (ou disciplina). Mesmo não tendo estudado, vou apresentar uma ressalva similar àquela acerca das linguagens indecidíveis. Um problema é da classe NP, por exemplo, se tem pelo menos uma instância que não pode ser resolvida em tempo polinomial por uma máquina determinística. No entanto, é possível que as instâncias do problema que interessam a você sejam mais simples, ou talvez o tamanho das instâncias não sejam proibitivos. Vimos um exemplo disso na disciplina de lógica, o problema de determinar a satisfatibilidade de fórmulas Booleanas pertence à classe NP, ele tem um custo exponencial no pior caso. Já a determinação da satisfatibilidade de fórmulas Booleanas em alguns formatos específicos, como 2-CNF e cláusulas de Horn, pertence à classe P, têm custo linear no pior caso. Por que simplesmente não convertamos as fórmulas para um desses formatos e usamos um algoritmo eficiente? Como visto naquela disciplina, o processo de conversão tem custo exponencial e o tamanho da fórmula também cresce de forma exponencial, no pior caso.

Dada uma ideia acerca do tipo de assunto estudado na teoria da complexidade, encerro aqui este texto. Espero que ele tenha ajudado vocês a se interessar pelo assunto, ou pelo menos tenha ajudado a acompanhar a disciplina.

Linha
Margem
Etcétera e tal
Pronto
Ponto
E ponto final

(Chico Buarque. Meu Caro Barão. In: Saltimbancos Trapalhães)